AWS ACADEMY CHALLENGE LABS · TRAINING ENVIRONMENT

V1 → V7

AWS Café

Cloud Architecture Journey

From a Static Website to a Scalable, Highly Available, Automated and Serverless Architecture

Amazon S3 | EC2 | RDS | VPC | ALB + Auto Scaling | CloudFormation | CodePipeline | Lambda | SNS

Seven stages · 189 evidence screenshots · architecture decisions, challenges and lessons

Contents

PART I — INTRODUCTION

1 Project Overview 4

PART II — THE ARCHITECTURE JOURNEY

2 V1 — Static Website on Amazon S3 7

3 V2 — Dynamic Website on Amazon EC2 14

4 V3 — Database Migration to Amazon RDS 23

5 V4 — Secure VPC Networking 35

6 V5 — Scalability & High Availability (ALB + Auto Scaling) 50

7 V6 — Infrastructure Automation (CloudFormation + CodePipeline) 63

8 V7 — Serverless Reporting (Lambda + SNS) 86

PART III — THE COMPLETE ARCHITECTURE

9 Final Architecture 103

10 Key Decisions & Lessons Learned 106

APPENDICES

A AWS Services Used 109

B Configuration Reference 112

C Evidence Index 113

PART I

Introduction

The business case, the roadmap and how this book is built.

CHAPTER 1

Project Overview

> Completed as part of AWS Academy Challenge Labs (training environment).

1.1 The Café Business Case

AWS Café is a fictional café used as the running story of the AWS Academy Cloud Architecting challenge labs. The owners are a retired baker and a retired accountant. They are not technical. Their daughter, who runs operations (supply chain, inventory and some programming), and a part-time employee with visual design skills are the people who build the technology. AWS consultants advise them: a solutions architect for databases and networking, a developer for security, and a SysOps specialist for automation, backup and disaster recovery. In every lab I worked in the role of the café's operations staff.

The café starts with no online presence. Each lab adds one new business requirement, and each requirement forces an architecture change: a website, then online ordering, then a managed database, then a secure network, then high availability, then automated deployment, and finally serverless reporting. This book follows that evolution and records what I built, what went wrong and why I chose each solution.

1.2 Architecture Roadmap (V1 → V7)

STAGE	BUSINESS REASON	TECHNICAL UPDATE
V1	Build a static website for a small business	Host the website on Amazon S3
V2	Dynamic content and online ordering	Web application and database on Amazon EC2
V3	Reduce database maintenance effort and secure the data	Separate web and database layers; migrate the database to Amazon RDS in private subnets
V4	Improve the security of the web application	VPC with public and private subnets
V5	Handle an expected traffic increase and stay highly available	Load balancer and Auto Scaling on EC2, instances across two Availability Zones
V6	Consistent deployments across Regions	CloudFormation templates (version-controlled) for network and application layers, deployed to another Region
V7	Reporting with lower operations effort and lower cost	Lambda functions connecting to RDS and generating scheduled reports

1.3 Scope & Methodology

- **Training environment.** All seven stages were completed in AWS Academy Challenge Labs. They are not production systems.
- **Separate lab environments.** Each lab ran in its own lab environment, so the stages are connected by the story and the design, not by one continuously running deployment.

- **My own words.** The lab instructions are not copied. Each chapter explains the requirement, my decisions and my problem-solving.
- **Only what I did.** The text and the screenshots record what I actually built and observed. Nothing is added that I did not do.
- **How each chapter is built.** Business requirement; the start and target architecture from the lab; my own before/after diagram; services used; implementation highlights; challenges and decisions; the full set of screenshots; the outcome.
- **Evidence.** Every screenshot has an ID (for example V4-E12) listed in Appendix C. Account IDs, ARNs, e-mail addresses and personal paths were masked.
- **Lab diagrams.** The architecture figures taken from the lab are labelled as AWS Academy reference diagrams.

PART II

The Architecture Journey

Seven stages, each driven by a new business requirement.

CHAPTER 2 · STAGE V1

V1 — Static Website on Amazon S3

> Completed as part of AWS Academy Challenge Labs (training environment).

Business Requirement

The café is a single-location business that gains customers mostly from foot traffic and word of mouth. It has no web presence and no cloud services yet. The owners want to make the café visible online with a simple website, and to do it in a way that protects the site's content and keeps storage costs under control.

Lab objectives

- Host a static website on Amazon S3
- Protect the data with at least one S3 data-protection feature
- Implement a data lifecycle strategy
- Implement a disaster recovery (DR) strategy

Target Architecture

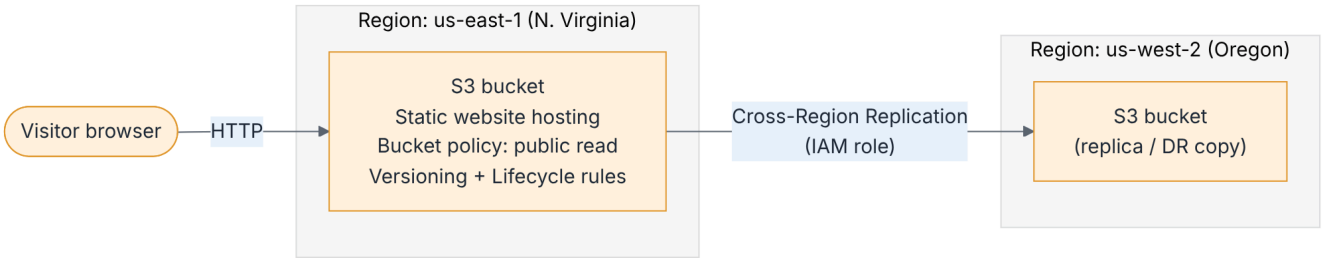
This diagram comes from the AWS Academy lab material and shows the design this stage builds. It is shown as reference; my own diagram of what I deployed follows in the next section.

Target architecture. AWS Academy lab diagram: the design this stage builds.

Before → After

	BEFORE	AFTER
Web presence	None	Public static website served from Amazon S3
Data protection	–	Versioning enabled (previous versions of the page are kept)
Storage cost control	–	Lifecycle rules for old versions

	BEFORE	AFTER
Disaster recovery	–	Cross-Region Replication to a bucket in another Region



AWS Services Used

SERVICE / FEATURE	ROLE IN V1
Amazon S3	Stores the website files (`index.html` , CSS, images)
S3 static website hosting	Serves the bucket content through an S3 website endpoint
Bucket policy	Allows public read-only access (`s3:GetObject`) so visitors can load the site
S3 Versioning	Keeps previous versions of objects; protects against overwrites
S3 Lifecycle rules	Moves old versions to a cheaper class and later deletes them
S3 Cross-Region Replication (CRR)	Copies new/changed objects to a bucket in a second Region (DR)
IAM role	Lets S3 replicate objects between the two buckets

Implementation Highlights

1. **Bucket and content.** Created an S3 bucket in `us-east-1` and uploaded the site files: `index.html` , a `css/` folder and an `images/` folder (9 files in the folders plus the HTML page). *(E01)*
2. **Static website hosting.** Enabled static website hosting in the bucket properties and used the bucket's website endpoint to open the site.
3. **Public access.** The site was not reachable at first (see Challenges). I turned off *Block all public access* and added a bucket policy that allows read-only `GetObject` on the bucket's objects. After that the site loaded from the website endpoint. *(E02, E03)*
4. **Versioning (data protection).** Enabled versioning on the bucket. To test it, I edited the page's HTML locally (changed a few background colors), re-uploaded `index.html` , and checked that S3 now kept both the old and the new version. *(E04, E05)*
5. **Lifecycle strategy.** Because versioning makes the bucket grow with every upload, I created two lifecycle rules that apply to the entire bucket and only target *noncurrent* (previous) versions: *(E06, E07, E08)* - `MovePreviousVersionsToStandardIA` — transitions previous versions to **Standard-IA** after **30 days** - `DeletePreviousVersions` — permanently deletes previous versions after **365 days**
6. **Disaster recovery.** Created a second bucket in **us-west-2 (Oregon)** and configured a replication rule on the source bucket (`CafeCrossRegionReplication` , scope: entire bucket, destination: the Oregon

bucket, using an IAM role). I then edited the HTML again, uploaded it to the source bucket, and confirmed that the new version appeared in the destination bucket. *(E09, E10)*

Challenges & Decisions

1. The website returned no content even though hosting was enabled. After enabling static website hosting, opening the website endpoint did not show the site. The cause was that public access was still blocked: *Block all public access* had been turned on when I created the bucket. To add a public-read bucket policy, I first had to turn that setting off, then add a policy that allows only `s3:GetObject` (read-only) on the bucket's objects. I chose read-only access deliberately so the public can view the site but cannot list, modify or delete anything.

2. Versioning solves one problem and creates another. Versioning protects the page from accidental overwrites, but every re-upload adds another stored version, so the bucket grows indefinitely. That is why I paired versioning with lifecycle rules: move old versions to a cheaper storage class, then delete them after a retention period.

3. Replication does not copy everything automatically. After I created the replication rule, the destination bucket contained only the `index.html` that I uploaded afterwards, not the `css/` and `images/` files that already existed. The console message on the rule explains this: replication applies only to **new** objects, and existing objects need a separate replication job. For a real DR setup I would run that job once so the second Region holds a complete copy of the site. *(E09, E10)*

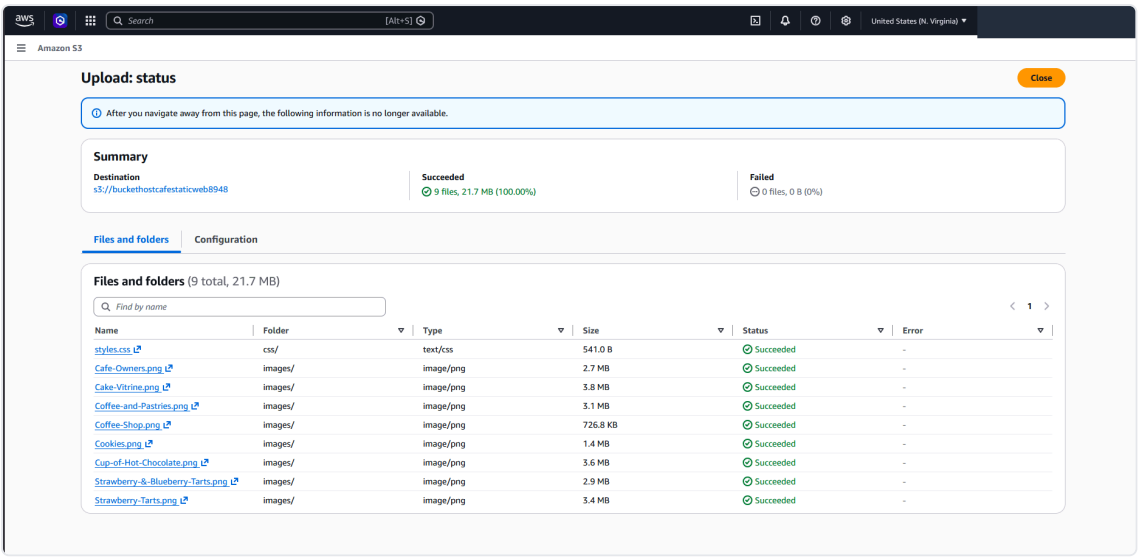
4. Deleting in the source did not delete in the destination. When I deleted an object from the source bucket, it was not removed from the destination bucket. This is consistent with S3's default replication behavior, where delete markers are not replicated unless that option is turned on, and it works in favor of DR: an accidental delete in the main bucket does not wipe out the copy in the second Region. *(Observed during the lab; no screenshot.)*

5. A design note about the lifecycle transition rule. The console warns that objects smaller than 128 KB are not transitioned between storage classes by default. The site's `index.html` is only about 2.9 KB, so in this small site the Standard-IA transition rule would not actually move those versions. The rule is still the right pattern for larger content, but it is worth knowing it has no effect on tiny files unless the minimum-size filter is changed. *(E06)*

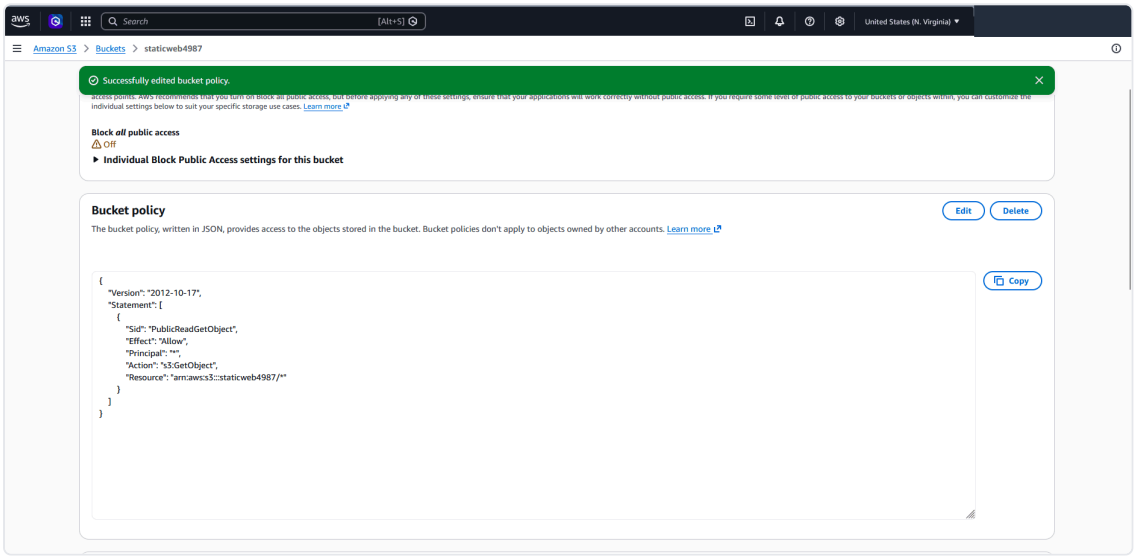
Validation & Evidence

All 10 screenshots of this stage, in lab order. Account IDs, ARNs, e-mail addresses and personal details were masked.

> **Note on evidence.** The V1 lab was run in two sessions, so the screenshots show two sets of bucket names (`buckethostcafestaticweb…` and `staticweb… / dststaticweb…`). Both sessions followed the same steps. Account IDs and user details were redacted from all screenshots.

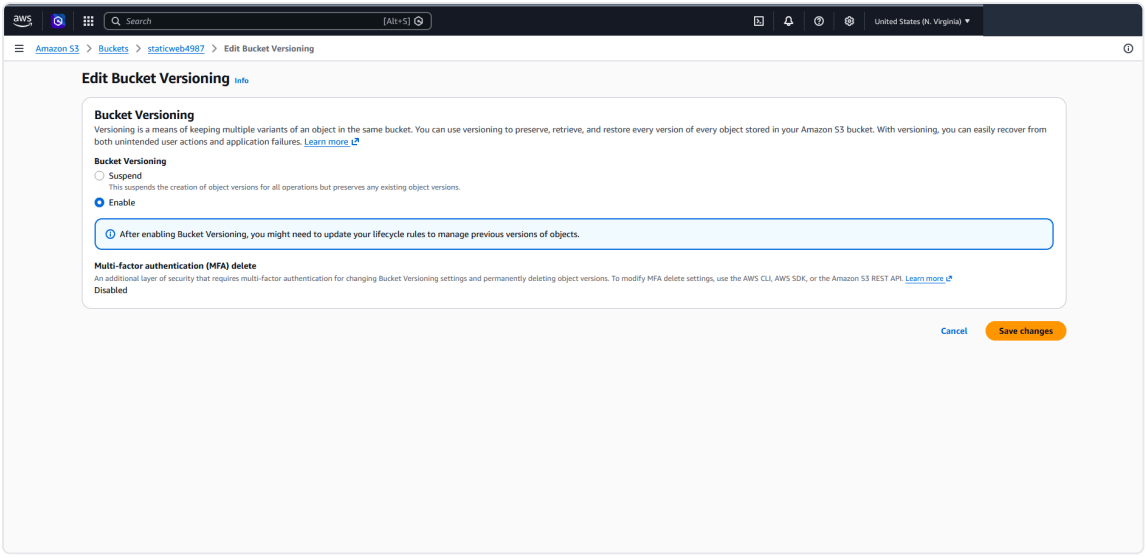


V1-E01 — Site files uploaded to the bucket (9 files, 0 failed)

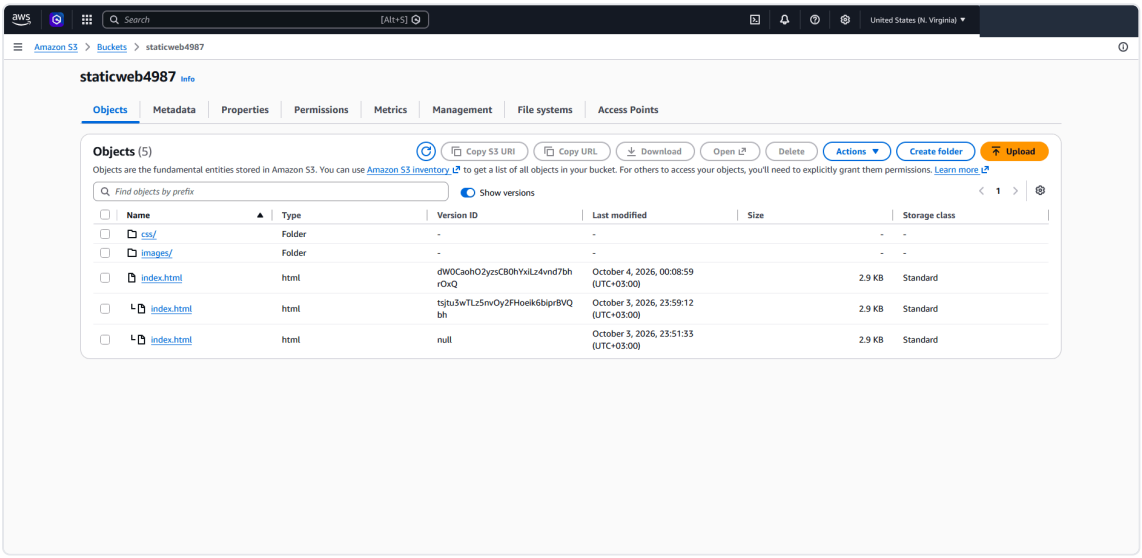


V1-E02 — Public-read bucket policy, Block public access off

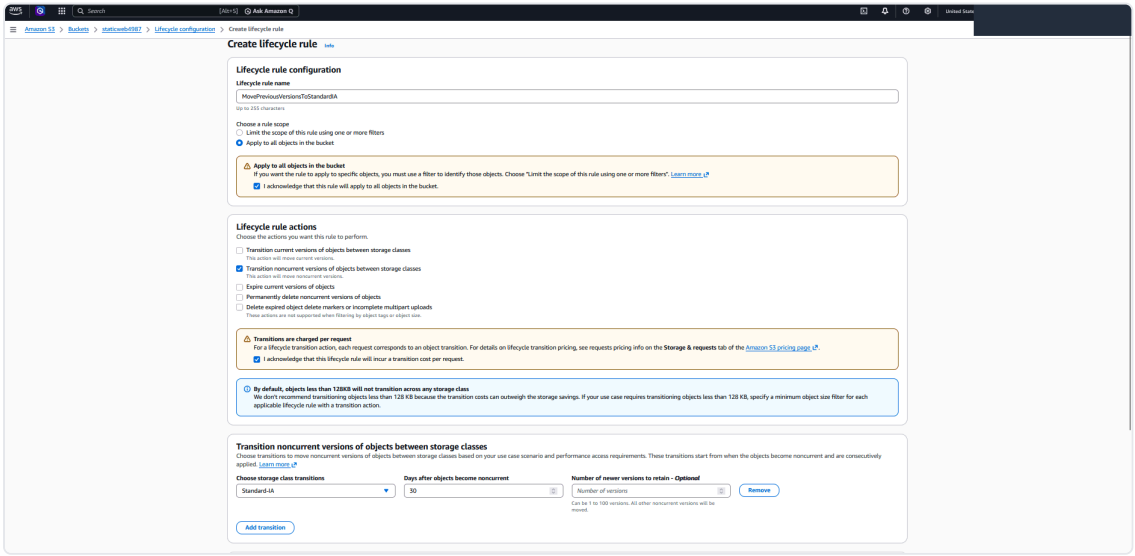
V1-E03 — Website live at the S3 website endpoint



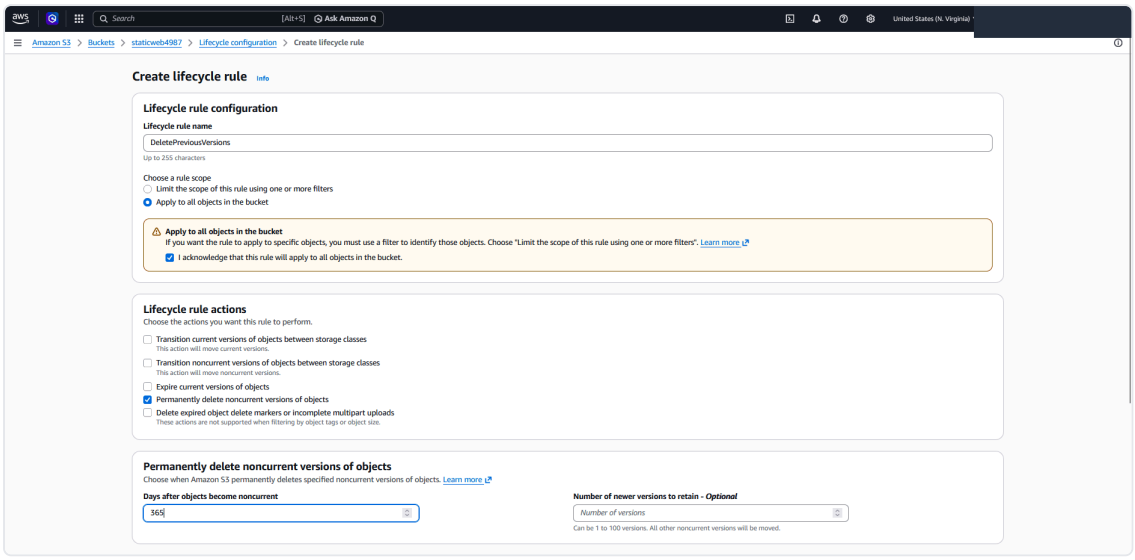
V1-E04 — Enabling bucket versioning



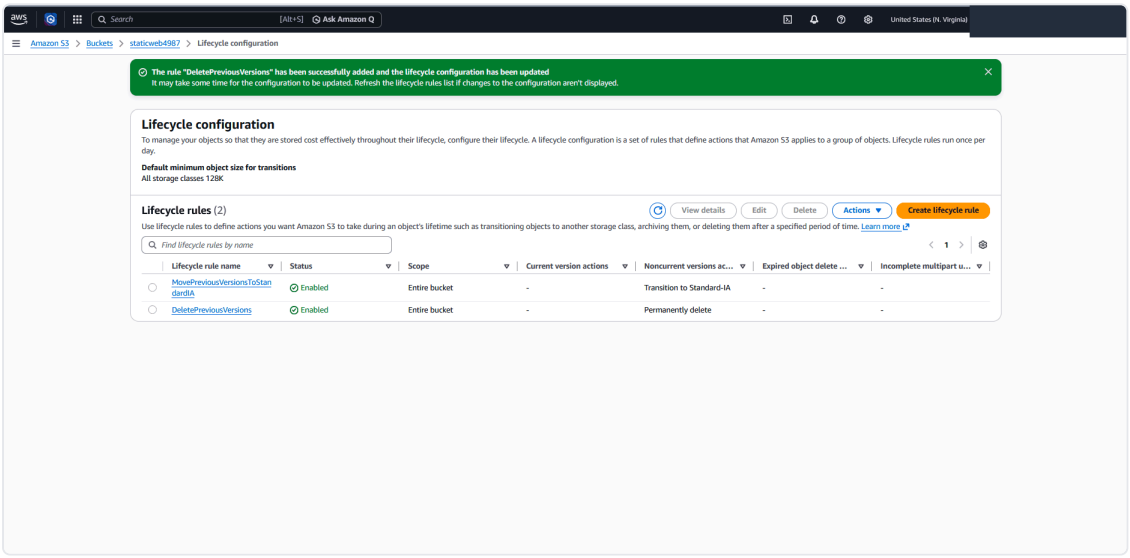
V1-E05 — Multiple versions of `index.html`



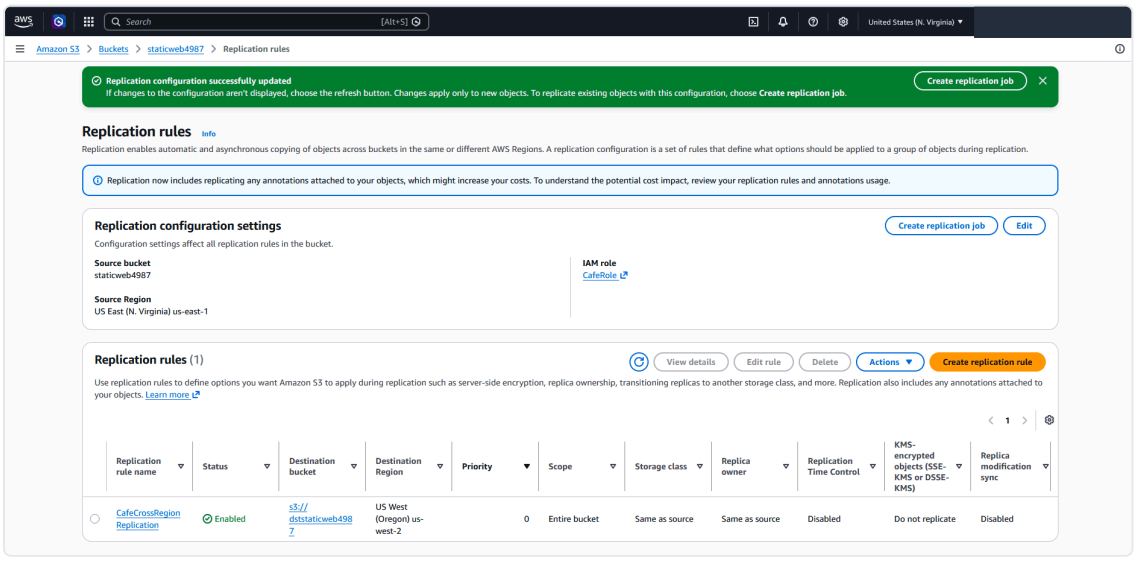
V1-E06 — Lifecycle rule: noncurrent versions → Standard-IA after 30 days



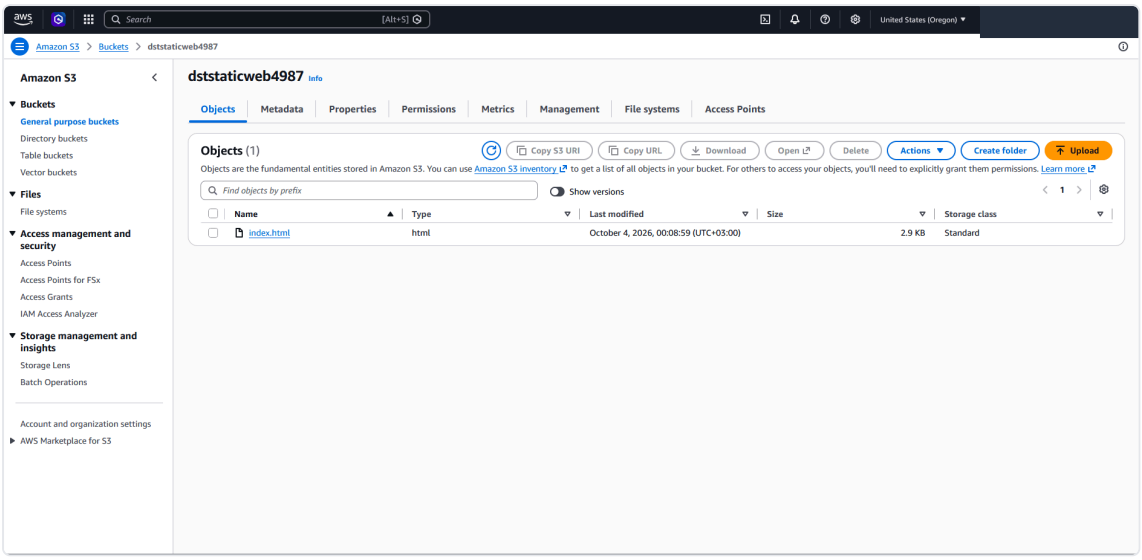
V1-E07 — Lifecycle rule: delete noncurrent versions after 365 days



V1-E08 — Both lifecycle rules enabled



V1-E09 — Cross-Region Replication rule



V1-E10 — Replicated object in the Oregon bucket

Outcome

The café went from having no online presence to a public website hosted on Amazon S3. The content is protected with versioning, old versions are managed by lifecycle rules, and a Cross-Region Replication rule keeps a copy of updated content in a second Region.

Limits of this version (and what the next versions address): the site is static, so customers cannot place orders (V2 introduces a dynamic application), and it is served over HTTP only. A natural improvement would be to put a CDN with HTTPS in front of the bucket.

CHAPTER 3 · STAGE V2

V2 — Dynamic Website on Amazon EC2

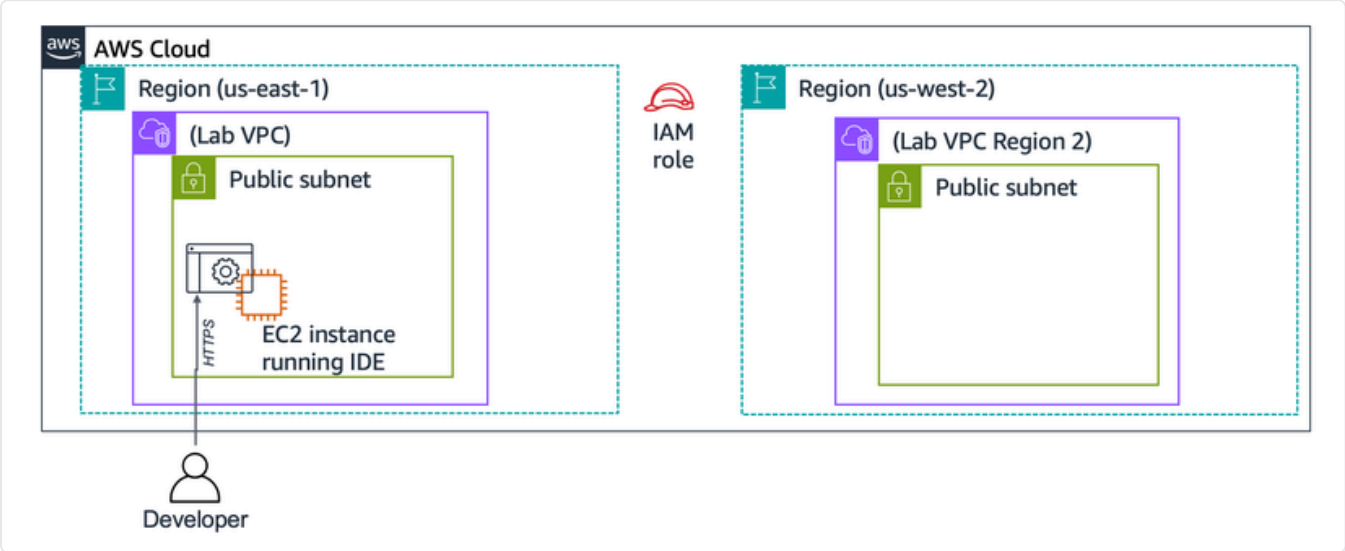
> Completed as part of AWS Academy Challenge Labs (training environment).

Business Requirement

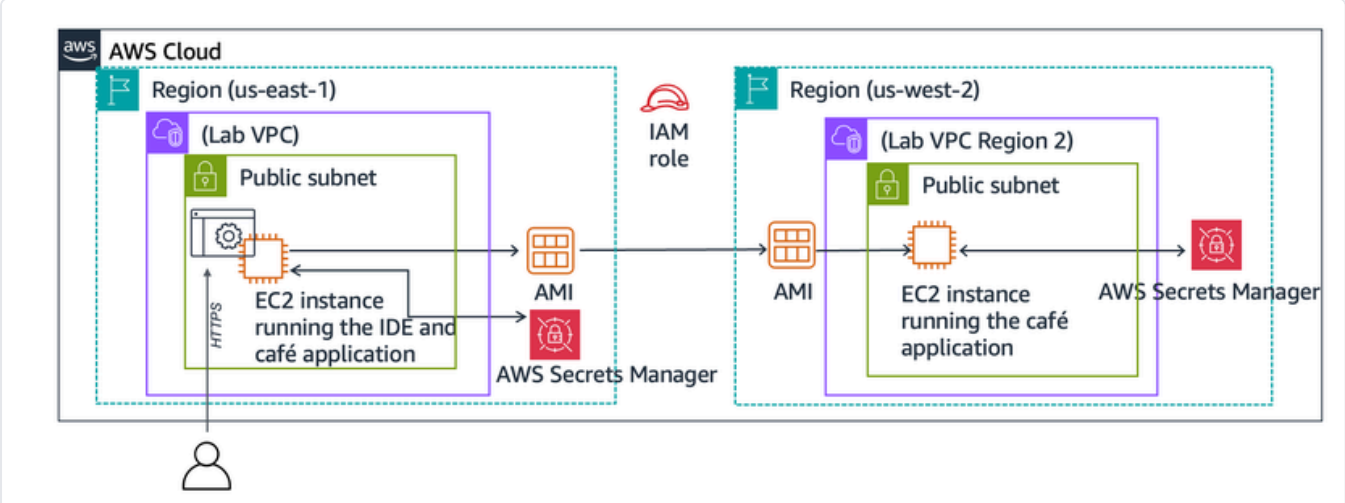
The café wanted to move beyond a static page and let customers browse a menu and place online orders. That requires a web server, application code, and a database. The café also wanted a separate **development** environment (to test) and a **production** environment in a different Region (for availability).

Start and Target Architecture

These two diagrams come from the AWS Academy lab material. They show where this stage starts and what it builds. They are shown here as reference; my own diagram of what I deployed follows in the next section.

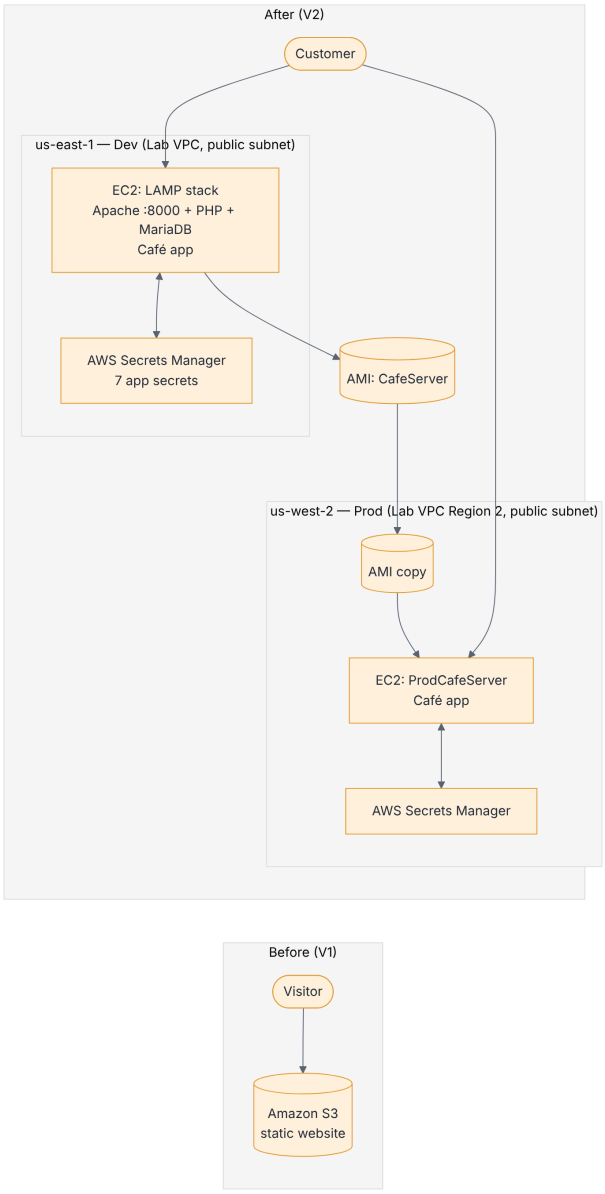


Starting architecture. AWS Academy lab diagram: where the stage begins.



Target architecture. AWS Academy lab diagram: the design this stage builds.

Before → After



AWS Services Used

SERVICE	ROLE IN THIS STAGE
Amazon EC2	Hosts the web server, the café application, and the database (dev and prod)
Security Groups	Allow inbound TCP 8000 so the site is reachable in the browser
AWS Secrets Manager	Stores the application's configuration and DB credentials outside the code
Amazon Machine Image (AMI)	Captures the configured dev server so it can be reproduced
IAM role (CafeRole)	Attached to the EC2 instances so the app can read its secrets

Implementation Highlights

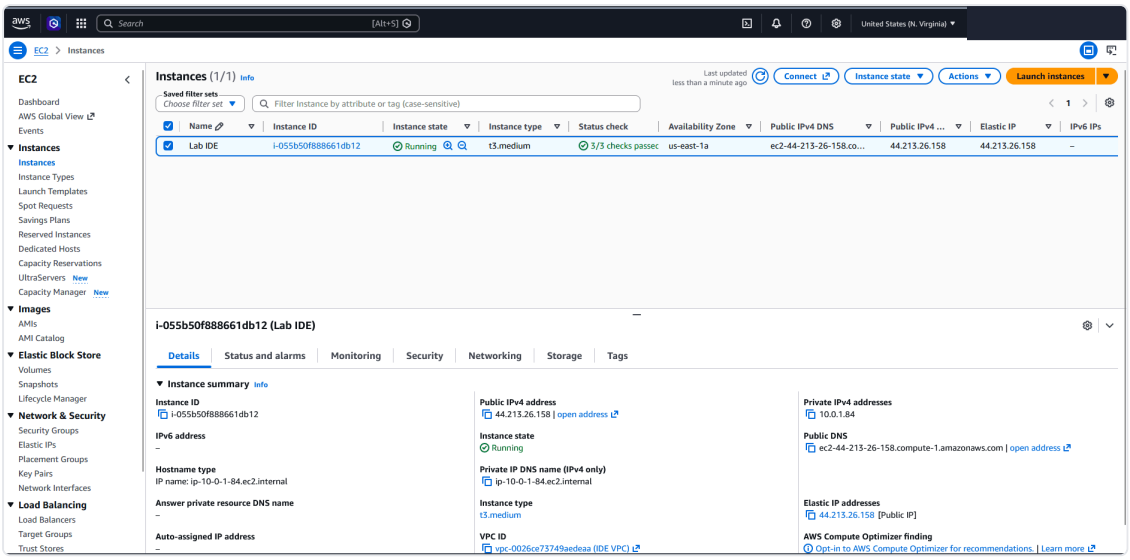
- **Web server:** Connected to the VS Code IDE running on the existing EC2 instance, changed Apache's listening port from 80 to 8000, and enabled it to start on boot. Confirmed PHP was available.
- **Database:** Installed MariaDB on the same instance, started it, and enabled it on boot.
- **Reachability test:** Created a simple `index.html` page and opened the instance's public address on port 8000. It loaded only after I added an inbound rule for TCP 8000 to the instance's security group.
- **Application:** Downloaded and unpacked the café application, the database scripts, and the AWS SDK for PHP.
- **Secrets:** Ran the setup script that creates seven parameters in Secrets Manager (DB user, password, name, URL, plus currency, time zone, and server-info settings). The PHP code reads these at runtime instead of hard-coding them.
- **Database setup:** Set the DB root password, created the café database, and checked the tables from the MySQL client (the products table held 9 menu items).
- **Time zone:** Set the PHP time zone, restarted Apache, then opened the café site and placed a test order.
- **AMI:** Created an AMI named `CafeServer` from the dev instance, copied it to **us-west-2**, and launched `ProdCafeServer` (t2.small) from it in the public subnet of Lab VPC Region 2, with the same IAM role.
- **Secrets in Region 2:** Secrets Manager is regional, so I edited the setup script (target Region and the prod instance's public DNS) to create the secrets in us-west-2, then connected to the prod instance over SSH.

Challenges & Decisions

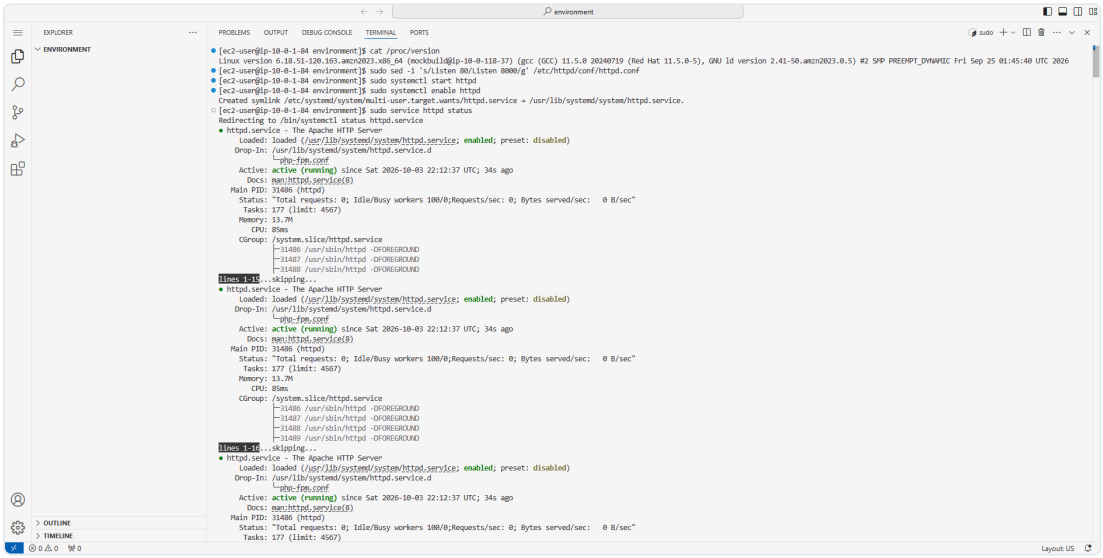
- **Challenge — the test page would not load.** After moving Apache to port 8000, the page did not open in the browser. I had not taken a screenshot of the failure, but the cause was the instance's security group, which did not allow inbound traffic on that port. **Fix:** I added an inbound rule for TCP 8000 to the security group, and the page loaded (V2-E04, V2-E05).
- **Decision — AMI instead of a manual rebuild:** Capturing the configured server as an AMI means prod is a copy of dev rather than a second hand-built install.
- **Decision — Secrets, not hard-coded credentials:** The app reads its settings from Secrets Manager. Because those secrets are regional, they had to be recreated in the second Region.

Validation & Evidence

All 16 screenshots of this stage, in lab order. Account IDs, ARNs, e-mail addresses and personal details were masked.

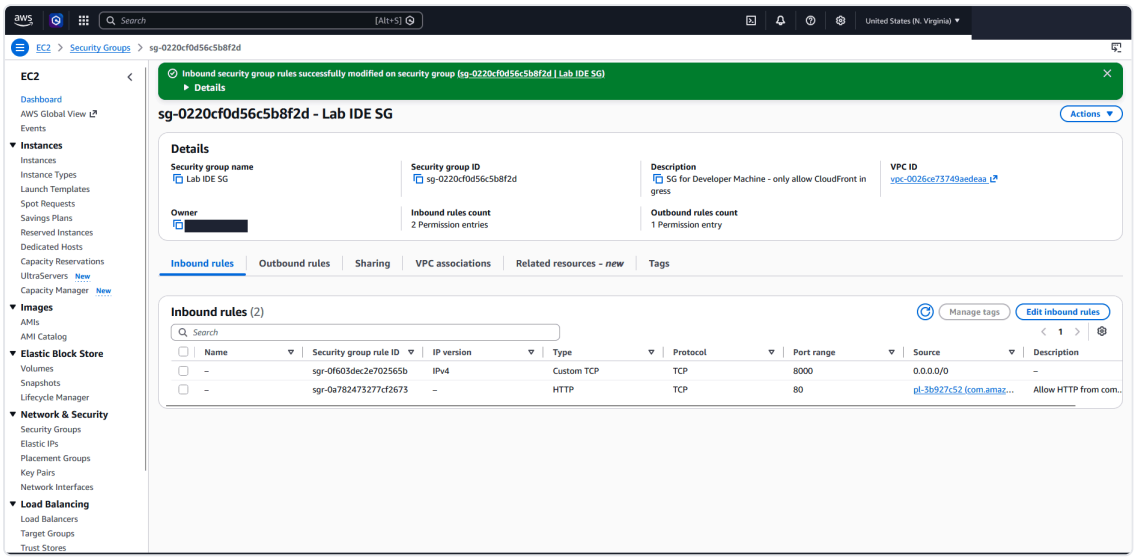


V2-E01 — Lab ide instance running



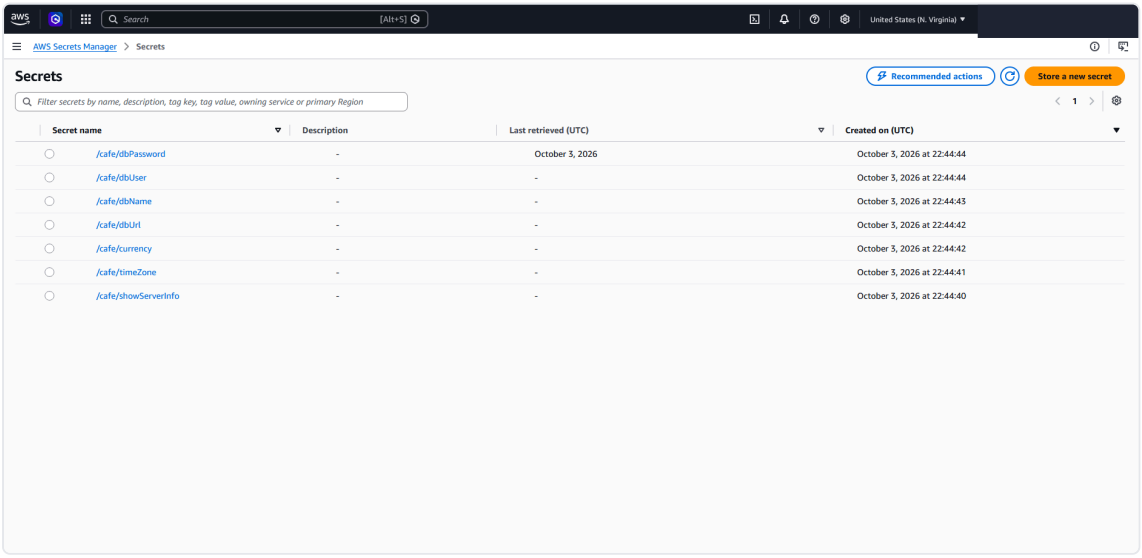
V2-E02 — Apache running port 8000

V2-E03 — Mariadb running

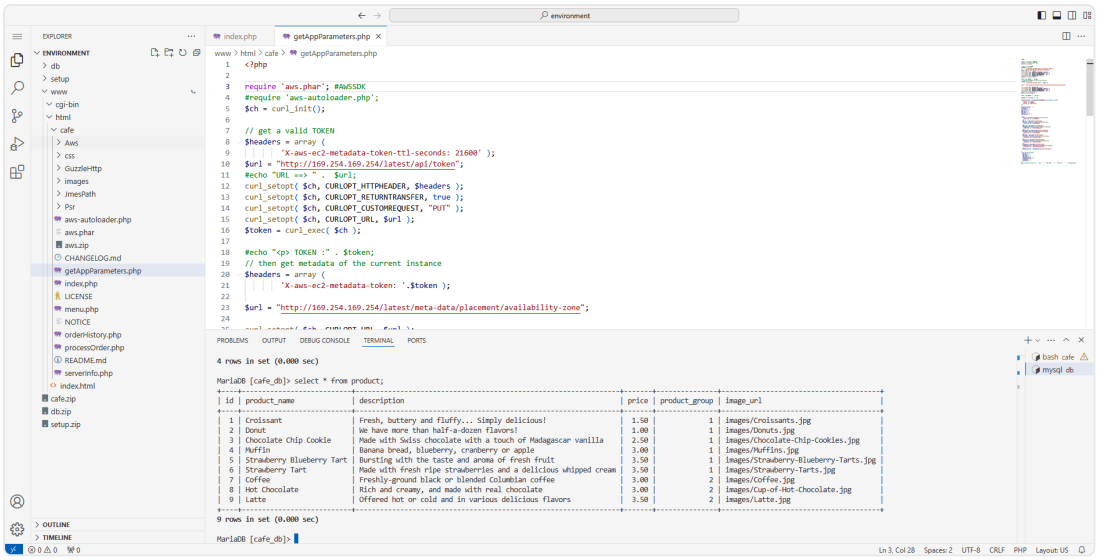


V2-E04 — Inbound rule for TCP 8000 added to the instance's security group

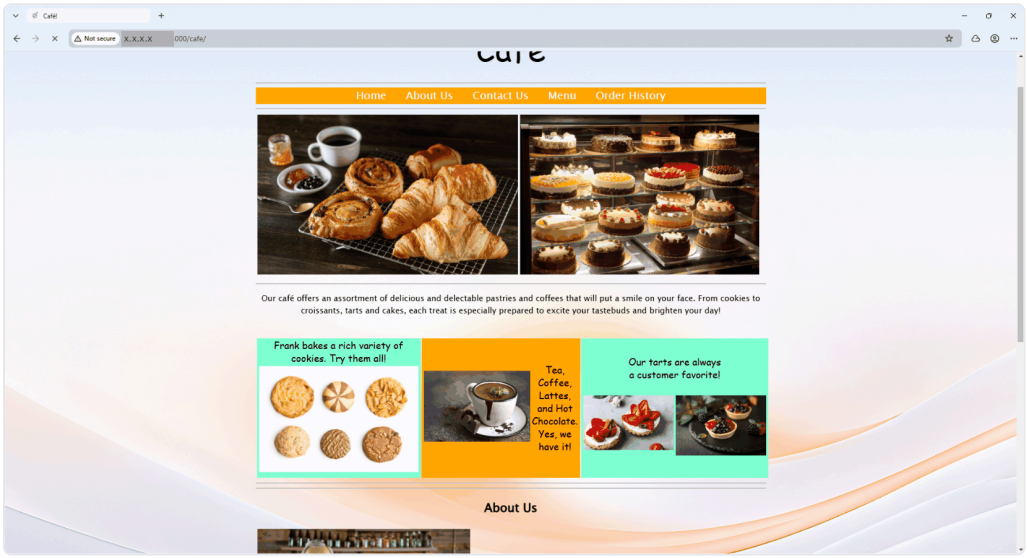
V2-E05 — Hello page port 8000



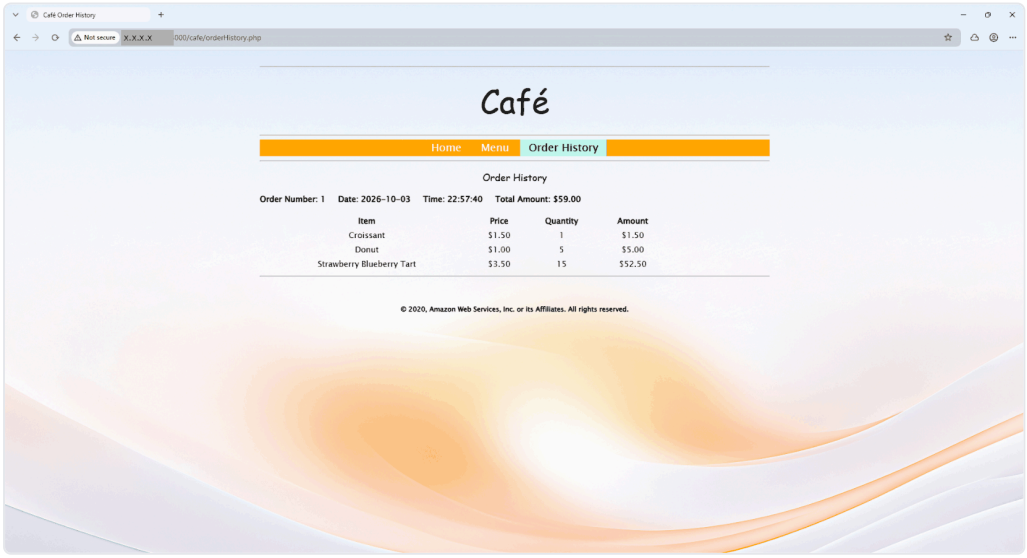
V2-E06 — Seven application secrets created in AWS Secrets Manager



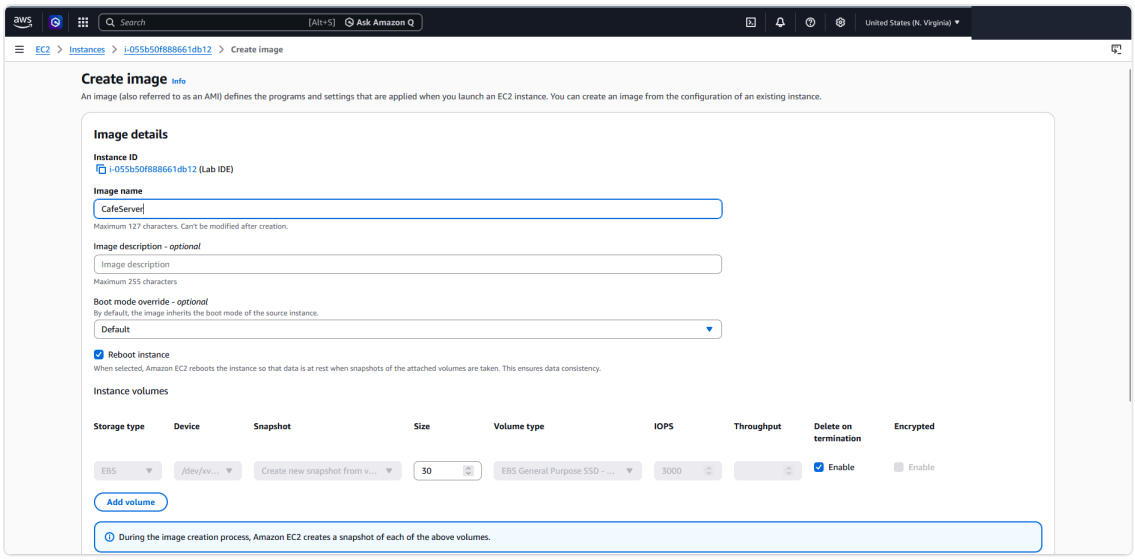
V2-E07 — Database menu table



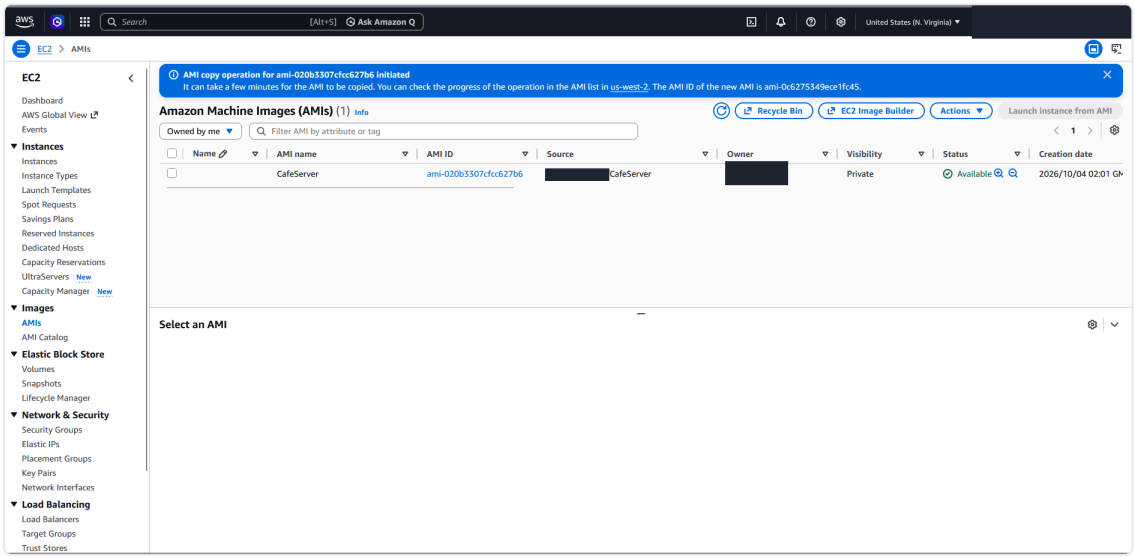
V2-E08 — Café home page served from the dev instance in us-east-1



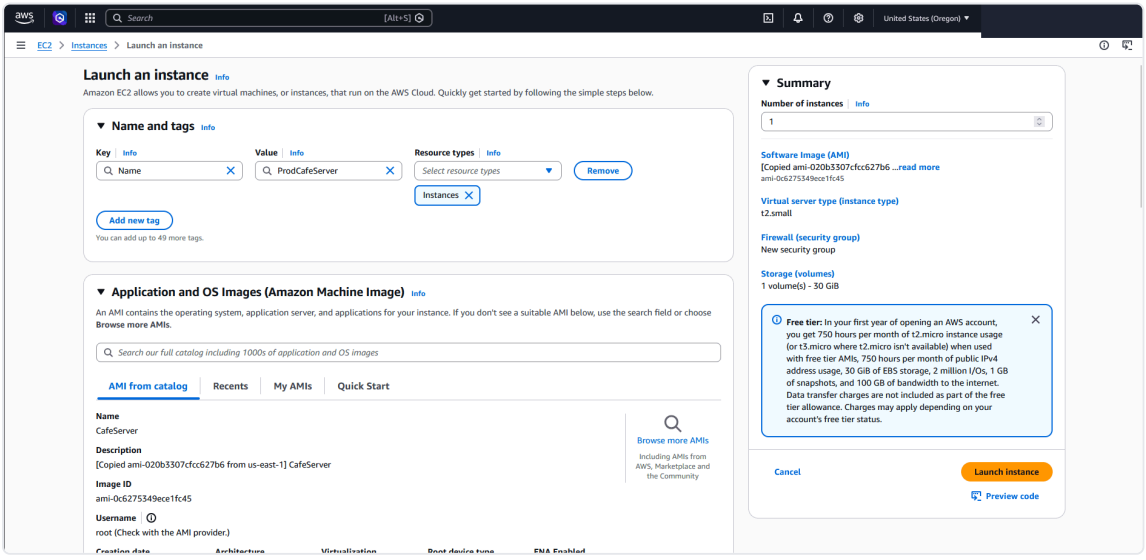
V2-E09 — Test order recorded on the Order History page (dev)



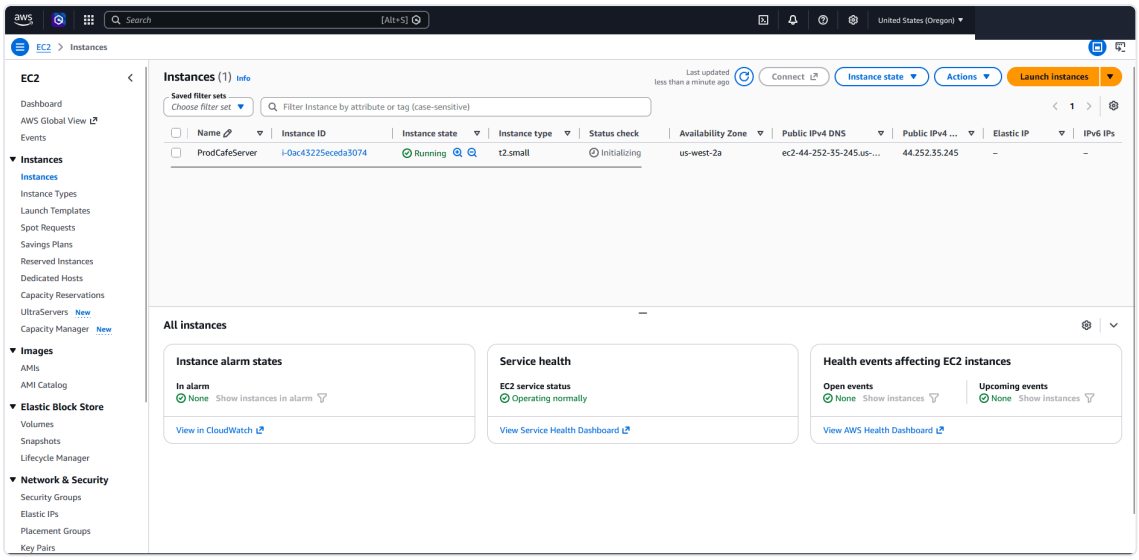
V2-E10 — Create ami cafeserver



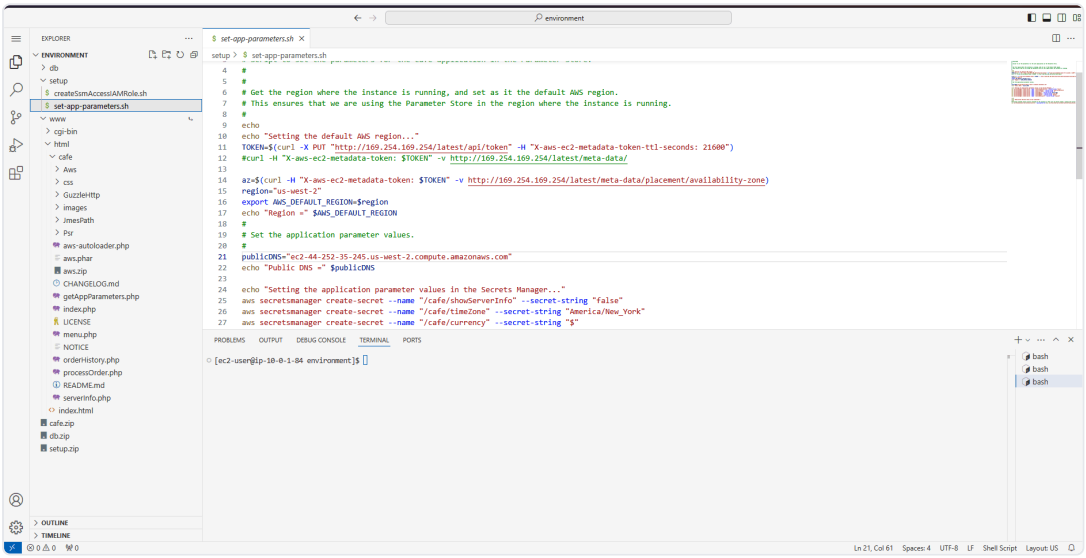
V2-E11 — `CafeServer` AMI available, with the copy to us-west-2 started



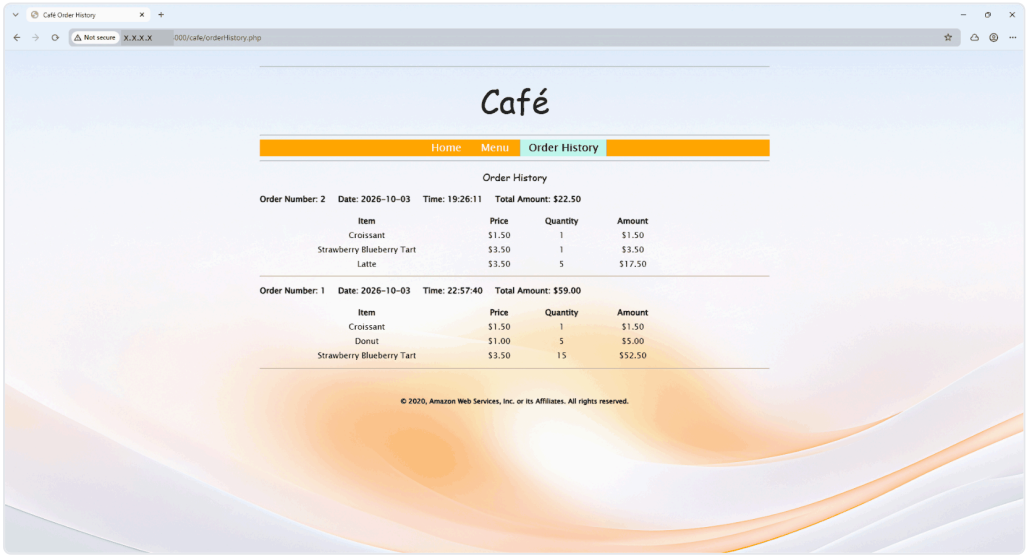
V2-E12 — Launch prodcafeserver from ami



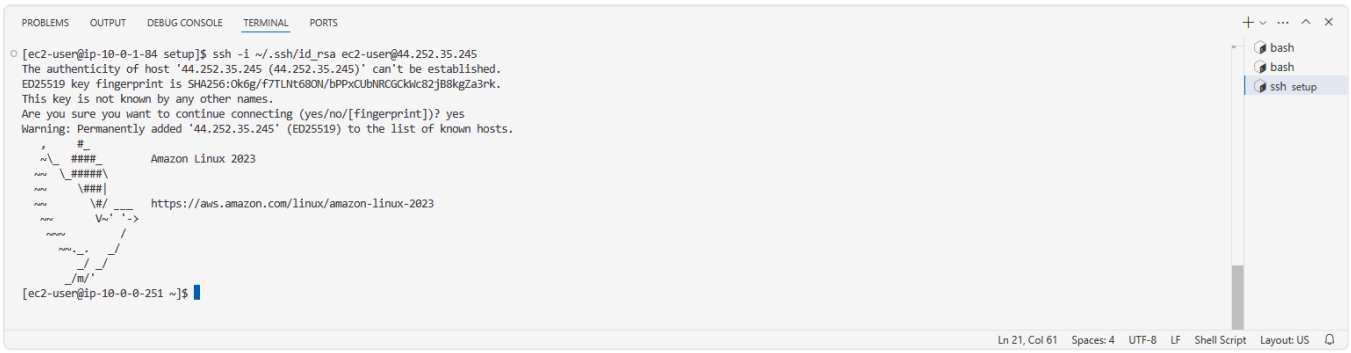
V2-E13 — Prod instance running oregon



V2-E14 — Set app parameters us west 2



V2-E15 — Café app running on `ProdCafeServer` in us-west-2, with its own order history



V2-E16 — Ssh to prod instance

Outcome

The café went from a static page to a working ordering application with its configuration kept in Secrets Manager. The same application now runs in two Regions, a dev environment in N. Virginia and a prod environment in Oregon, both created from one AMI.

CHAPTER 4 · STAGE V3

V3 — Database Migration to Amazon RDS

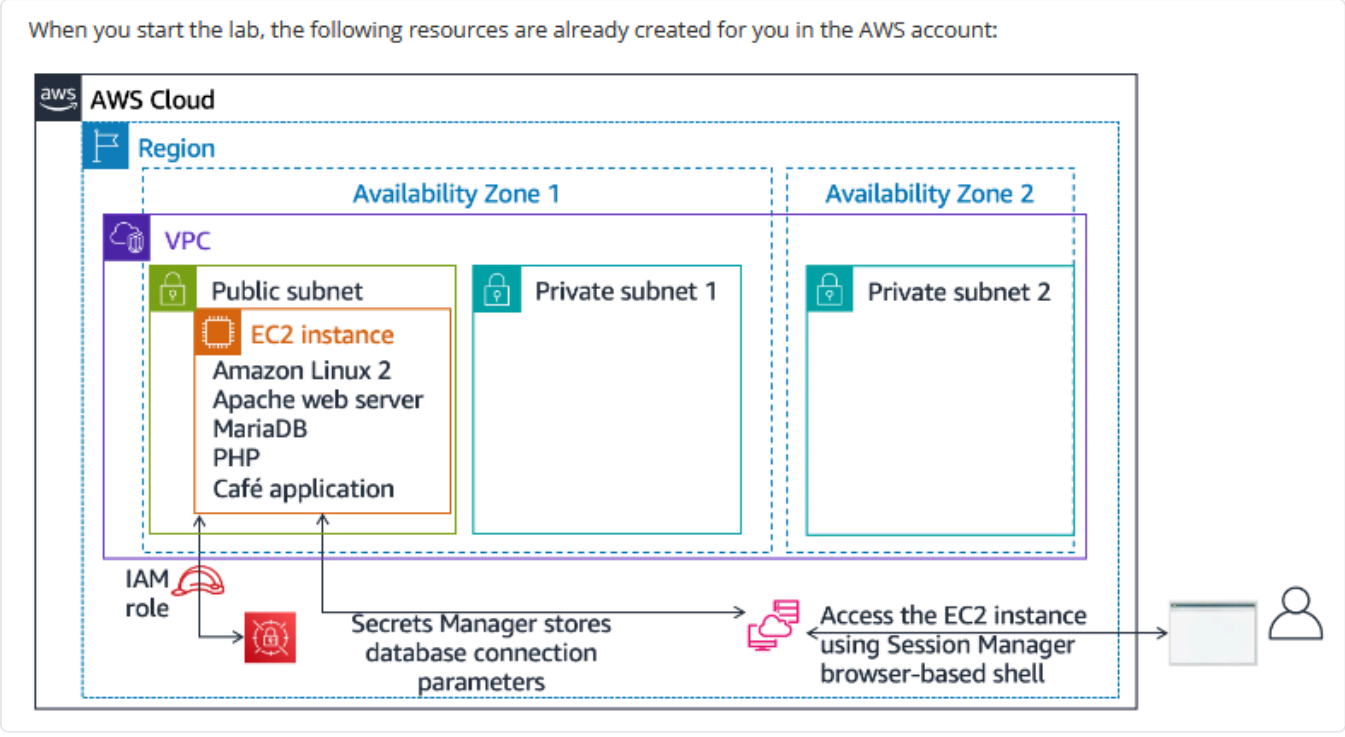
> Completed as part of AWS Academy Challenge Labs (training environment).

Business Requirement

After the café's online ordering went live (V2), the web server and the database lived on the same EC2 instance, so the owners had to maintain both together. Sofía wanted a database that is easier to maintain and gives durability, scalability and good performance, without losing the order history already stored. The requirement for this stage: **move the café database to a managed service and keep the application working with all existing data.**

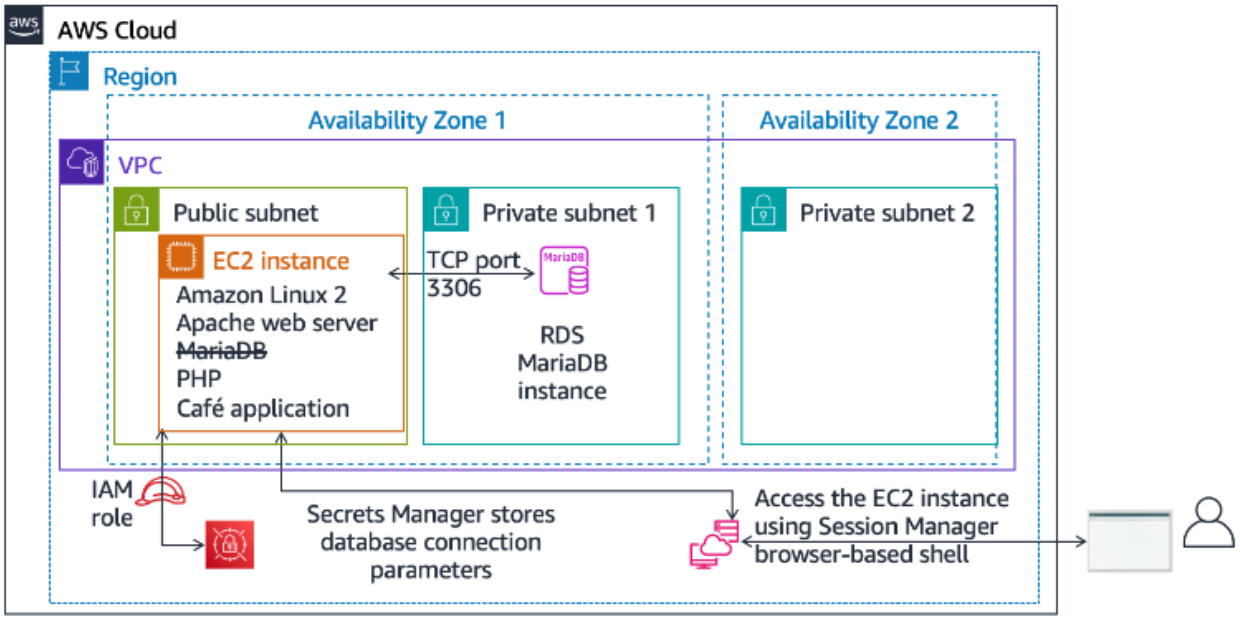
Start and Target Architecture

These two diagrams come from the AWS Academy lab material. They show where this stage starts and what it builds. They are shown here as reference; my own diagram of what I deployed follows in the next section.



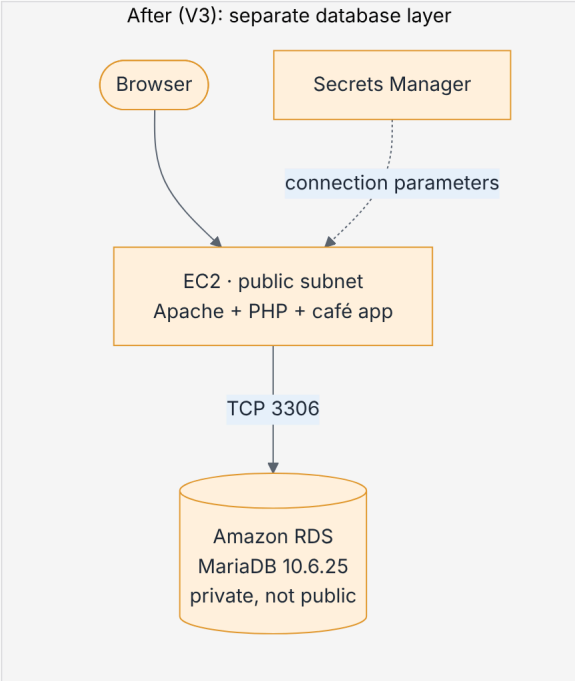
Starting architecture. AWS Academy lab diagram: where the stage begins.

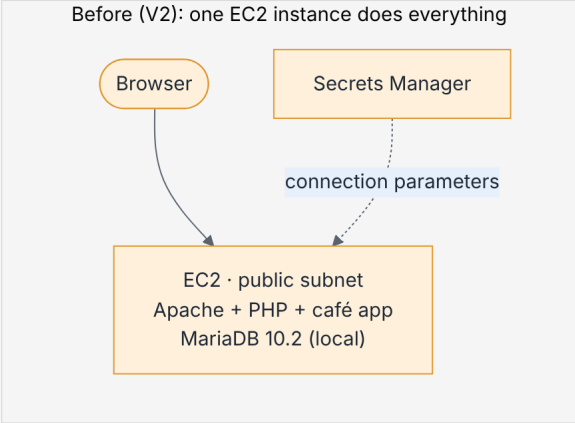
At the end of this lab, your architecture will look like the following example:



Target architecture. AWS Academy lab diagram: the design this stage builds.

Before → After





AWS Services Used

SERVICE	ROLE IN THIS STAGE
Amazon RDS (MariaDB)	Managed database replacing the local MariaDB on EC2
Amazon EC2	Hosts the café web application (database removed from it)
AWS Secrets Manager	Holds the DB connection parameters the PHP code reads
AWS Systems Manager (Session Manager)	Browser-based shell into the EC2 instance
VPC security groups	Control which source can reach the database on port 3306

Implementation Highlights

- **Reviewed what existed.** I tested the running café site (placed an order, checked Order History) and opened a Session Manager shell on the EC2 instance. The local MariaDB 10.2.38 held `cafe_db` with four tables (`order`, `order_item`, `product`, `product_group`) and 25 orders at that point.
- **Created the RDS instance.** `CafeDatabase`: MariaDB 10.6.25, `db.t3.micro`, placed in the Lab VPC using `lab-db-subnet-group`, not publicly accessible, attached to the `dbSG` security group.
- **Exported and imported the data.** I dumped `cafe_db` with `mysqldump` into `CafeDbDump.sql` and loaded it into RDS with the `mysql` client, then queried RDS to confirm the same 25 orders were there.
- **Switched the application by configuration only.** No PHP changes: I updated the `dbUrl`, `dbUser` and `dbPassword` secrets in Secrets Manager to point at RDS, stopped the local MariaDB service, and confirmed the site still worked.

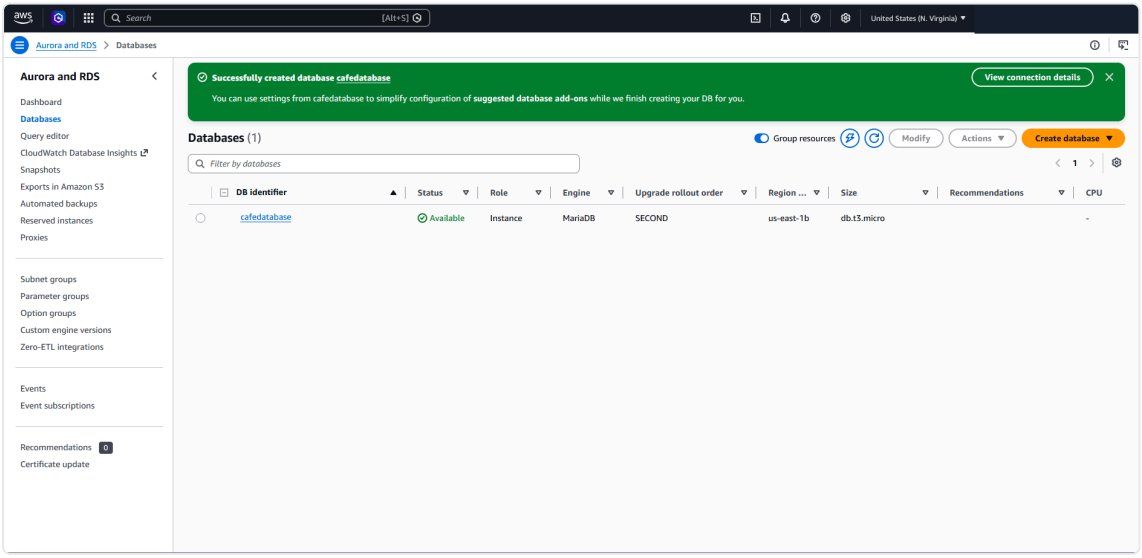
Challenges & Decisions

- **Decision — database access limited by security group.** My first connection from the EC2 instance to RDS failed with `ERROR 2002` (E12). I added an inbound rule on `dbSG` for MySQL/Aurora TCP 3306 with a **security group as the source** rather than an open IP range (E13), then confirmed with `nmap` that port 3306 was open before logging in (E14).
- **Observation — Order History while the local database was stopped.** One screenshot shows "Connection failed: Connection refused" on the Order History page (E17), and the terminal shows `mariadb` being stopped and started (E19).

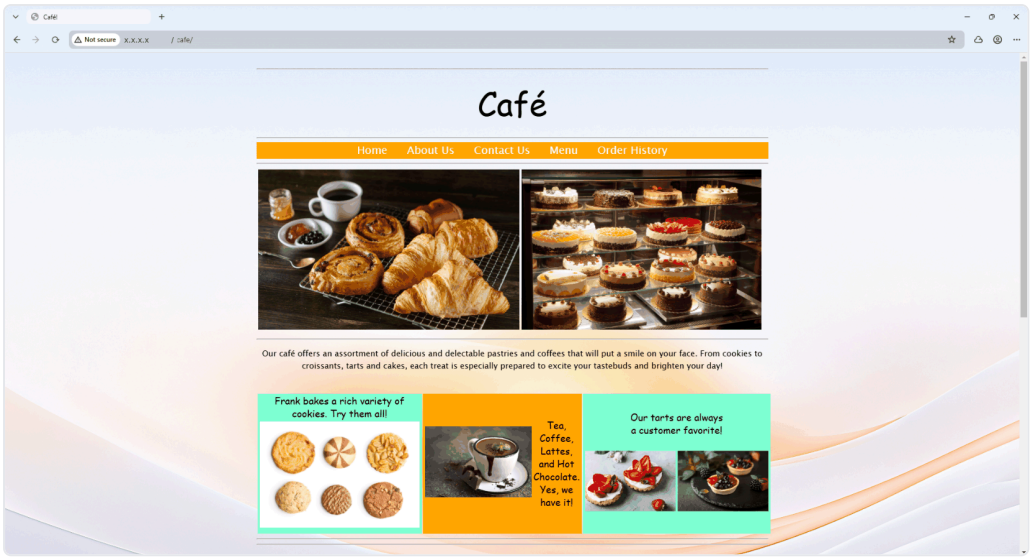
Validation & Evidence

All 22 screenshots of this stage, in lab order. Account IDs, ARNs, e-mail addresses and personal details were masked.

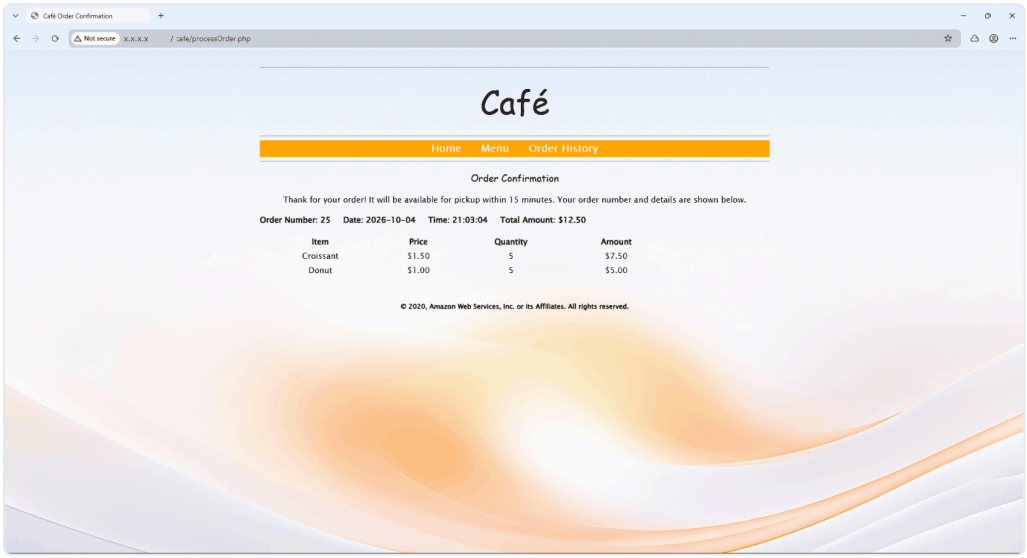
Task 1–2 — Existing application and RDS instance



V3-E01 — RDS console: `CafeDatabase` created and showing Available (MariaDB, db.t3.micro, us-east-1)

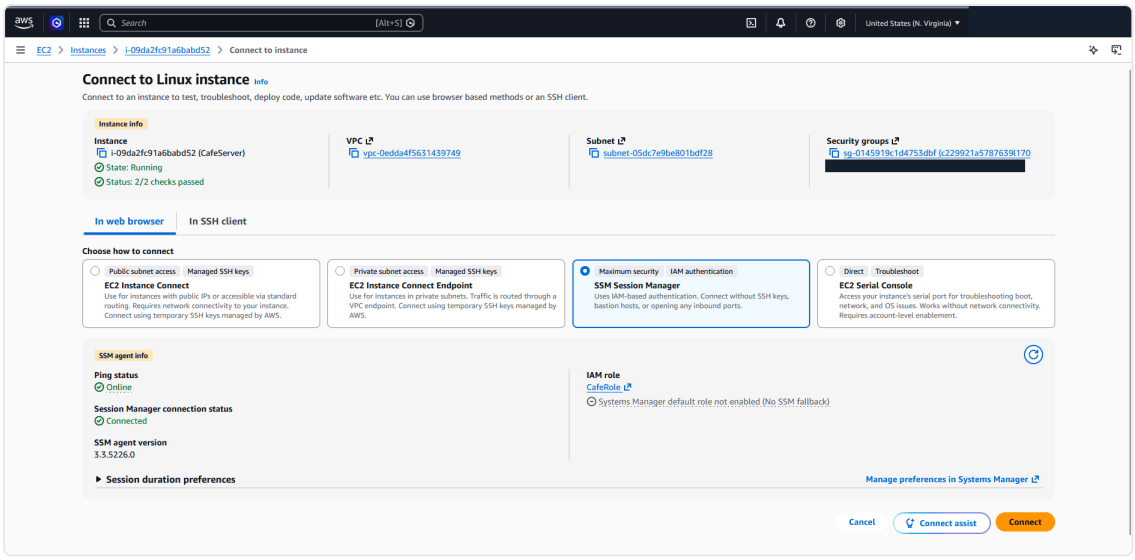


V3-E02 — Café website home page running on the EC2 instance (public IP masked)



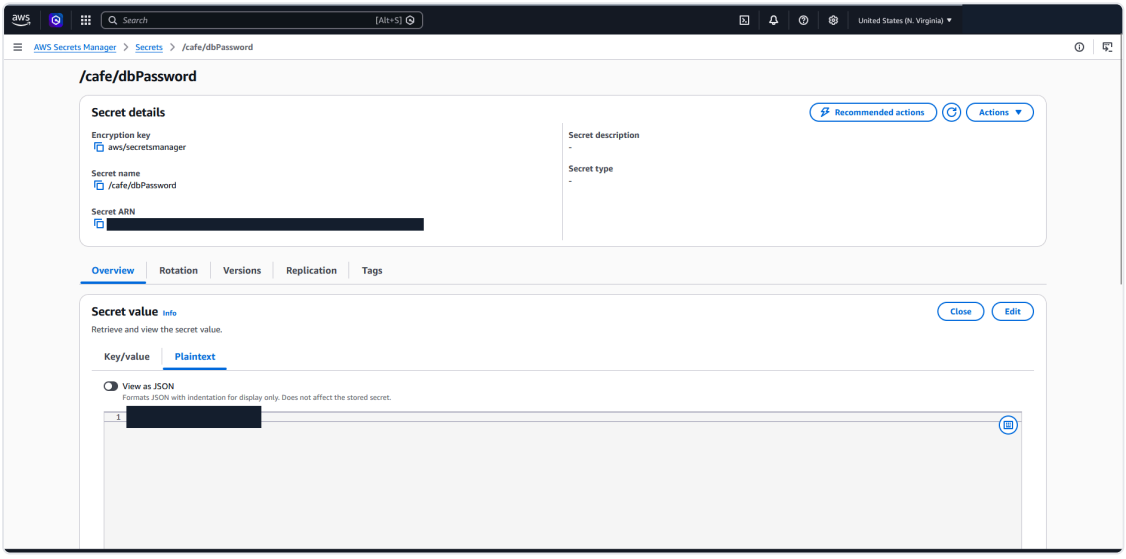
V3-E03 — Order Confirmation page after placing a test order (order #25)

V3-E04 — Order History before migration: original orders plus my test order, all served from the local database

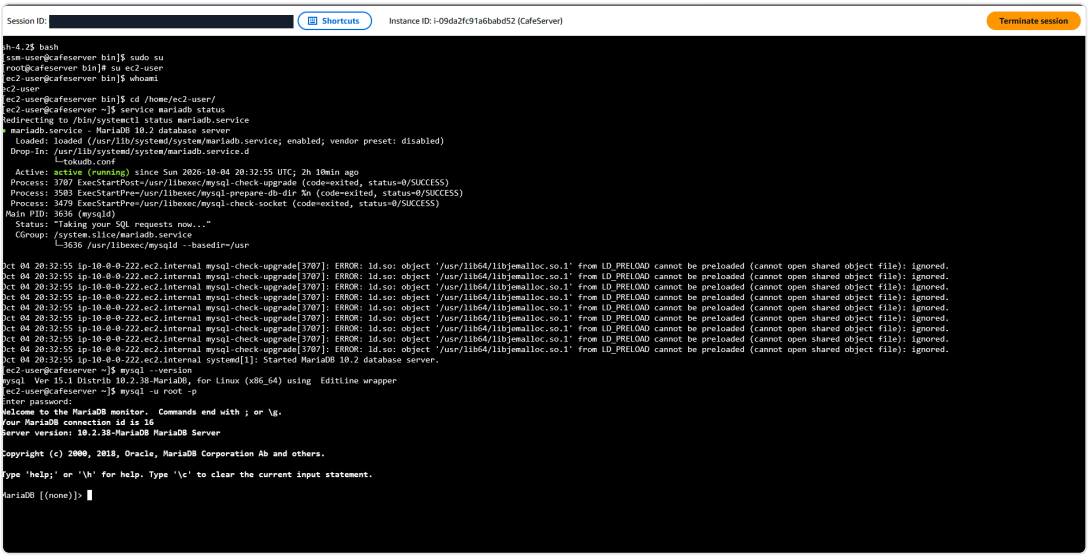


V3-E05 — Connect to instance dialog with Session Manager selected (browser-based shell, no SSH keys)

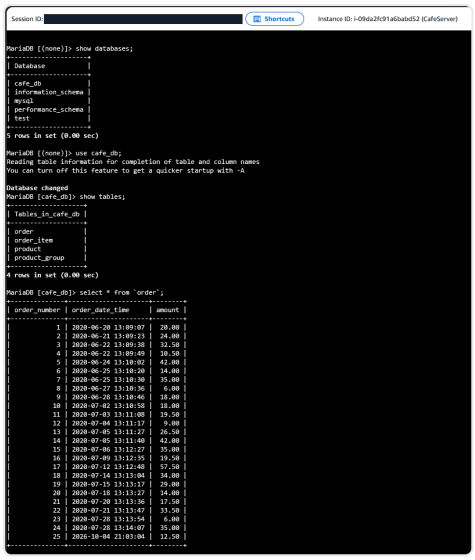
Task 3 — Source database and export



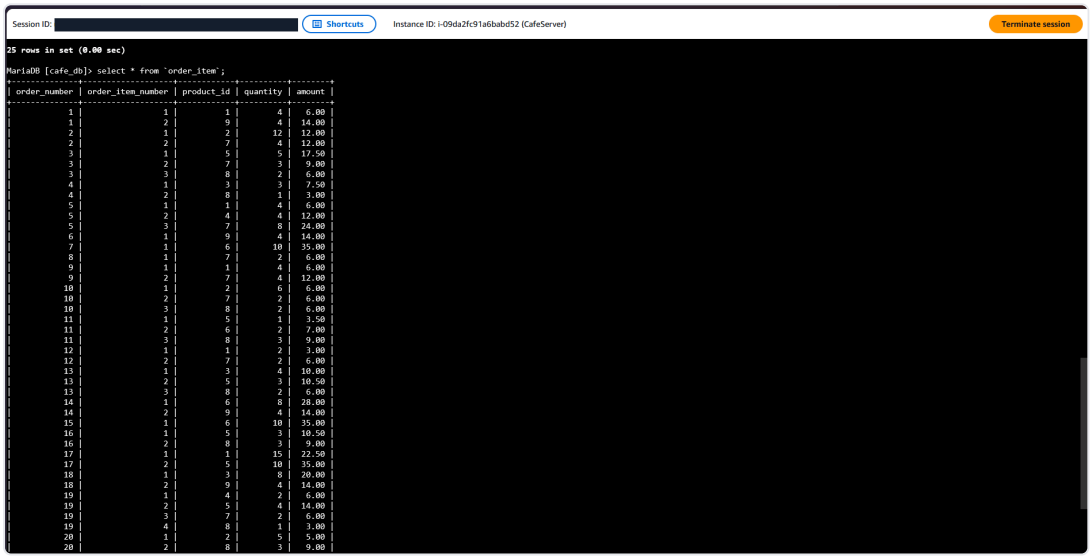
V3-E06 — Secrets Manager `/cafe/dbPassword` secret (ARN and value masked)



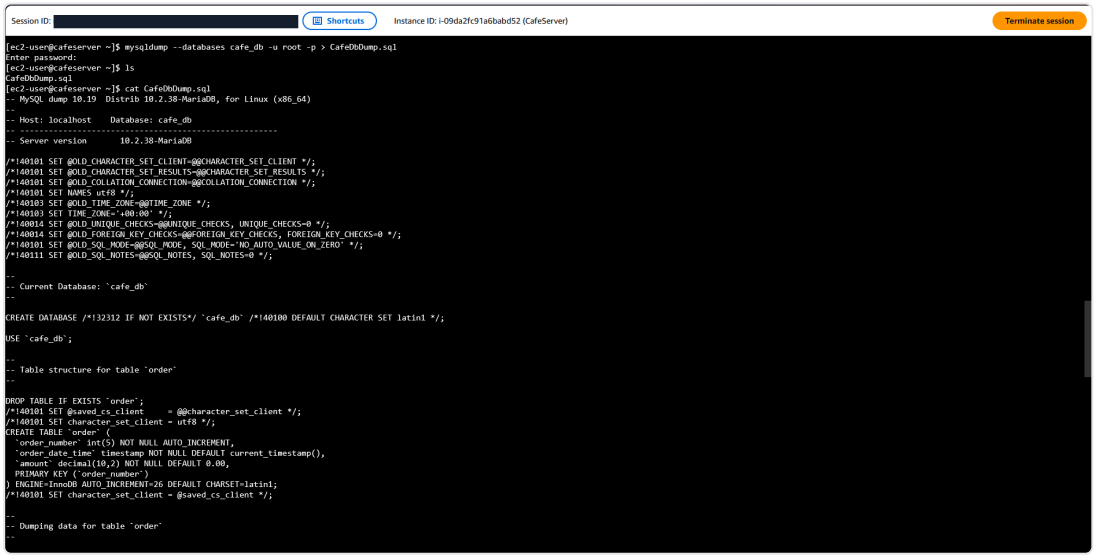
V3-E07 — Terminal: local MariaDB 10.2.38 running on the EC2 instance and root login to the SQL client



V3-E08 — Source database: `cafe_db` with its four tables and 25 rows in `order`

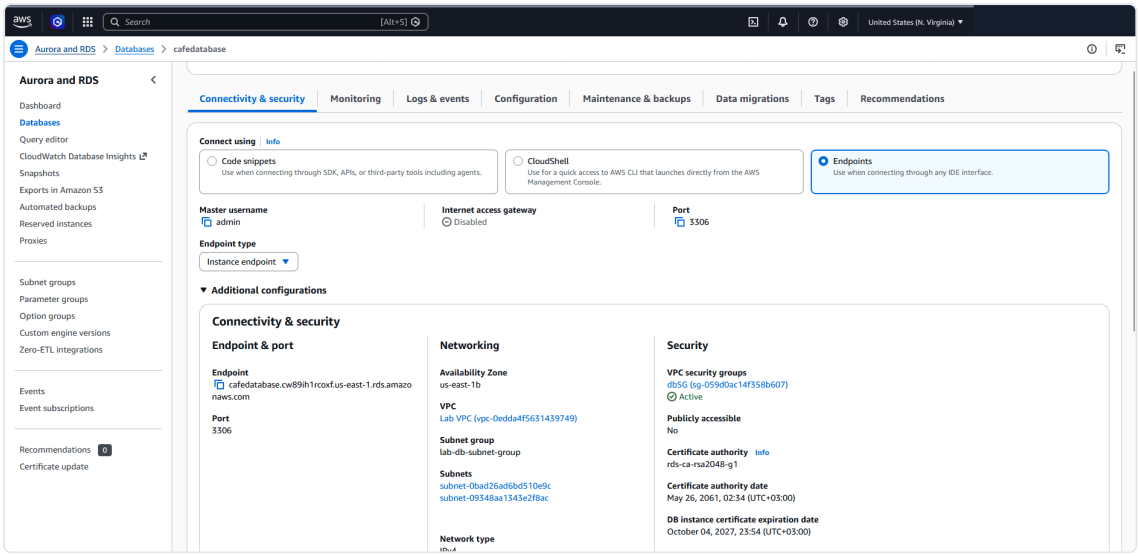


V3-E09 — Source database: rows in `order_item` (items, quantities, prices per order)

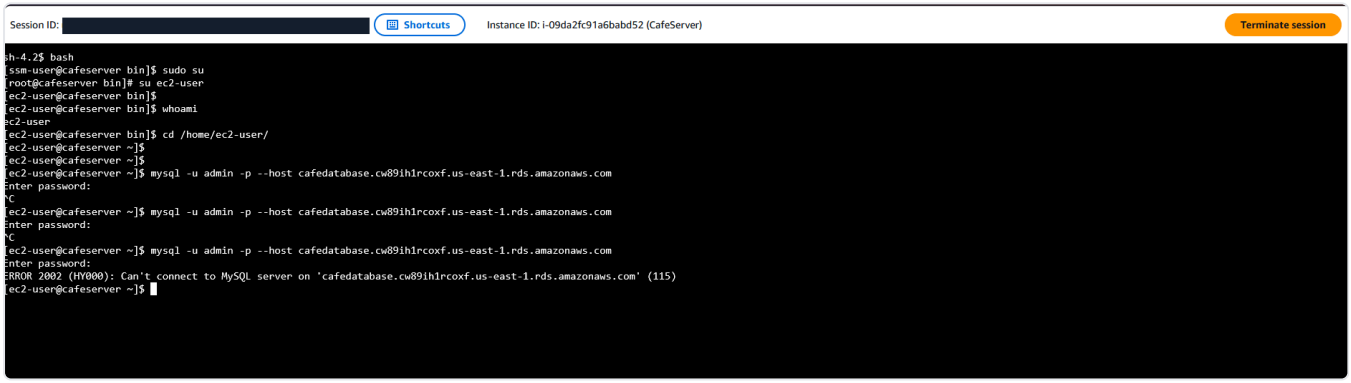


V3-E10 — `mysqldump` creating `CafeDbDump.sql` and a `cat` of the start of the dump file

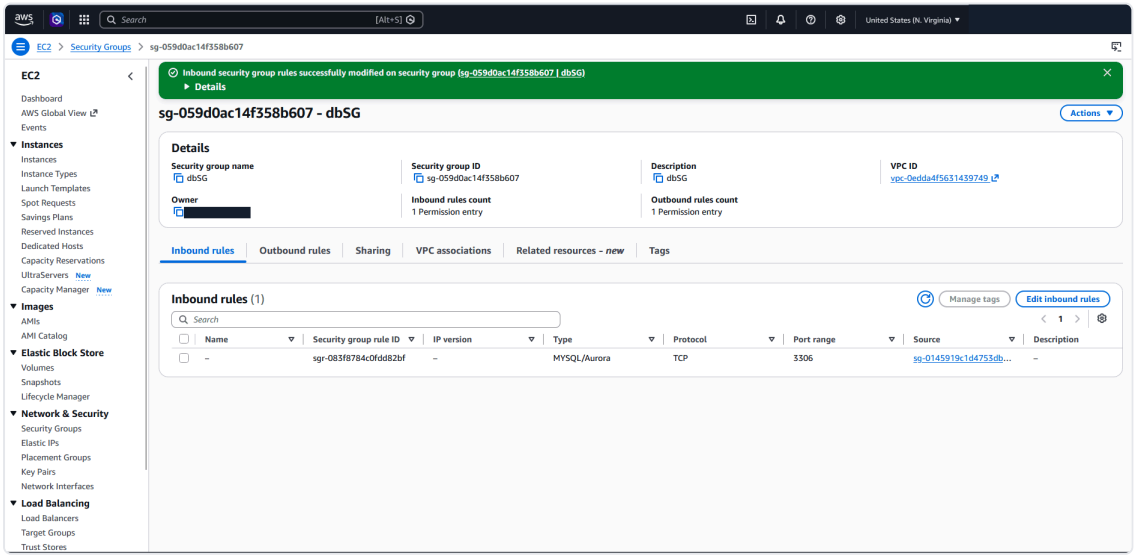
Task 4 — Connecting EC2 to RDS



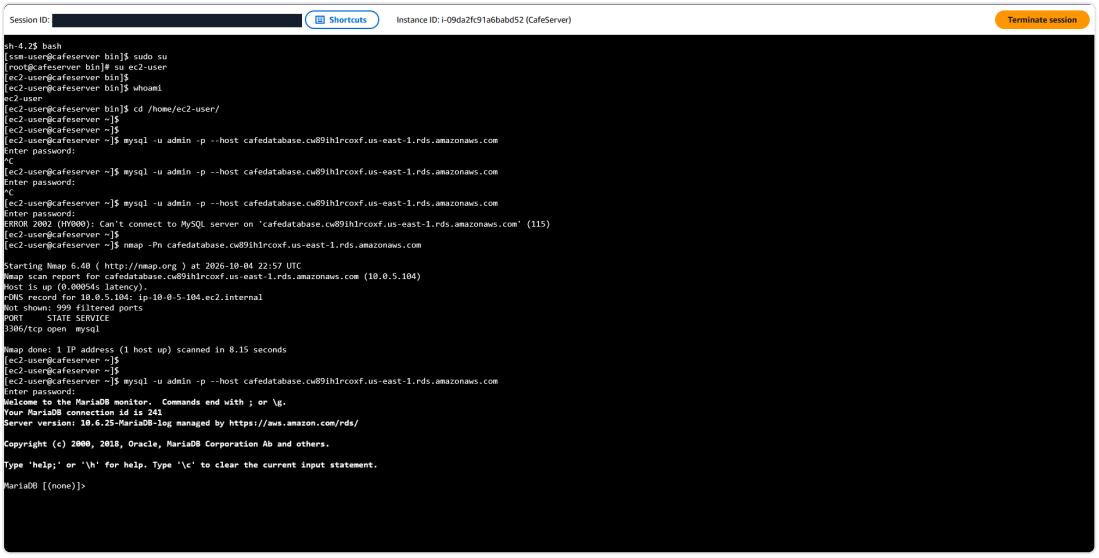
V3-E11 — RDS connectivity details: endpoint, port 3306, Lab VPC, `lab-db-subnet-group`, `dbSG`, Publicly accessible = No



V3-E12 — First connection attempt from EC2 to RDS fails with ERROR 2002 (can't connect)

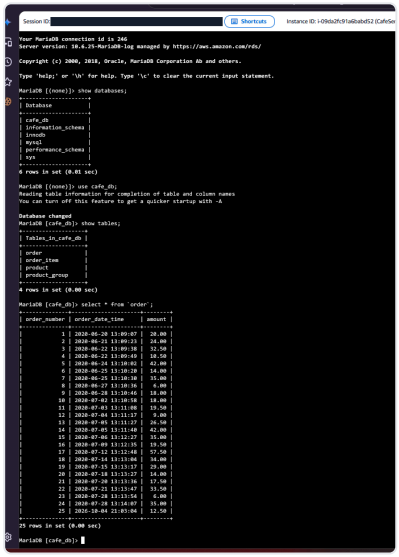


V3-E13 — `dbSG` inbound rule: MySQL/Aurora, TCP 3306, source = a security group



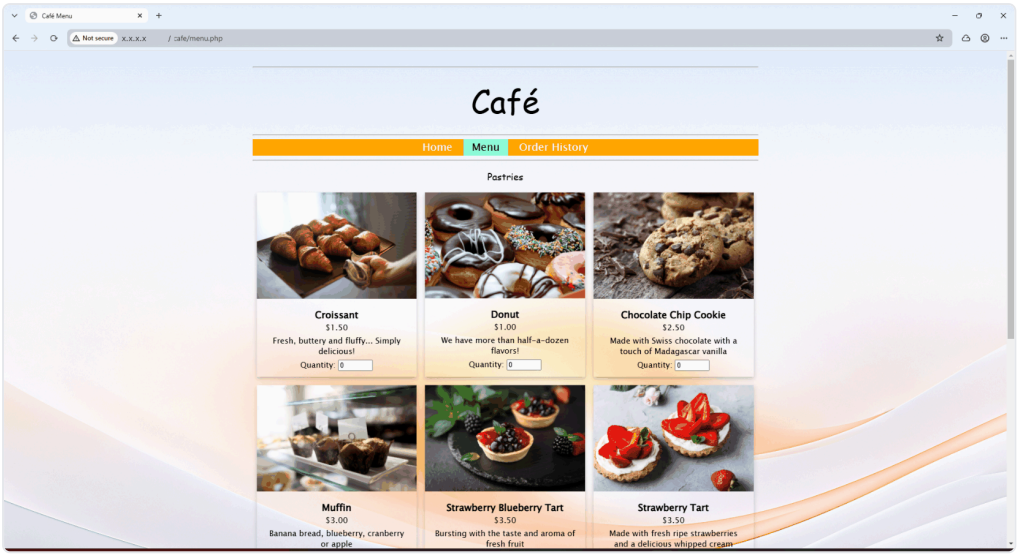
V3-E14 — `nmap` shows 3306/tcp open on the RDS endpoint, then a successful login (server 10.6.25-MariaDB)

Task 5 — Import and verification

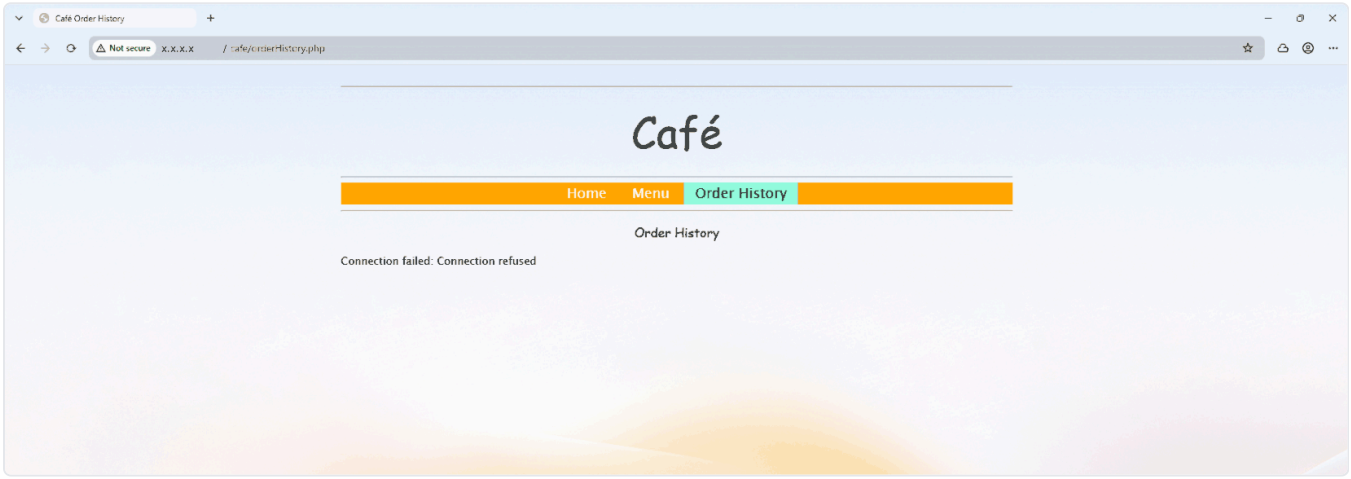


V3-E15 — RDS after the import: `cafe_db` present, four tables, 25 orders

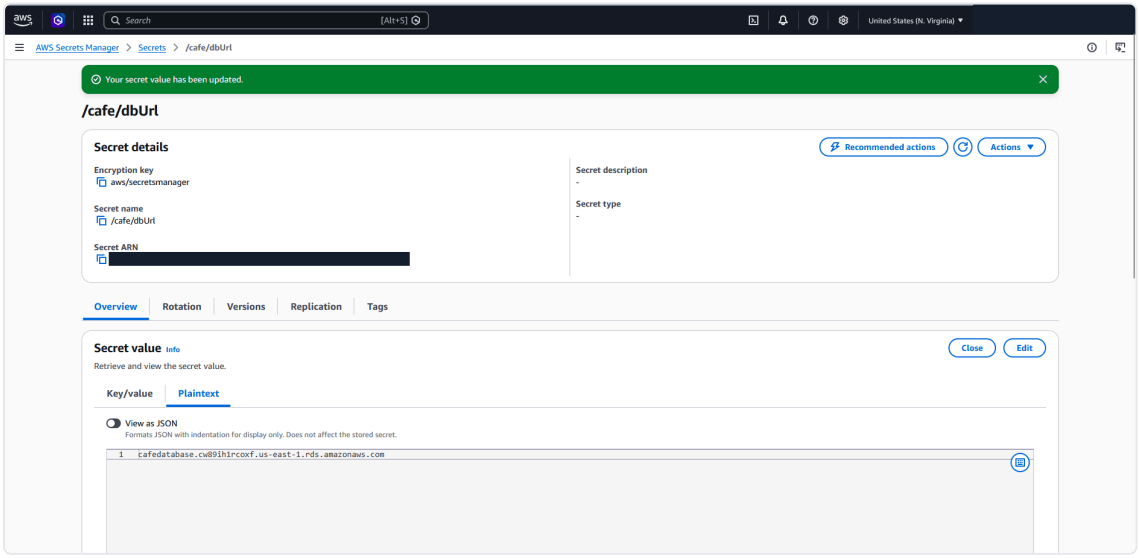
Task 6 — Switching the application to RDS (in the order they appear in my screenshot file)



V3-E16 — Café menu page (`menu.php`) used to test the application

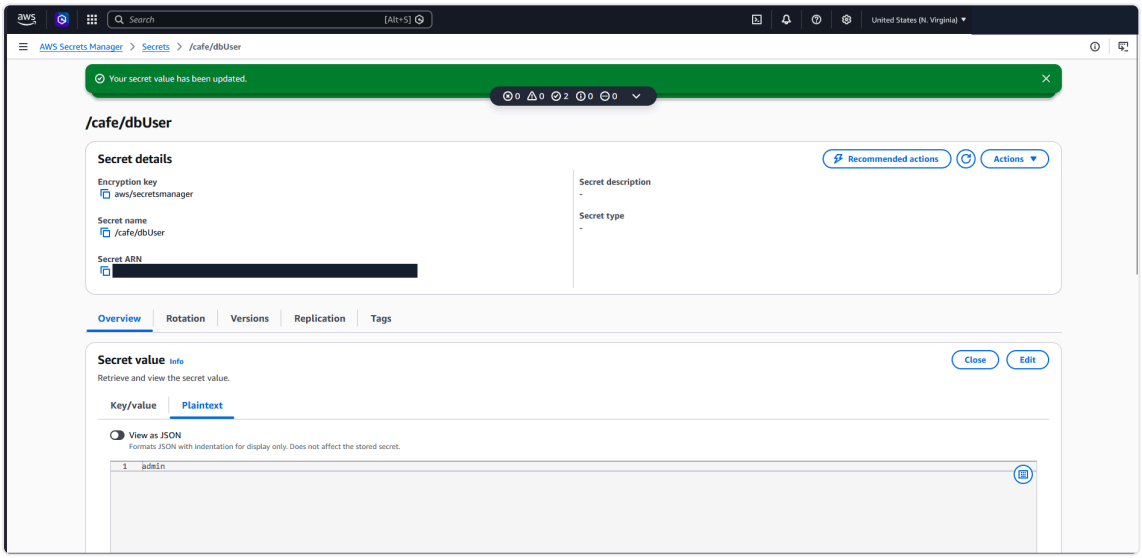


V3-E17 — Order History page showing "Connection failed: Connection refused"

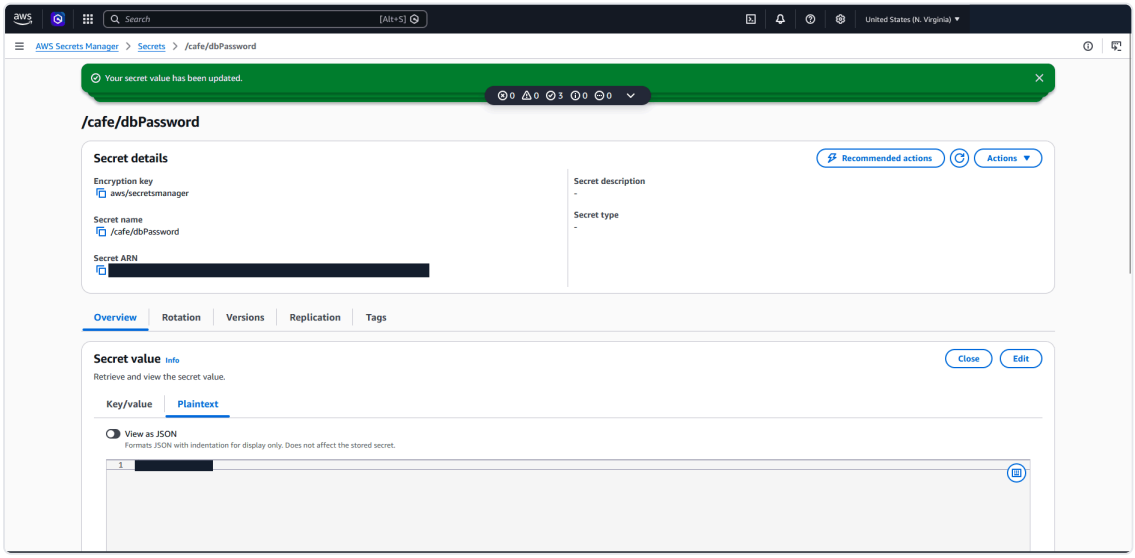


V3-E18 — Secrets Manager `/cafe/dbUrl` updated with the RDS endpoint

V3-E19 — Terminal: end of the RDS verification output (25 rows), then `sudo service mariadb stop` on the EC2 instance



V3-E20 — Secrets Manager `/cafe/dbUser` updated (`admin`)



V3-E21 — Secrets Manager `/cafe/dbPassword` updated (value masked)

V3-E22 — Final Order History: new order #26 placed after the switch, with all migrated orders still present

Outcome

The café's order data now lives in a managed Amazon RDS MariaDB instance in the private database subnets, reachable only from the application's security group. The application kept all 25 existing orders and accepted new ones (order #26) after the switch, with no code changes.

CHAPTER 5 · STAGE V4

V4 — Secure VPC Networking

> Completed as part of AWS Academy Challenge Labs (training environment).

Business Requirement

After moving the database to a private subnet (V3), the café team wanted to improve the security of the application layer as well. Their AWS advisor recommended running the application server in a private subnet that is not reachable from the internet, and giving administrators access only through a **bastion host** (jump box). The private server still has to download patches, so it needs outbound internet access without being exposed to inbound connections.

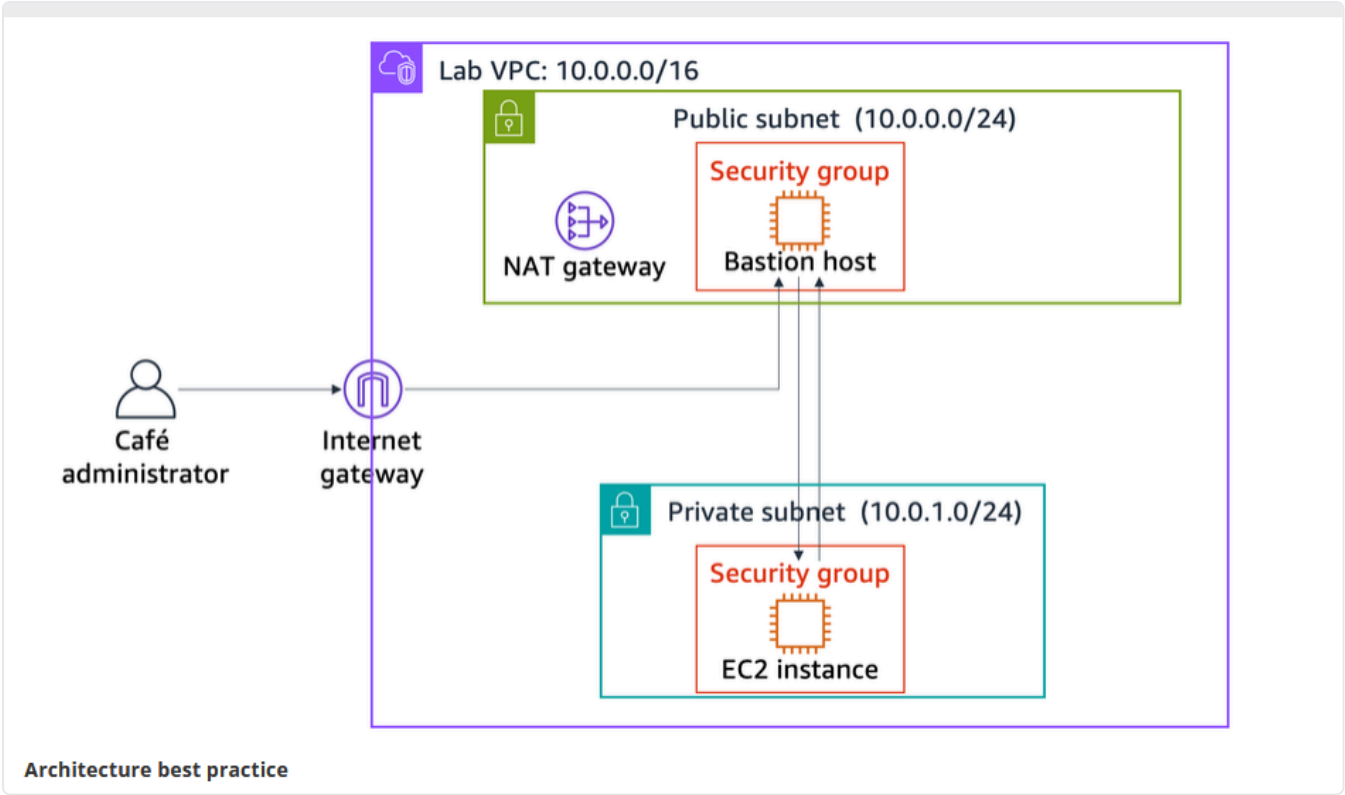
The team also wanted a **non-production VPC** where they could build and test this design, plus an extra security layer around the private subnet using a **custom network ACL**.

Lab objectives

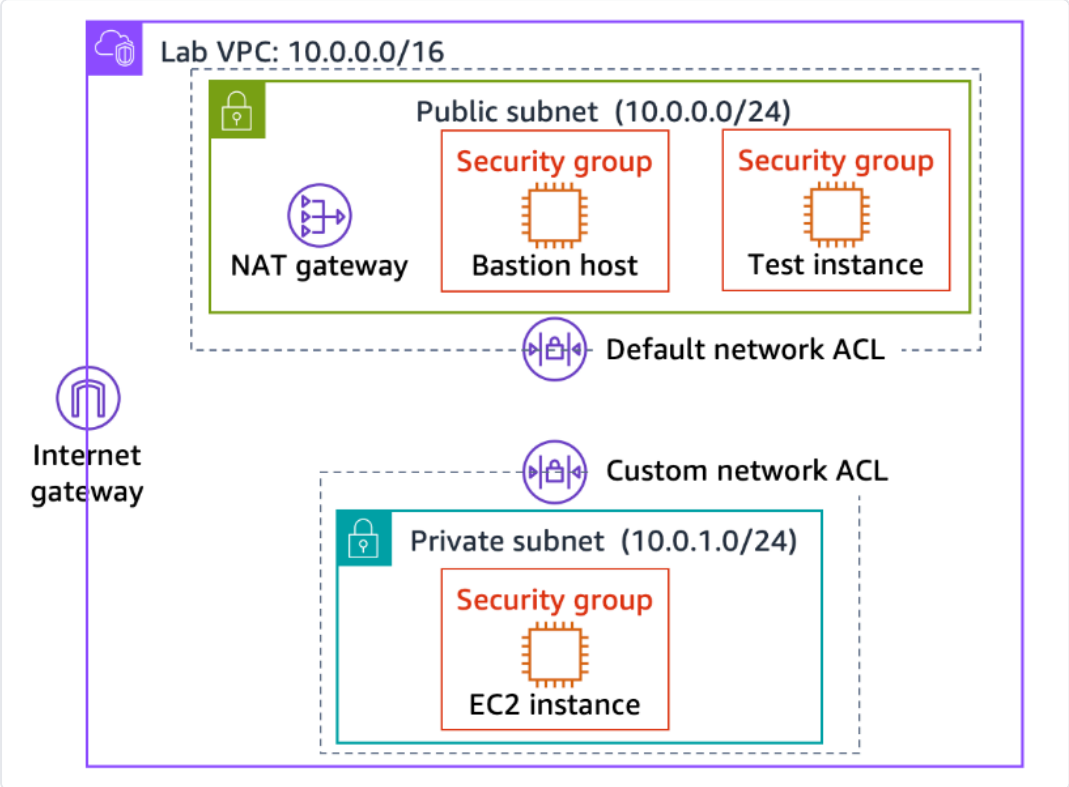
- Create a VPC environment that allows secure connections to private resources
- Let private resources reach the internet without being publicly reachable
- Add security layers (NAT gateway, security groups, network ACL) around the private resources

Start and Target Architecture

These two diagrams come from the AWS Academy lab material. They show where this stage starts and what it builds. They are shown here as reference; my own diagram of what I deployed follows in the next section.



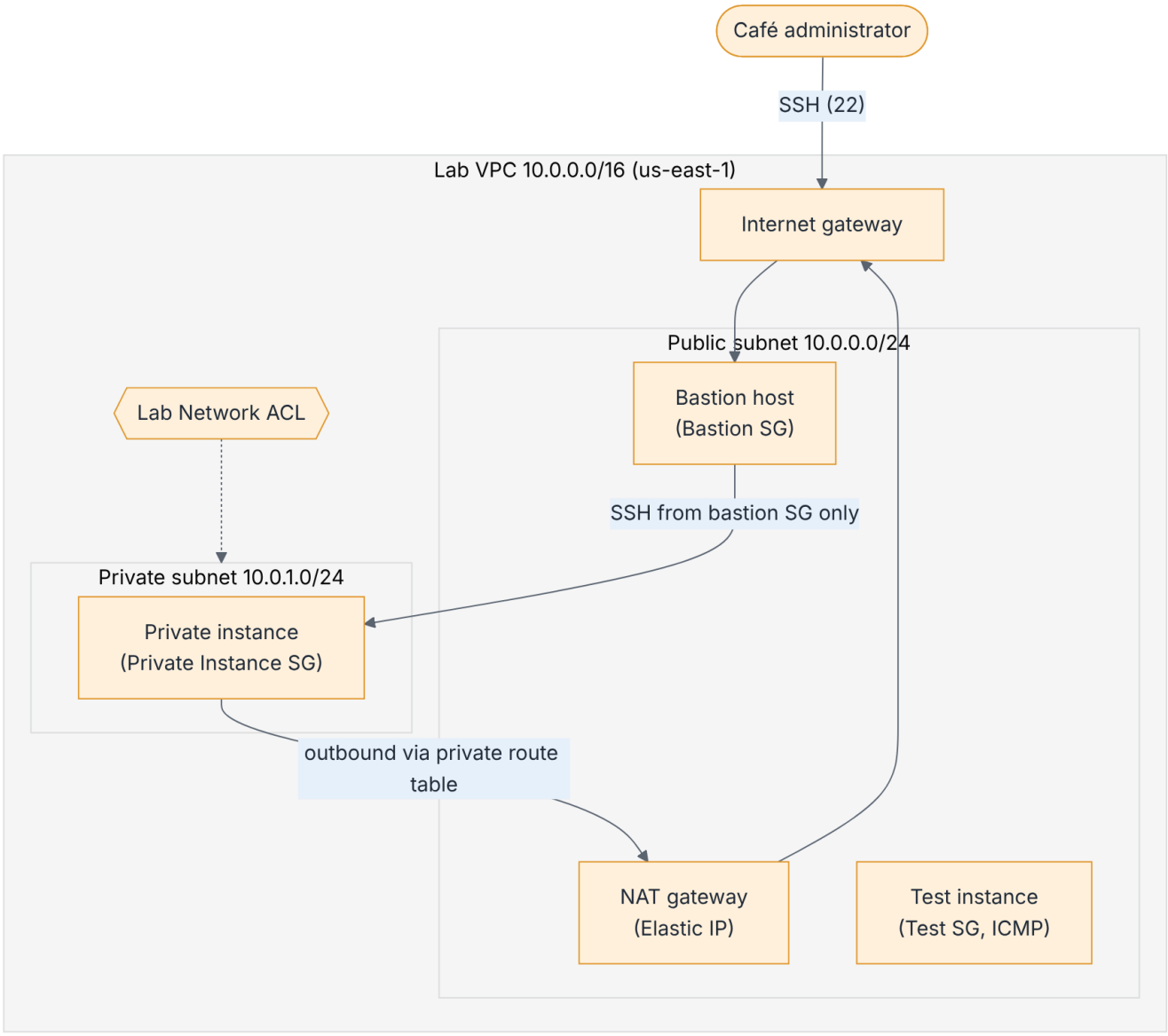
Bastion access to the private subnet. AWS Academy lab diagram: how an administrator reaches the private subnet.



Target architecture. AWS Academy lab diagram: the design this stage builds.

Before → After

	BEFORE	AFTER
Network	A VPC with no usable subnets	Public subnet `10.0.0.0/24` and private subnet `10.0.1.0/24` in the Lab VPC (`10.0.0.0/16`)
Internet access (public)	None	Internet gateway attached; public route `0.0.0.0/0 → IGW`
Admin access	–	SSH only through a bastion host in the public subnet
Private resources	–	Private instance with no public IP, SSH allowed only from the bastion's security group
Outbound access from private subnet	–	NAT gateway in the public subnet; private route `0.0.0.0/0 → NAT`
Extra layer	–	Custom network ACL tested by blocking ICMP between instances



AWS Services Used

SERVICE / FEATURE	ROLE IN V4
Amazon VPC	The isolated network (`10.0.0.0/16`) holding all resources
Subnets (public + private)	Separate internet-facing and internal resources
Internet gateway	Connects the public subnet to the internet
Route tables	Public route to the IGW; separate private route table pointing to the NAT gateway
NAT gateway + Elastic IP	Lets private instances start outbound connections (patches, updates) without inbound exposure
Amazon EC2	Bastion host, private instance and test instance (Amazon Linux 2023, `t2.micro`)
Security groups	Instance-level firewall (SSH from my IP to bastion; SSH from bastion SG to private instance; ICMP for the test instance)
Network ACL	Subnet-level, stateless firewall layer with explicit allow/deny rules
Key pairs / SSH (PuTTY)	Key-based SSH access to the bastion and, through it, to the private instance

Implementation Highlights

1. **Public subnet and internet access.** Created a public subnet (`10.0.0.0/24`) in the Lab VPC, created a new internet gateway (`igw-new`) and attached it to the VPC, then added a route `0.0.0.0/0 → internet gateway` to the route table that serves the public subnet. *(E01, E02, E03)*
2. **Bastion host.** Launched an EC2 instance (Amazon Linux 2023, `t2.micro`) in the public subnet and connected to it over SSH (port 22) from my machine using PuTTY and a `.ppk` key. *(E04)*
3. **Private subnet.** Created a second subnet (`10.0.1.0/24`) for private resources. *(E05)*
4. **NAT gateway.** Created `Lab NAT Gateway` in the public subnet with an Elastic IP, so that instances in the private subnet can reach the internet. *(E06)*
5. **Separate route table for the private subnet.** Created `Private Route Table` in the Lab VPC, added `0.0.0.0/0 → NAT gateway`, and associated it explicitly with the private subnet. The public subnet keeps using the VPC's main route table (which has the internet gateway route). *(E07, E08, E09)*
6. **Private instance.** Launched an EC2 instance in the private subnet with a new key pair (`vockey2`) and a security group (`Private Instance SG`) that allows SSH (22) **only from the bastion host's security group**. It has no public IP. *(E09)*
7. **SSH pass-through.** Configured the PuTTY session so I could SSH into the bastion and from there into the private instance, without copying the private key onto the bastion. *(E10)*
8. **Connectivity test.** From the private instance, `ping 8.8.8.8` succeeded, which confirms outbound internet access through the NAT gateway. *(E10)*
9. **Network ACL.** Created `Lab Network ACL` for the Lab VPC. A custom ACL starts by denying everything, so I added allow-all rules (rule 100) in both directions, plus a lower-numbered **deny ICMP** rule (rule 99) for the private instance's address (`10.0.1.142/32`). *(E12, E13)*

10. **ACL test.** Launched a `Test Instance` in the public subnet (security group allowing ICMP), confirmed that the private instance could ping it, then added the deny rule and confirmed the ping stopped working (12 packets sent, 0 received). *(E11, E14)*

Challenges & Decisions

1. I lost SSH access to the bastion host after my home IP changed. I had limited the bastion's SSH rule to **My IP** (the right security choice). After my home router restarted, my public IP changed and SSH stopped working. I updated the access by adding a new inbound rule for the new IP, because editing the existing one did not work for me in the console. *(No screenshot.)* **What I learned:** restricting SSH to a single IP is good practice, but with a dynamic home IP it breaks easily. Things I would do next time: remove the old rule once the new one works (so stale IPs don't stay allowed), and in a real environment use AWS Systems Manager Session Manager or a VPN instead of opening port 22 at all.

2. Why a bastion host and not a public IP on the private server. The application server should be reachable only by admins. The private instance has no public IP, and its security group accepts SSH only from the bastion's security group (a reference to the group, not an IP range). So there is exactly one way in, and that door is a single hardened host.

3. The NAT gateway needs both a public subnet and a private route. Creating the NAT gateway alone did not give the private instance internet access. Two more pieces were needed: the NAT gateway must sit in the **public** subnet (so it can use the internet gateway), and the **private subnet's route table** must send `0.0.0.0/0` to the NAT gateway. That is why I created a separate `Private Route Table` instead of reusing the main one. After that, `ping 8.8.8.8` worked from the private instance. *(E07, E08, E10)*

4. Security groups vs. network ACLs. Security groups are stateful and instance-level; network ACLs are stateless and subnet-level. A custom ACL starts by denying all traffic, and rules are evaluated in number order, so the deny rule must have a **lower number (99) than the allow-all rule (100)** to take effect. Because ACLs are stateless, I also needed a matching rule in the outbound direction, not only inbound. *(E12, E13)*

5. Things I noticed while documenting this lab (for honesty and next time):

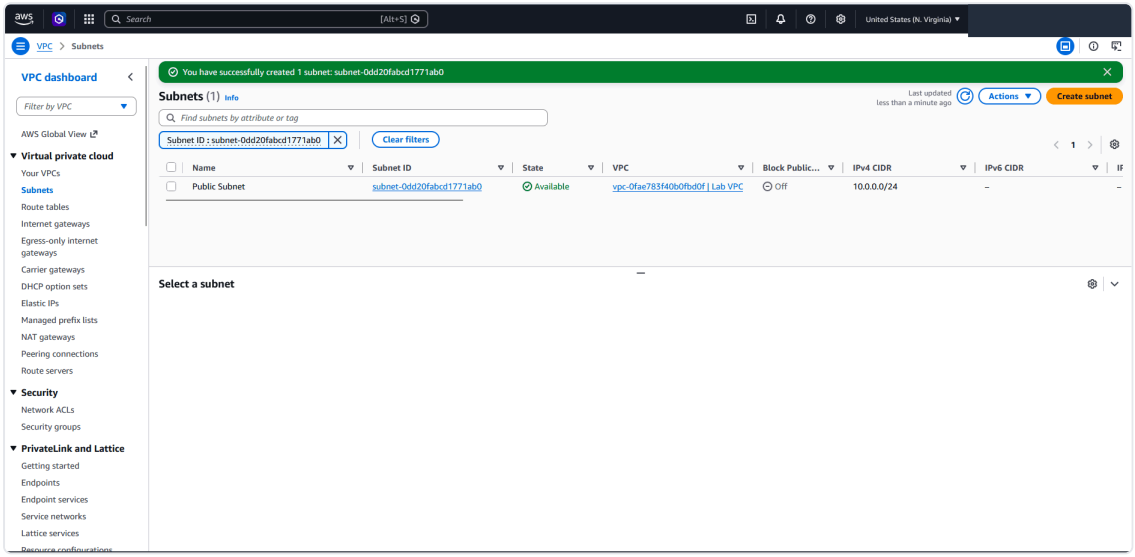
- **The deny rule uses the private instance's address (`10.0.1.142/32`).** The lab's wording asks for a rule that denies ICMP to the *Test Instance's* `/32` address. My rules reference the *private instance's* address instead, in both directions. The test result is the same (ping to the Test Instance stops), but the rule matches the opposite end of the conversation. *(E12, E13)*
- **The custom ACL ended up associated with both subnets.** The lab expects the custom ACL to protect the private subnet only, while the public subnet stays on the default ACL. In my account the Lab Network ACL shows two associated subnets (public and private). That is likely why the ping was blocked at both ends. In a real design I would associate it only with the private subnet. *(E15)*
- **The internet route for the public subnet lives in the VPC's main route table.** This works, but a safer pattern is to keep the main route table private and create an explicit public route table, so a new subnet is never public by accident.

Validation & Evidence

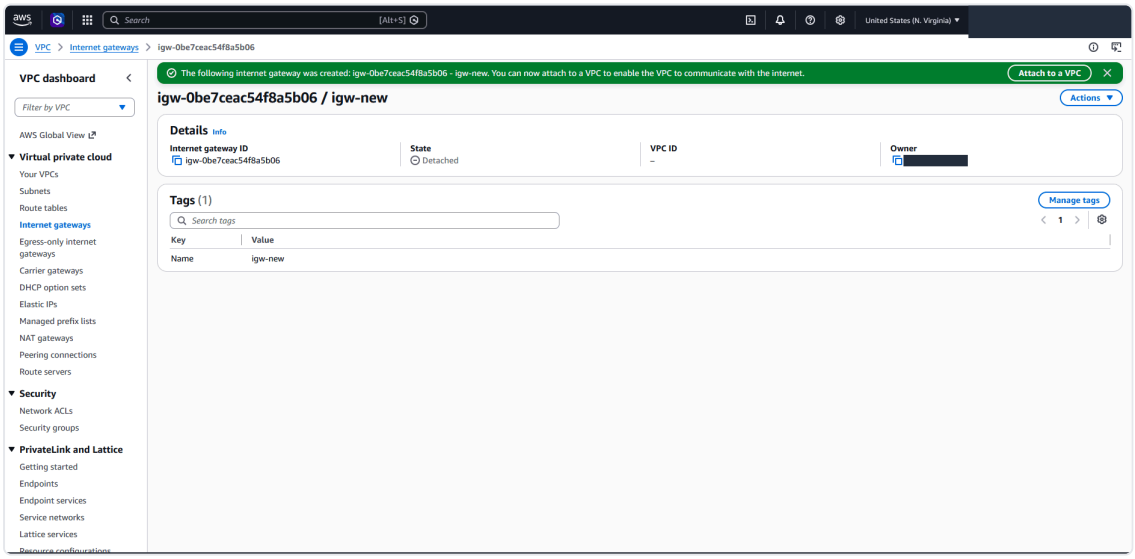
All 27 screenshots of this stage, in lab order. Account IDs, ARNs, e-mail addresses and personal details were masked.

> **Note on evidence.** Account IDs, user details, ARNs, personal file paths and my own public IP address were redacted from all screenshots. The public IPs visible in some images belong to temporary lab instances that no longer exist.

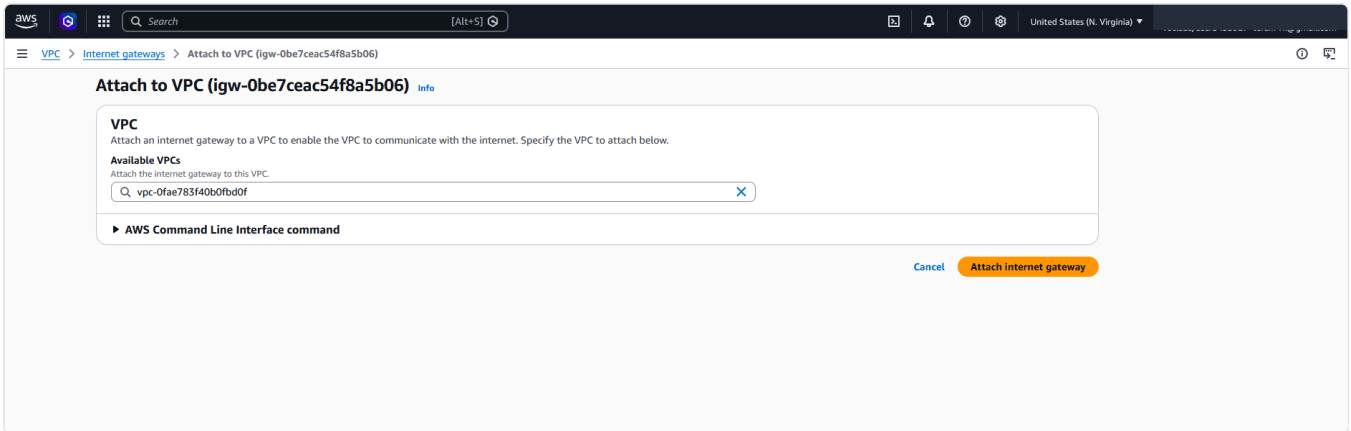
Task 1 — Public subnet and internet gateway



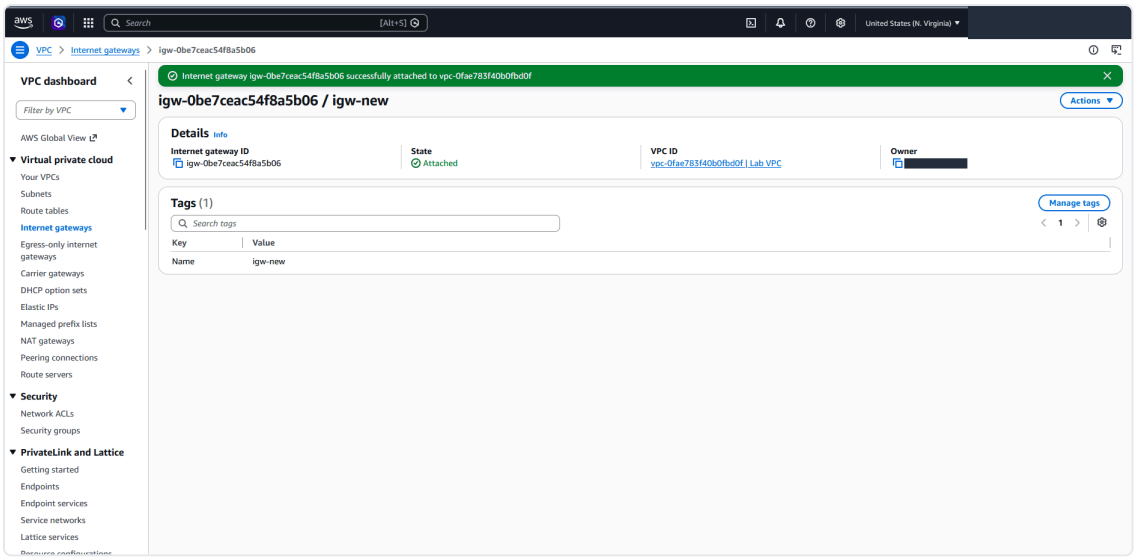
V4-E01 — Public subnet created.



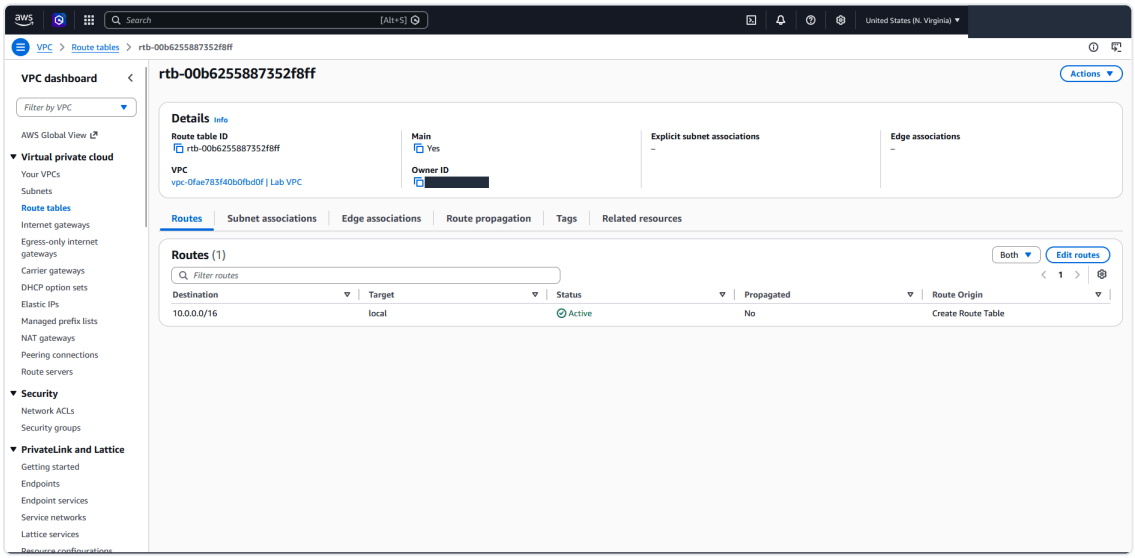
V4-E16 — Internet gateway created.



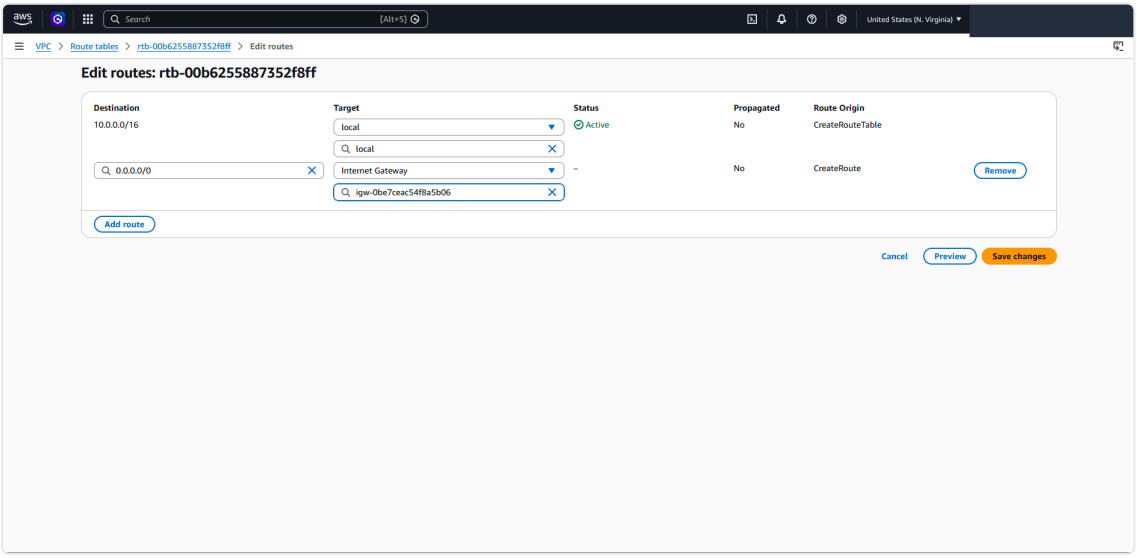
V4-E17 — Attaching the internet gateway to the Lab VPC.



V4-E02 — Internet gateway attached.

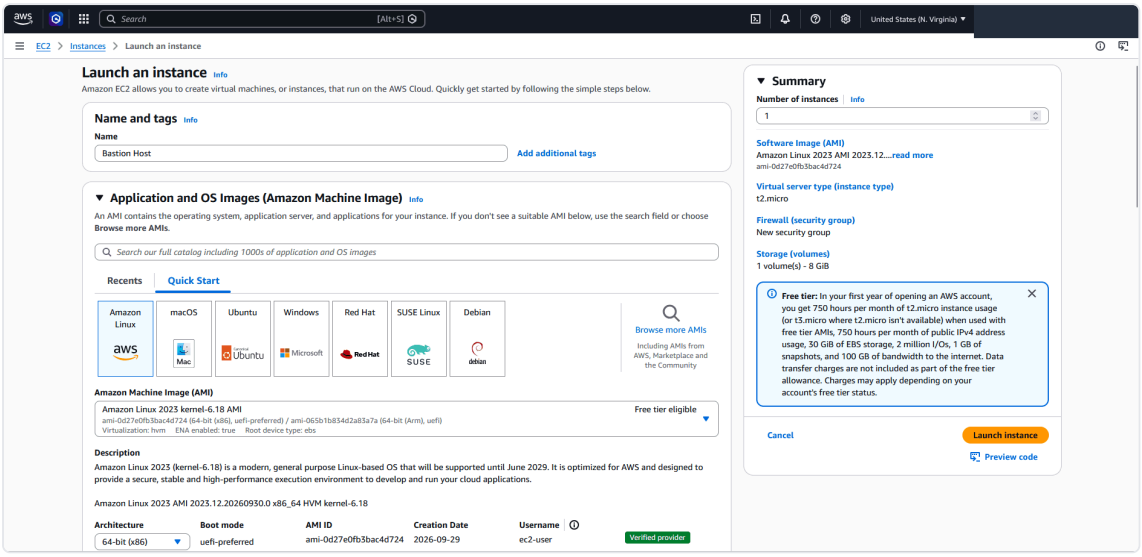


V4-E18 — Main route table before adding the internet route.

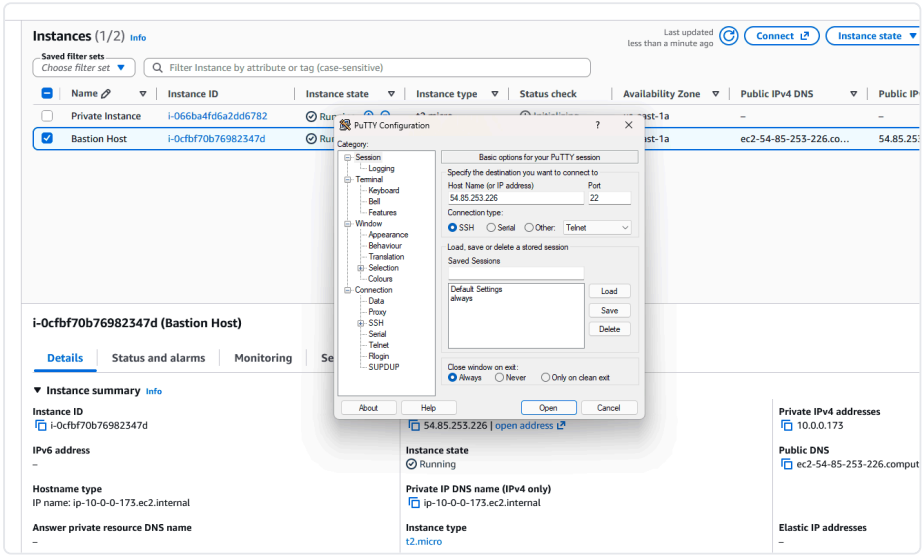


V4-E03 — Route `0.0.0.0/0` to the internet gateway.

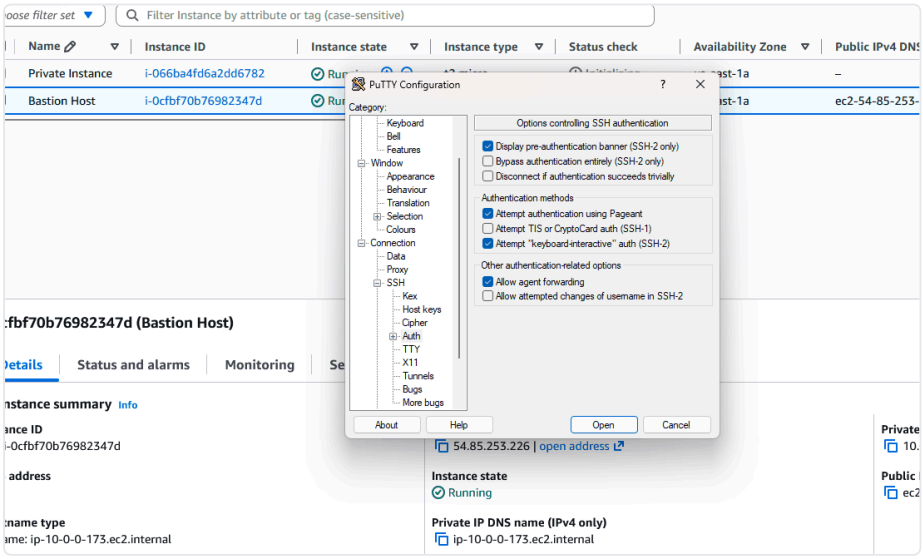
Tasks 2–3 — Bastion host



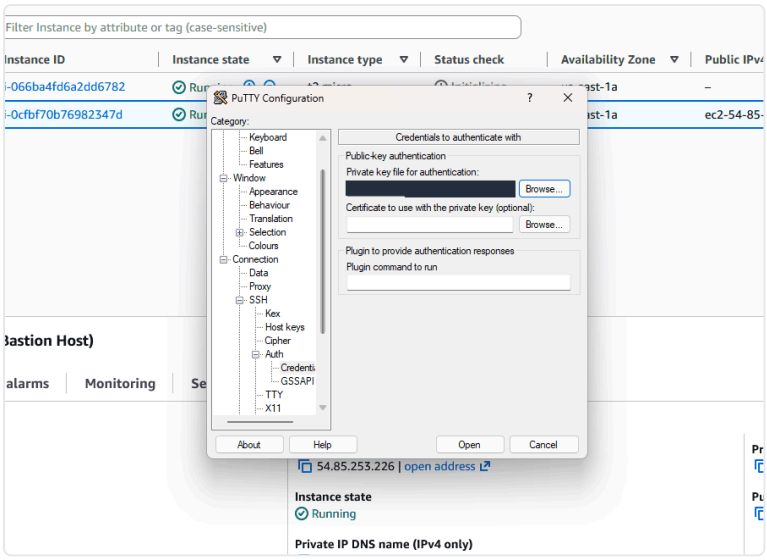
V4-E19 — Launching the bastion host.



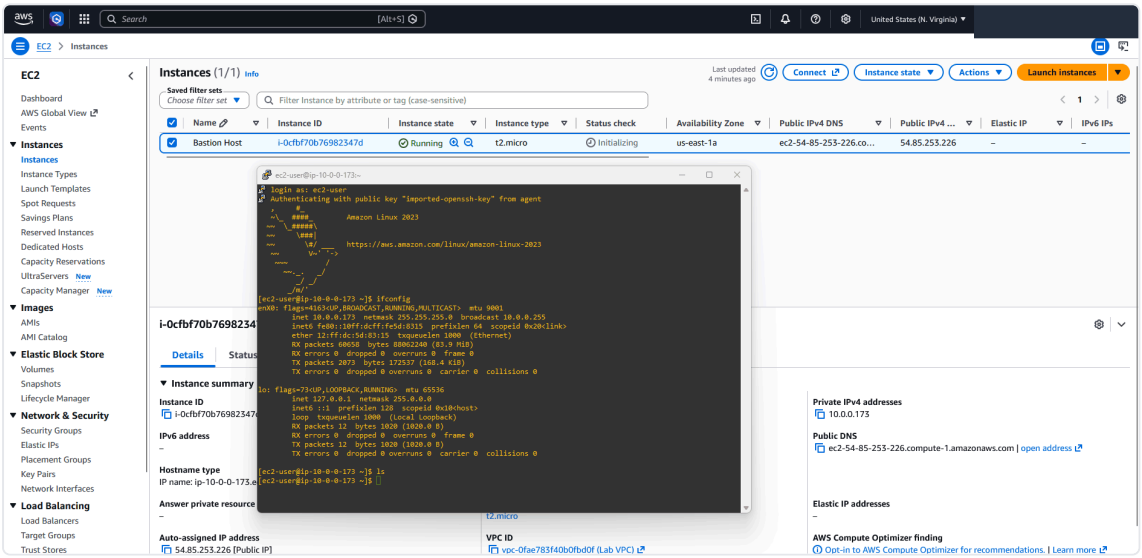
V4-E23 — PuTTY session to the bastion host.



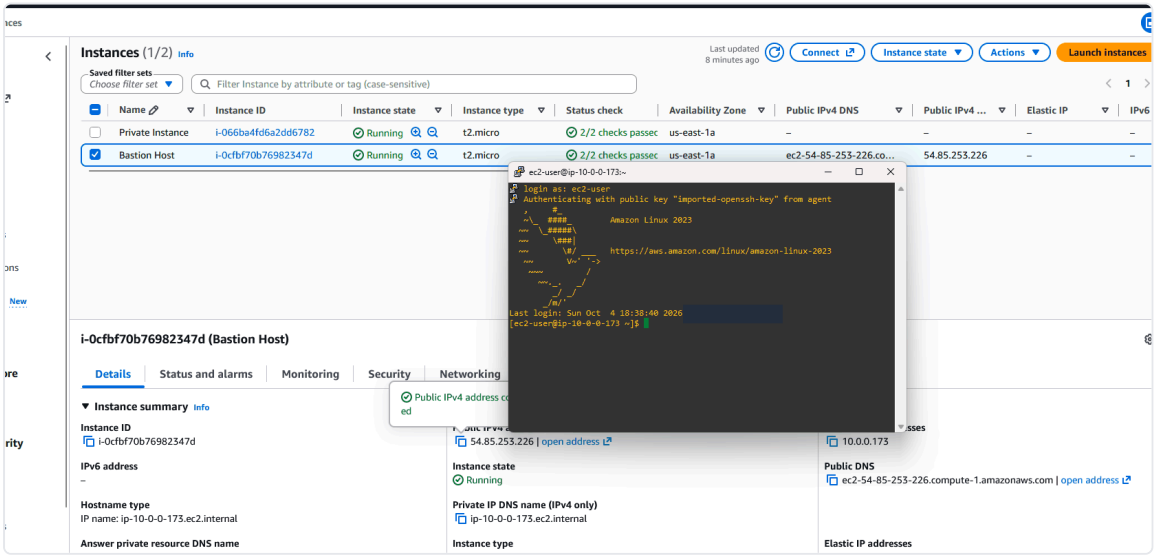
V4-E24 — PuTTY authentication options (agent forwarding).



V4-E25 — PuTTY private key setting (file path redacted).

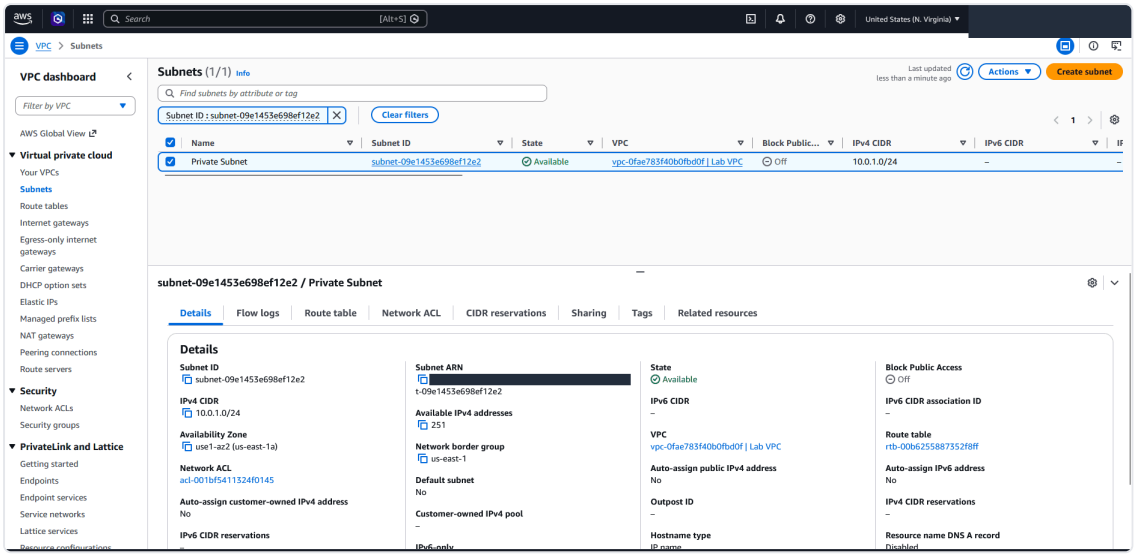


V4-E04 — Bastion host running with an open SSH session.

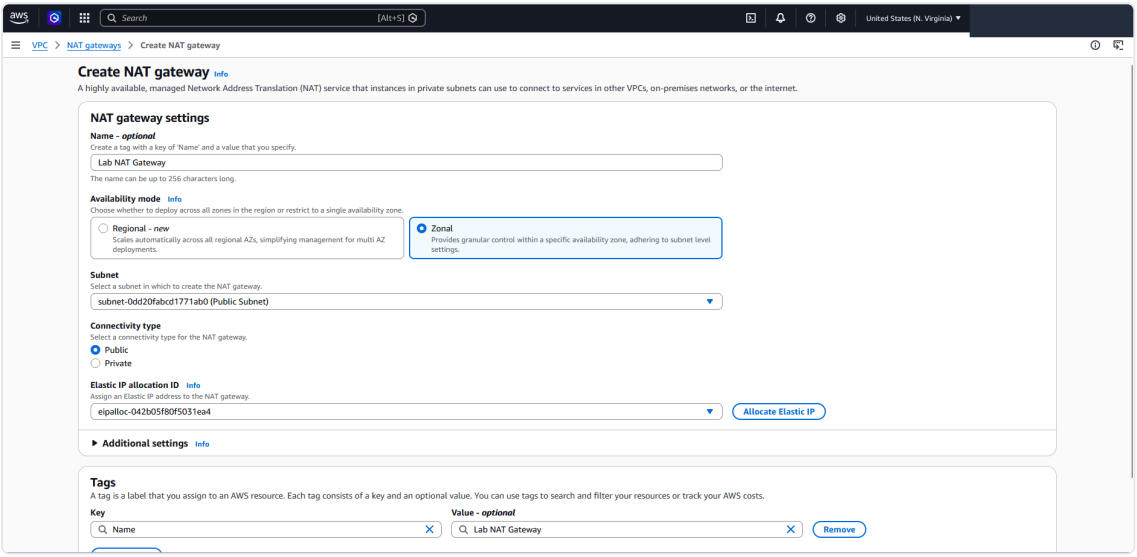


V4-E26 — Logged in to the bastion host.

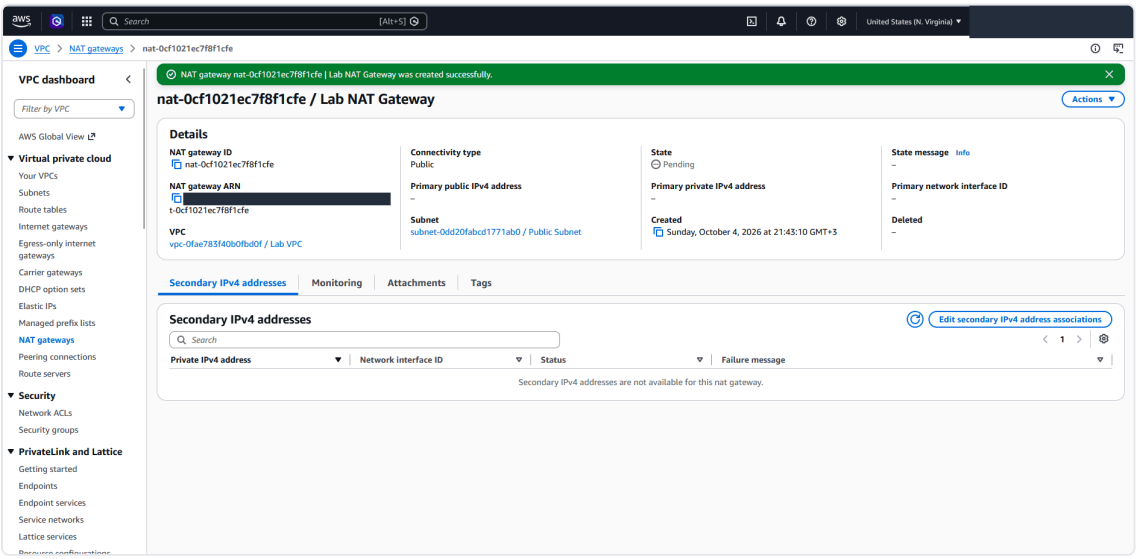
Tasks 4–5 — Private subnet and NAT gateway



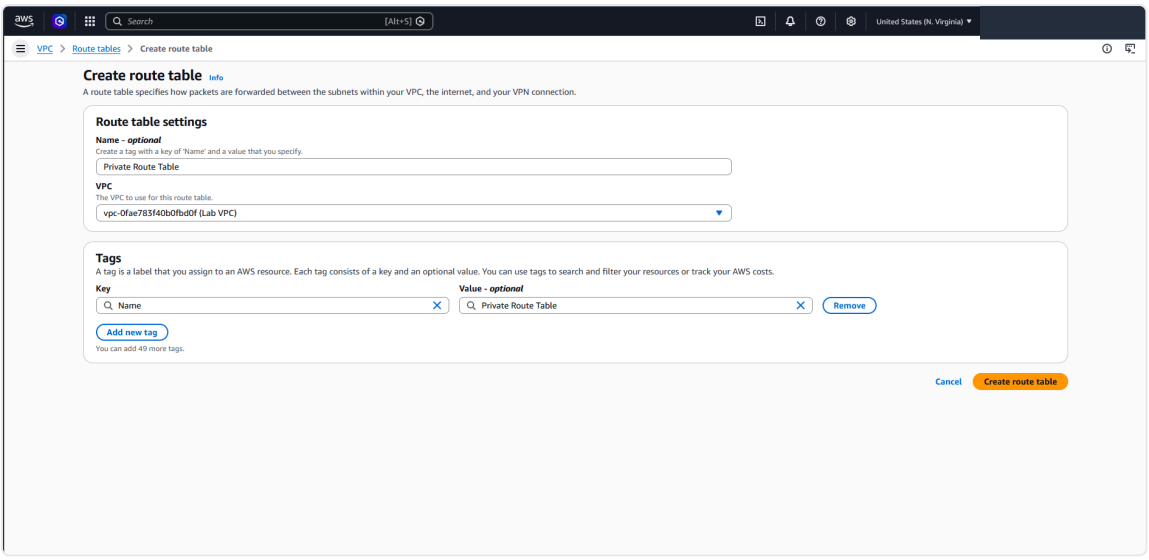
V4-E05 — Private subnet created.



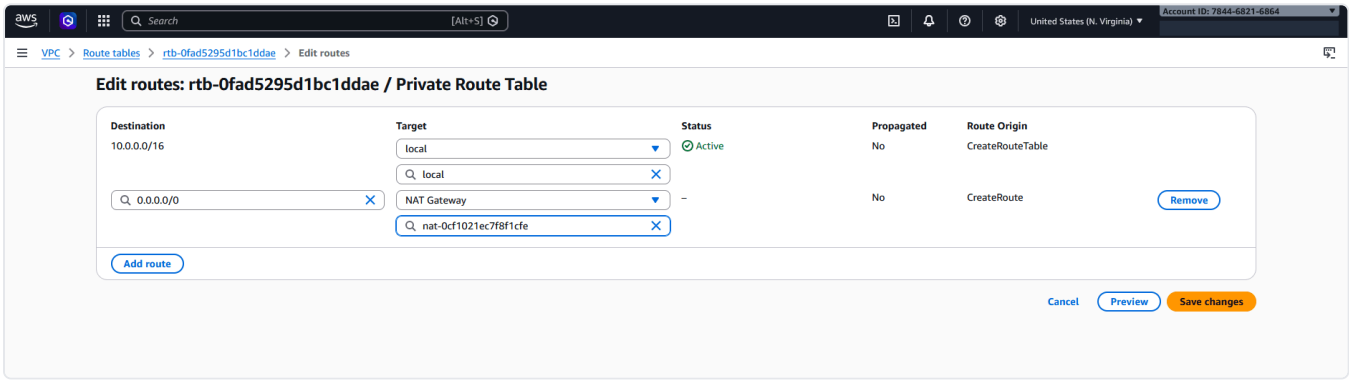
V4-E20 — Creating the NAT gateway.



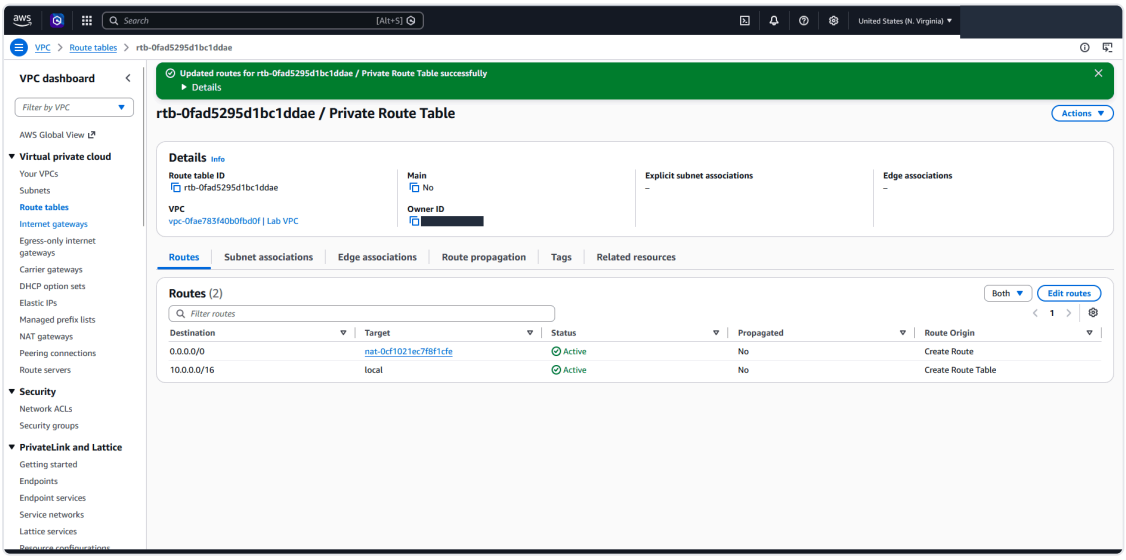
V4-E06 — NAT gateway created.



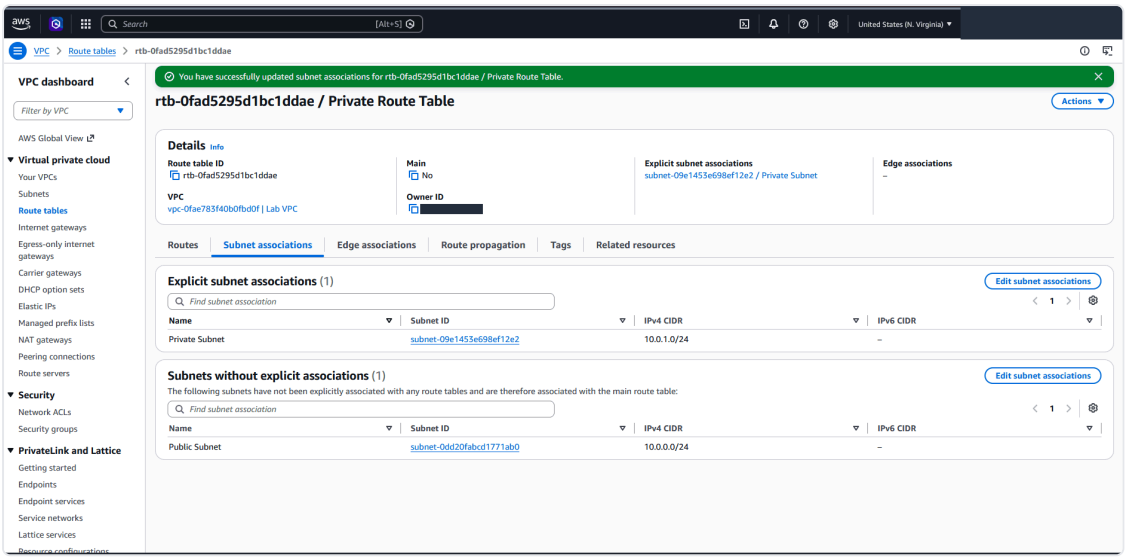
V4-E21 — Creating the Private Route Table.



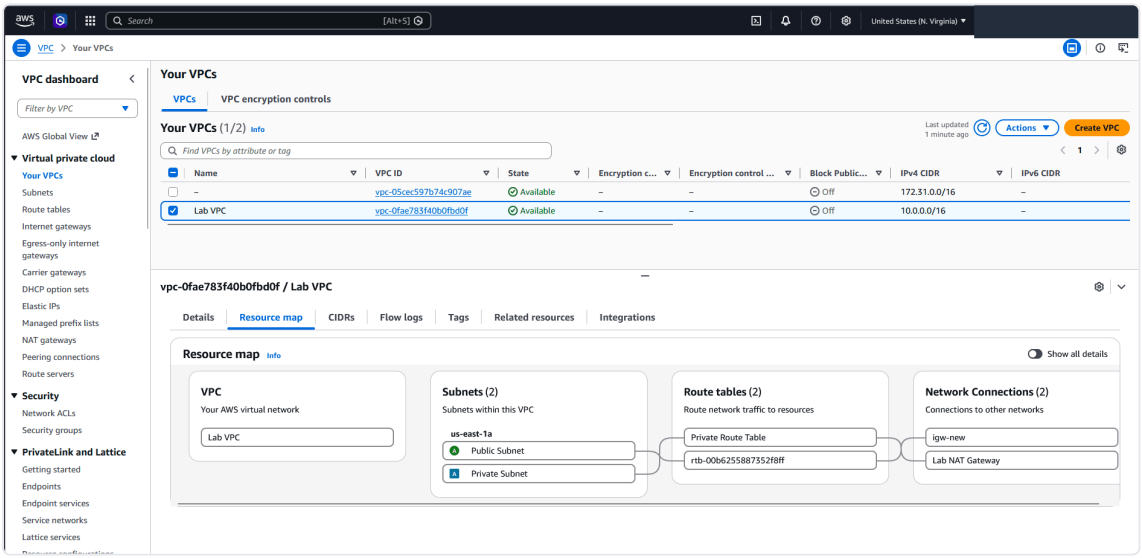
V4-E22 — Adding the route to the NAT gateway.



V4-E07 — Private route table with the NAT route.

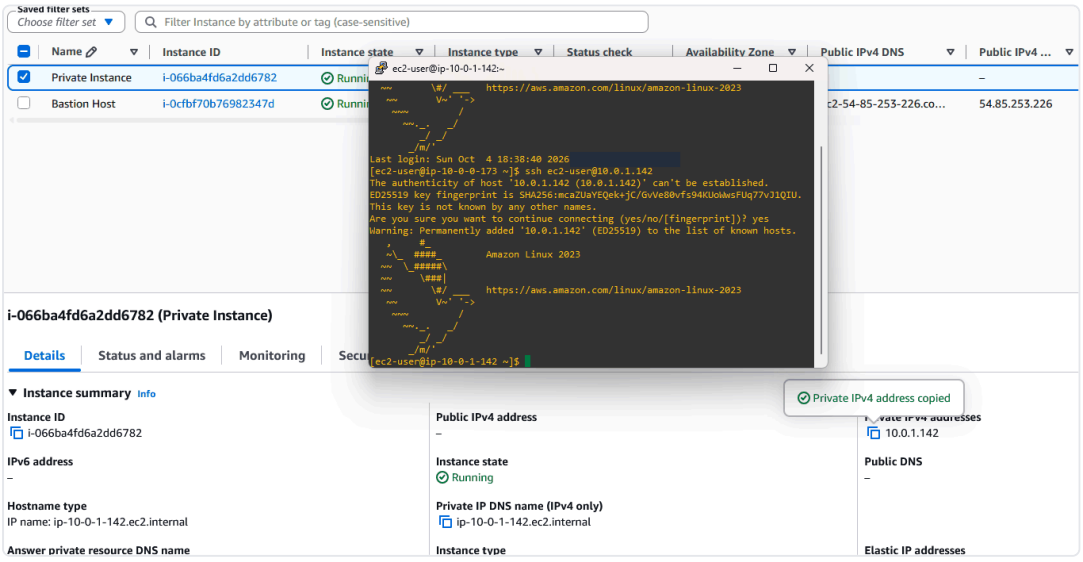


V4-E08 — Private route table associated with the private subnet.

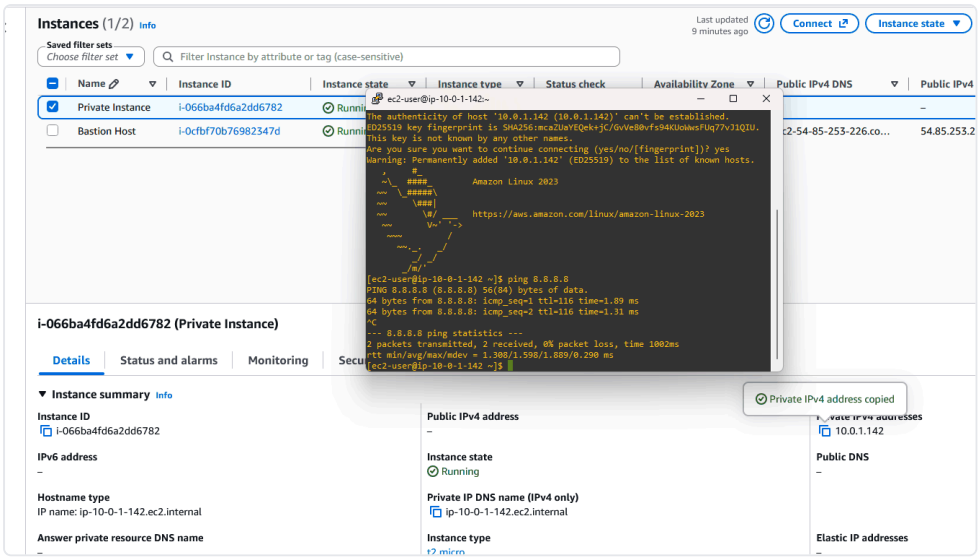


V4-E09 — VPC resource map.

Tasks 6–8 — Private instance and SSH pass-through

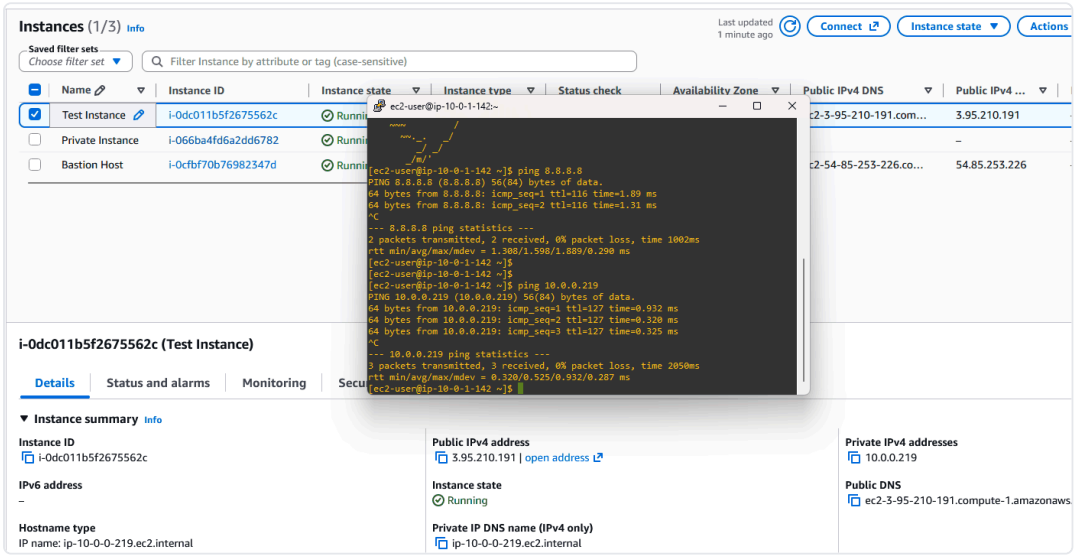


V4-E27 — SSH hop from the bastion host to the private instance.

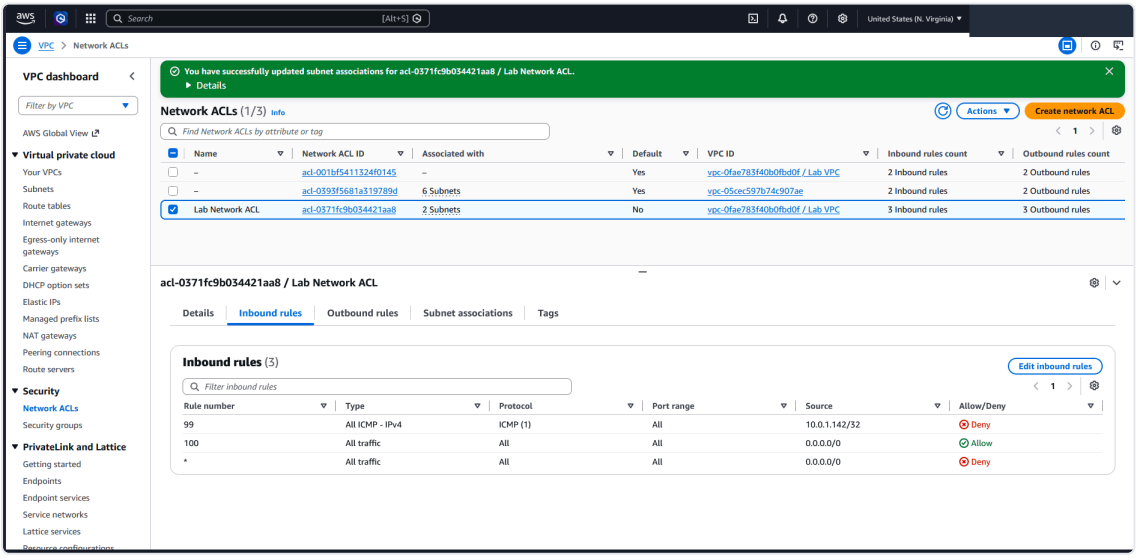


V4-E10 — Private instance reaches the internet through the NAT gateway.

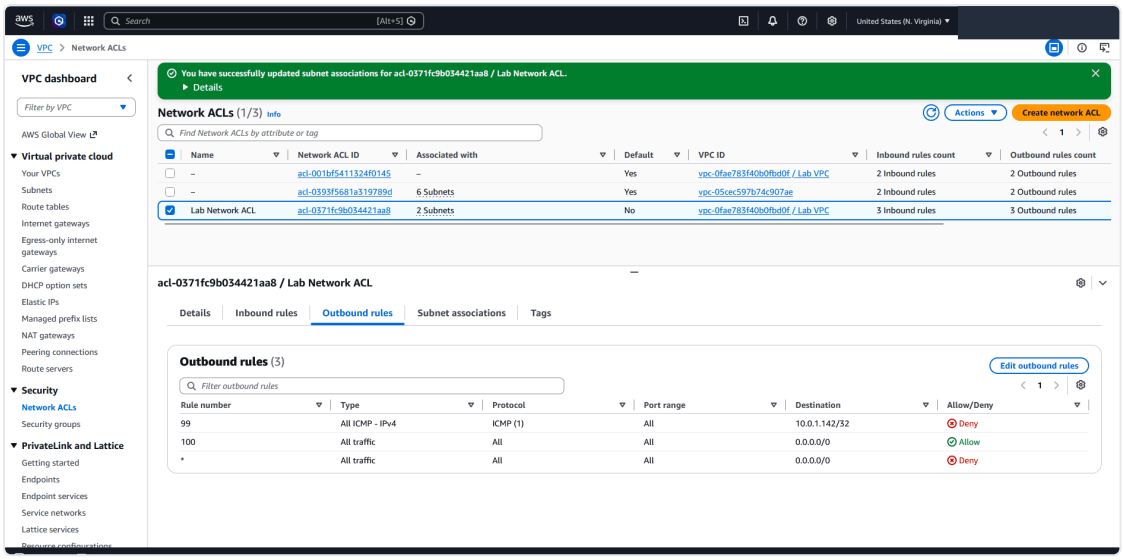
Tasks 9–10 — Network ACL



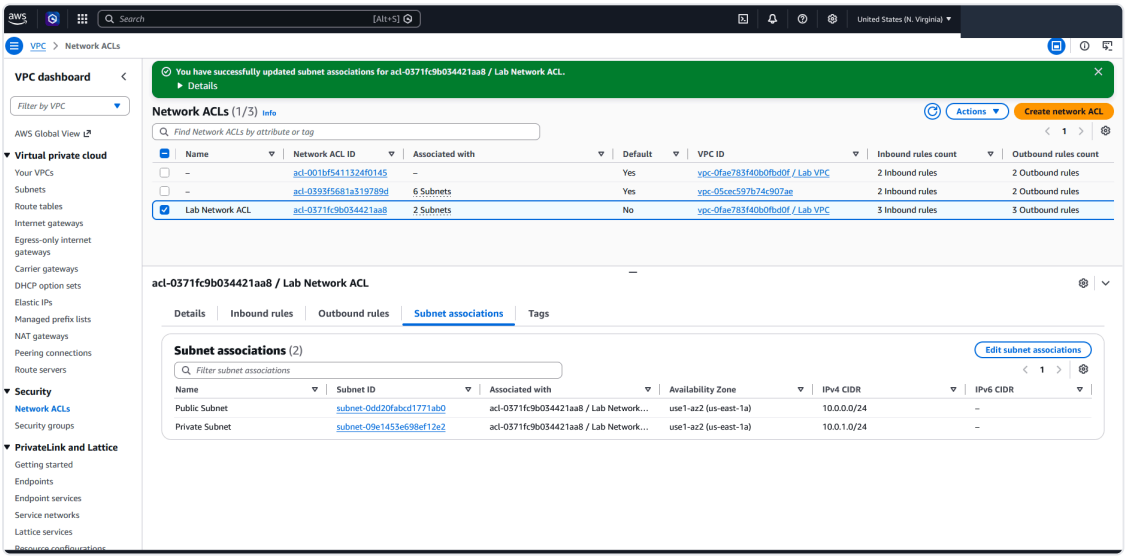
V4-E11 — Ping to the Test Instance works before the deny rule.



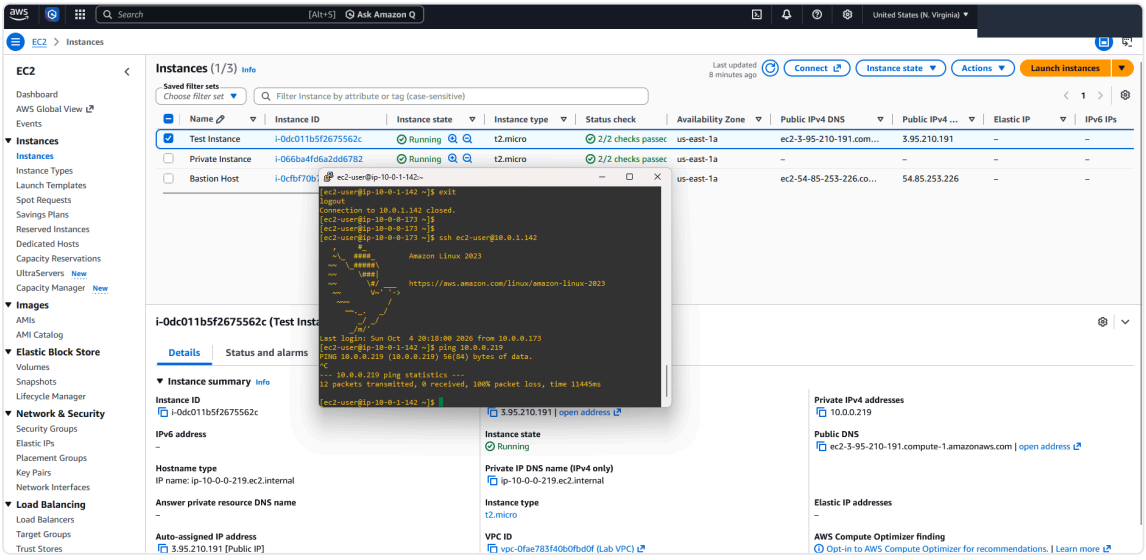
V4-E12 — Network ACL inbound rules.



V4-E13 — Network ACL outbound rules.



V4-E15 — Subnets associated with the Lab Network ACL.



V4-E14 — Ping blocked by the network ACL.

Outcome

The café now has a VPC with a public tier (bastion host, NAT gateway) and a private tier (private instance). Private resources have no public IP, are reachable only through the bastion host, can still reach the internet for updates through the NAT gateway, and sit behind an additional network ACL layer that I tested by blocking and observing traffic.

Limits of this version (and what the next versions address): a single Availability Zone means one failure can take the whole site down, and a single instance cannot absorb a traffic spike. V5 addresses this with a load balancer, Auto Scaling and a second Availability Zone.

CHAPTER 6 · STAGE V5

V5 — Scalability & High Availability (ALB + Auto Scaling)

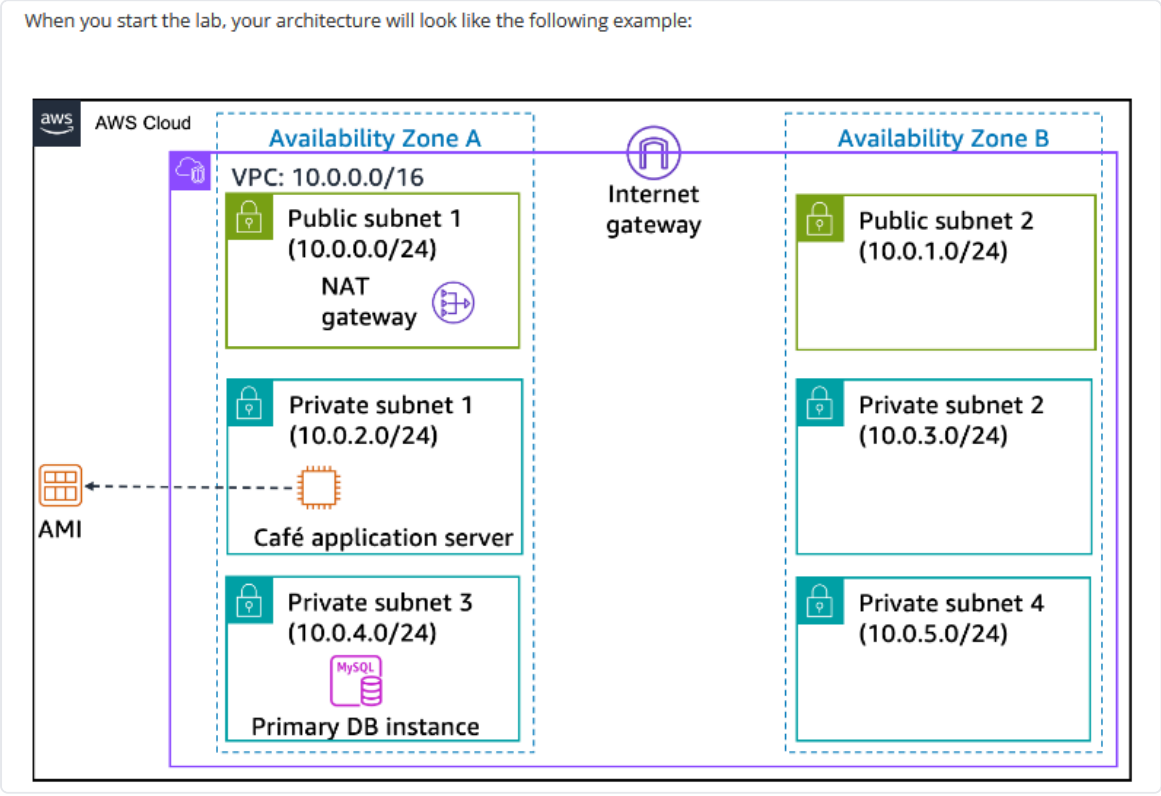
> Completed as part of AWS Academy Challenge Labs (training environment).

Business Requirement

The café was about to be featured on a TV food show, which could bring a short spike of up to tens of thousands of visitors. A single web server in a single Availability Zone could not absorb that traffic, so the website had to stay responsive, scale up and down with demand, and survive the loss of one Availability Zone.

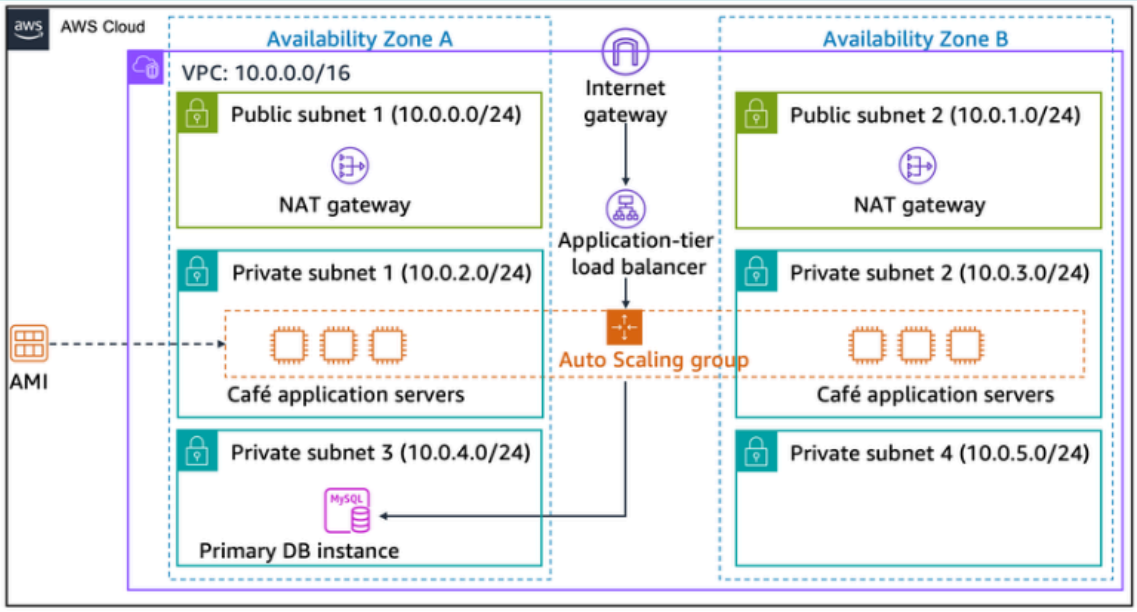
Start and Target Architecture

These two diagrams come from the AWS Academy lab material. They show where this stage starts and what it builds. They are shown here as reference; my own diagram of what I deployed follows in the next section.



Starting architecture. AWS Academy lab diagram: where the stage begins.

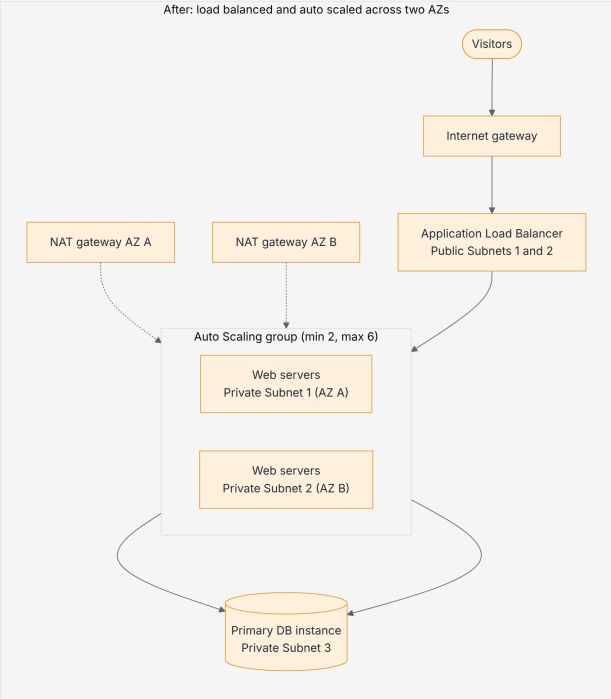
At the end of this lab, your architecture should look like the following example:

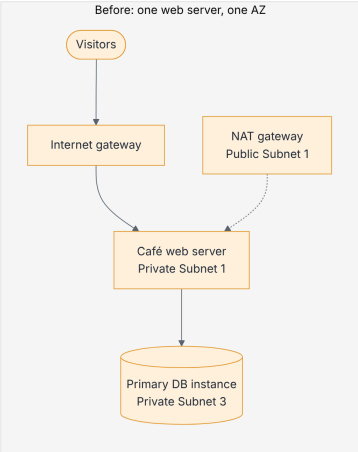


Note: In this challenge lab, step-by-step instructions are not provided for most tasks. You must figure out how to complete the tasks on your own.

Target architecture. AWS Academy lab diagram: the design this stage builds.

Before → After





AWS Services Used

SERVICE	ROLE IN THIS STAGE
Amazon VPC (subnets, route tables, NAT gateway)	Second NAT gateway so private instances in the second AZ can reach the internet
Amazon EC2 (AMI, launch template)	Repeatable definition of the café web server (`CafeWebAppServer-temp` , t2.micro)
Amazon EC2 Auto Scaling	Keeps 2 to 6 web servers running across two AZs and scales on average CPU
Elastic Load Balancing (Application Load Balancer)	Public entry point that spreads HTTP requests across the private web servers

SERVICE	ROLE IN THIS STAGE
Amazon EC2 security groups	`CafeAPPELBallowHTTP` allows HTTP (port 80) from the internet to the load balancer
AWS Systems Manager Session Manager	Shell access to a web server to run the CPU stress test

Implementation Highlights

- **Network first.** The web servers sit in private subnets, so the second AZ needed its own NAT gateway before any instance could start there. I created a zonal NAT gateway (`natgw-az2`) in Public Subnet 2 and pointed the default route of Private Route Table 2 to it (E02, E04, E05).
- **One template for every server.** I built a launch template from the café AMI (`Cafe WebServer Image`) with a t2.micro instance type, so every instance the group starts is identical (E06-E09).
- **Auto Scaling group across two AZs.** The group uses Private Subnet 1 and Private Subnet 2 with desired 2, minimum 2 and maximum 6 instances, and scales on average CPU utilization (E10-E13).
- **Load balancer in the public subnets.** The Application Load Balancer `CafeAppELB` is internet-facing, sits in the two public subnets, and forwards HTTP:80 to the target group `CafeAPPELB-Targetgroup`. A dedicated security group opens port 80 to the internet. The Auto Scaling group was then attached to the target group so new instances register automatically (E14-E18).
- **Test under load.** I opened the site through the load balancer DNS name, then used Session Manager to run a CPU stress test on one instance and watched the group add servers until it reached six (E19-E23).

Challenges & Decisions

No failures are recorded for this stage in my notes or screenshots; the decisions that shaped it were:

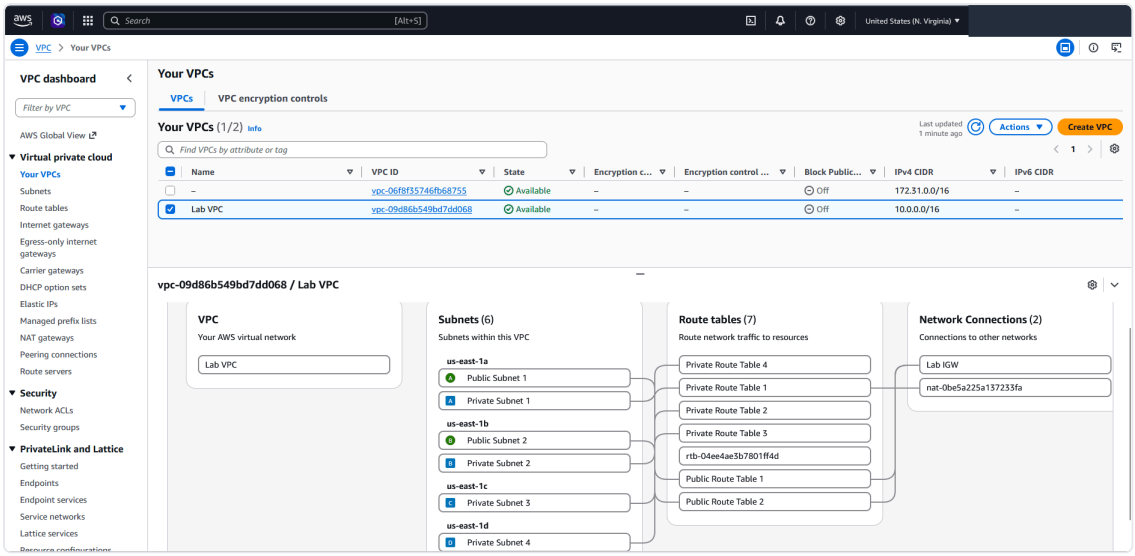
- **Web servers stay in private subnets** and only the load balancer is public, so the application servers are never directly reachable from the internet.
- **One NAT gateway per AZ.** A single NAT gateway in AZ A would have been a single point of failure for outbound traffic from AZ B, which would defeat the high availability goal.
- **Minimum of 2 instances** so there is always one server in each AZ, with a maximum of 6 to cap cost during the traffic spike.
- **Scaling on CPU** with a target of 25% average utilization and a 60 second warm-up (from my lab notes), which made the scale-out easy to trigger with a short stress test.

Validation & Evidence

All 23 screenshots of this stage, in lab order. Account IDs, ARNs, e-mail addresses and personal details were masked.

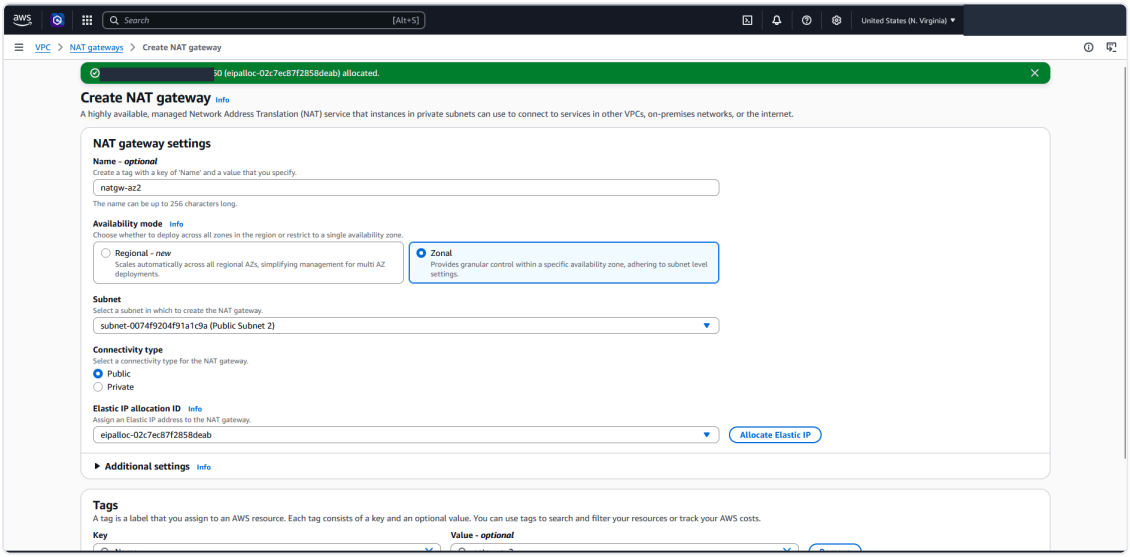
> Evidence note: the screenshots come from more than one lab session (the Lab VPC ID is not the same in E01-E05 as in E14-E18). Account details and public IPs were masked.

Task 1 - Inspect the environment

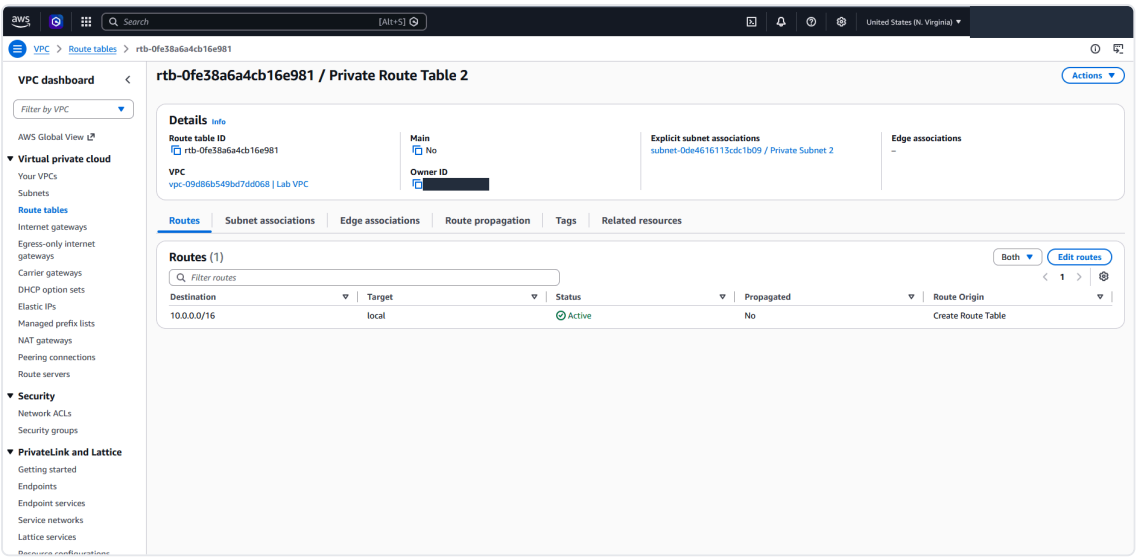


V5-E01 — Lab VPC resource map before any change: 4 private subnets, public subnets in two AZs, one NAT gateway.

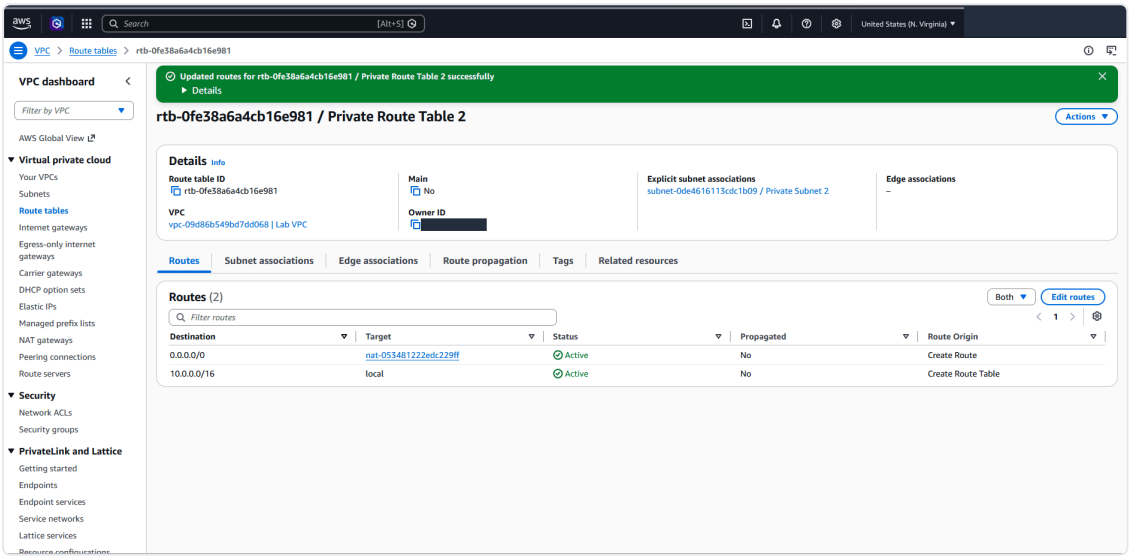
Task 2 - Second NAT gateway



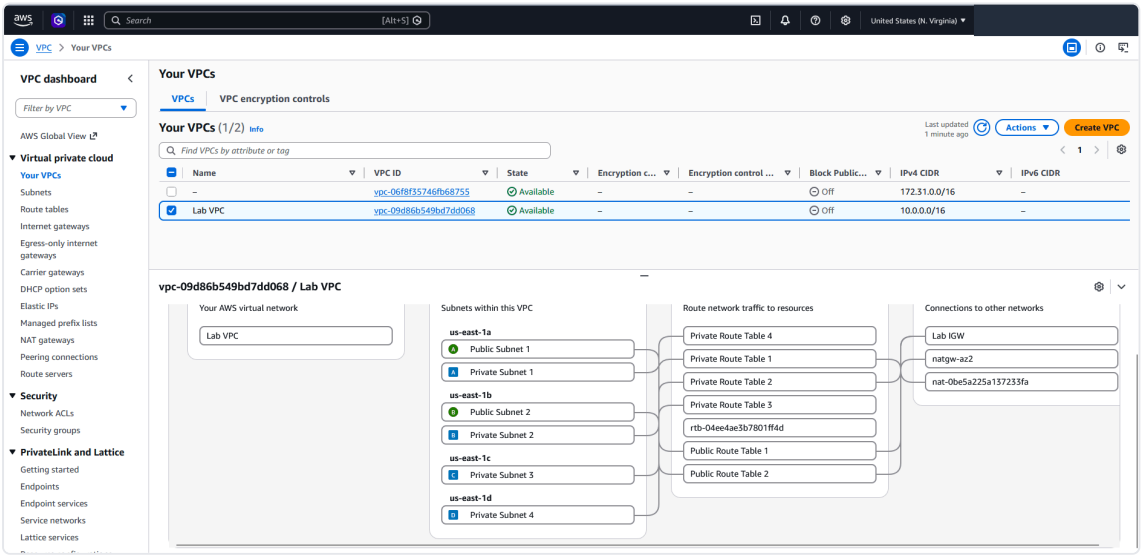
V5-E02 — Creating a zonal NAT gateway (natgw-az2) in Public Subnet 2 with its own Elastic IP.



V5-E03 — Private Route Table 2 before the change: only the local route.

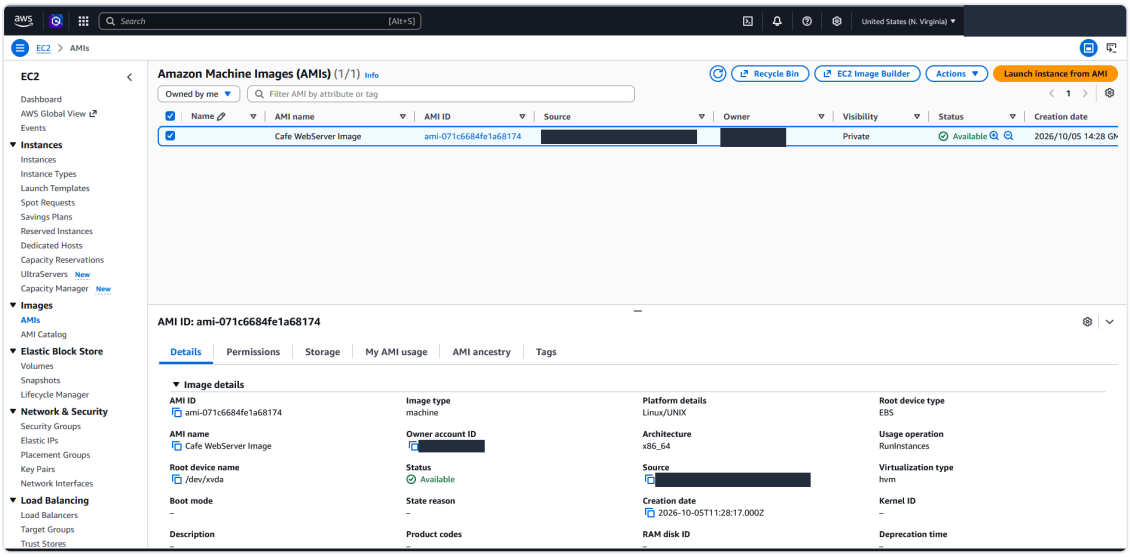


V5-E04 — Private Route Table 2 after adding 0.0.0.0/0 pointing to the new NAT gateway.

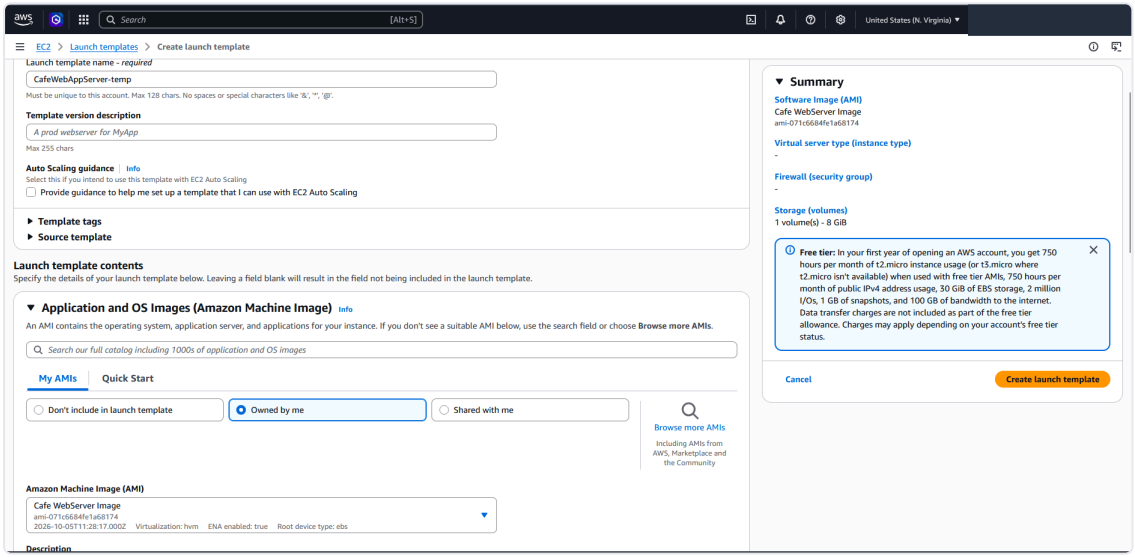


V5-E05 — VPC resource map after the change: second NAT gateway (natgw-az2) is attached to the network.

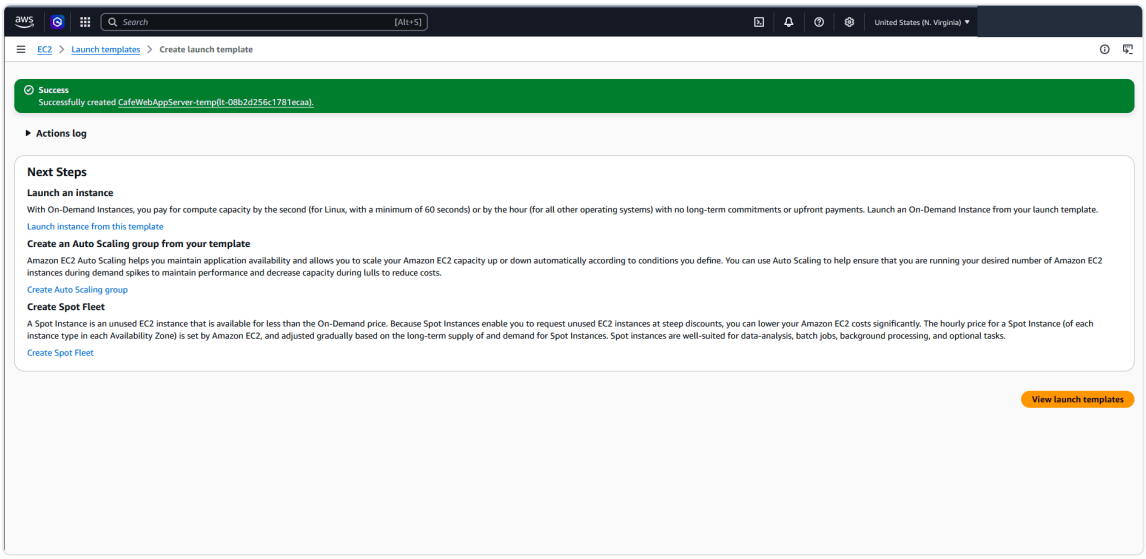
Task 3 - Launch template



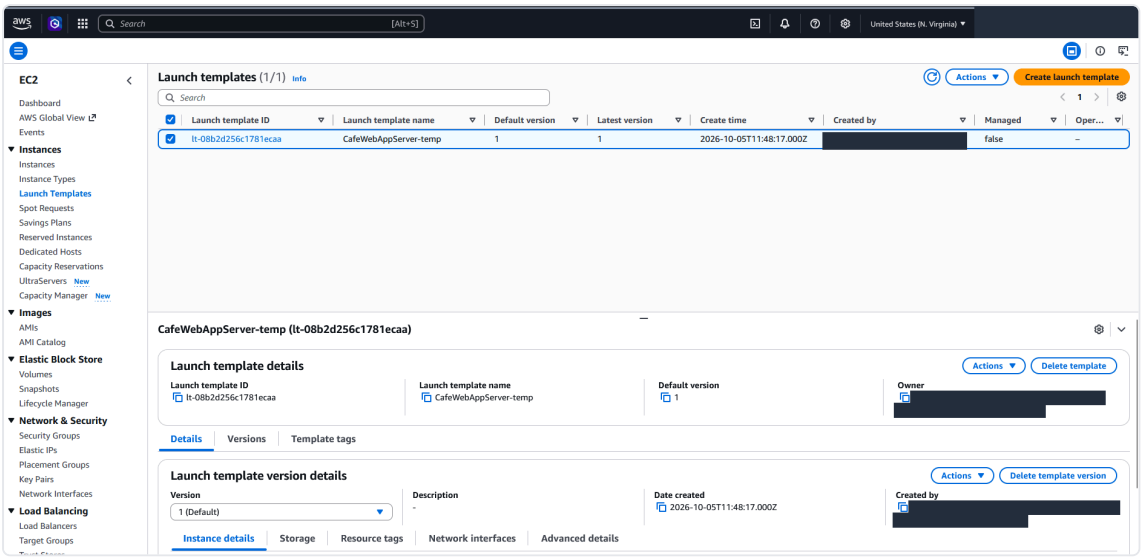
V5-E06 — The Cafe WebServer Image AMI is available (created from the café web server during lab setup).



V5-E07 — Launch template form: name, My AMIs, Cafe WebServer Image selected.

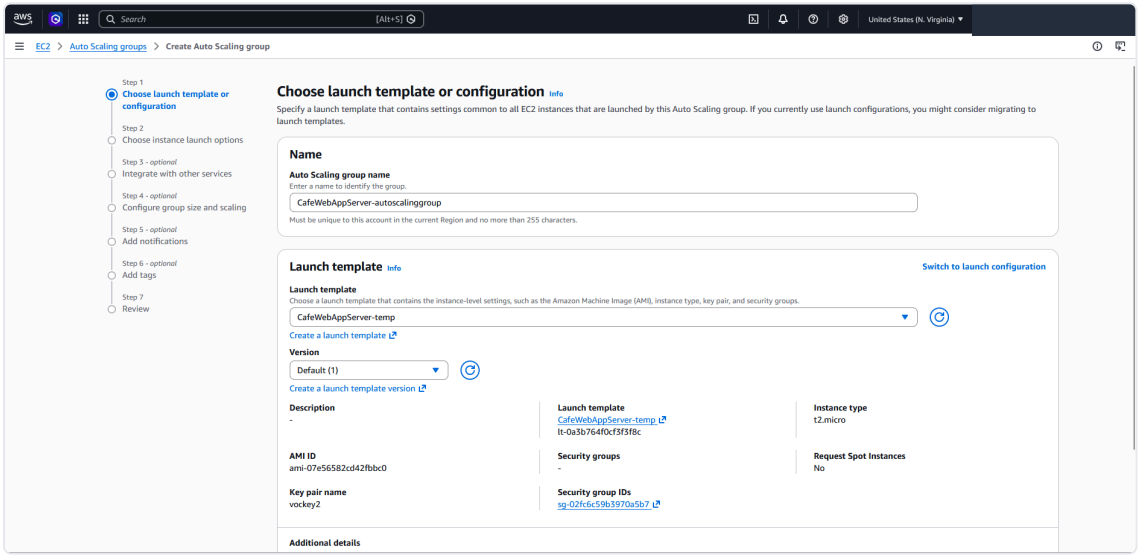


V5-E08 — Launch template CafeWebAppServer-temp created successfully.

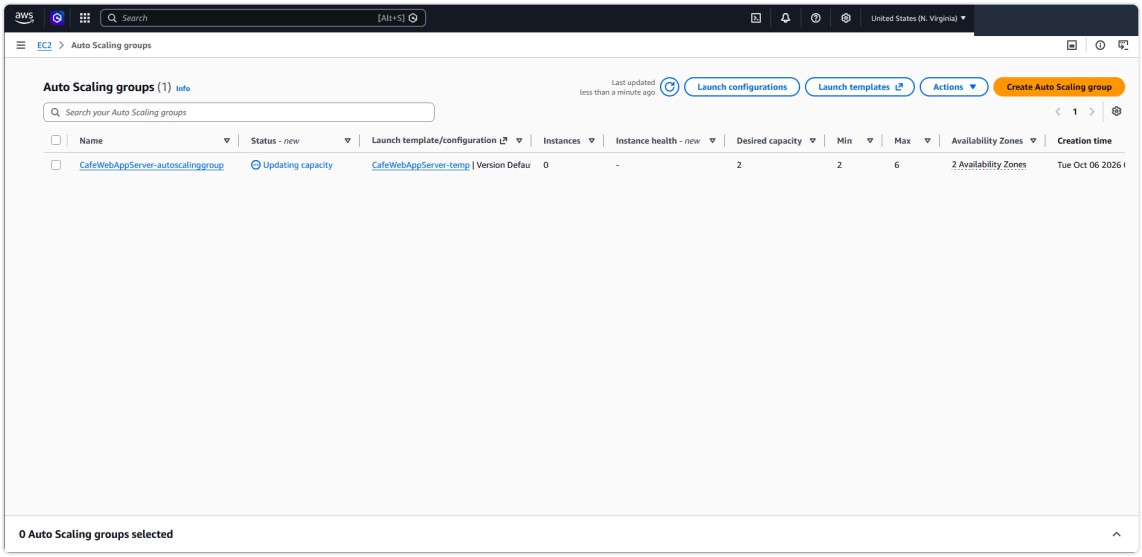


V5-E09 — Launch templates list and details (version 1, default).

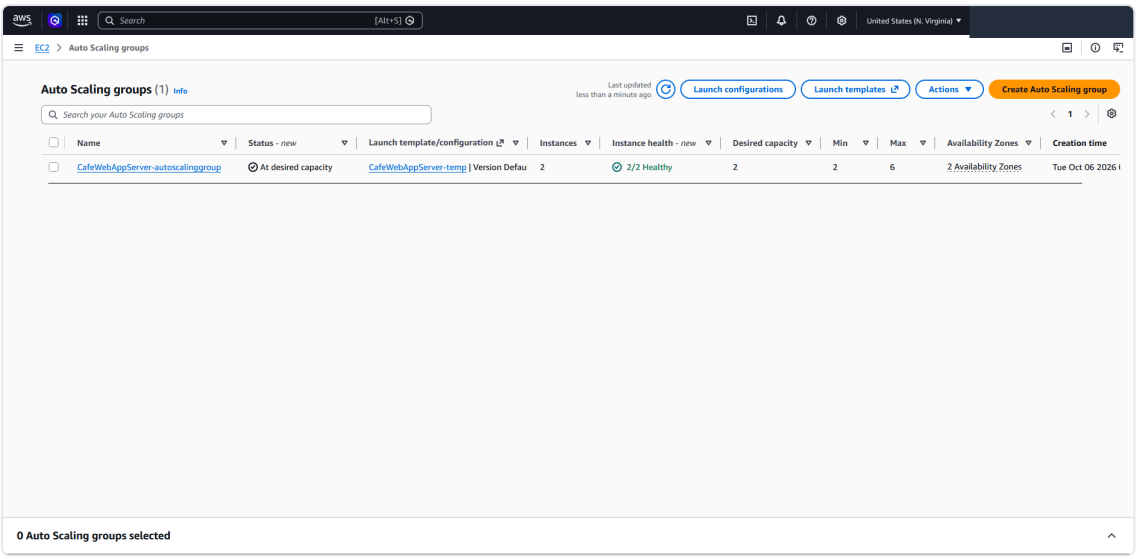
Task 4 - Auto Scaling group



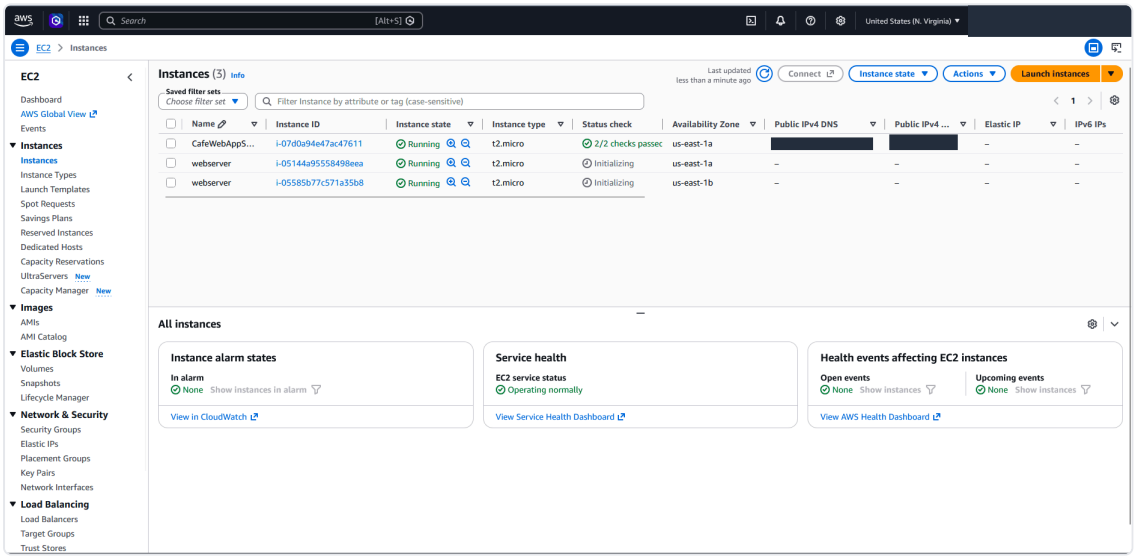
V5-E10 — Create Auto Scaling group: name and launch template selected (t2.micro, key pair, security group).



V5-E11 — Auto Scaling group created: desired 2, min 2, max 6, two Availability Zones, capacity updating.

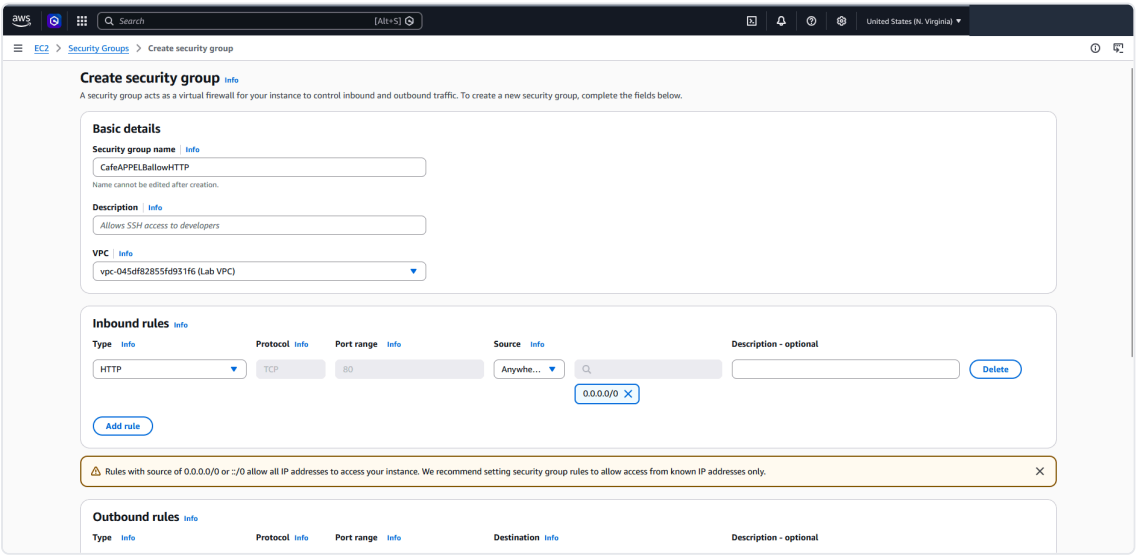


V5-E12 — Auto Scaling group at desired capacity: 2 instances, 2/2 healthy.

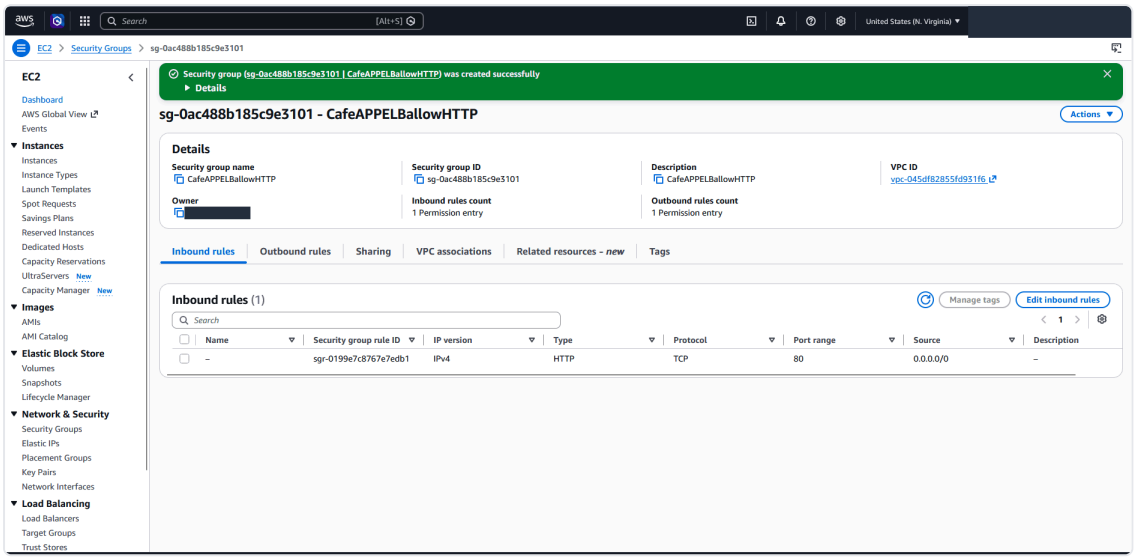


V5-E13 — EC2 console: two new "webserver" instances launching, one in us-east-1a and one in us-east-1b.

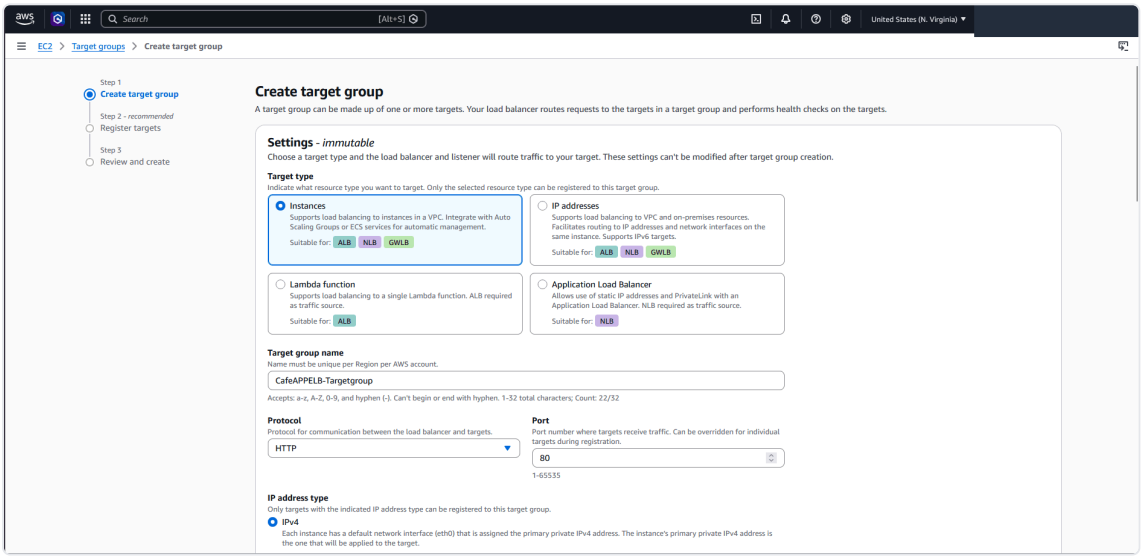
Task 5 - Load balancer



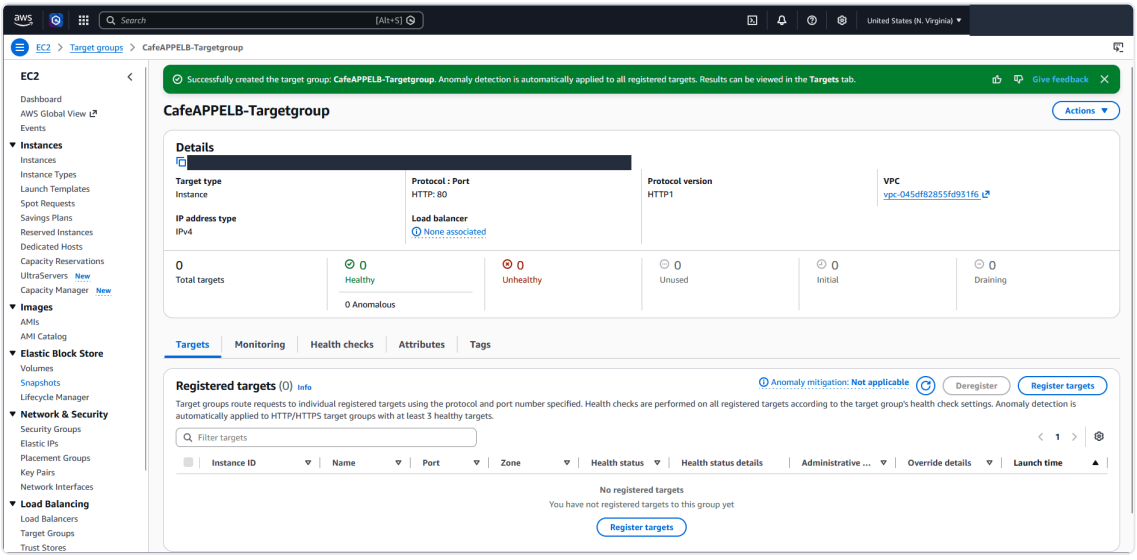
V5-E14 — Creating the new security group for the load balancer (inbound HTTP from anywhere).



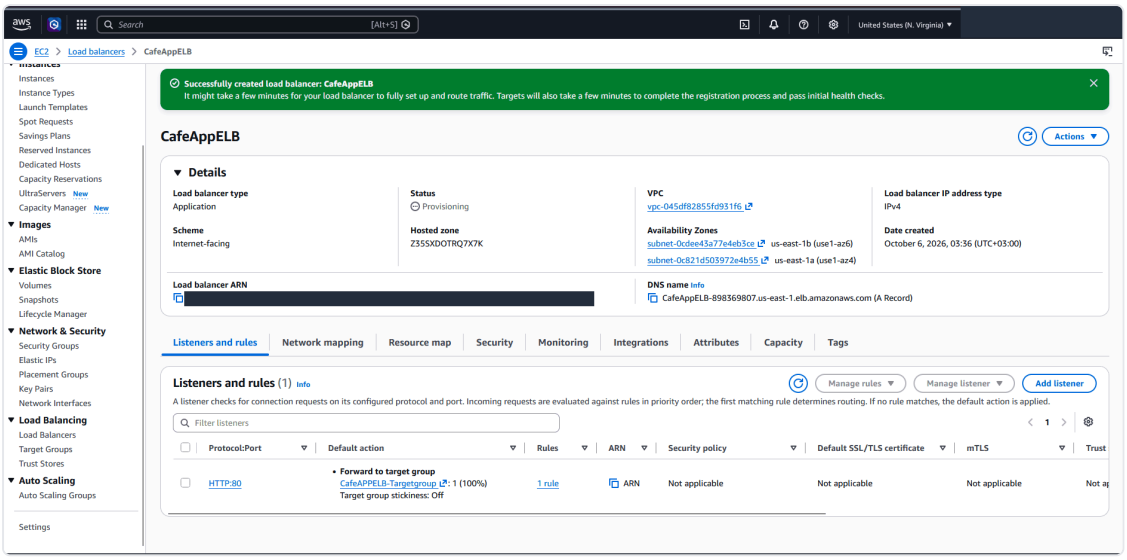
V5-E15 — Security group CafeAPPELBallowHTTP created: one inbound rule, HTTP port 80.



V5-E16 — Creating the target group: instance targets, HTTP port 80.

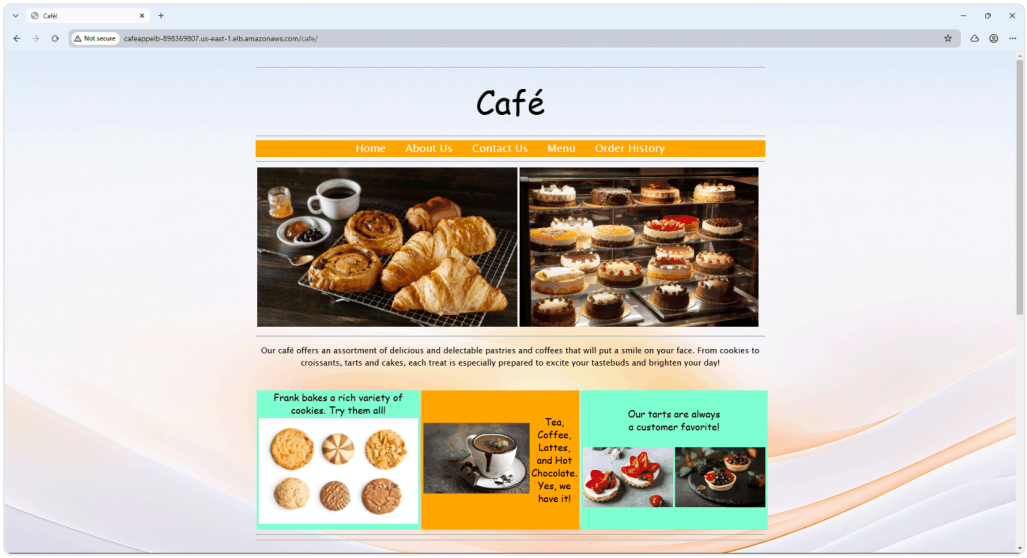


V5-E17 — Target group CafeAPPELB-Targetgroup created, no targets registered yet.

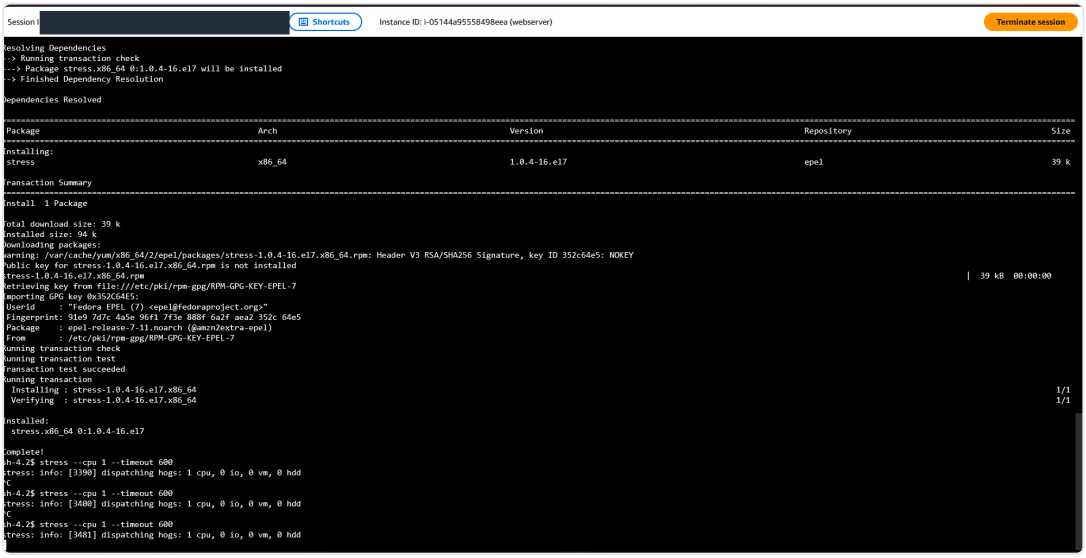


V5-E18 — Application Load Balancer CafeAppELB created, internet-facing, across two AZs, HTTP:80 listener forwarding to the target group.

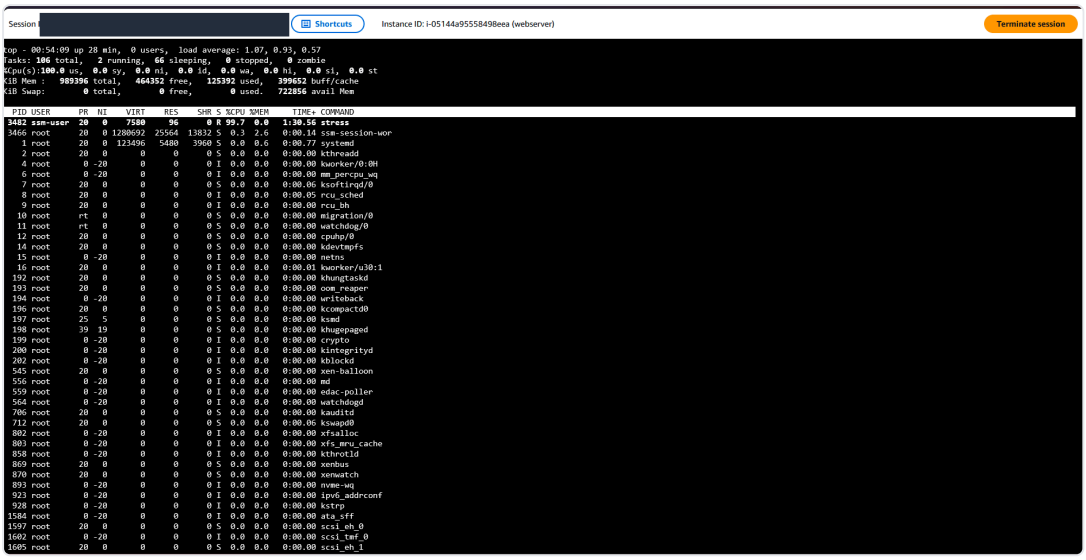
Task 6 - Testing load balancing and scaling



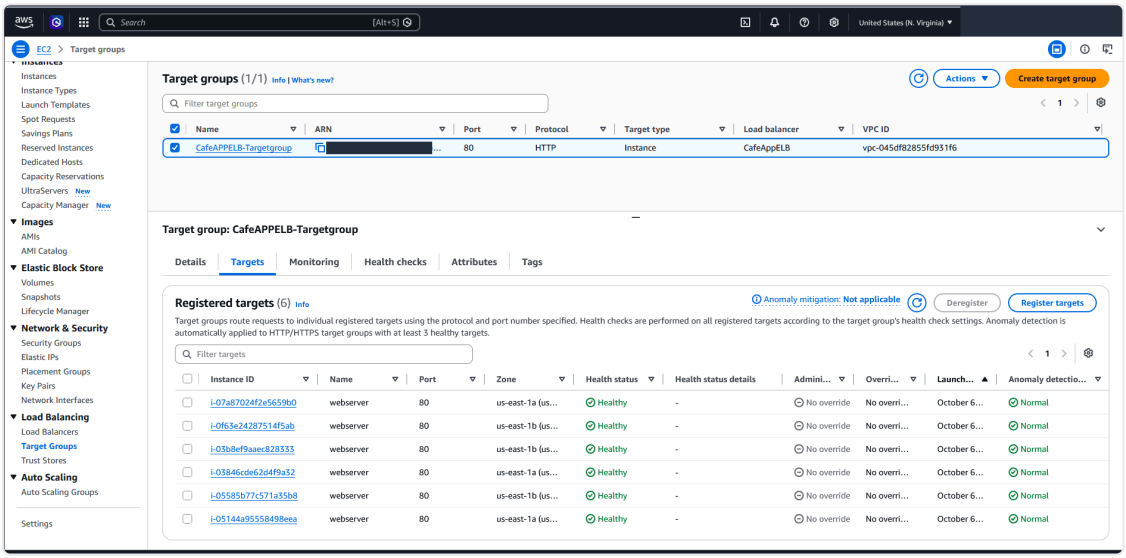
V5-E19 — Café web application loading through the load balancer DNS name at /cafe.



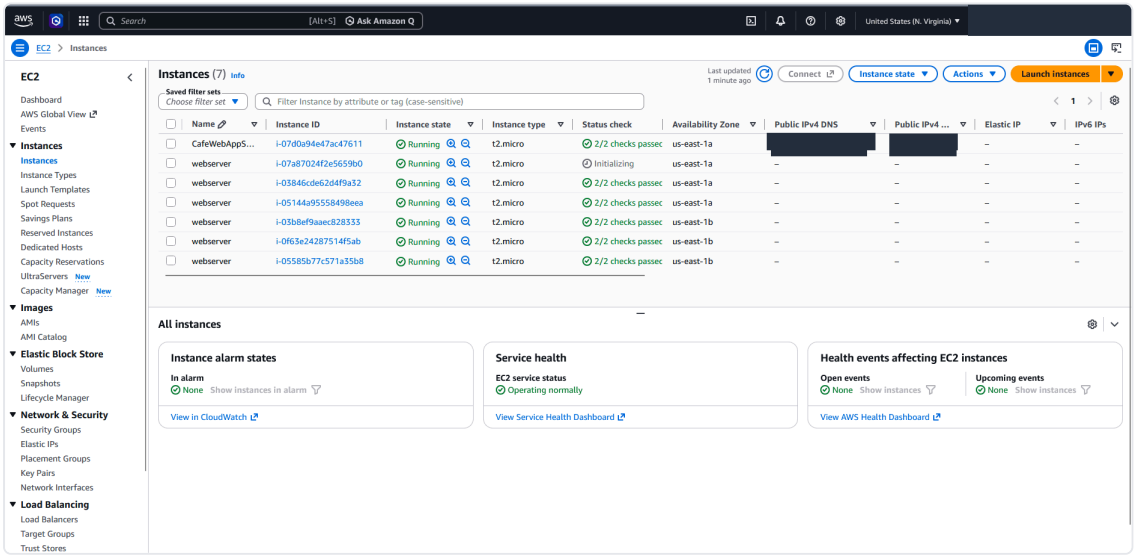
V5-E20 — Session Manager session on a web server: installing the stress tool and starting the CPU test.



V5-E21 — Top output during the test: the stress process is using about 100% of one CPU.



V5-E22 — Target group with six registered targets, all healthy.



V5-E23 — EC2 console after scale-out: six "webserver" instances running across us-east-1a and us-east-1b.

Outcome

The café website now runs behind an Application Load Balancer on an Auto Scaling group spread over two Availability Zones. Under a CPU stress test the group grew from 2 to 6 healthy web servers behind the load balancer without any manual action.

CHAPTER 7 · STAGE V6

V6 — Infrastructure Automation (CloudFormation + CodePipeline)

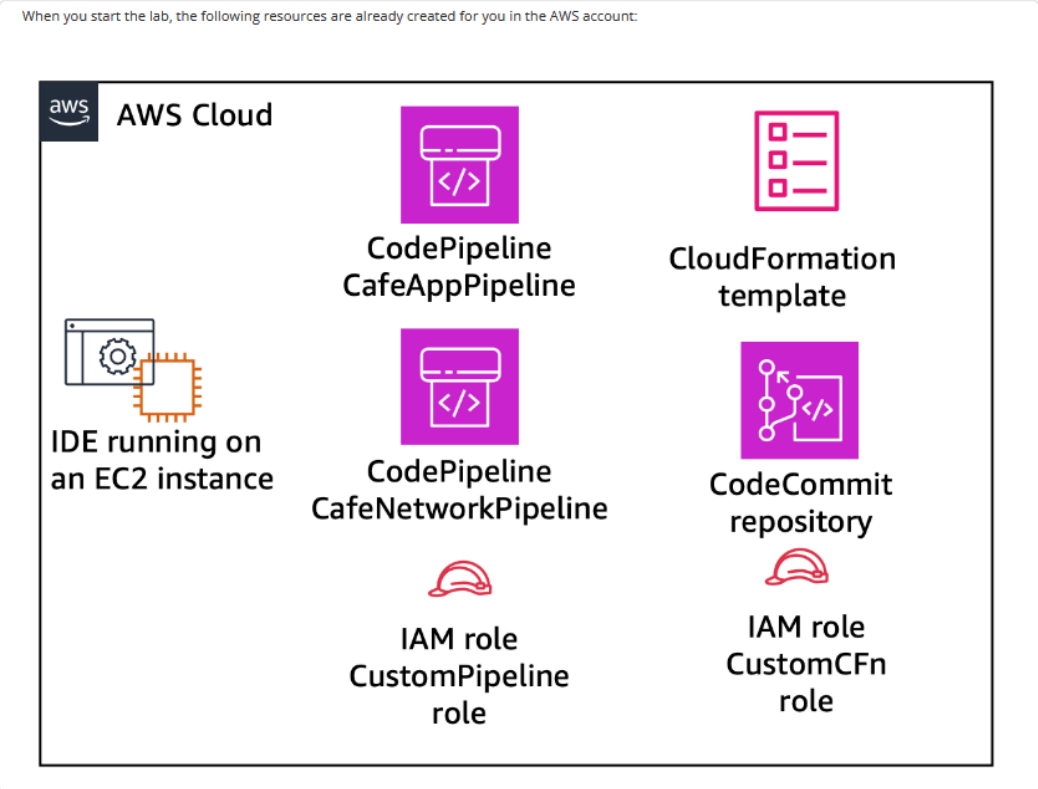
> Completed as part of AWS Academy Challenge Labs (training environment).

Business Requirement

The café wanted to open in new Regions and keep development and production identical. Building everything by hand in the console was slow and impossible to repeat reliably, so the infrastructure had to become code: version-controlled templates, deployed by a pipeline, and reusable in another Region.

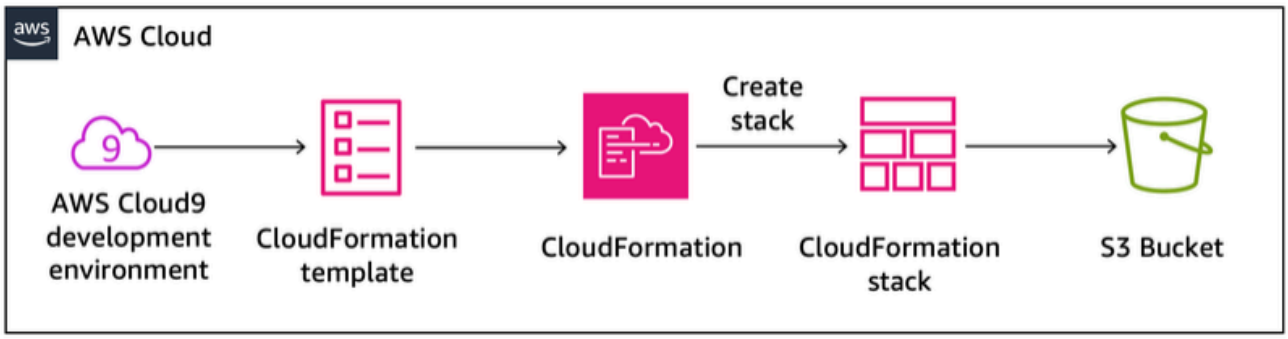
Start and Target Architecture

These two diagrams come from the AWS Academy lab material. They show where this stage starts and what it builds. They are shown here as reference; my own diagram of what I deployed follows in the next section.



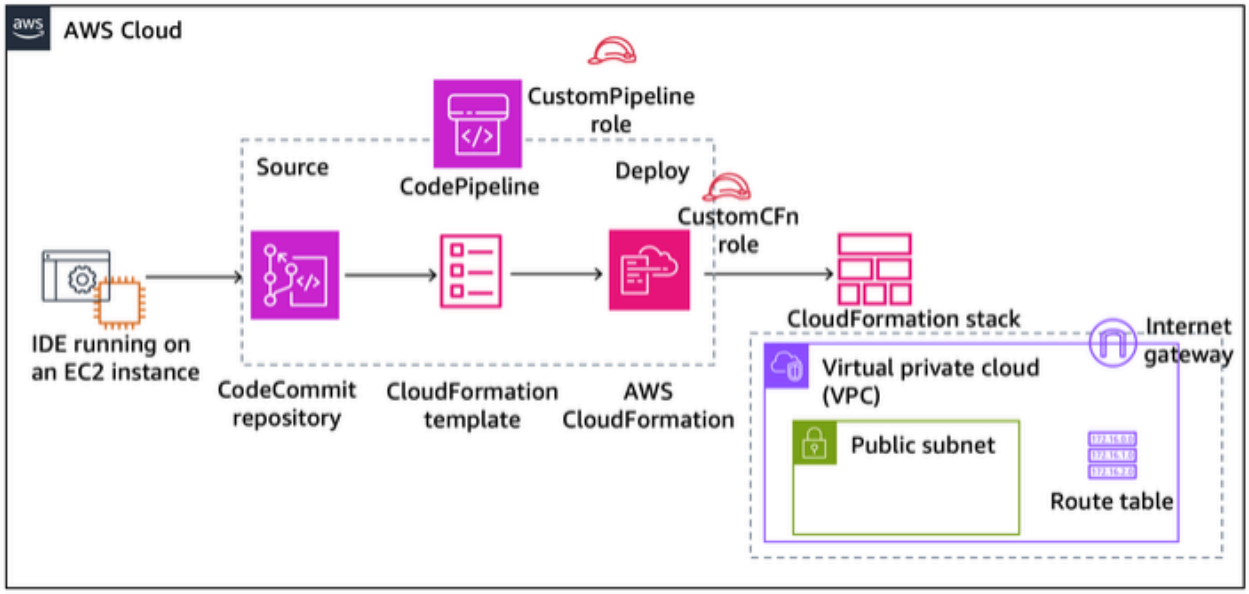
Starting resources. AWS Academy lab diagram: the resources available at the start of the lab.

The following diagram illustrates the actions that you just completed.



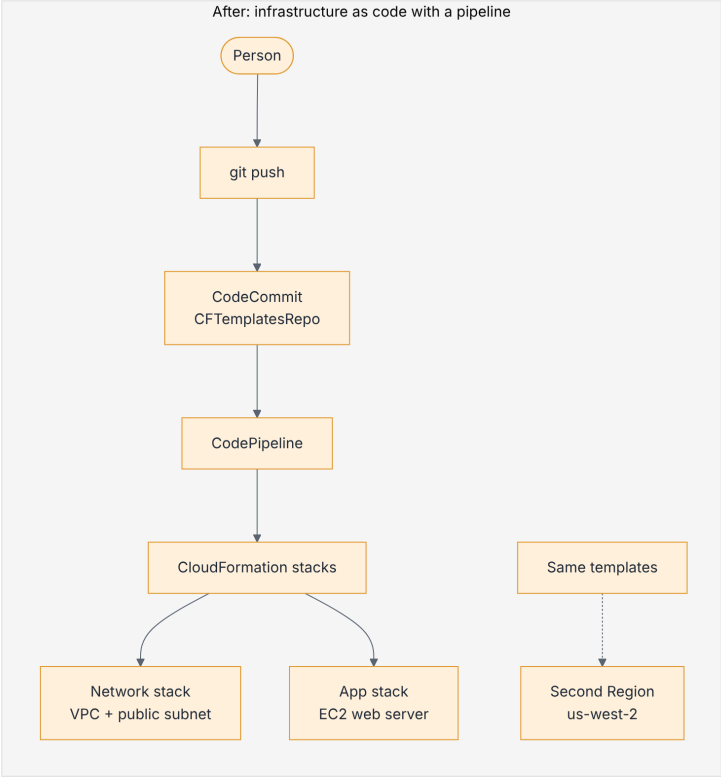
Creating a stack from a template. AWS Academy lab diagram: the flow used for the first CloudFormation stack.

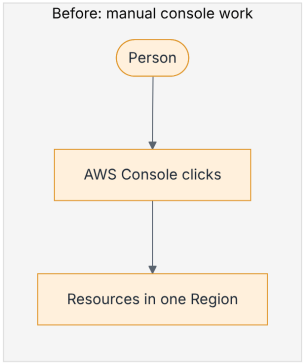
The following diagram illustrates how you will activate this pipeline and what the pipeline will do. It also shows some of the AWS account resources that the resulting CloudFormation stack will create or update.



Network pipeline flow. AWS Academy lab diagram: how a Git push drives the network pipeline.

Before → After





AWS Services Used

SERVICE	ROLE IN THIS STAGE
AWS CloudFormation	Defines the S3 bucket, the network layer and the application layer as templates and stacks
AWS CodeCommit	Git repository (`CFTemplatesRepo`) that stores and versions the templates
AWS CodePipeline	`CafeNetworkPipeline` and `CafeAppPipeline` deploy a stack whenever a template is pushed
Amazon EC2 and VPC	Resources created by the templates (Cafe VPC, public subnet, security group, web server)
Amazon S3	First practice stack (static website bucket) and storage for the template used in the second Region

SERVICE	ROLE IN THIS STAGE
AWS CLI and Git	Creating and validating stacks, and pushing templates from the lab IDE

Implementation Highlights

- **First stack from scratch.** I wrote `S3.yaml` and created the `CreateBucket` stack with the CLI, then updated it with a retain policy, static website configuration and a website URL output (E04-E16).
- **Templates under version control.** I cloned `CFTemplatesRepo` from CodeCommit into the IDE and used Git for every change (E17-E18).
- **Network through a pipeline.** I created `cafe-network.yaml`, committed and pushed it, and `CafeNetworkPipeline` built the `update-cafe-network` stack with the Cafe VPC and public subnet (E19-E28).
- **Outputs for reuse.** I added exported outputs (subnet ID and VPC ID) to the network template and pushed again, so the pipeline updated the stack (E29-E33).
- **Application through a pipeline.** `cafe-app.yaml` adds an instance-type parameter and a `CafeInstance` web server that imports the subnet from the network stack. After the push, `CafeAppPipeline` created the `update-cafe-app` stack and the café site came up (E34-E41).
- **Second Region.** I created the network stack in us-west-2 with the CLI, created the `cafe-oregon` key pair, uploaded the app template to S3 and created `cafe-app-oregon` from the console with `t3.micro`. The same unmodified template produced a working café in Oregon (E42-E55).

Challenges & Decisions

No failures are recorded for this stage in my notes or screenshots; the decisions that shaped it were:

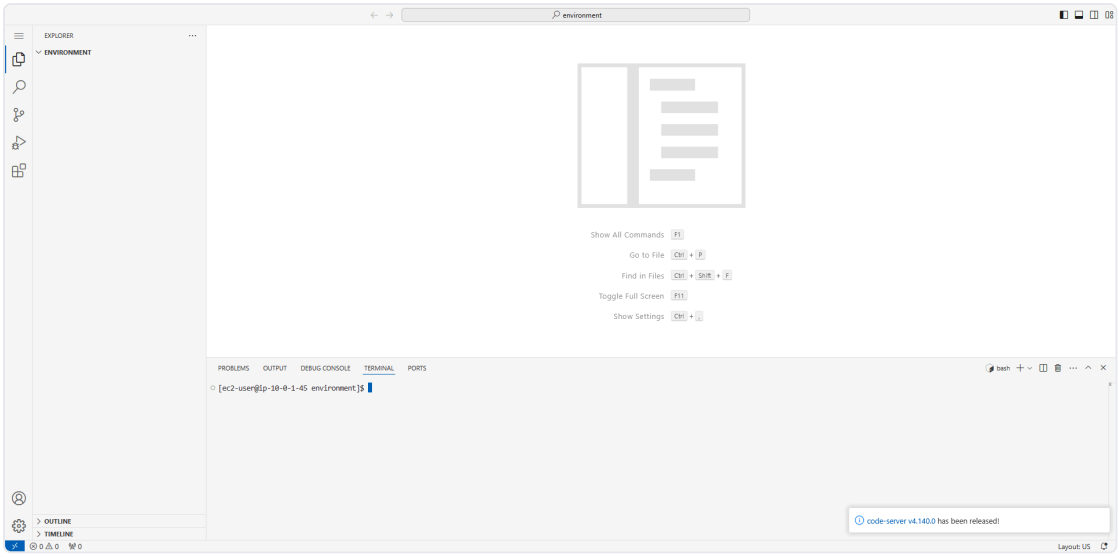
- **Pipeline instead of manual runs** for the network and application stacks, so every change is a reviewed Git commit.
- **Two stacks (network and app)** linked through exported outputs, so the application template never hard-codes IDs.
- **Instance type as a parameter and a Region map for the key pair**, which let the same template run in another Region with different settings (`t3.micro` and `cafe-oregon` in Oregon).

Validation & Evidence

All 55 screenshots of this stage, in lab order. Account IDs, ARNs, e-mail addresses and personal details were masked.

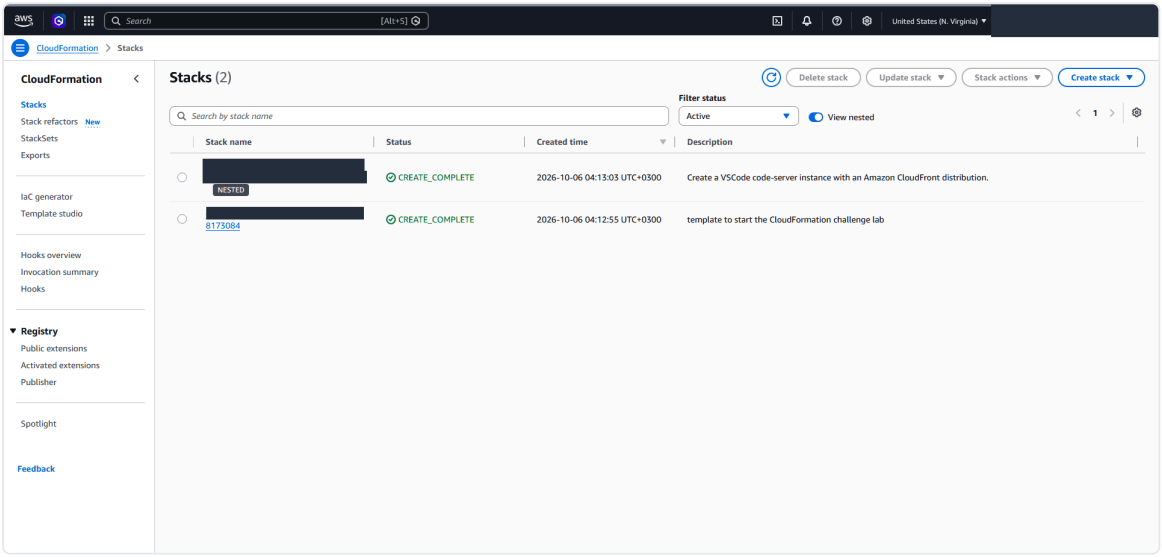
> Evidence note: the us-west-2 screenshots come from a different account ID than the us-east-1 ones. Account details and public IPs were masked.

Task 1 - IDE

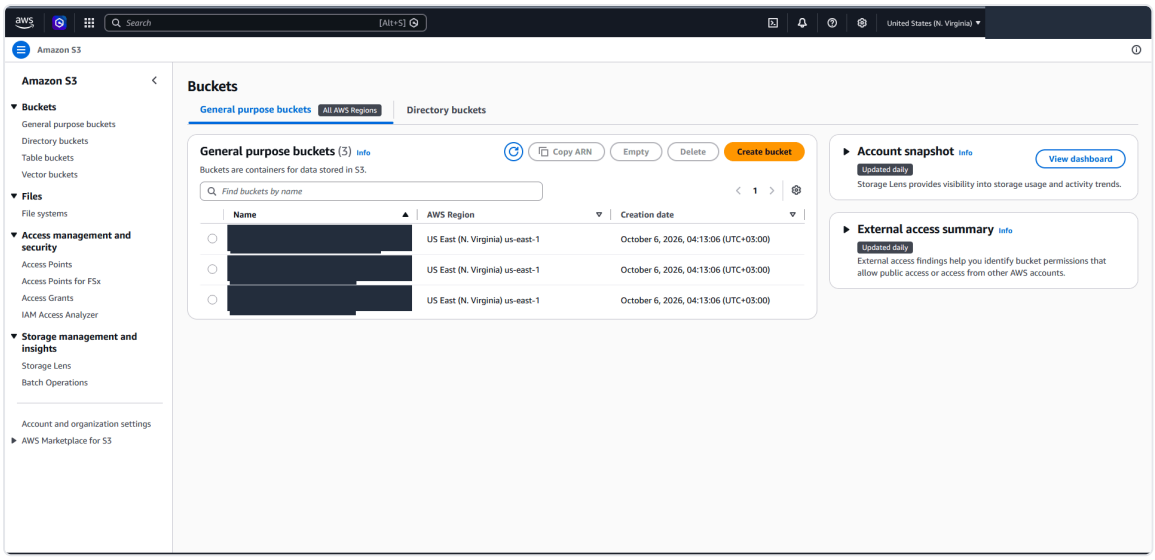


V6-E01 — Empty VS Code IDE (code-server) on the lab EC2 instance, terminal ready.

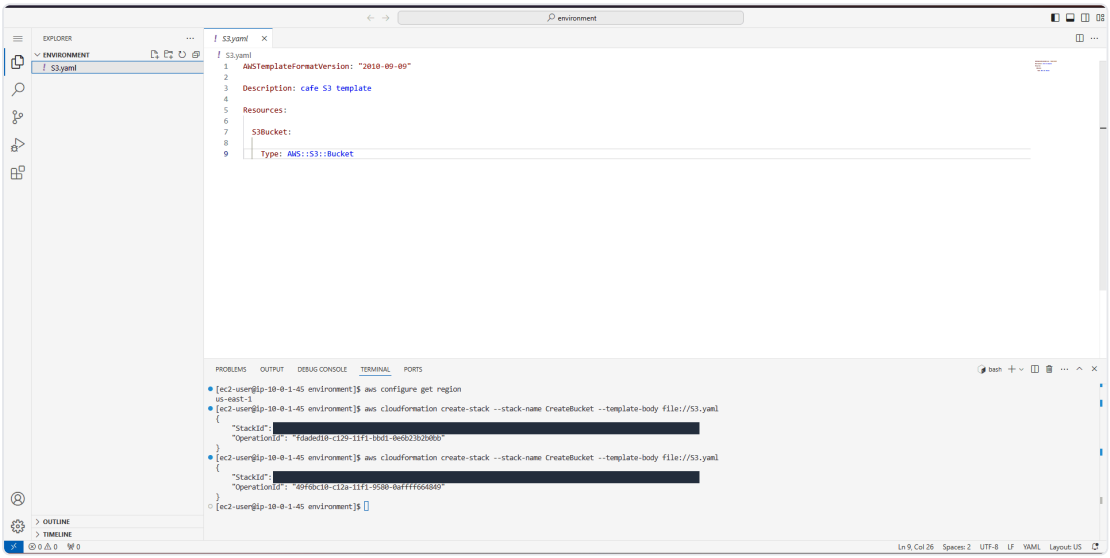
Task 2 - First stack from scratch (S3 bucket)



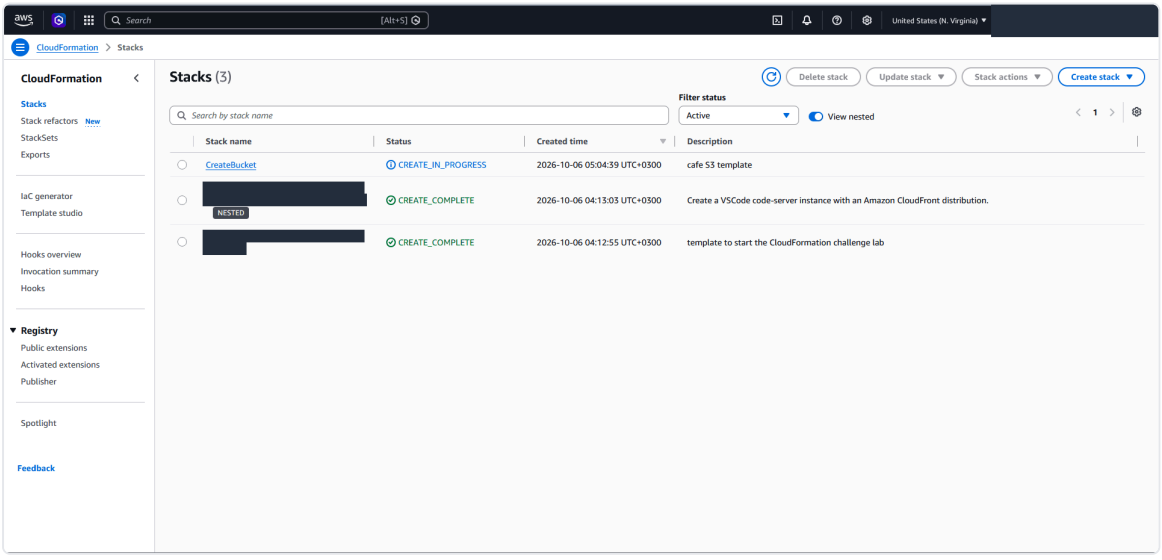
V6-E02 — Initial CloudFormation stacks created by the lab setup.



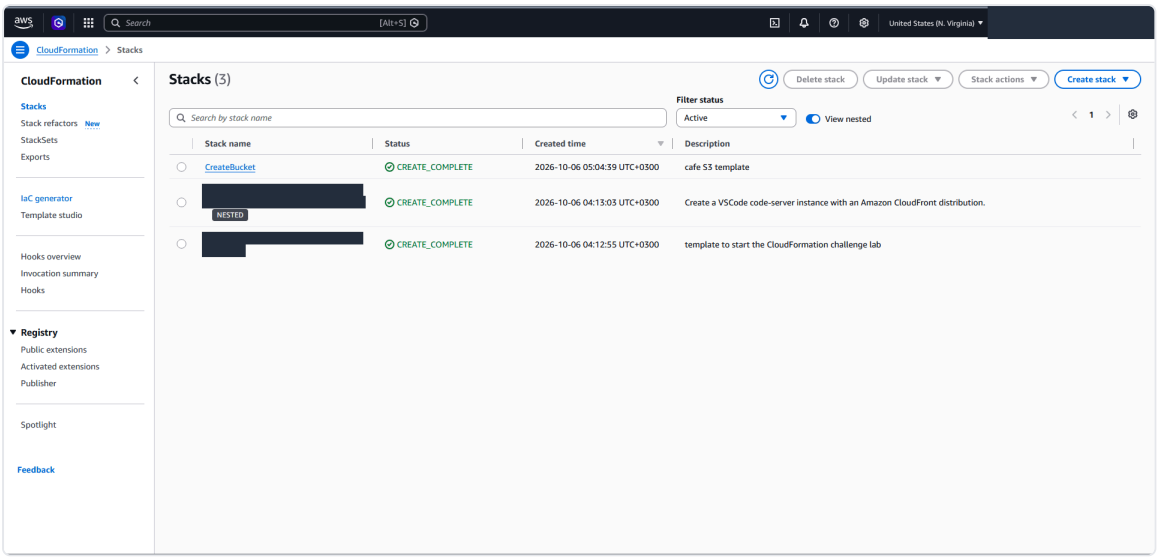
V6-E03 — Initial S3 buckets created by the lab setup.



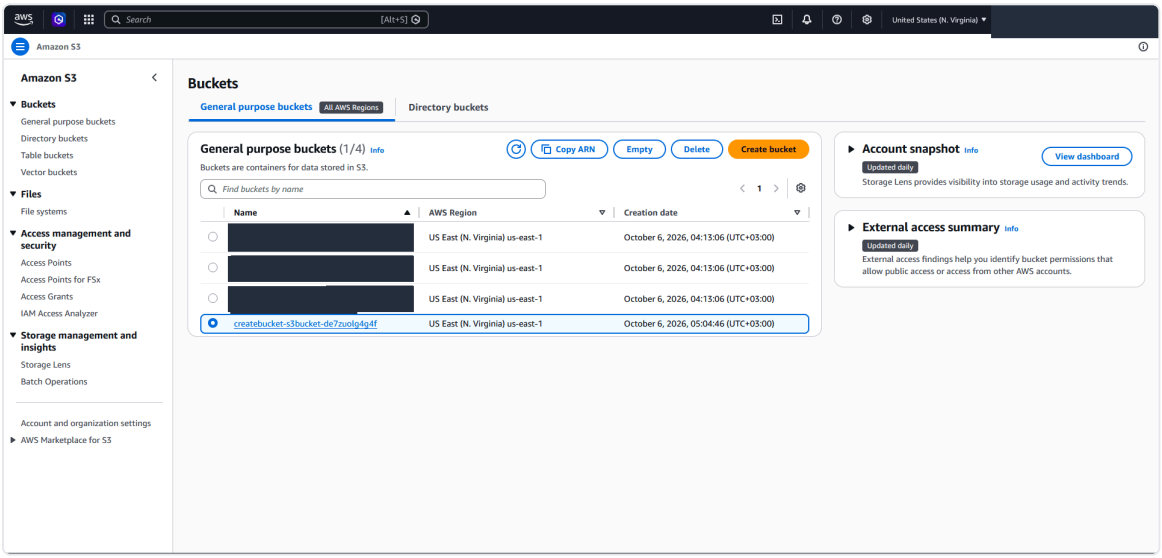
V6-E04 — S3.yaml (bucket only) and the first create-stack CLI runs.



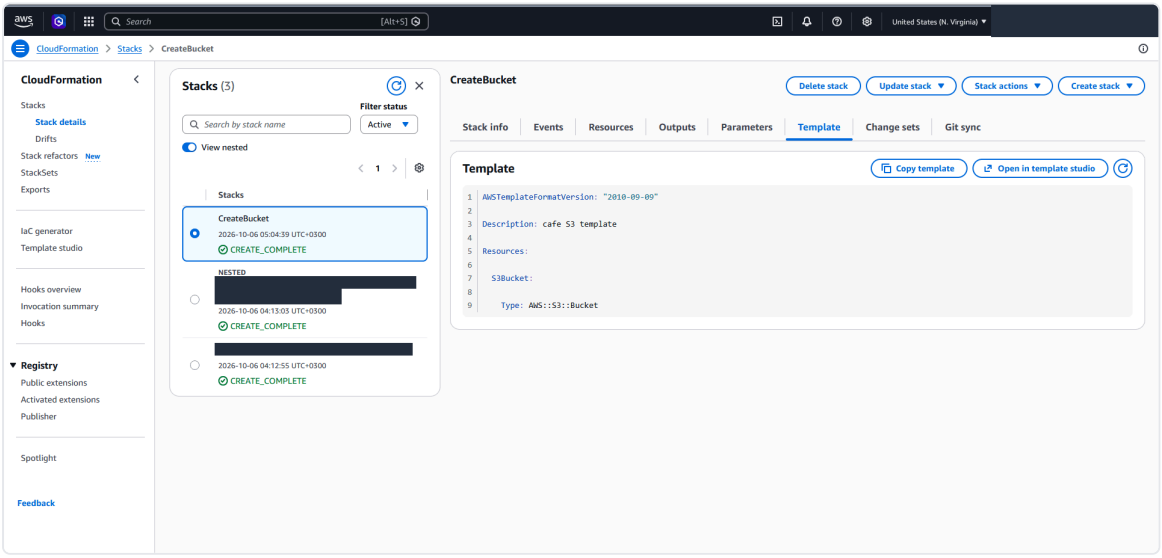
V6-E05 — CreateBucket stack creation in progress.



V6-E06 — CreateBucket stack CREATE_COMPLETE.

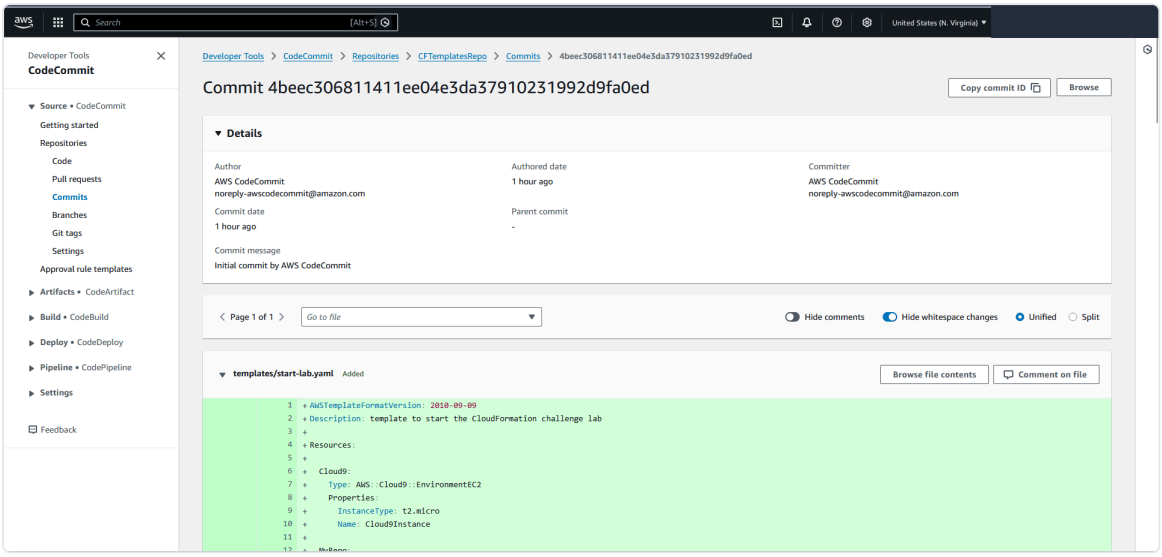


V6-E07 — S3 bucket created by the stack.

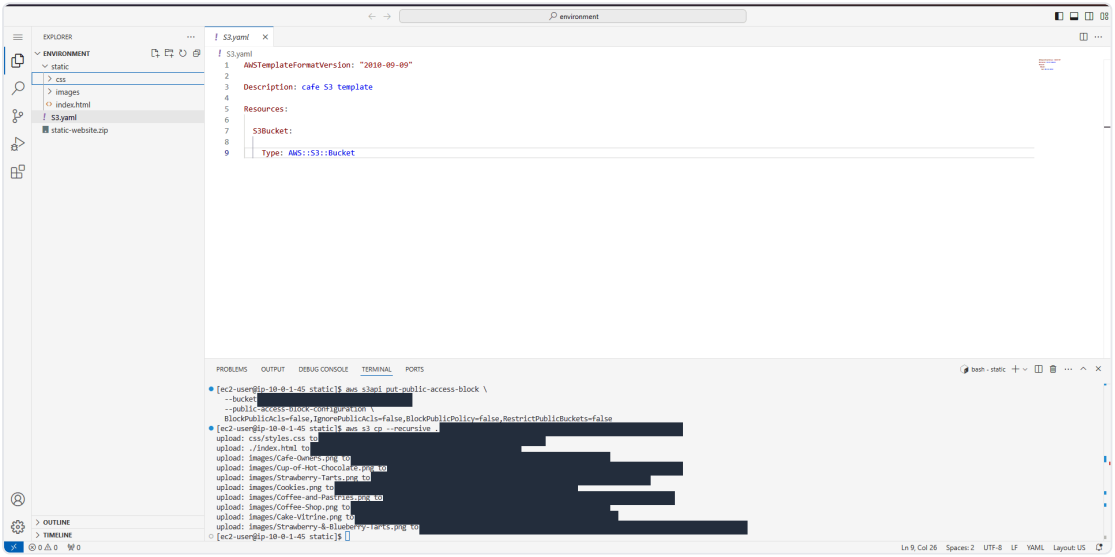


V6-E08 — Template tab of the CreateBucket stack.

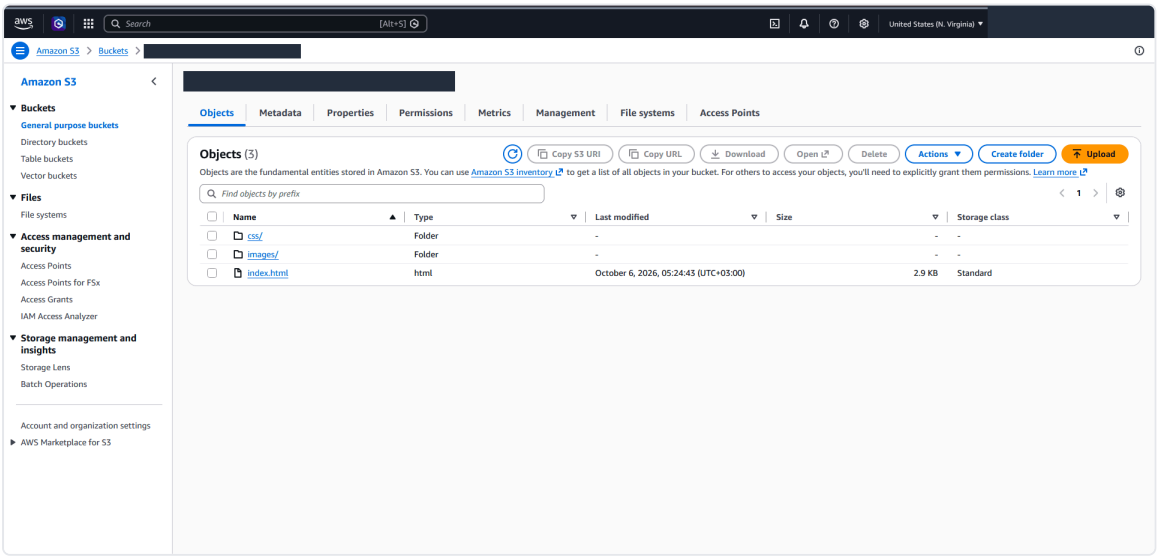
Task 3 - Website hosting and stack update



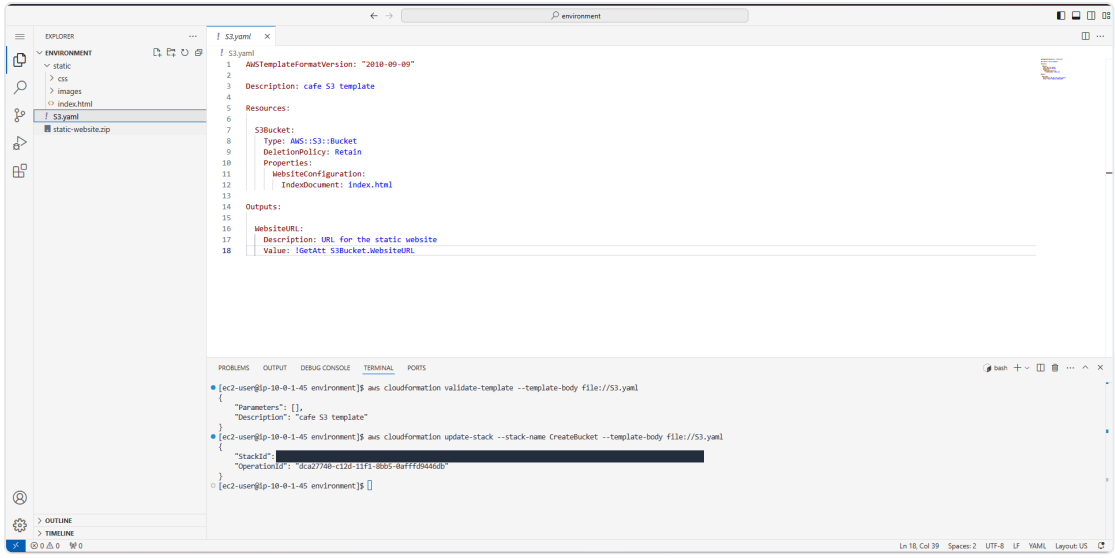
V6-E09 — CodeCommit initial commit of the CFTemplatesRepo repository.



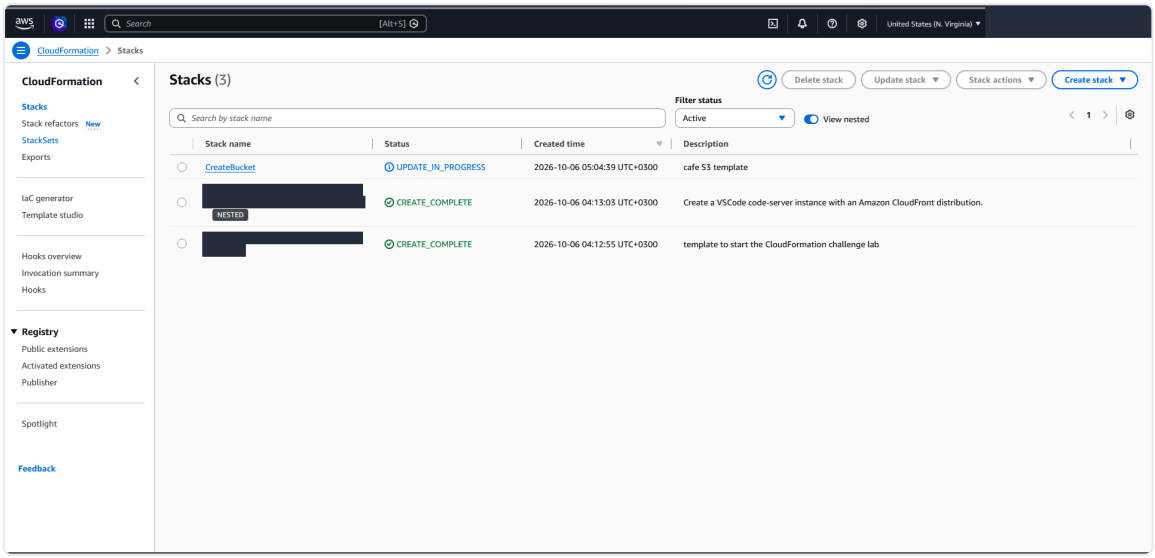
V6-E10 — Website files downloaded and copied to the bucket with the AWS CLI.



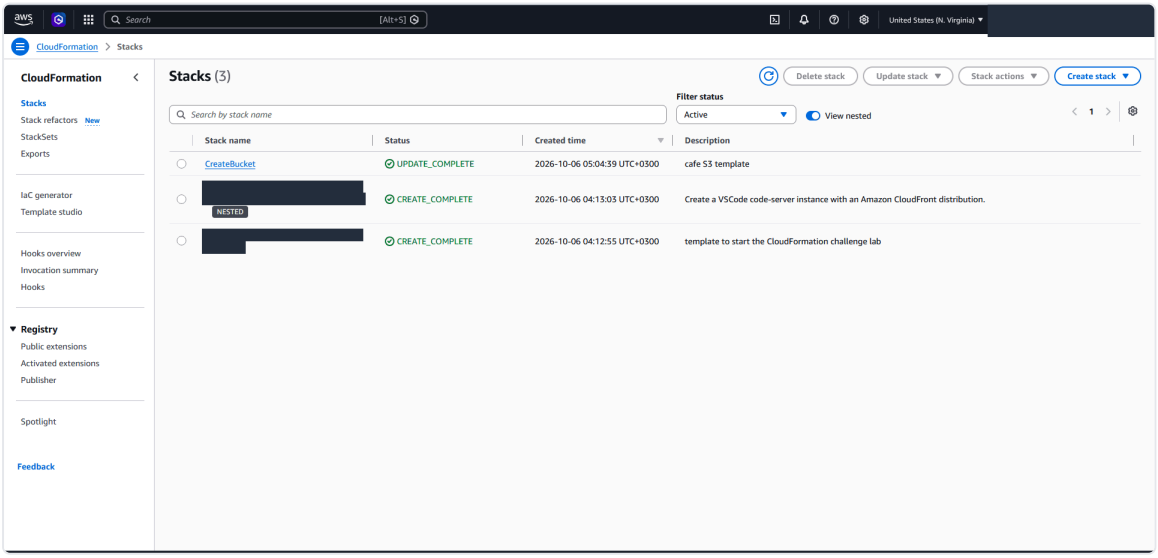
V6-E11 — Website objects (index.html, css/, images/) in the bucket.



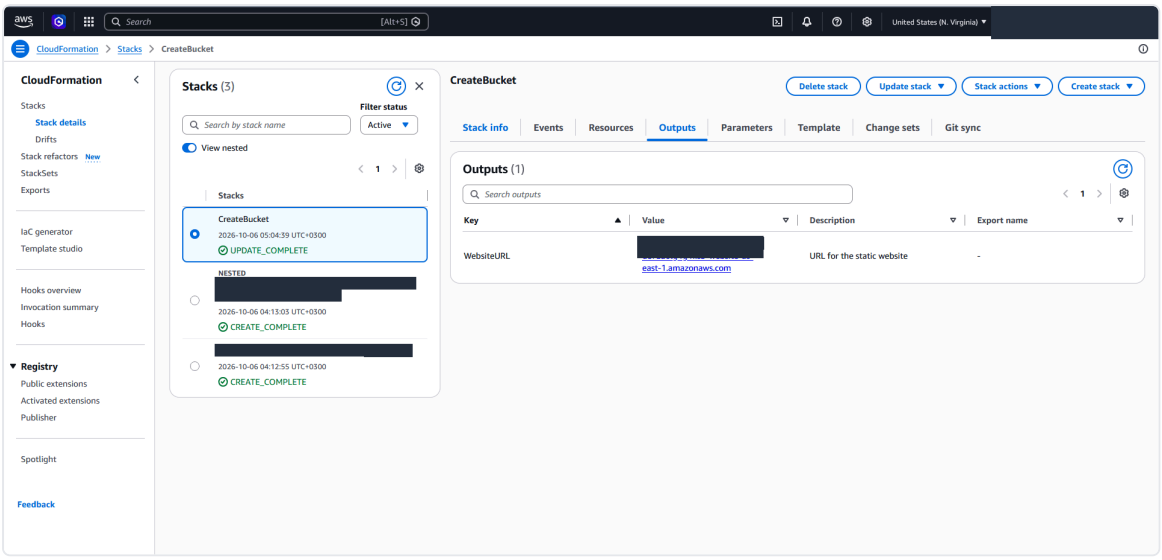
V6-E12 — S3.yaml updated with retain policy, website configuration and URL output; template validated and stack update started.



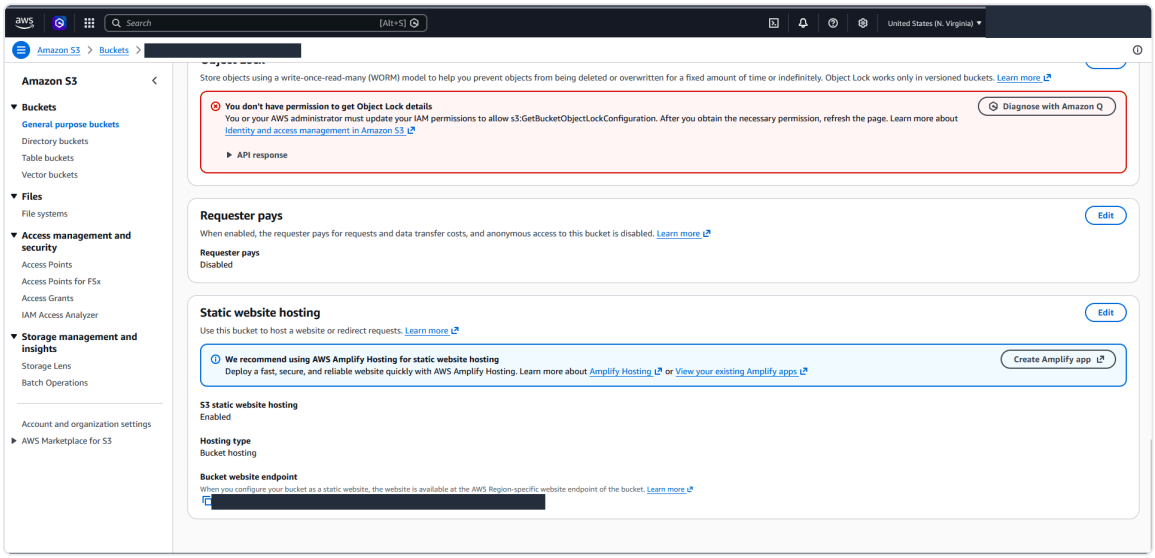
V6-E13 — CreateBucket stack UPDATE_IN_PROGRESS.



V6-E14 — CreateBucket stack UPDATE_COMPLETE.

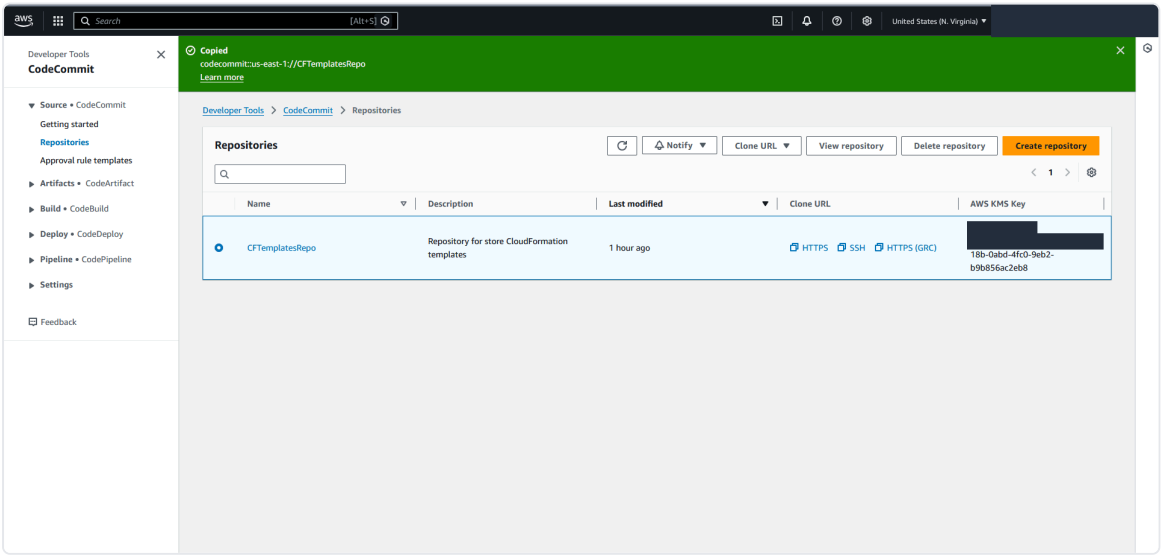


V6-E15 — Stack Outputs tab showing the WebsiteURL.

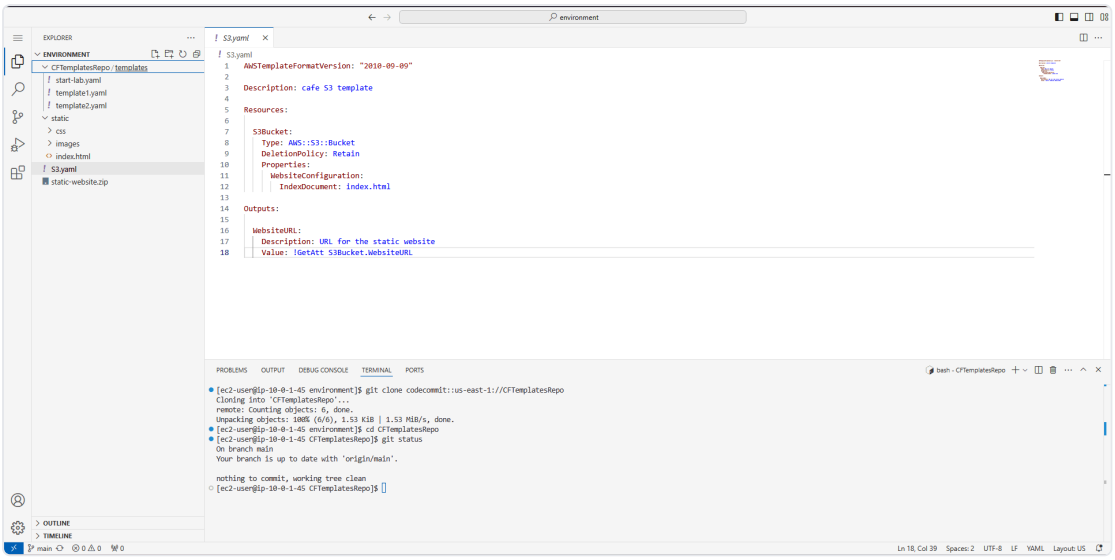


V6-E16 — Static website hosting enabled on the bucket.

Task 4 - CodeCommit repository

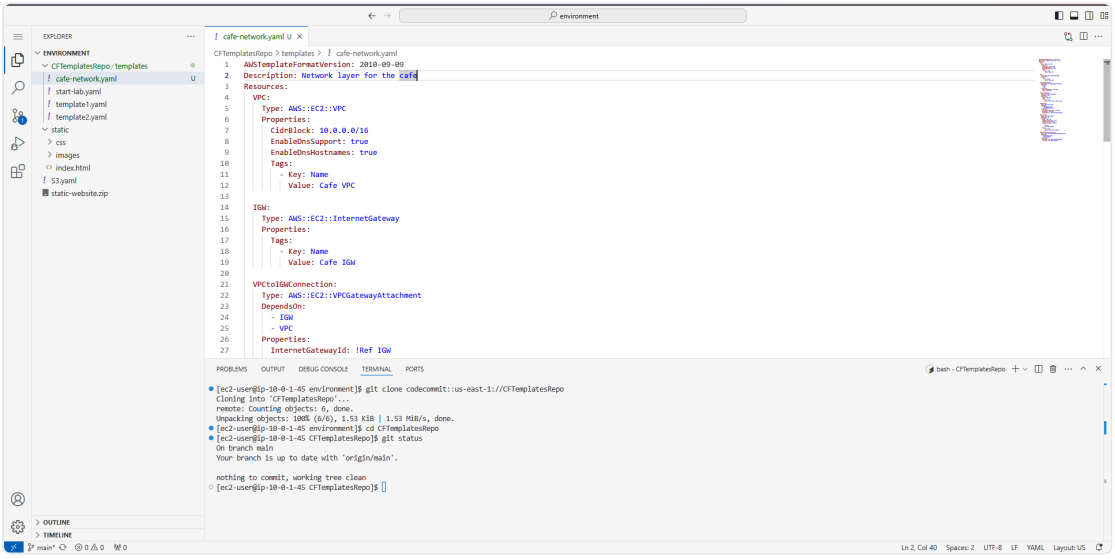


V6-E17 — CFTemplatesRepo repository in CodeCommit with the clone URL copied.

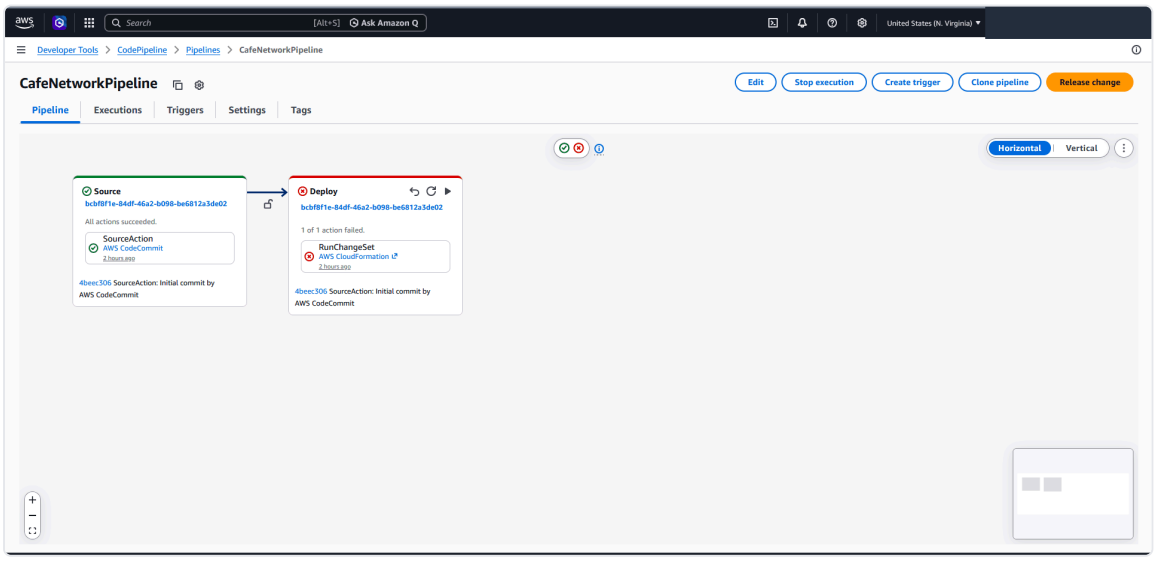


V6-E18 — Git clone and git status: working tree clean.

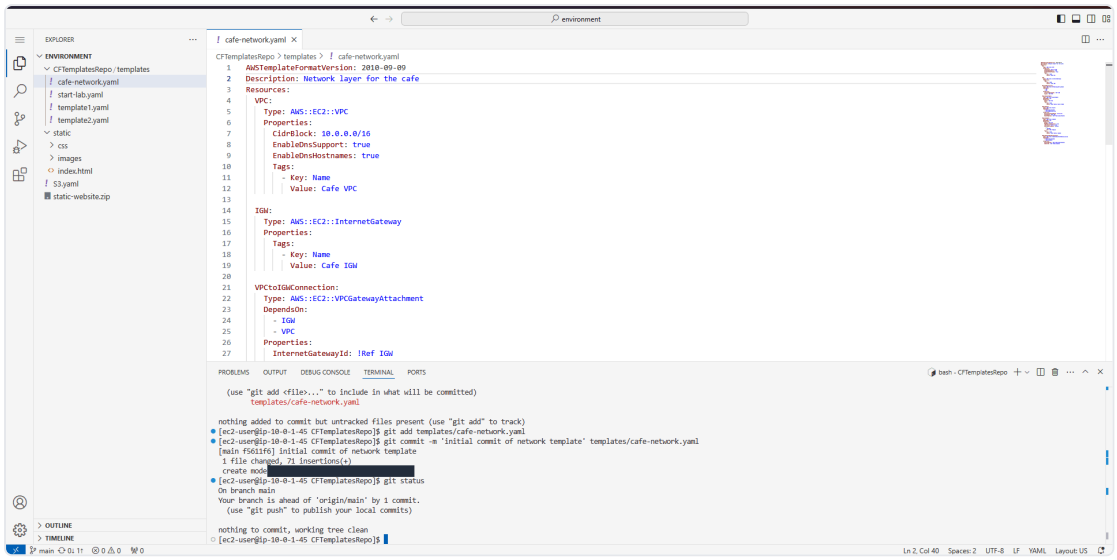
Task 5 - Network layer through the pipeline



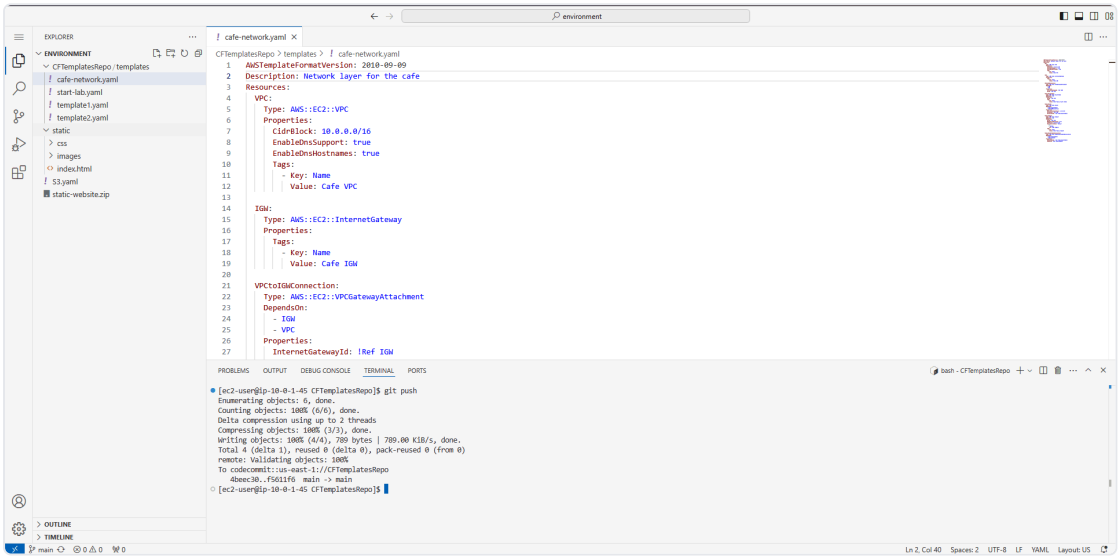
V6-E19 — Cafe-network.yaml created from the template with the new description.



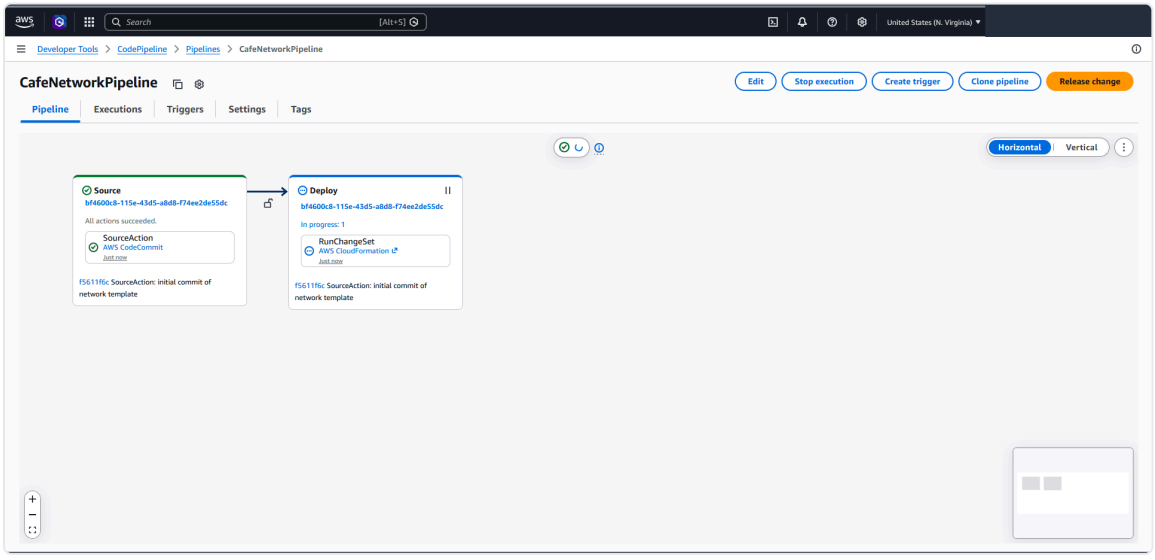
V6-E20 — CafeNetworkPipeline before the first push (Deploy shows Failed, as expected before a template exists).



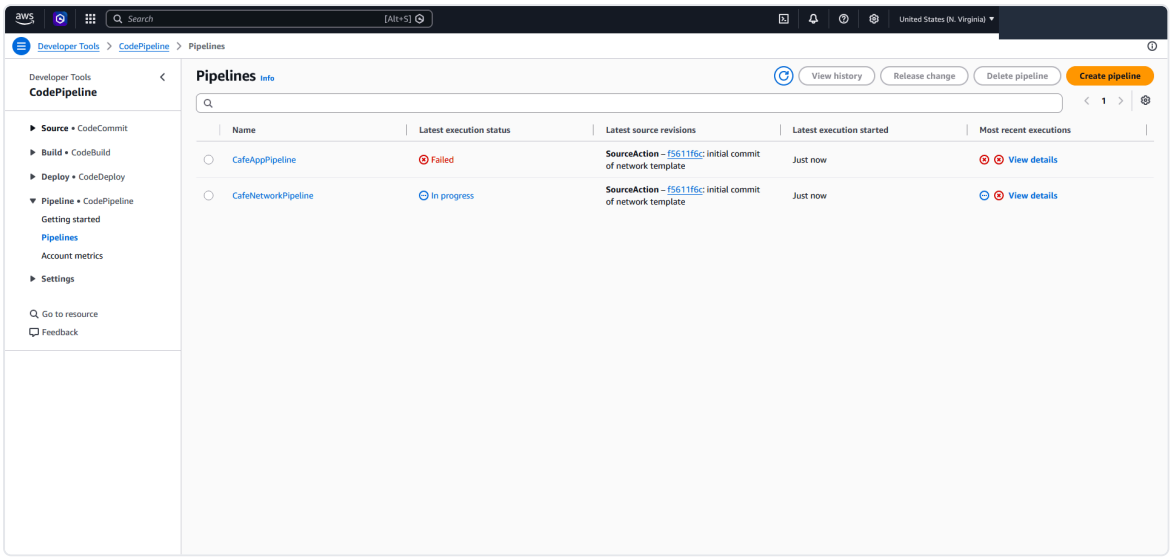
V6-E21 — Git add and git commit of the network template.



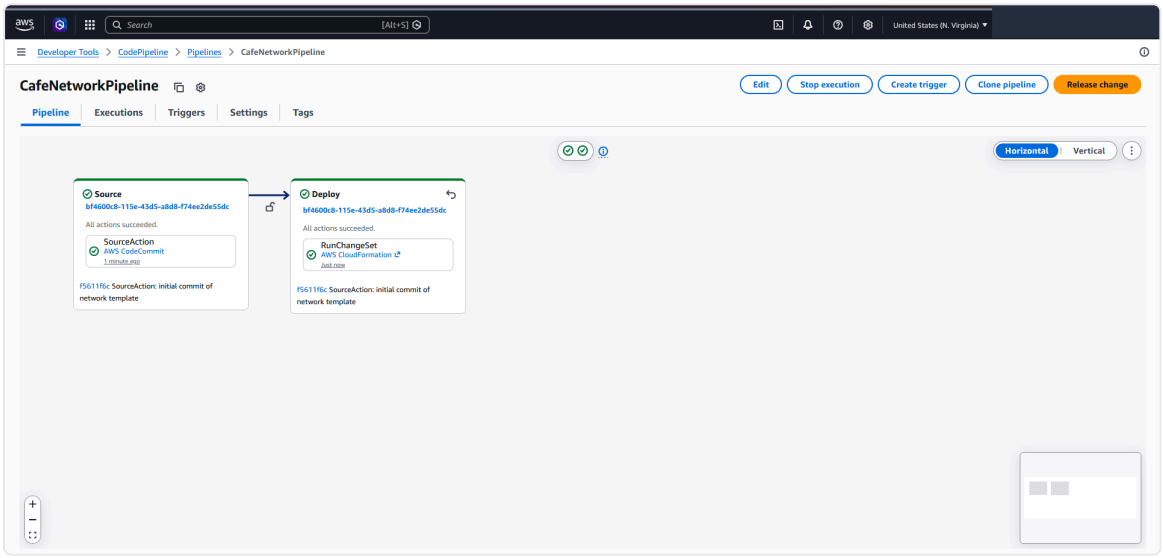
V6-E22 — Git push of the network template to CodeCommit.



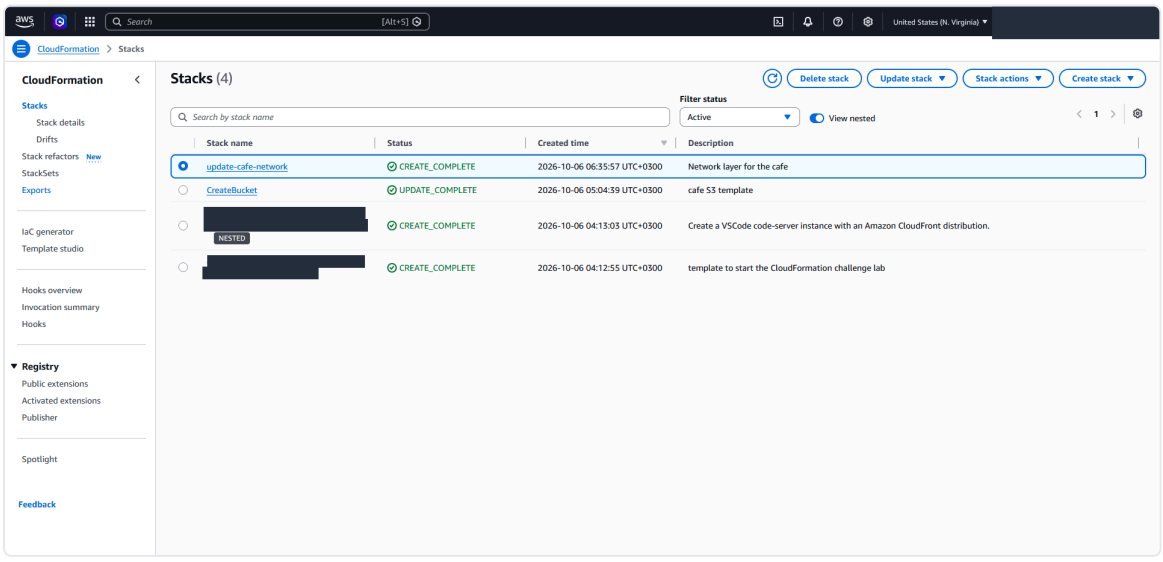
V6-E23 — CafeNetworkPipeline triggered: Deploy stage in progress.



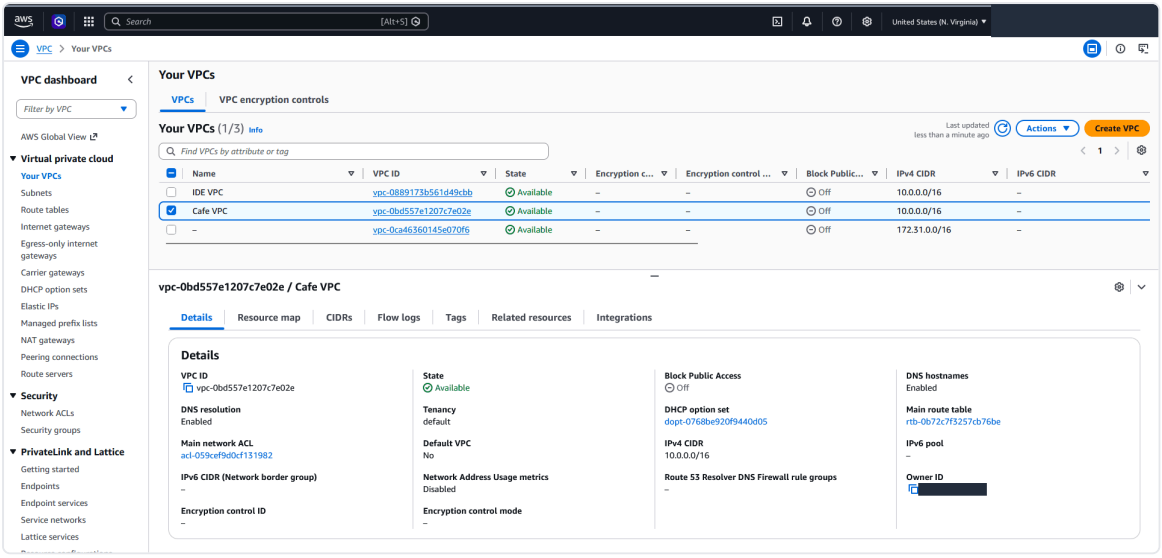
V6-E24 — Pipelines list: CafeNetworkPipeline In progress, CafeAppPipeline Failed (no template yet).



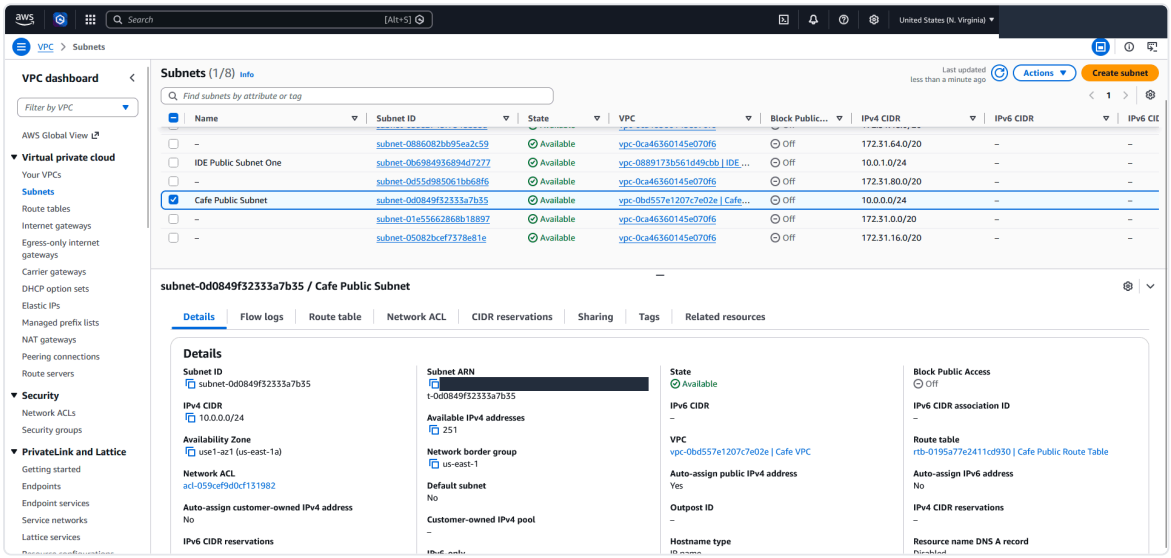
V6-E25 — CafeNetworkPipeline Deploy stage finishing.



V6-E26 — Update-cafe-network stack CREATE_COMPLETE.

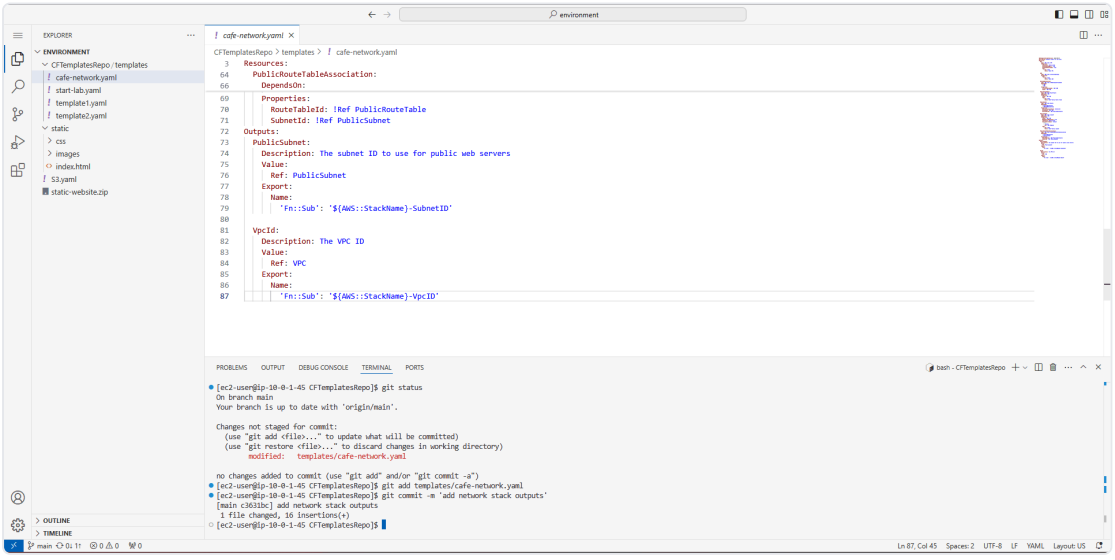


V6-E27 — Cafe VPC created by the pipeline.

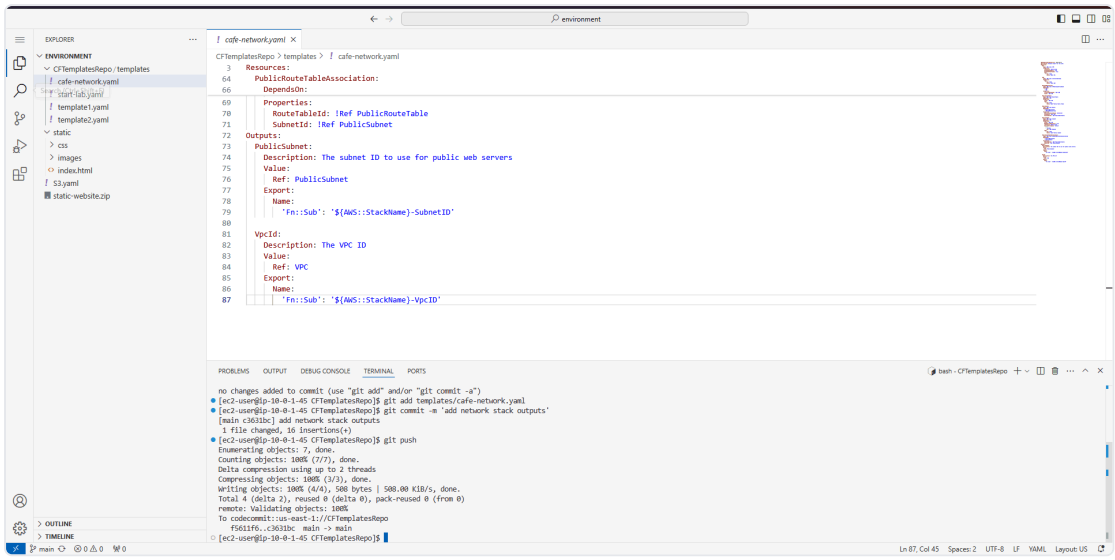


V6-E28 — Cafe Public Subnet created by the pipeline.

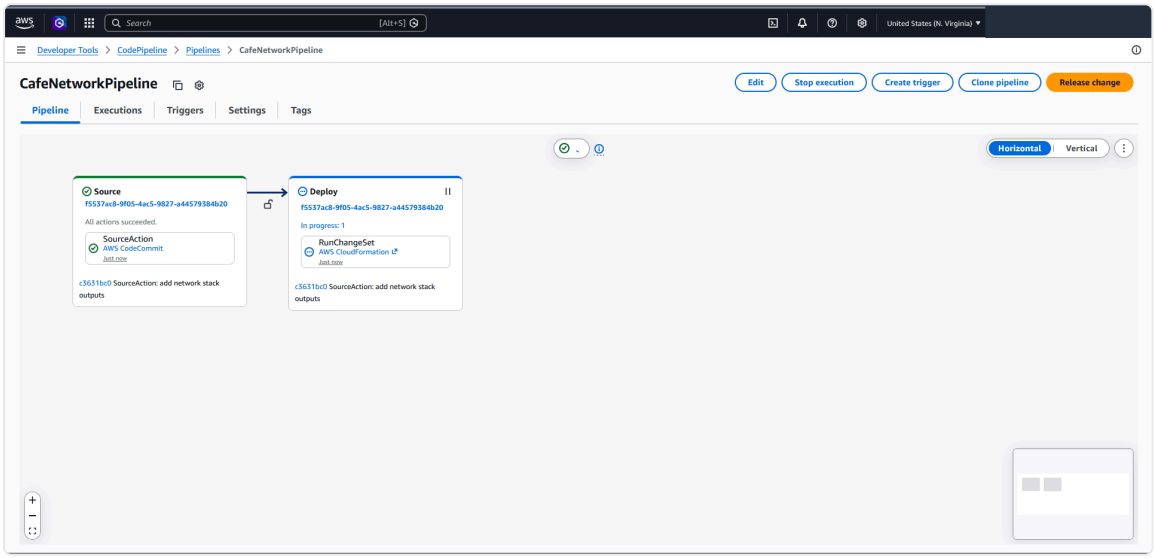
Task 6 - Network stack outputs



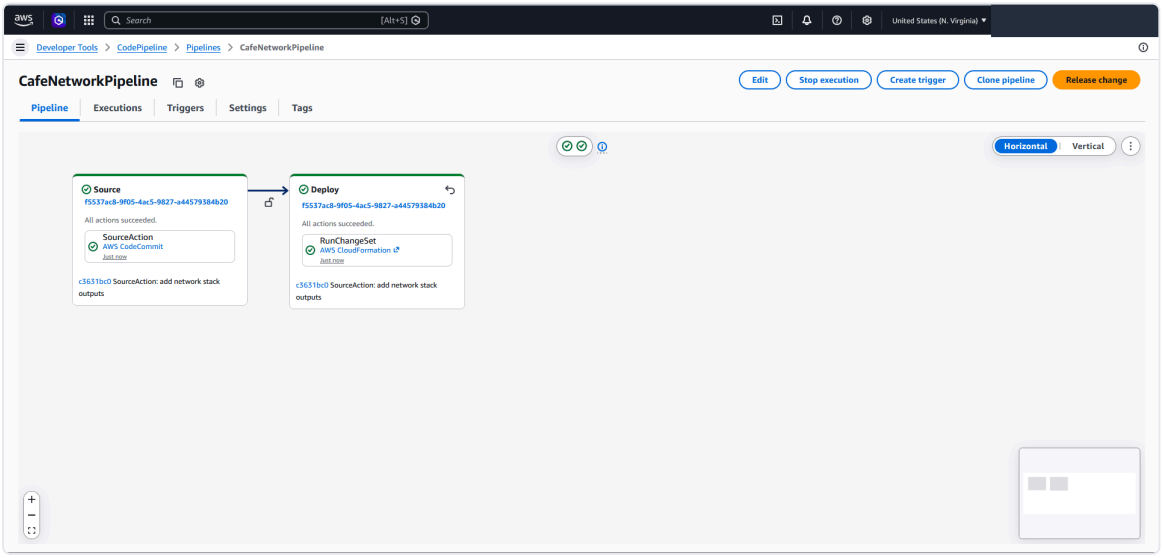
V6-E29 — Outputs added to cafe-network.yaml and committed.



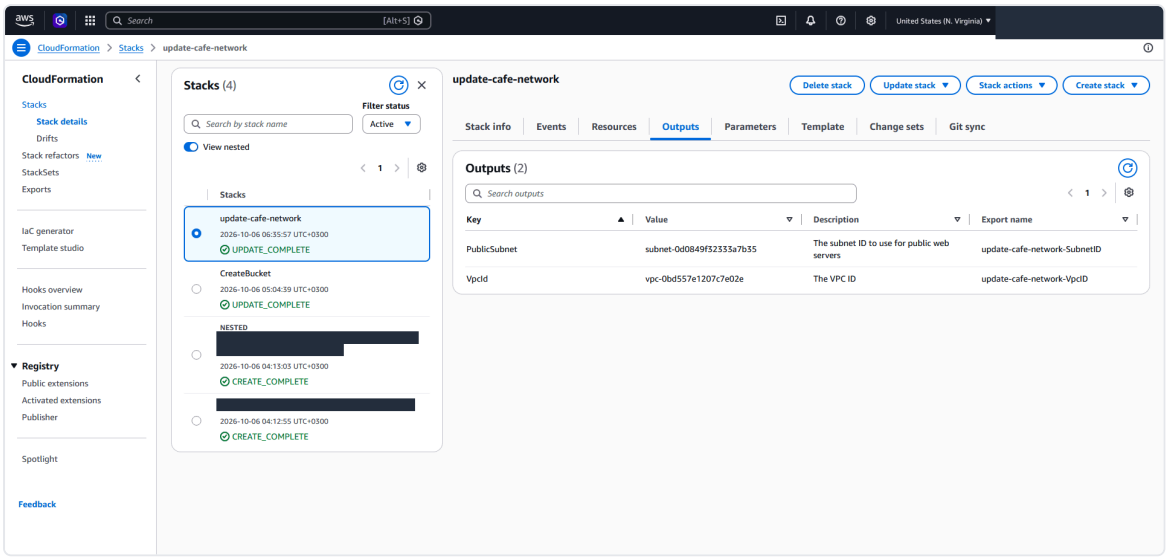
V6-E30 — Git push of the network outputs.



V6-E31 — CafeNetworkPipeline running the update.

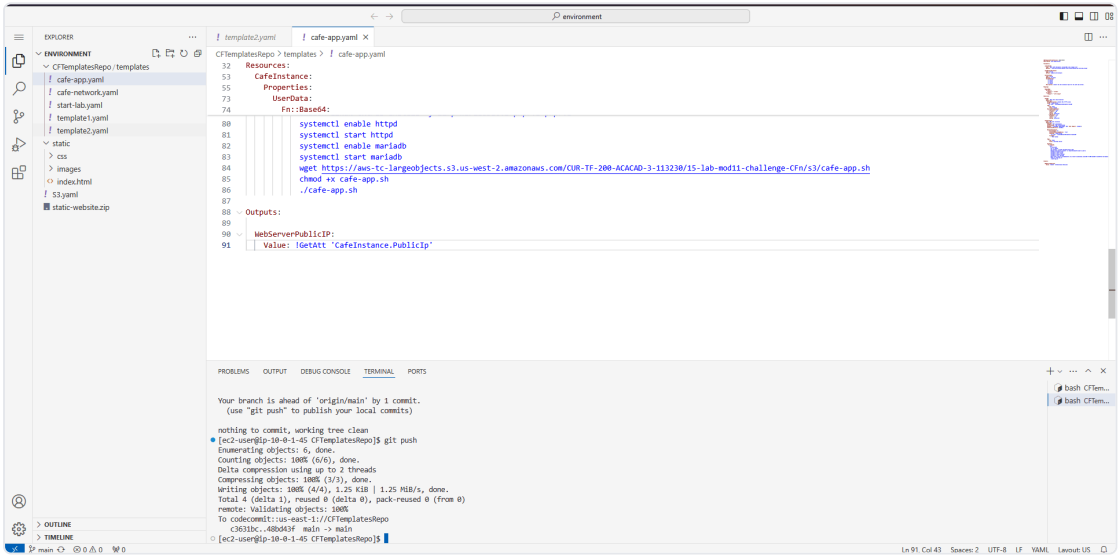


V6-E32 — CafeNetworkPipeline update succeeded.

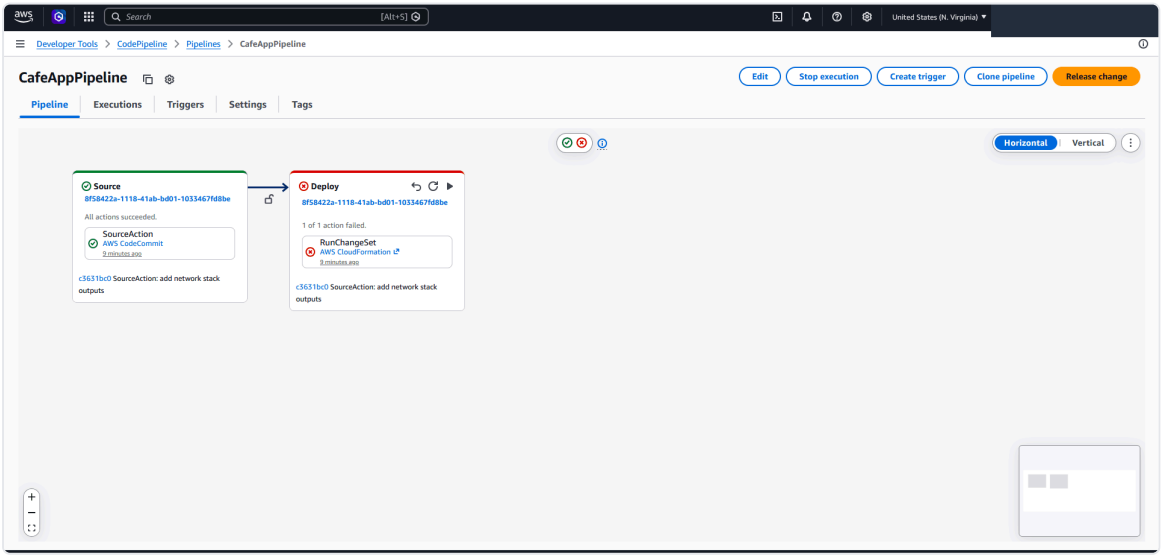


V6-E33 — Update-cafe-network outputs: PublicSubnet and VpcId with export names.

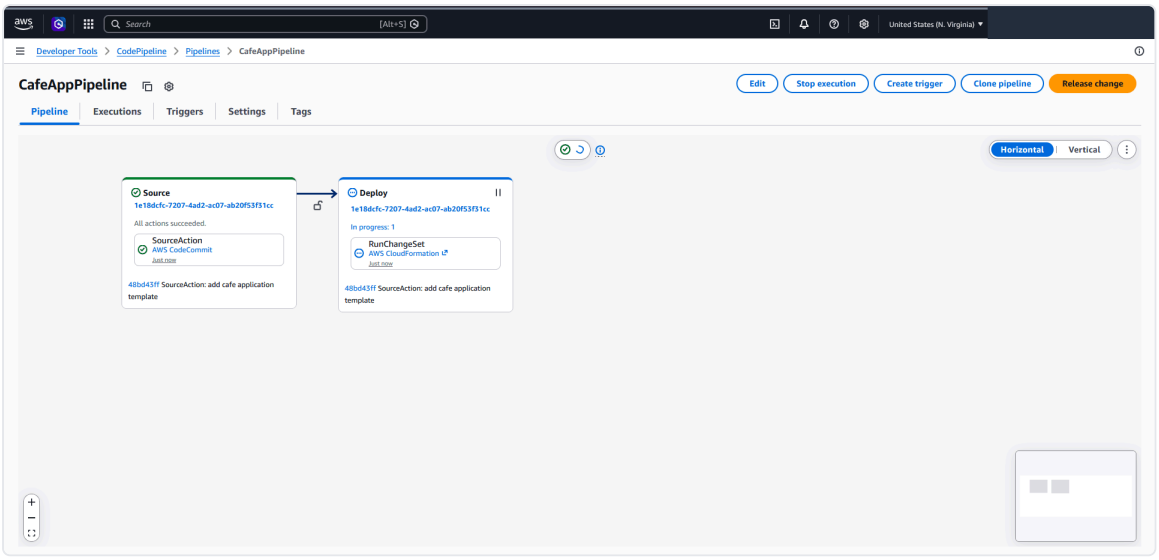
Task 7 - Application stack through the pipeline



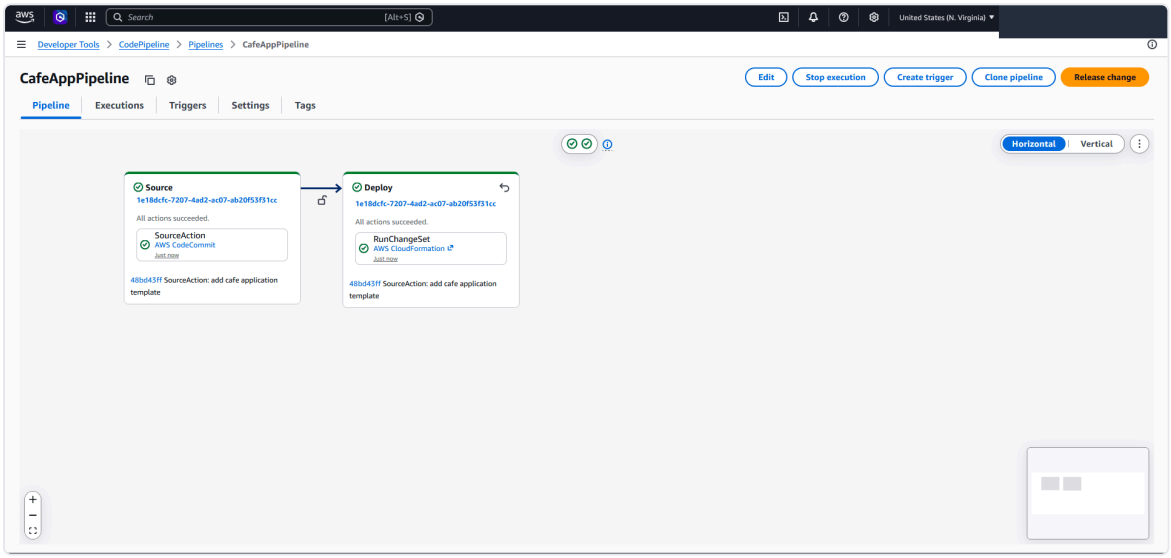
V6-E34 — Cafe-app.yaml (instance, parameter, user data) and git push.



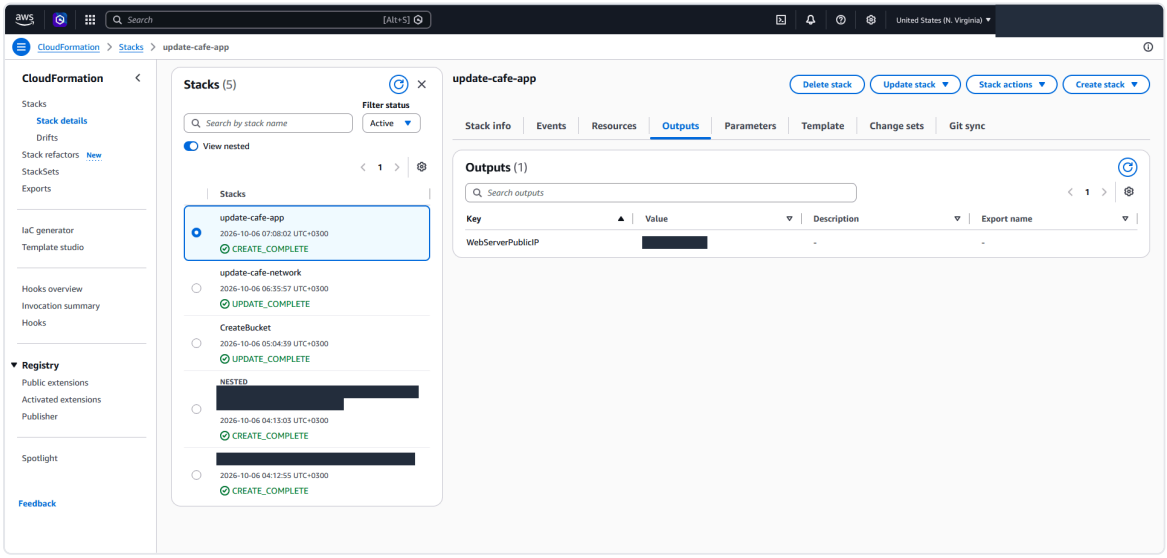
V6-E35 — CafeAppPipeline Deploy stage failed on the first run.



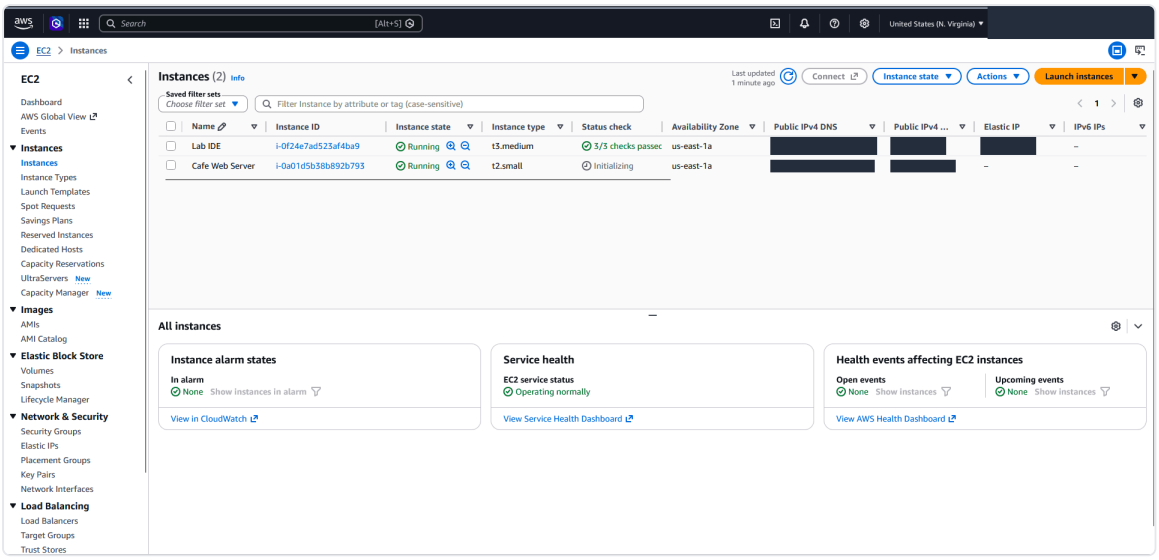
V6-E36 — CafeAppPipeline running again.



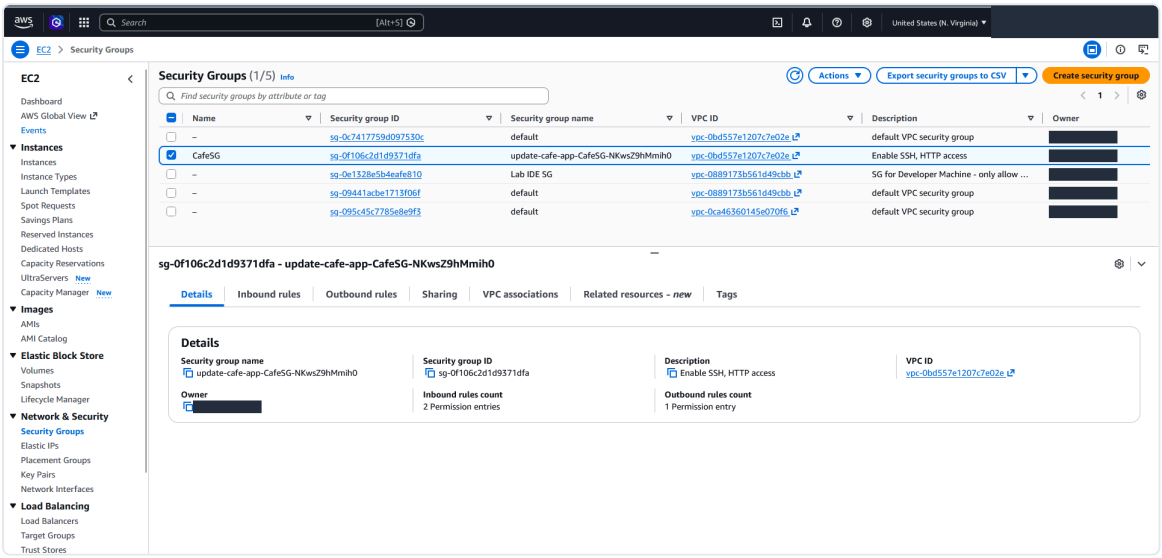
V6-E37 — CafeAppPipeline succeeded.



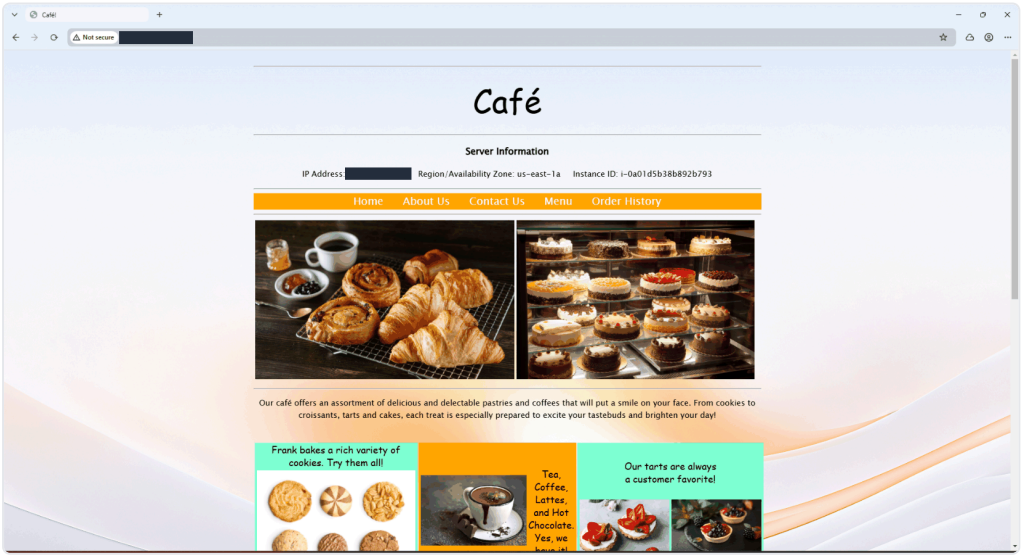
V6-E38 — Update-cafe-app stack CREATE_COMPLETE with the WebServerPublicIP output.



V6-E39 — EC2 console: Cafe Web Server (t2.small) created by the stack next to the Lab IDE instance.

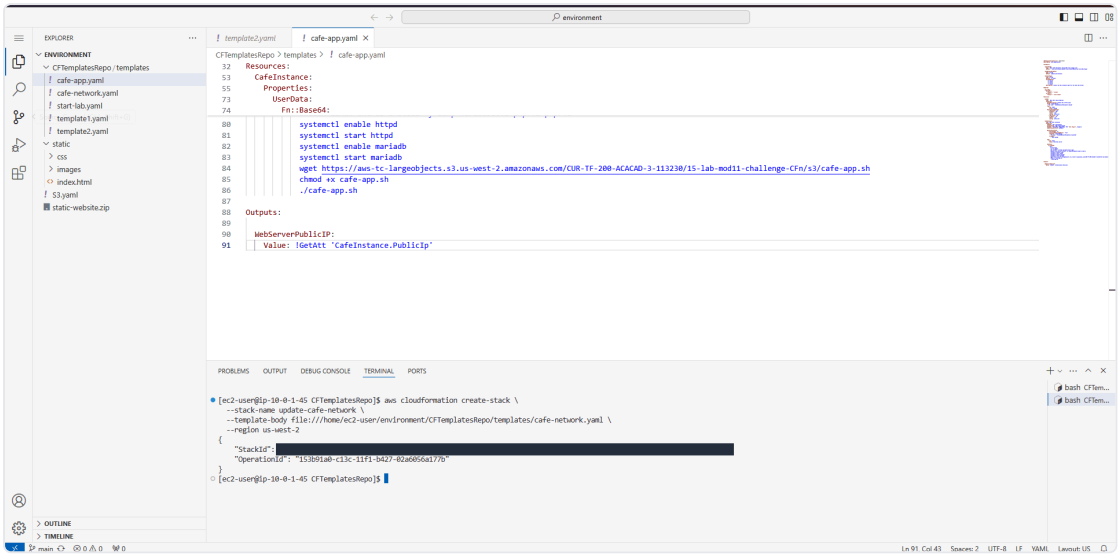


V6-E40 — CafeSG security group created by the app stack.

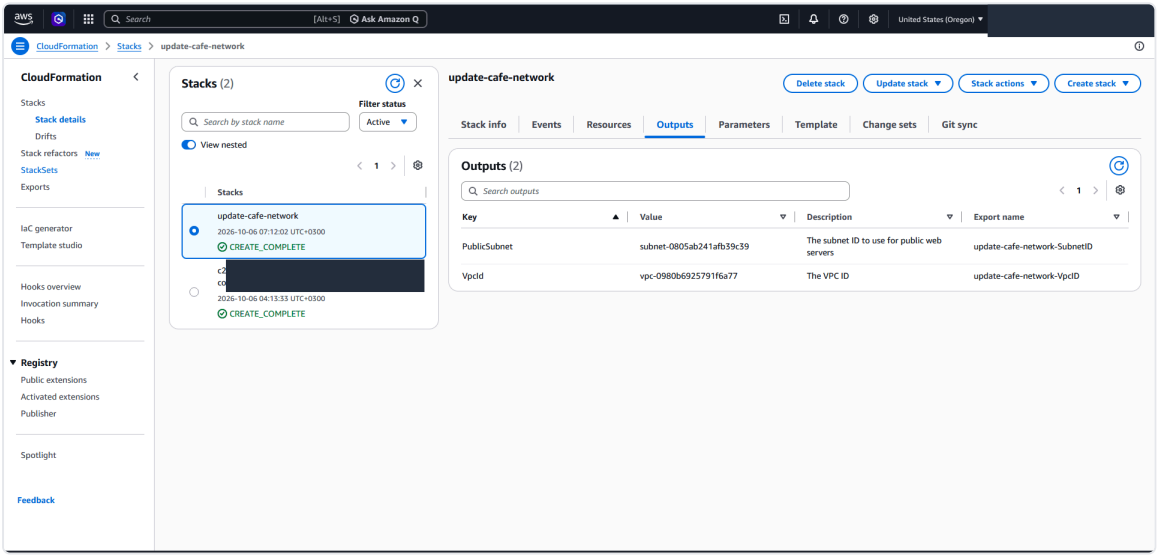


V6-E41 — Café website running in us-east-1 (server information shows Region/AZ).

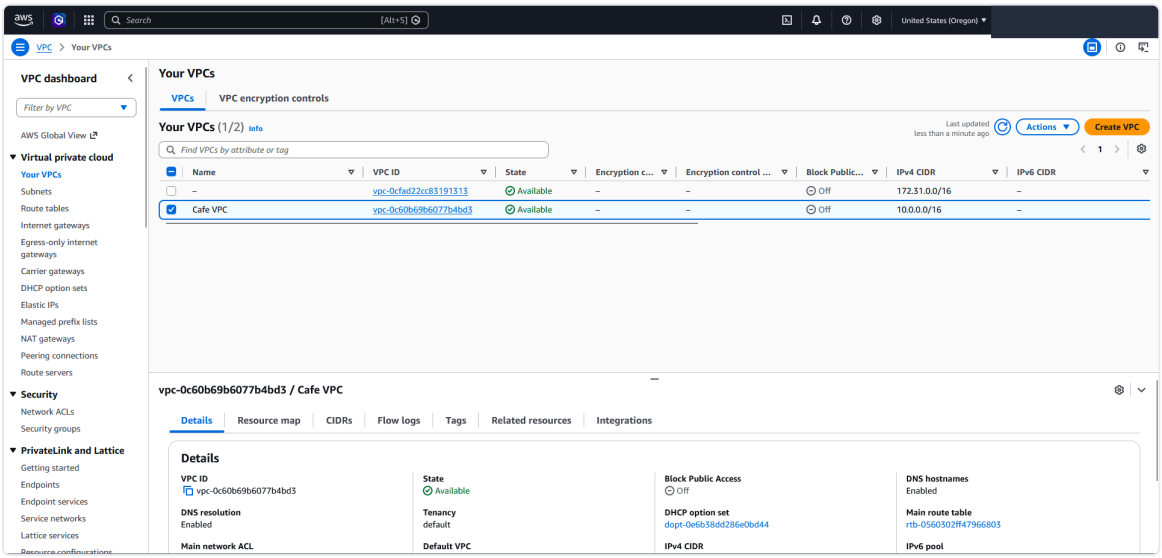
Task 8 - Second Region (us-west-2)



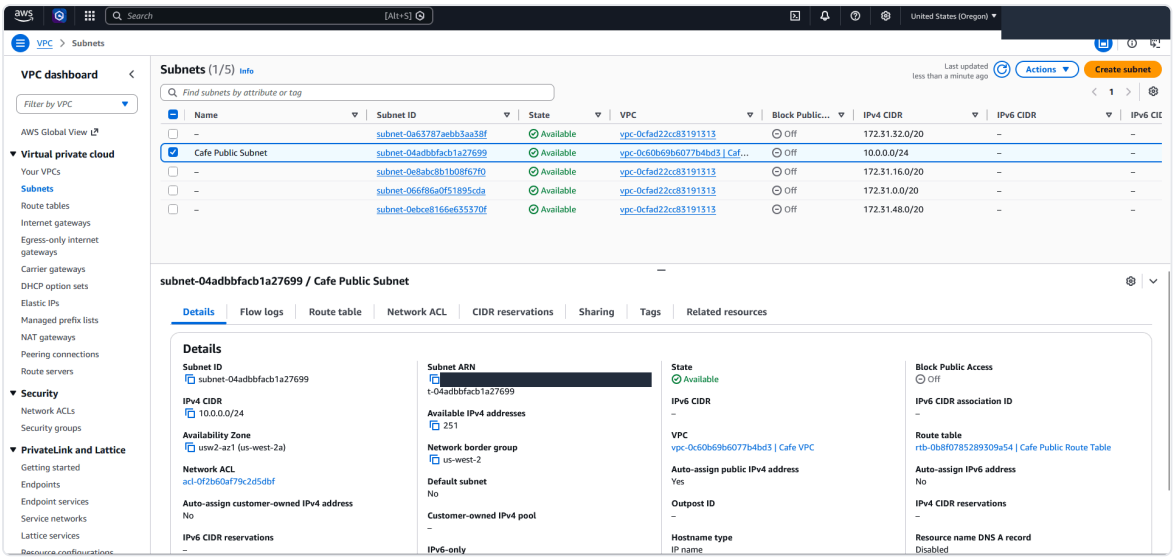
V6-E42 — CLI command creating the update-cafe-network stack in us-west-2.



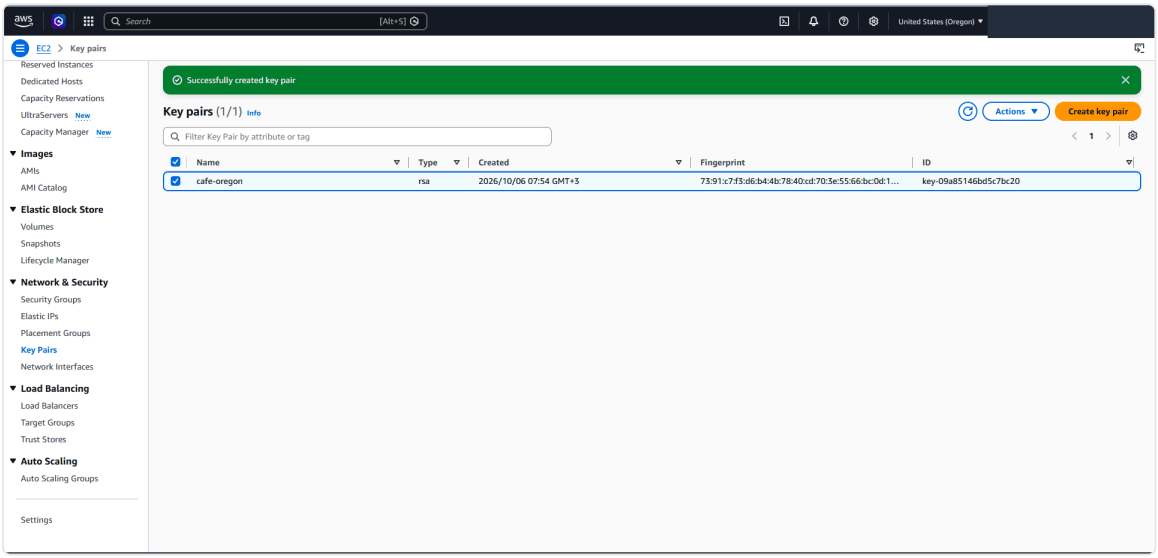
V6-E43 — Update-cafe-network stack in Oregon with its outputs.



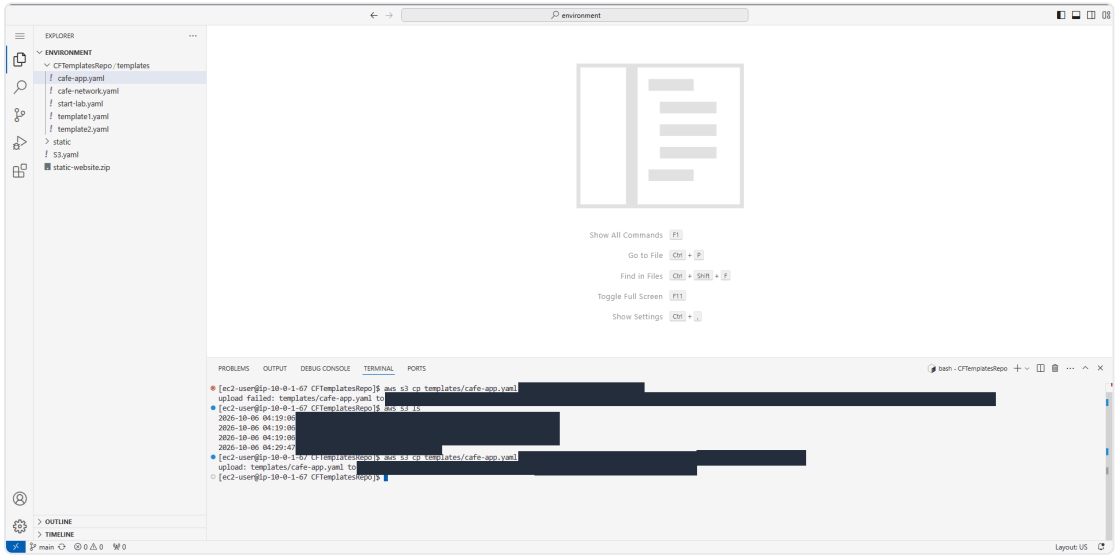
V6-E44 — Cafe VPC in us-west-2.



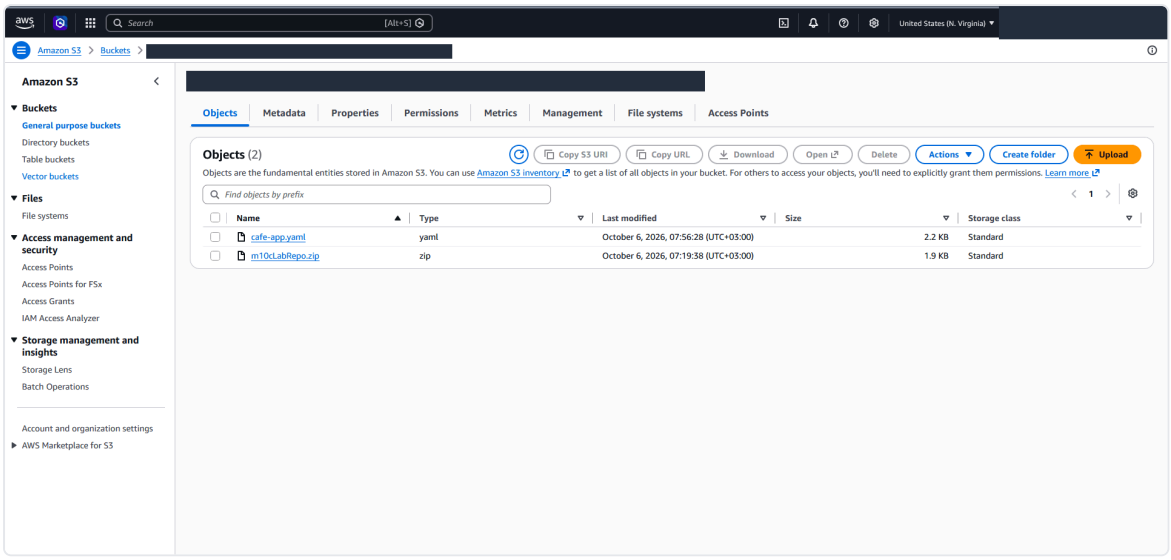
V6-E45 — Cafe Public Subnet in us-west-2.



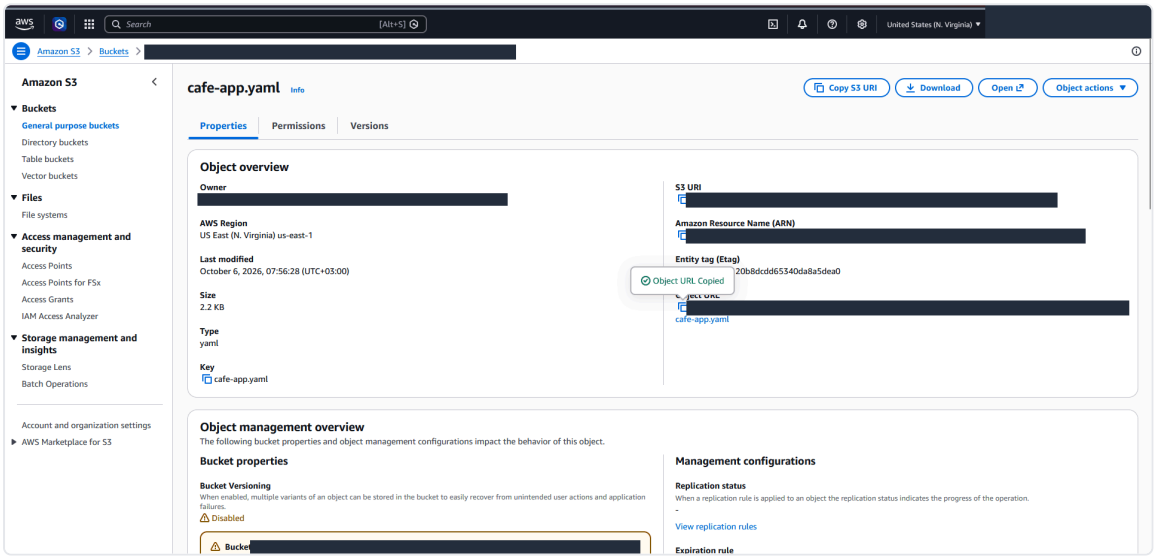
V6-E46 — Key pair cafe-oregon created in Oregon.



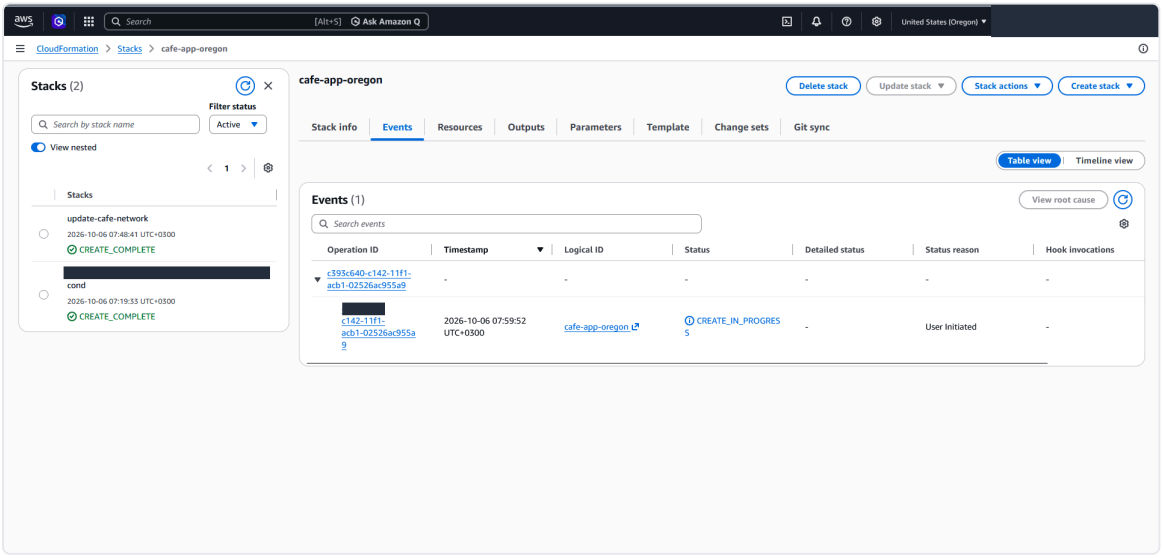
V6-E47 — Uploading cafe-app.yaml to the repository bucket: first attempt failed (NoSuchBucket), second succeeded.



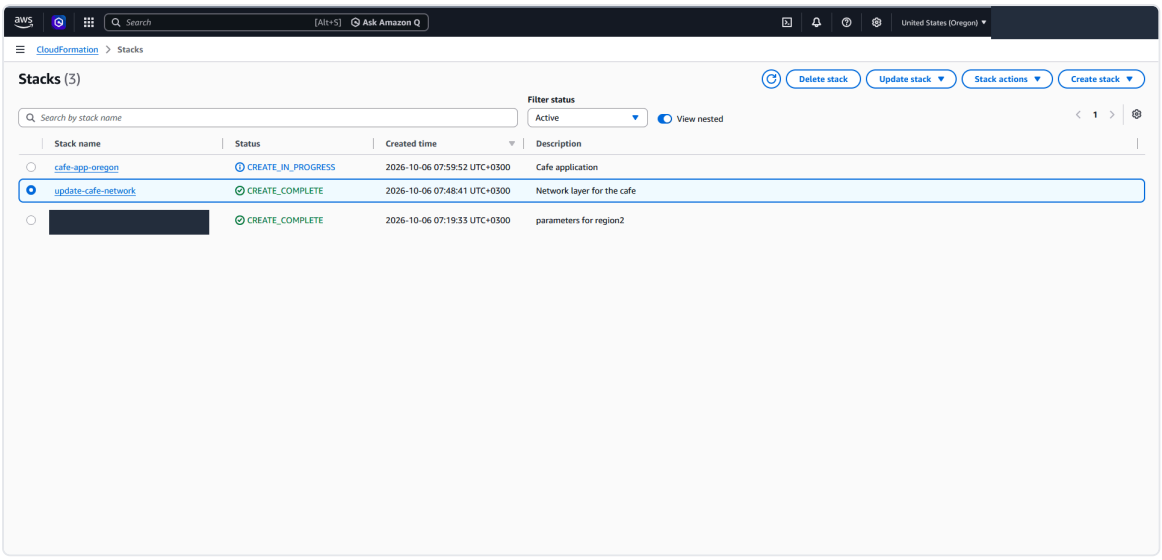
V6-E48 — Repository bucket containing cafe-app.yaml.



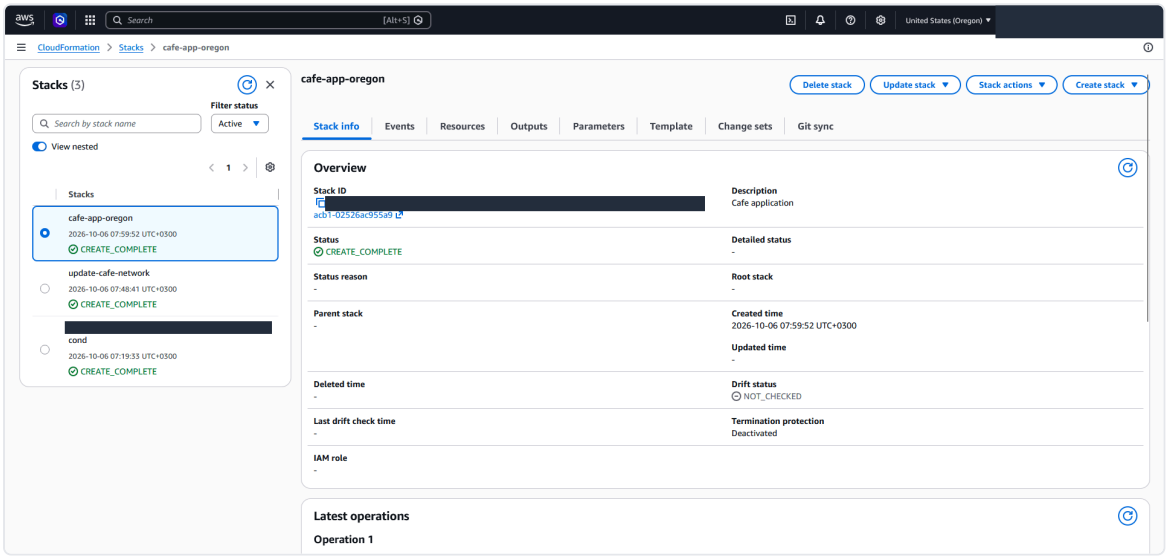
V6-E49 — Cafe-app.yaml object details with the object URL copied.



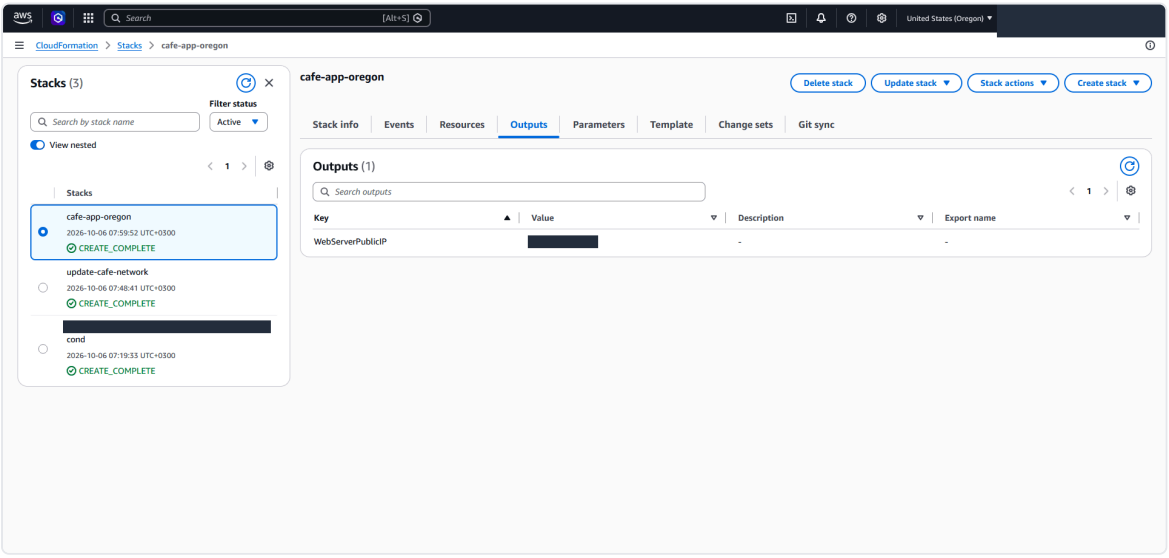
V6-E50 — Cafe-app-oregon stack creation in progress.



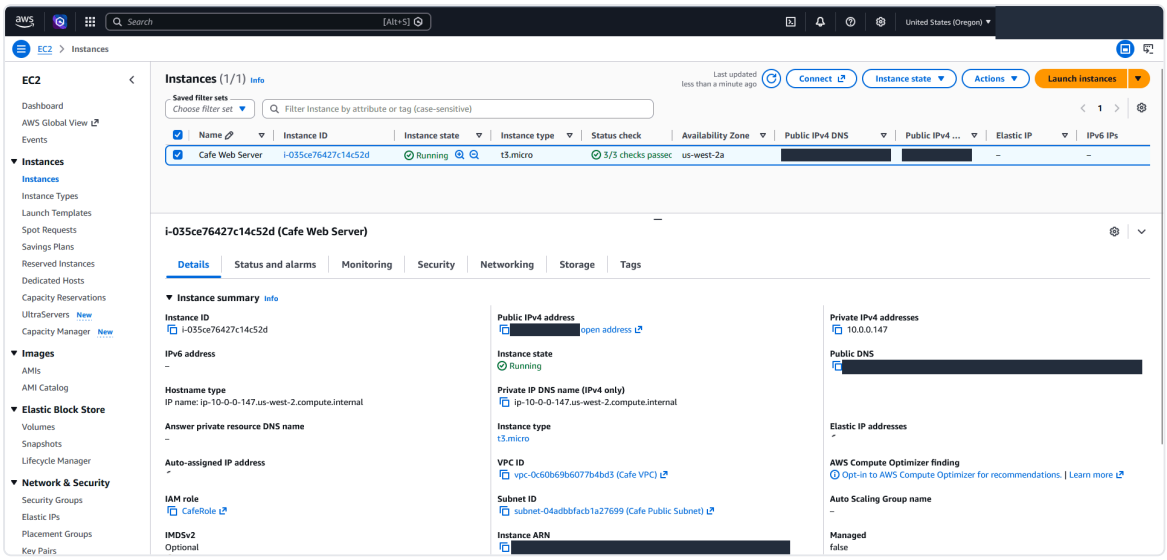
V6-E51 — Stacks list in Oregon.



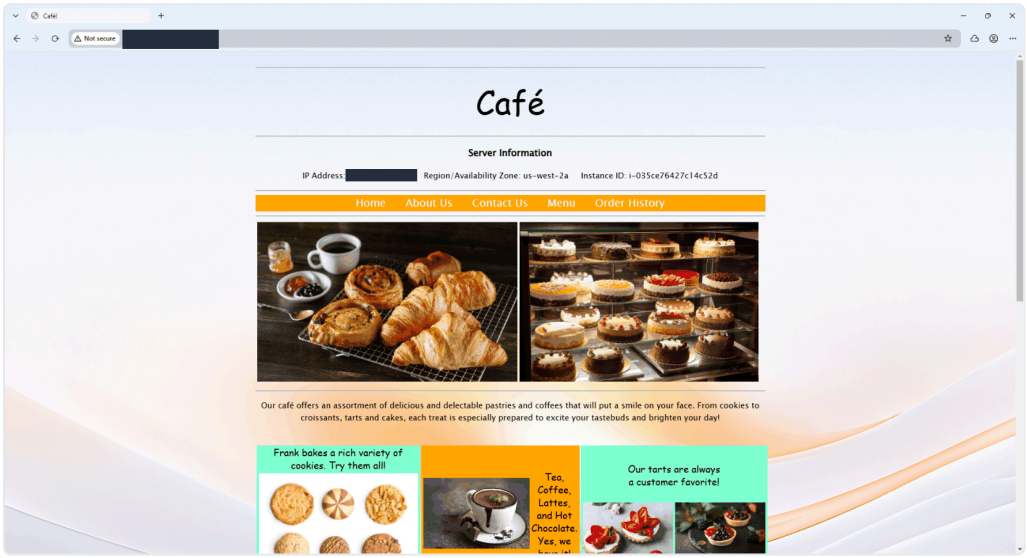
V6-E52 — Cafe-app-oregon stack CREATE_COMPLETE.



V6-E53 — Cafe-app-oregon outputs (WebServerPublicIP).



V6-E54 — Oregon Cafe Web Server instance (t3.micro).



V6-E55 — Café website running in us-west-2 (server information shows the Oregon AZ).

Outcome

The café's network and web server are now defined as CloudFormation templates in CodeCommit and deployed by pipelines on every push. The same templates created a second working copy of the café in us-west-2.

CHAPTER 8 · STAGE V7

V7 — Serverless Reporting (Lambda + SNS)

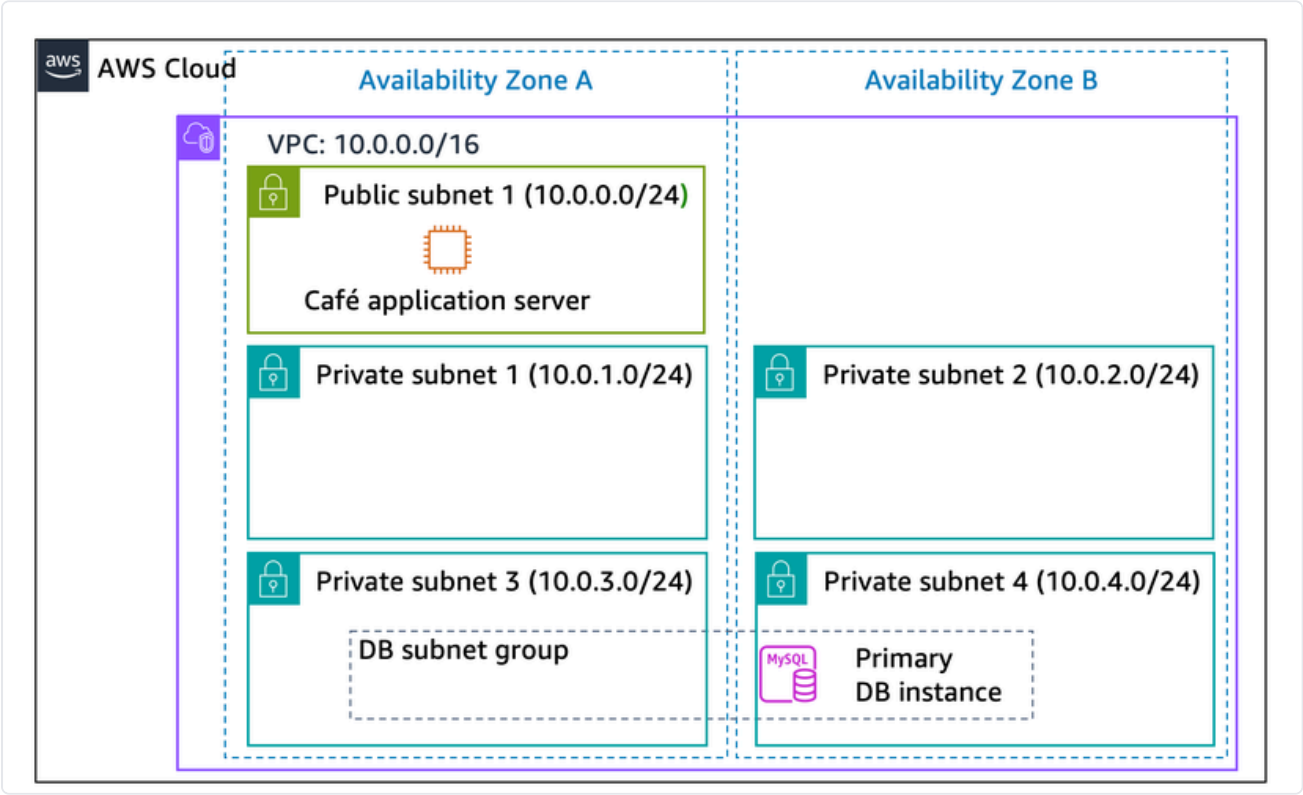
> Completed as part of AWS Academy Challenge Labs (training environment).

Business Requirement

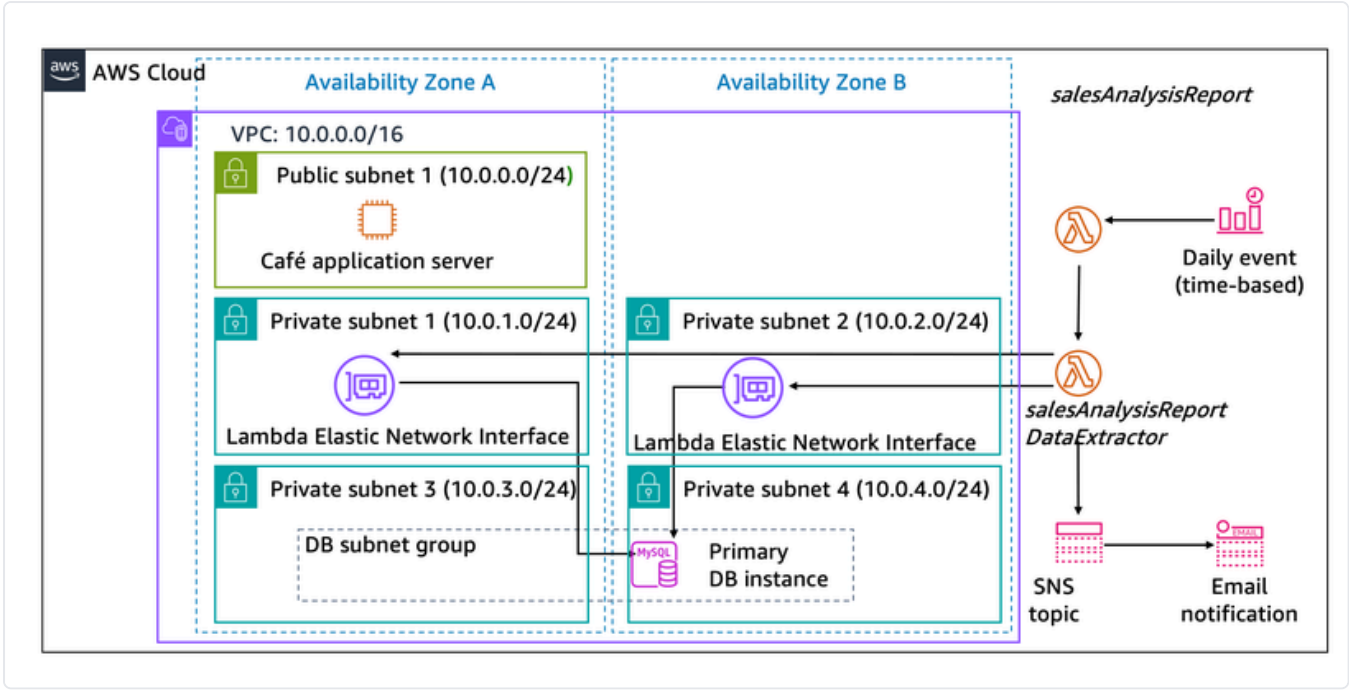
The café is growing and the owners want a **daily sales report** by email, to plan ingredient orders and see the effect of promotions. A cron job on the web server worked but slowed the production site, and a dedicated EC2 instance would sit idle almost all day. The decision: move report generation to **AWS Lambda** (pay only while it runs) and deliver the result with **Amazon SNS**.

Start and Target Architecture

These two diagrams come from the AWS Academy lab material. They show where this stage starts and what it builds. They are shown here as reference; my own diagram of what I deployed follows in the next section.

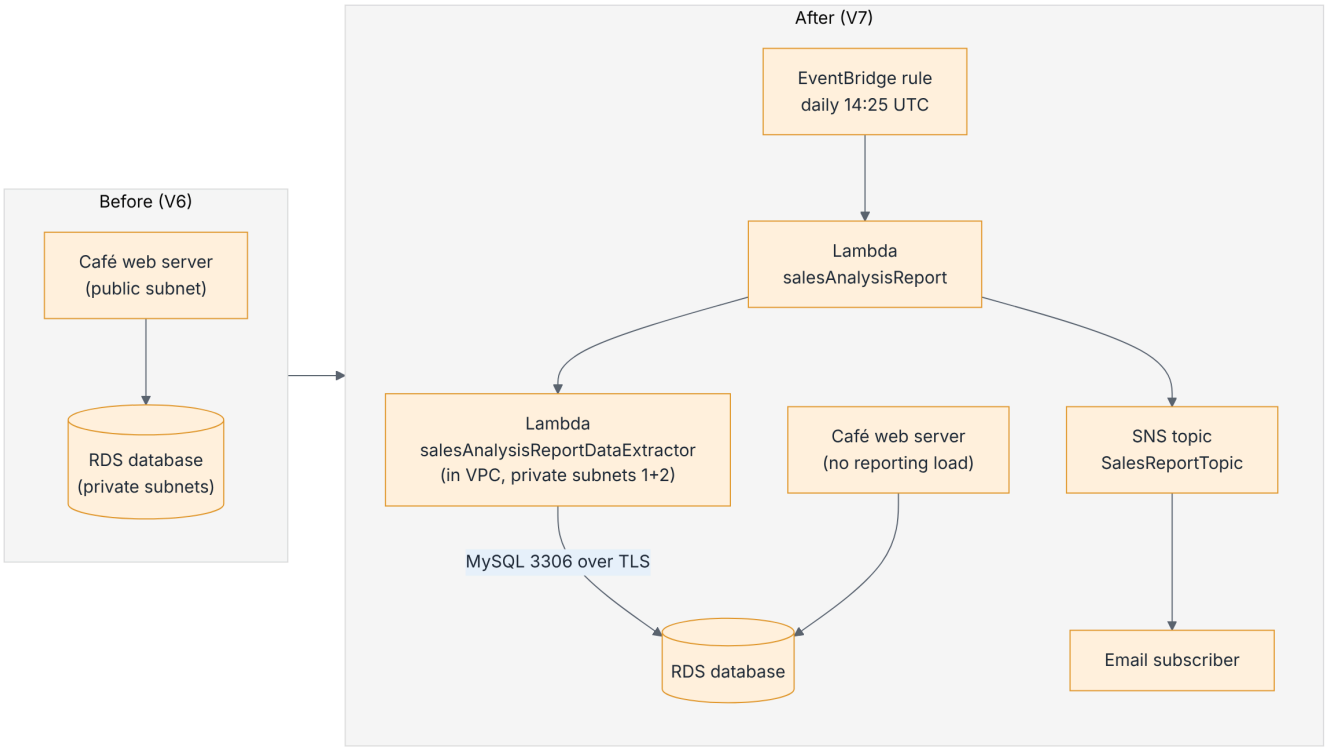


Starting architecture. AWS Academy lab diagram: where the stage begins.



Target architecture. AWS Academy lab diagram: the design this stage builds.

Before → After



AWS Services Used

SERVICE	ROLE IN V7
AWS Lambda	Two functions: a data extractor (inside the VPC) and the report generator
Amazon VPC / Security Groups	`LambdaSG` for the extractor; `DatabaseSG` updated to accept it

SERVICE	ROLE IN V7
Amazon RDS	Source of the café sales data (reached privately)
Amazon SNS	Topic `SalesReportTopic` with an email subscription
Amazon EventBridge	Scheduled rule that runs the report daily
Amazon CloudWatch Logs	Debugging the failed test runs
AWS IAM	Execution roles for the functions and `mySchedulerRole` for the schedule

Implementation Highlights

- **Network access for Lambda:** created `LambdaSG` (all outbound) in the Lab VPC and added a second MySQL/Aurora (3306) inbound rule on `DatabaseSG` whose source is `LambdaSG`, so only the function, not the whole network, can reach the database.
- **DataExtractor:** a Lambda attached to Private subnet 1 and 2 with `LambdaSG`, 128 MB memory, 30 s timeout, using the pre-made `salesAnalysisReportDERole`. It must live in the VPC because the database is in private subnets.
- **Report function:** `salesAnalysisReport`, 128 MB, 30 s, role `salesAnalysisReportRole`. It is not placed in the VPC.
- **SNS:** standard topic `SalesReportTopic` (display name *Sales Report Topic*); its ARN is passed to the report function as the `topicARN` environment variable; an email subscription was created and confirmed from the inbox.
- **Schedule:** EventBridge rule `dailySalesReportRule` with a cron expression for 14:25 UTC every day, targeting `salesAnalysisReport` through the existing `mySchedulerRole`.

Challenges & Decisions

Problem 1 — first test failed. Running `salesAnalysisReport` returned *"No body in returned data"*. The report function's own log only repeated that message, so the real cause was upstream.

Problem 2 — database refused the connection. The DataExtractor log showed it failed to connect to the database with error 3159: *connections using insecure transport are prohibited while* `--require_secure_transport=ON`. The network path (security groups, subnets) was therefore fine; the database required TLS.

Fix. The extractor package was updated with the RDS CA bundle (`global-bundle.pem`) and the connection code now passes it as the SSL certificate. After that the test returned `statusCode 200` and the report email arrived.

Decisions

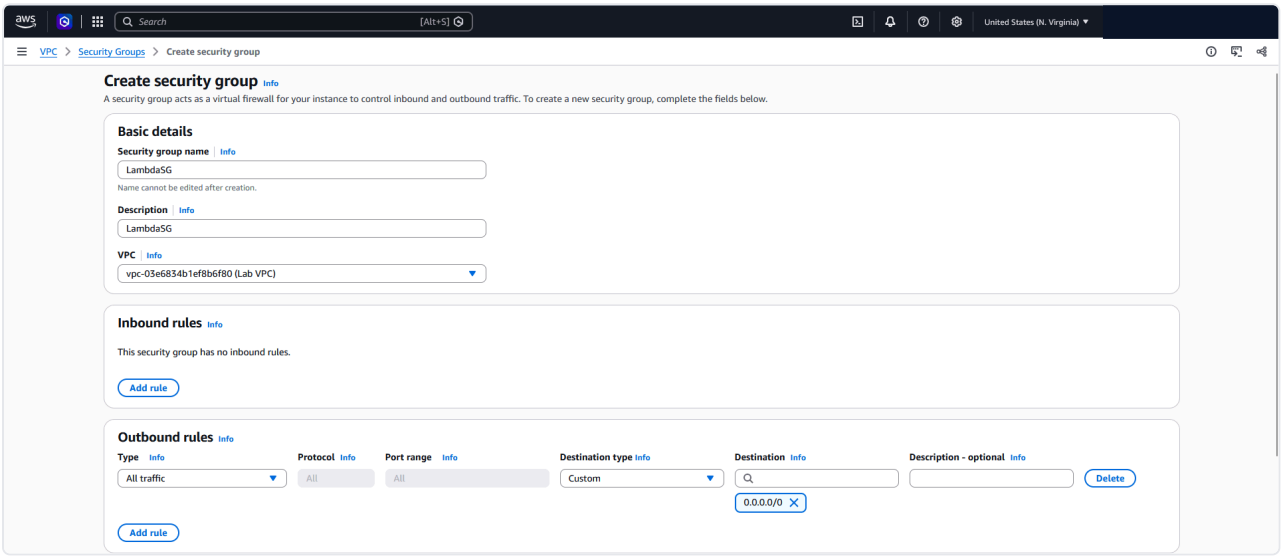
- Security group *source* = `LambdaSG` instead of a CIDR range: least privilege and no dependency on IP addresses.
- Extractor in two private subnets (two AZs); report function left outside the VPC because it only needs to call the extractor and SNS, not the database.

- Lambda instead of cron-on-EC2 or a separate instance: no impact on the web server and no 24/7 cost.

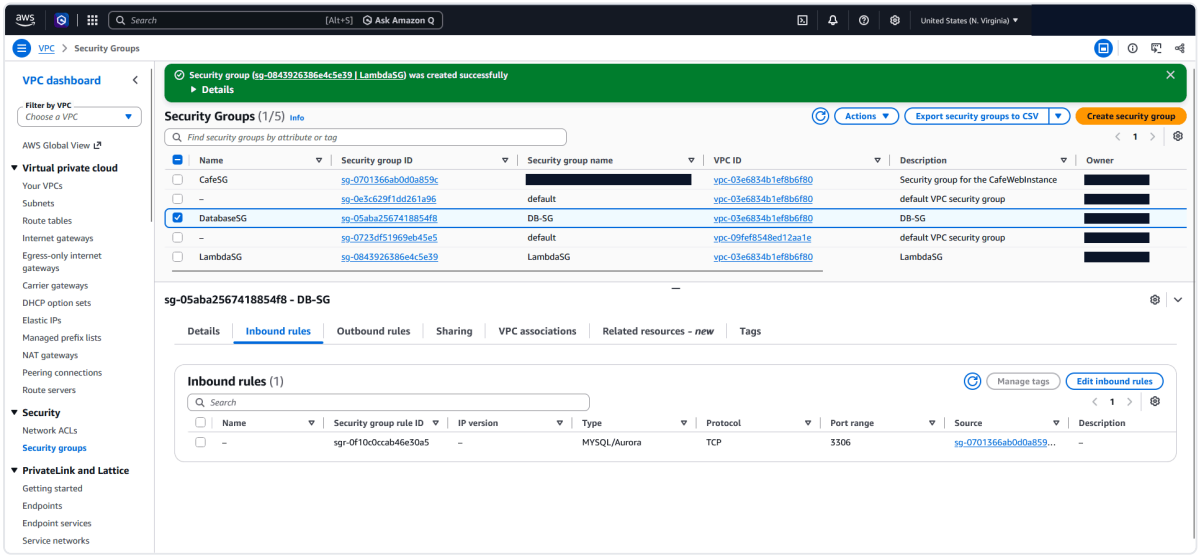
Validation & Evidence

All 36 screenshots of this stage, in lab order. Account IDs, ARNs, e-mail addresses and personal details were masked.

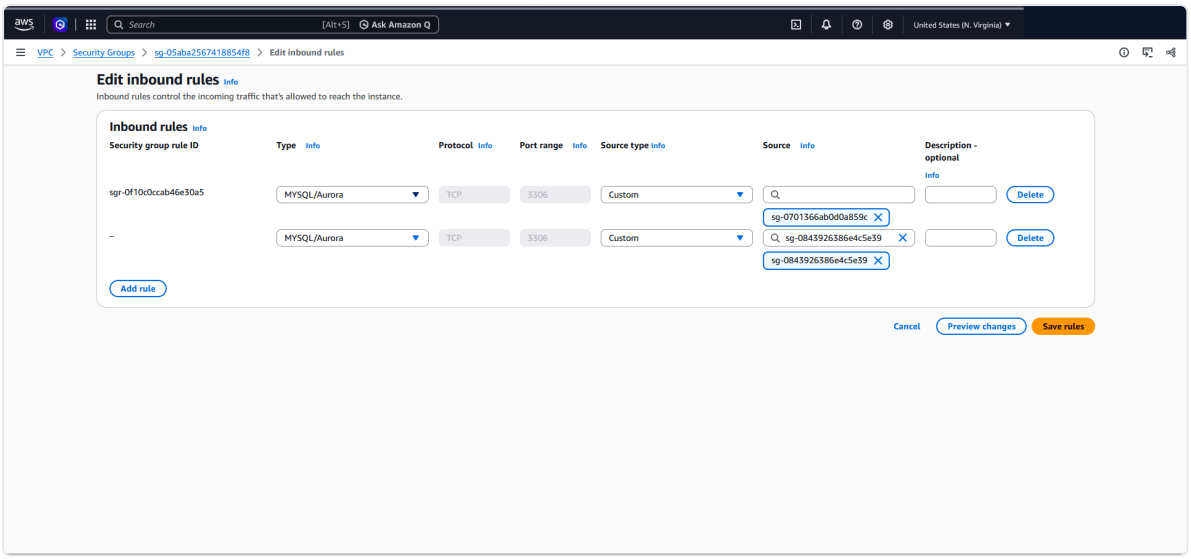
Task 2 — Security groups



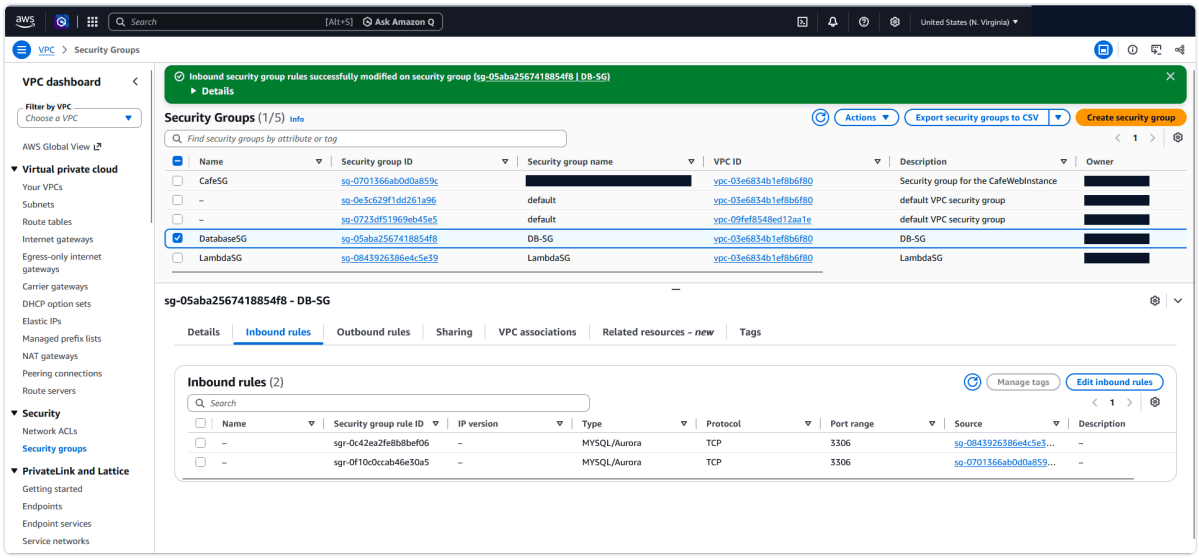
V7-E01 — Creating the LambdaSG security group in the Lab VPC (all outbound allowed)



V7-E02 — LambdaSG created

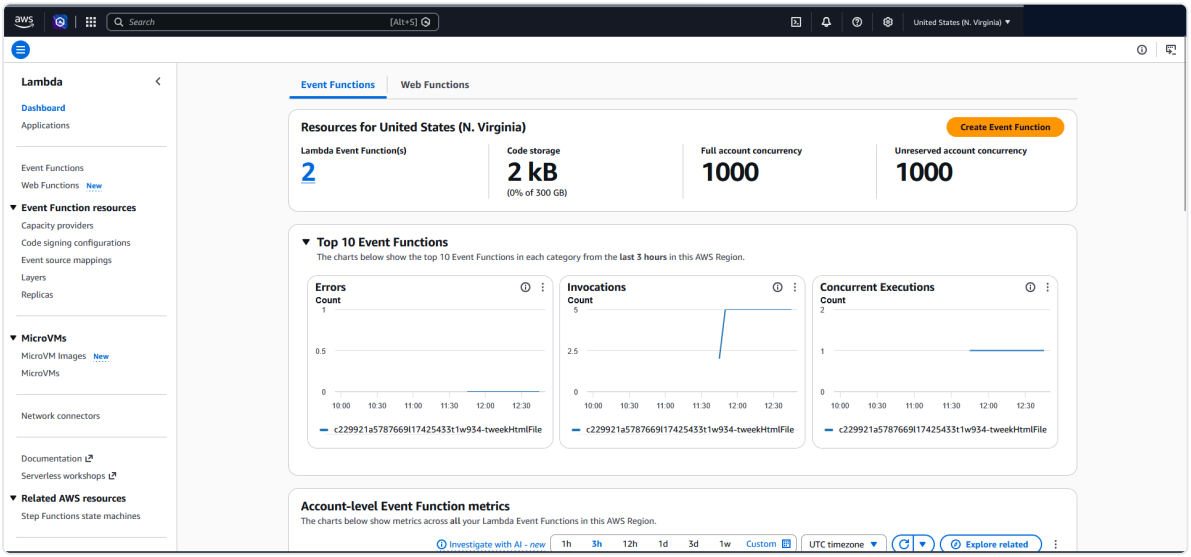


V7-E03 — Editing the database security group to add a second inbound rule

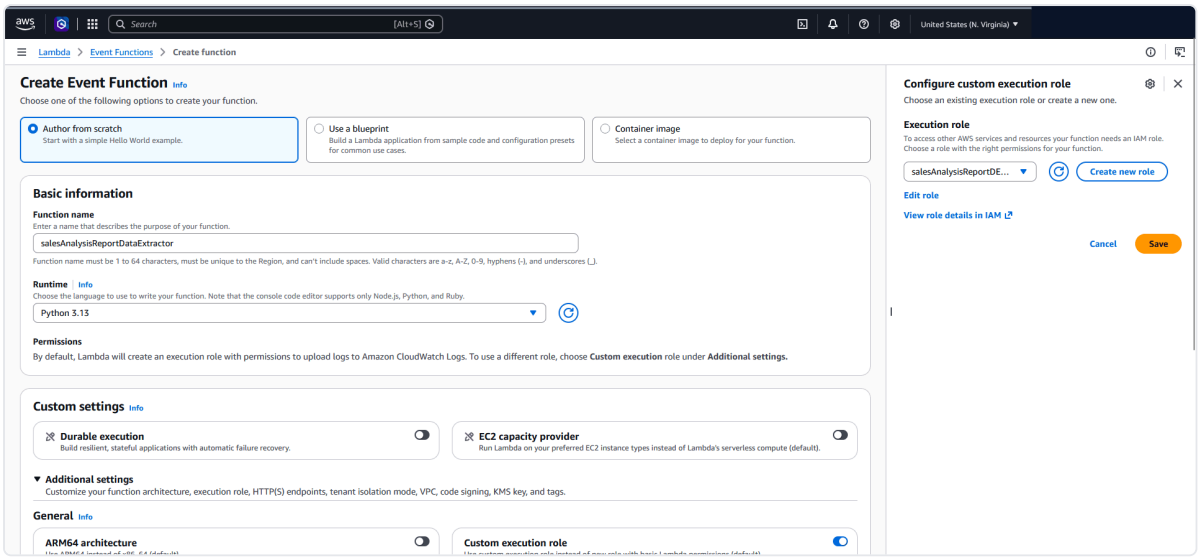


V7-E04 — DatabaseSG now allows MySQL/Aurora (3306) from both the café app SG and LambdaSG

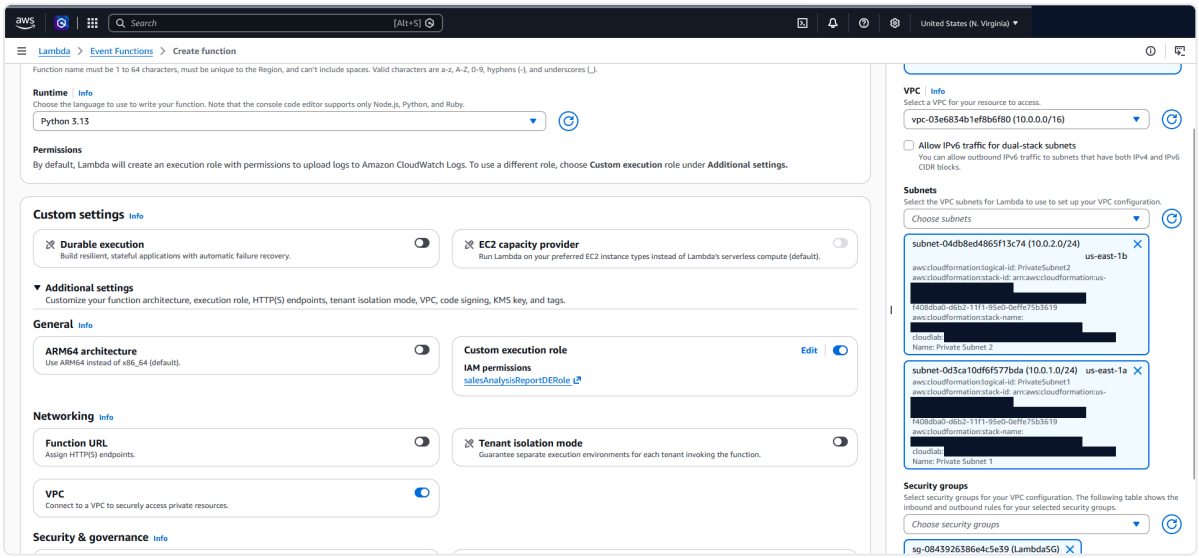
Task 2 — DataExtractor function



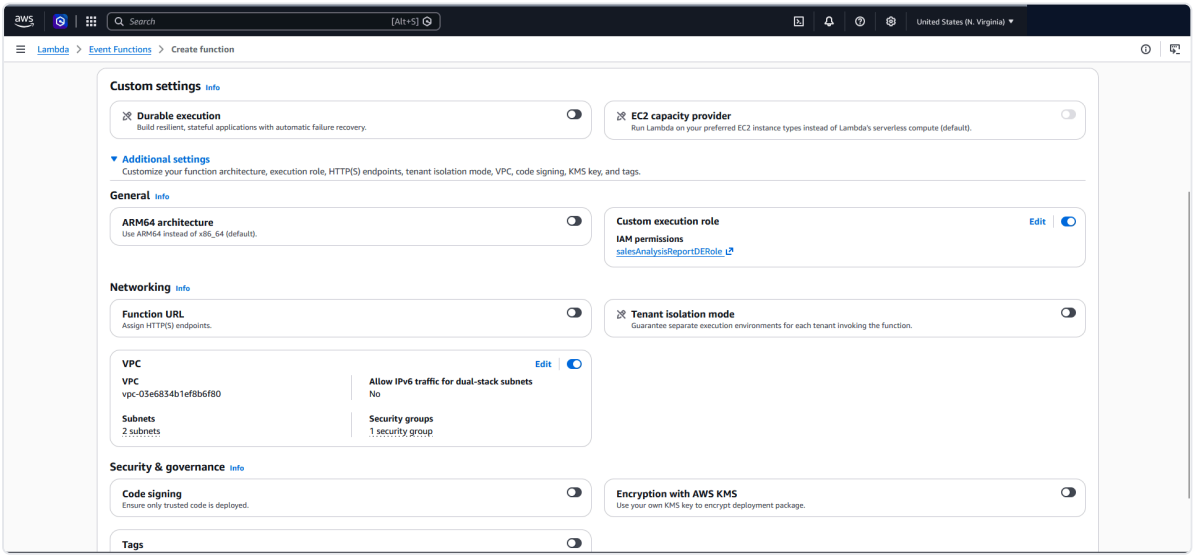
V7-E05 — Lambda console before creating the functions



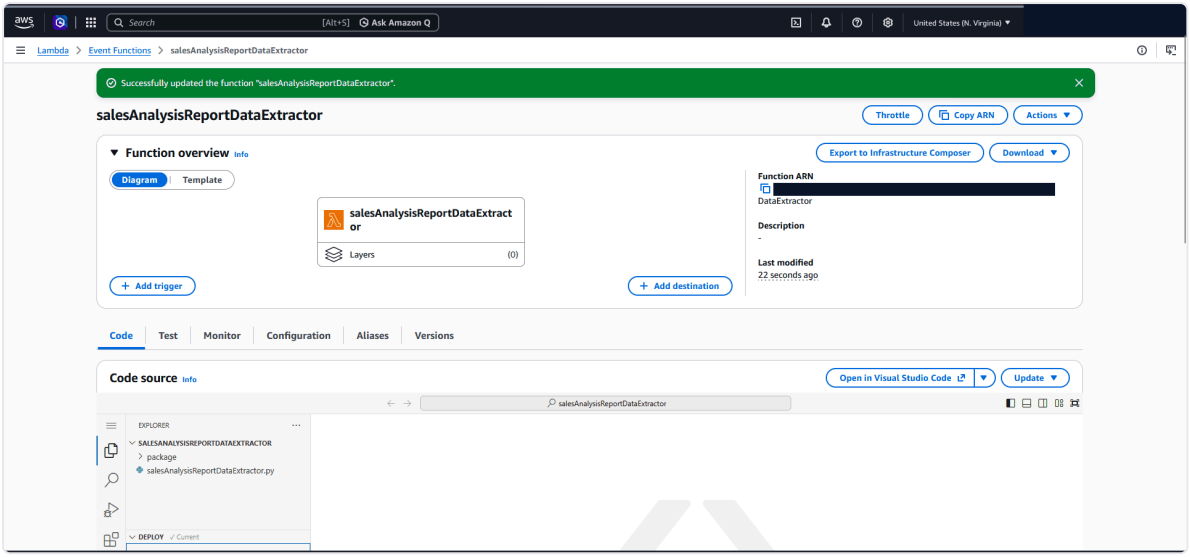
V7-E06 — Creating salesAnalysisReportDataExtractor with the custom execution role



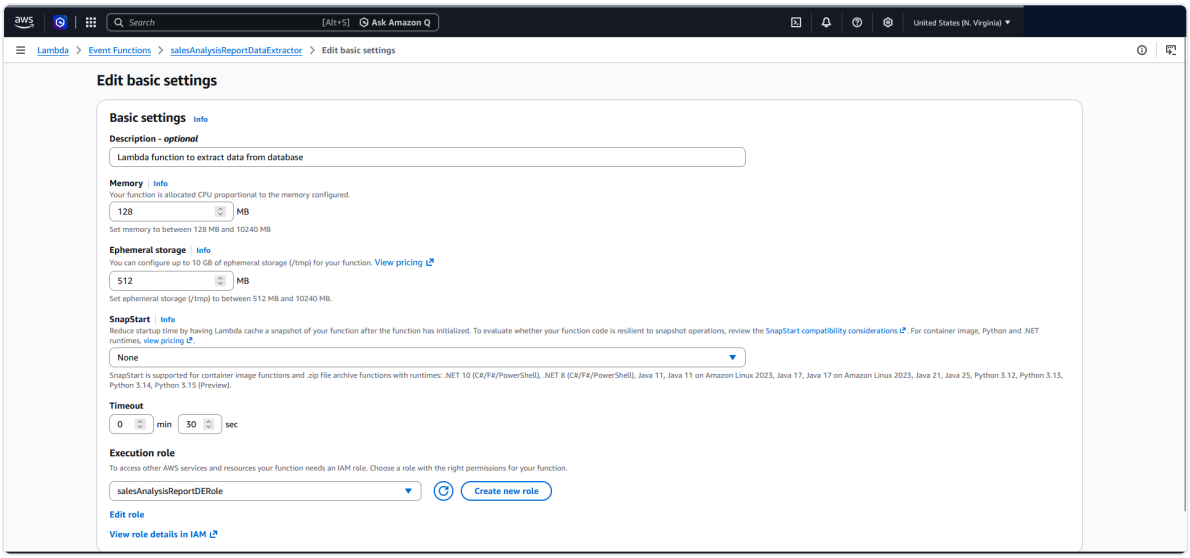
V7-E07 — Attaching the function to Lab VPC, Private subnet 1 + 2 and LambdaSG



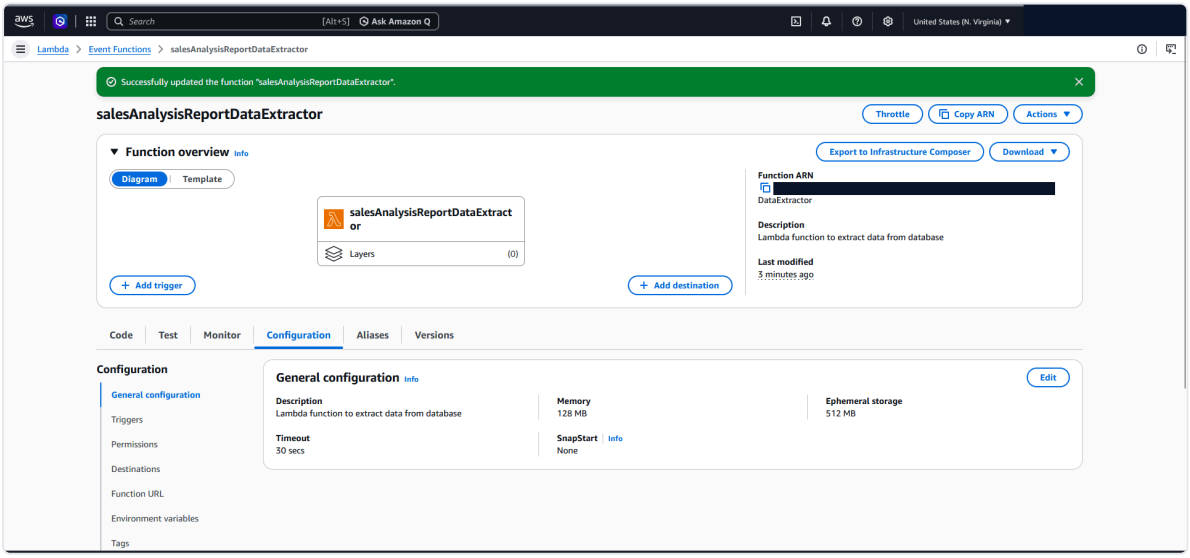
V7-E08 — VPC configuration summary (2 subnets, 1 security group)



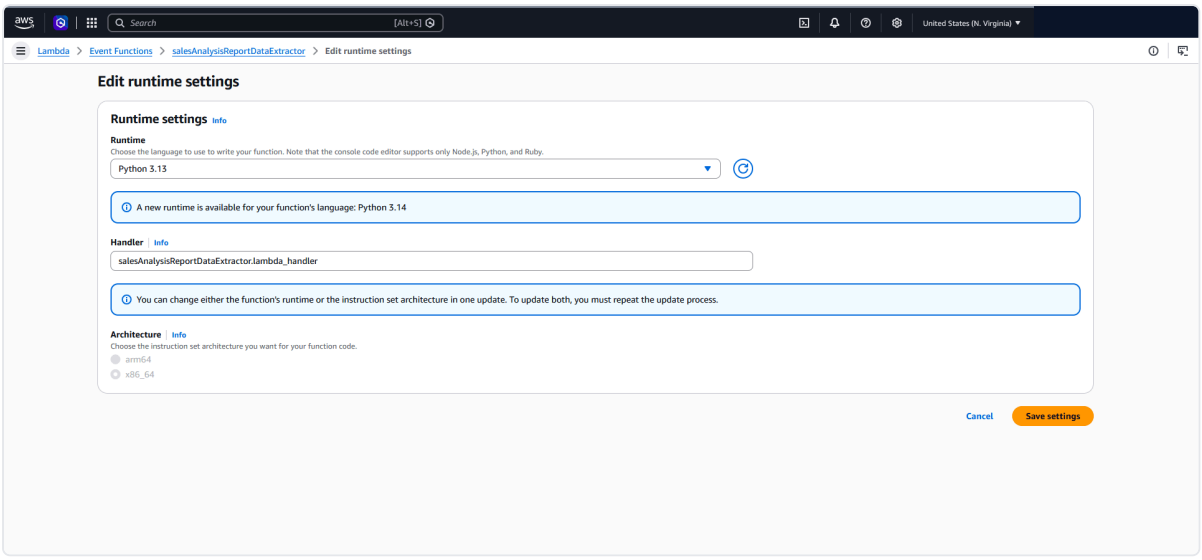
V7-E09 — Function code package uploaded



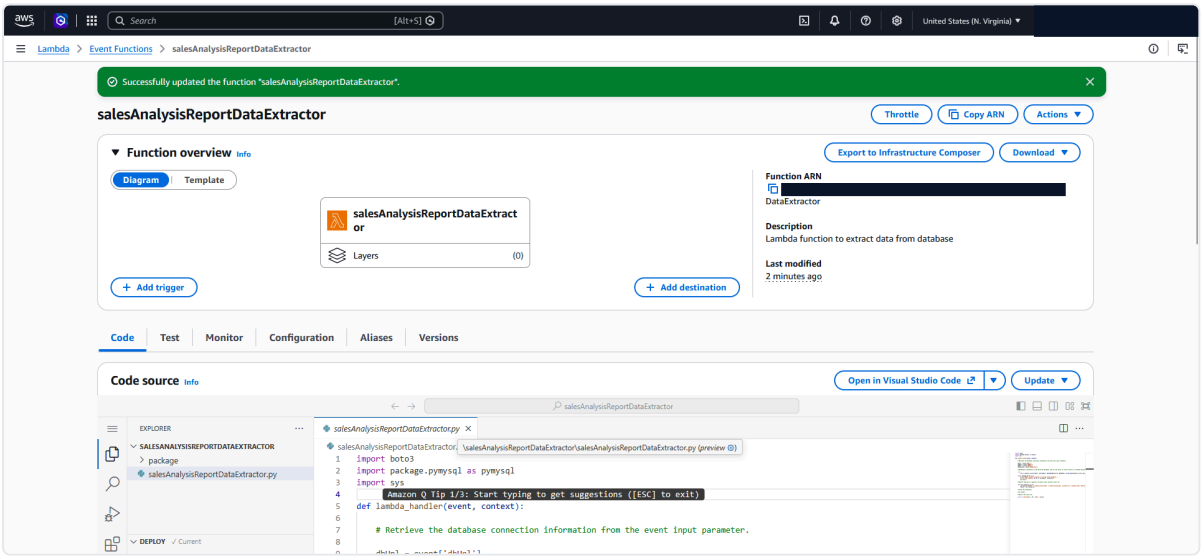
V7-E10 — Basic settings: 128 MB memory, 30 s timeout, salesAnalysisReportDERole



V7-E11 — General configuration confirmed

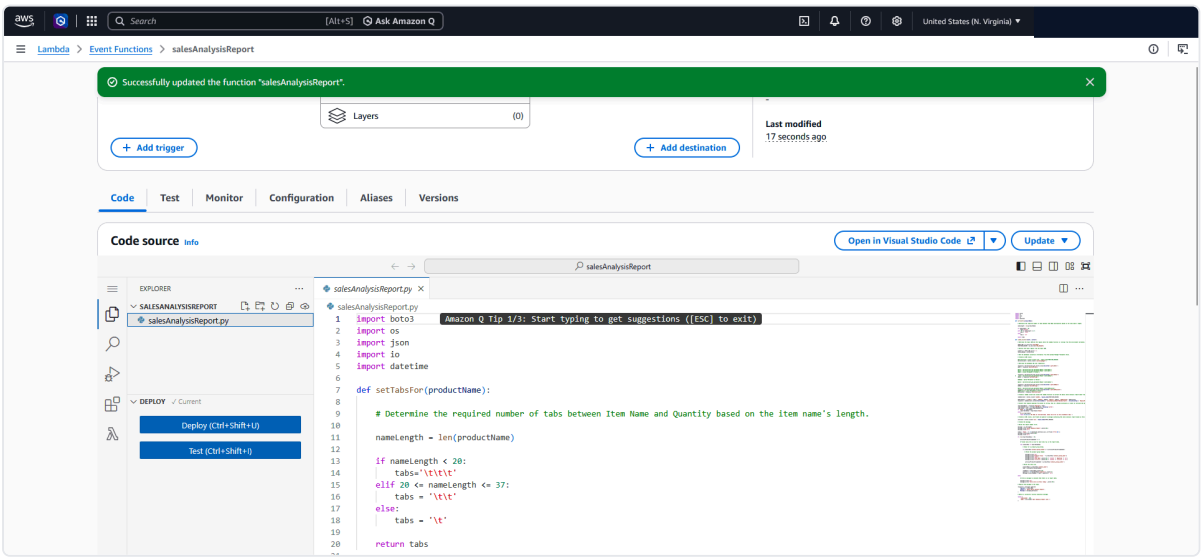


V7-E12 — Runtime and handler settings

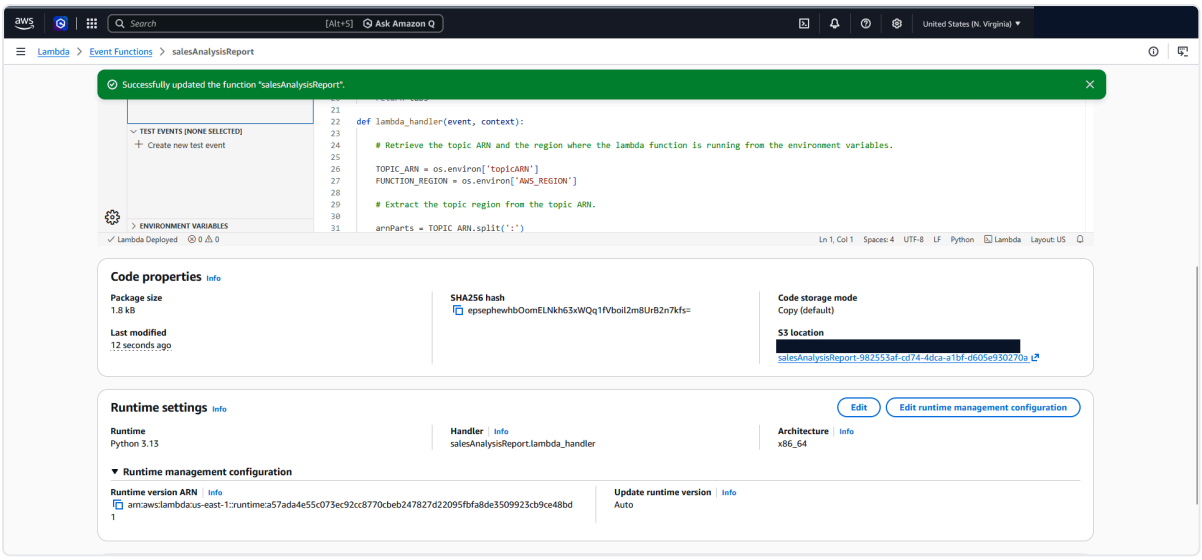


V7-E13 — Handler code in the console editor

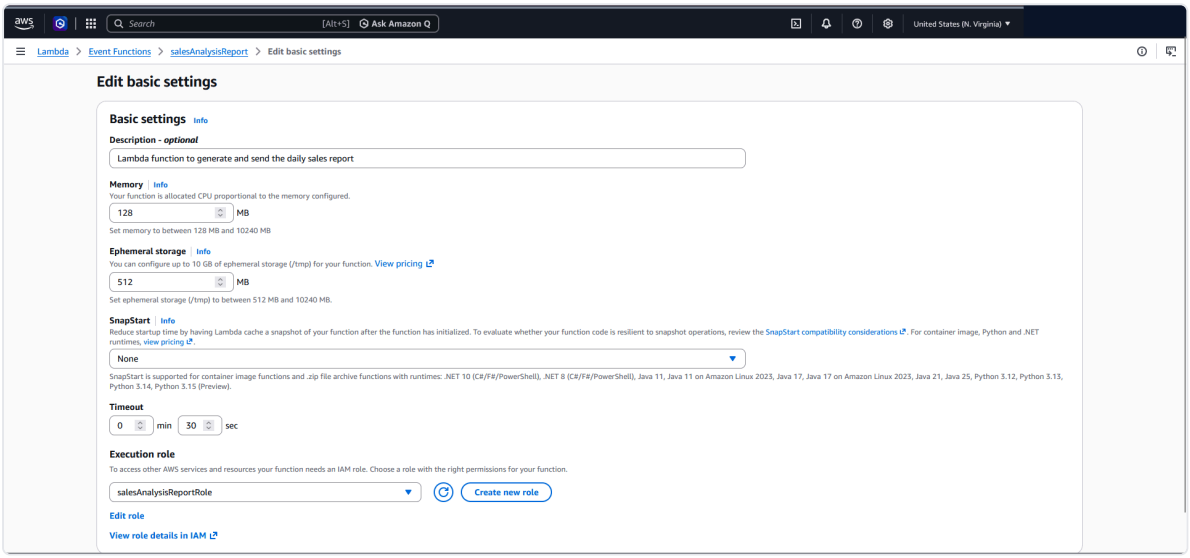
Task 3 — salesAnalysisReport function



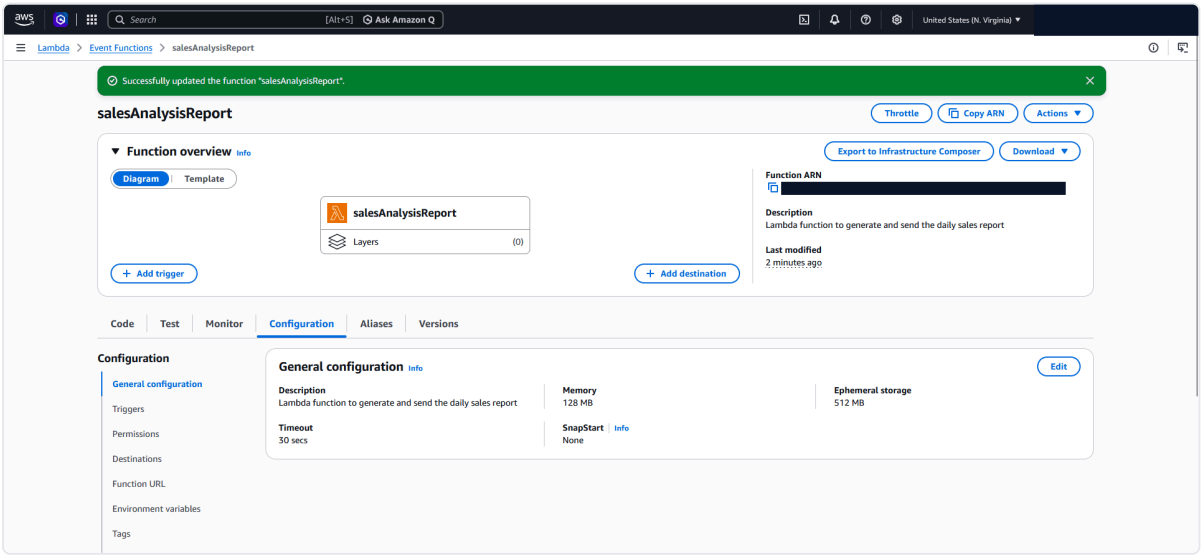
V7-E14 — Report function code uploaded



V7-E15 — Runtime, handler and code properties

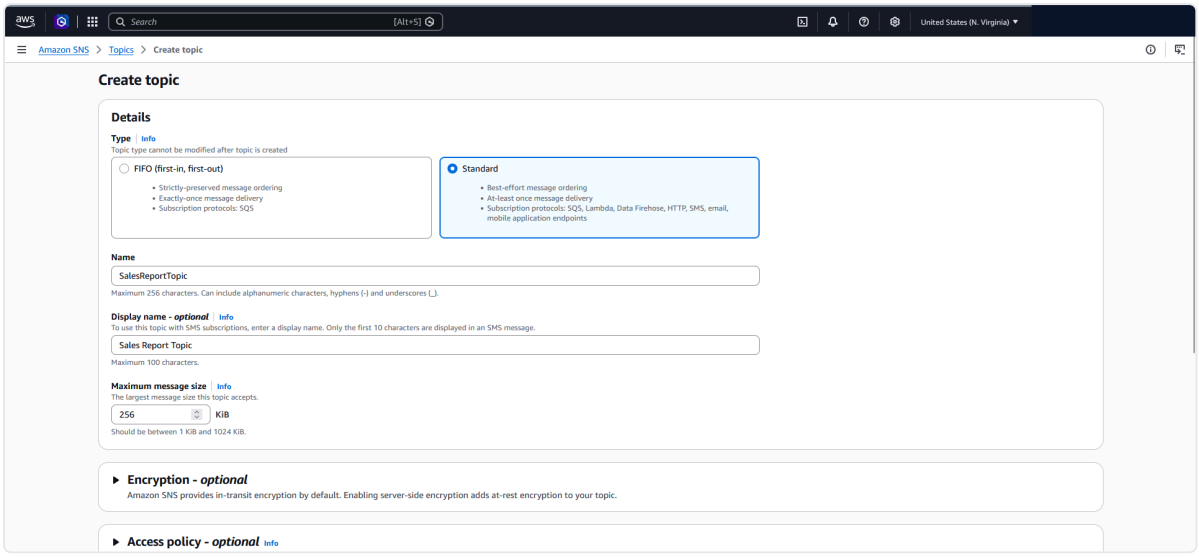


V7-E16 — Basic settings: 128 MB, 30 s, salesAnalysisReportRole

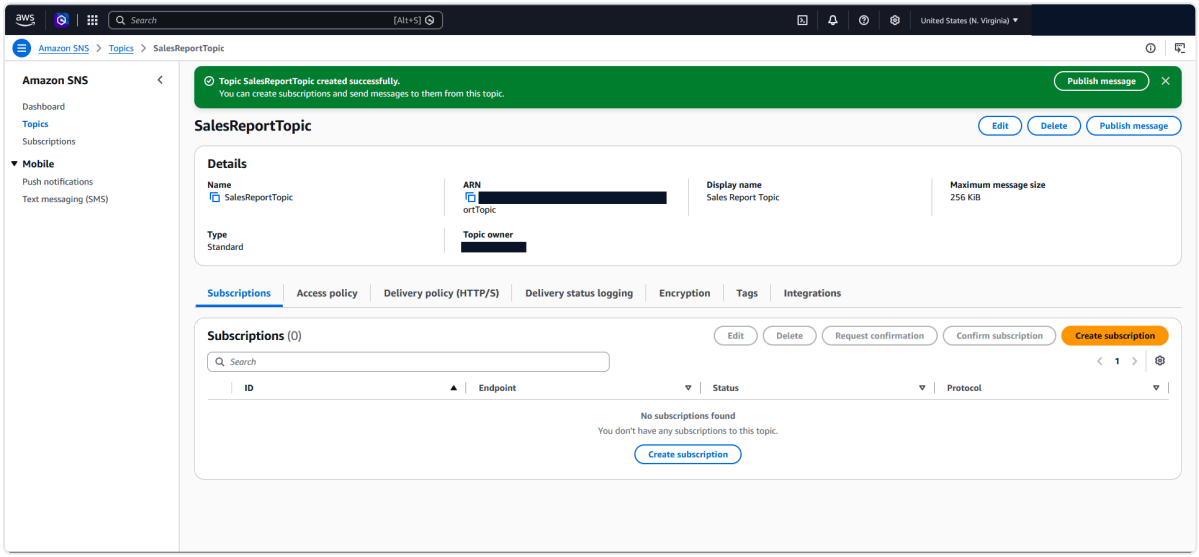


V7-E17 — General configuration confirmed

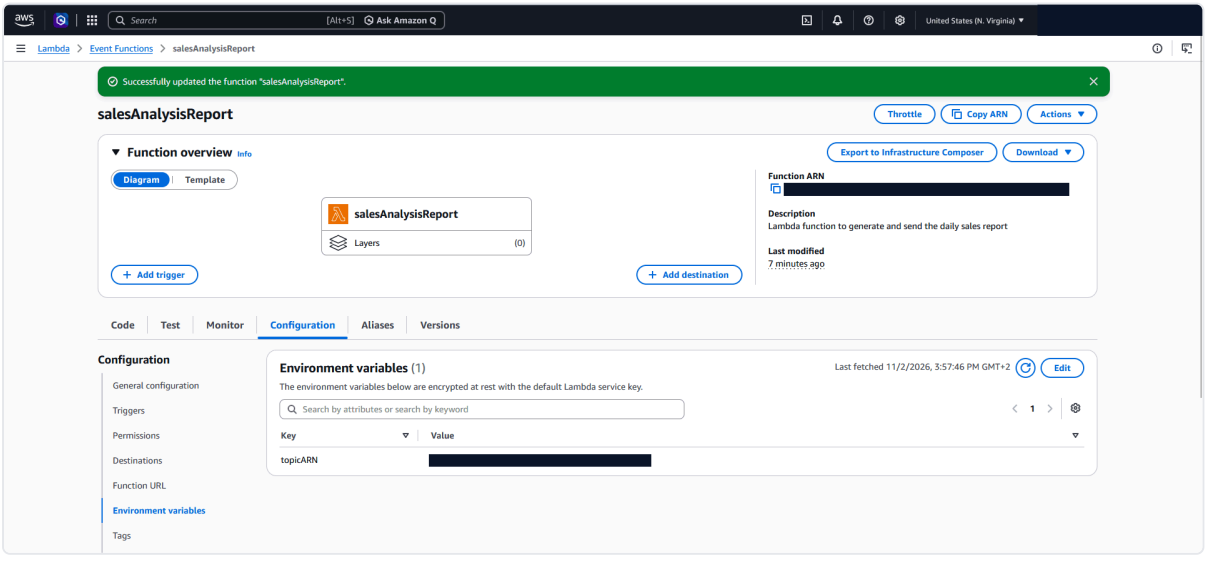
Task 4 — SNS topic



V7-E18 — Creating the standard SNS topic SalesReportTopic

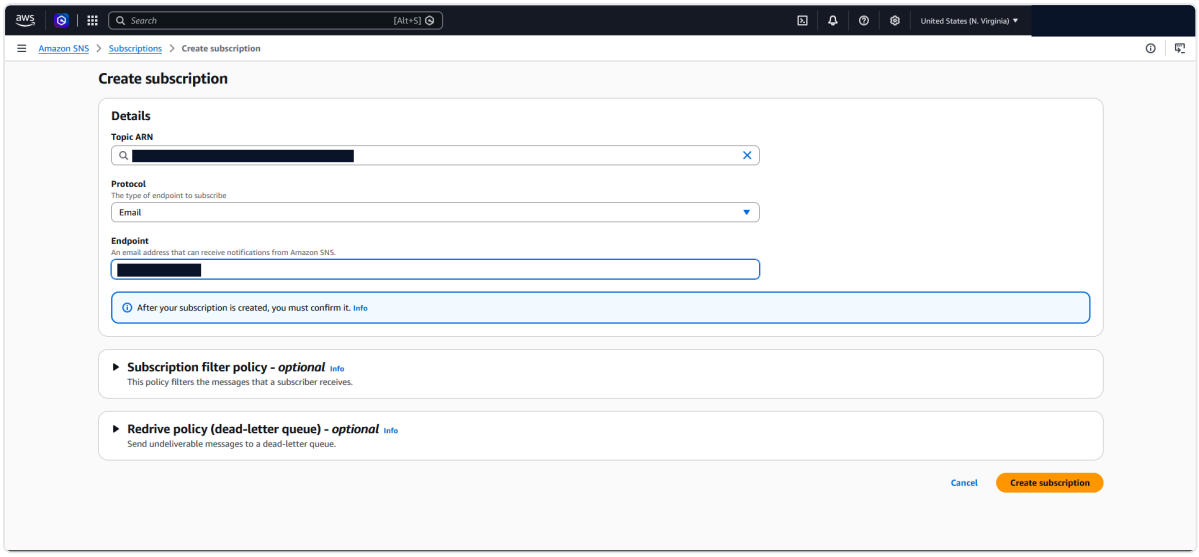


V7-E19 — Topic created, no subscriptions yet

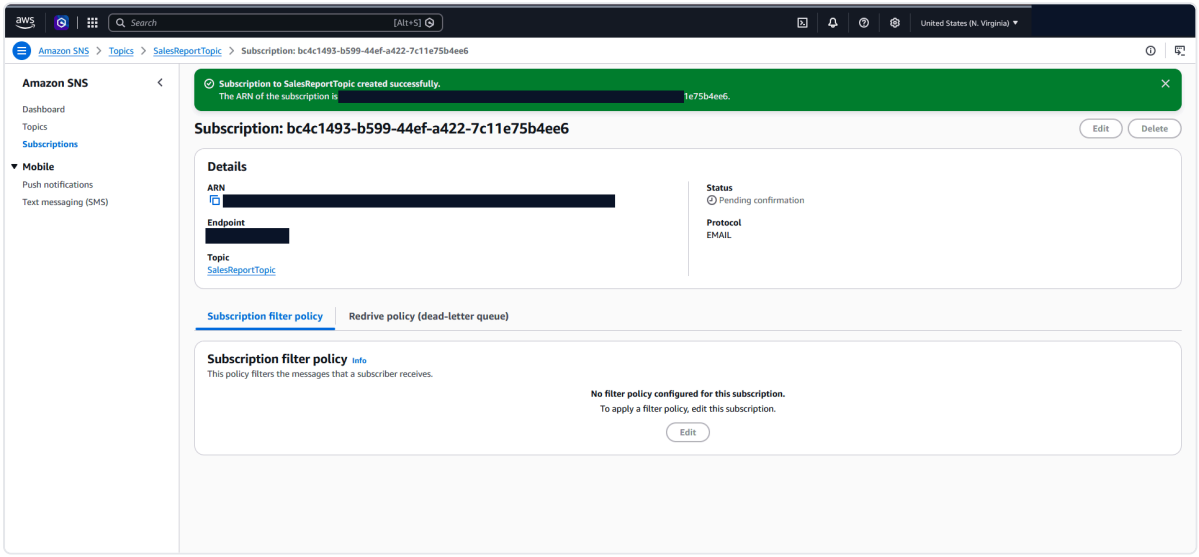


V7-E20 — TopicARN environment variable set on salesAnalysisReport

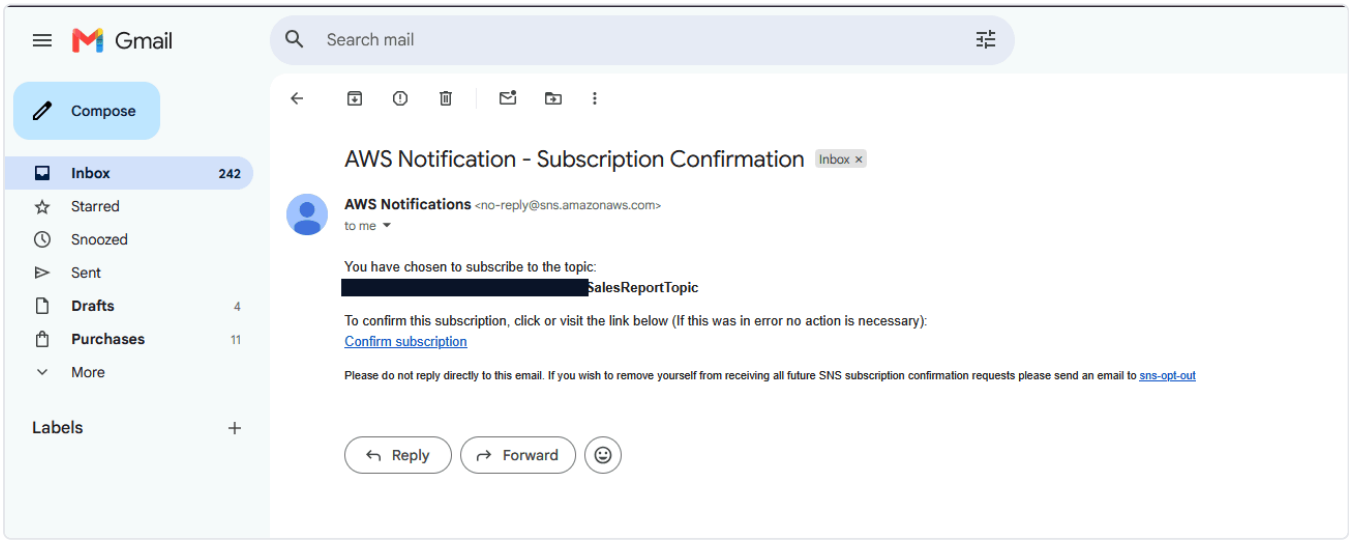
Task 5 — Email subscription



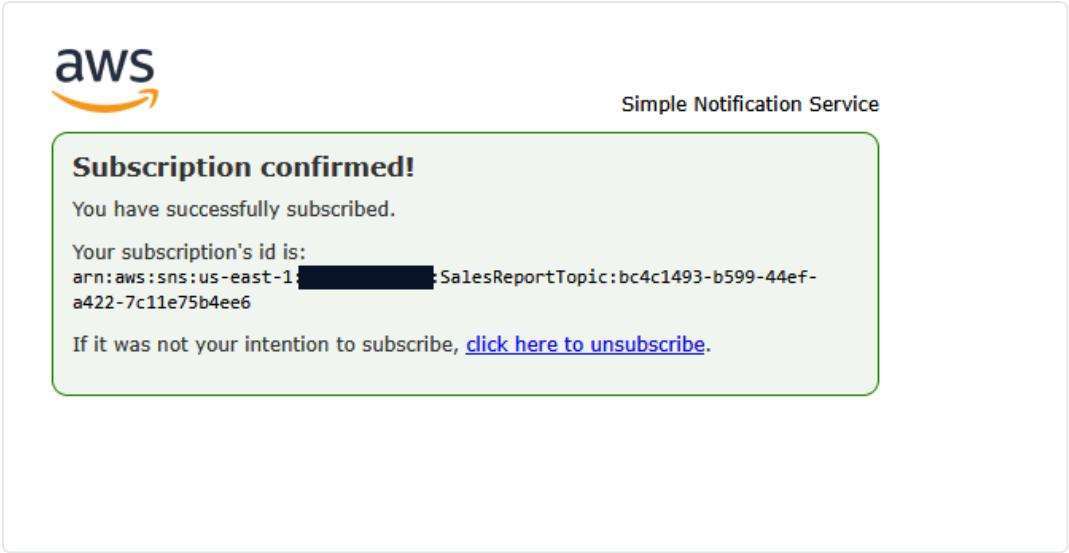
V7-E21 — Creating the email subscription on the topic



V7-E22 — Subscription created, status Pending confirmation

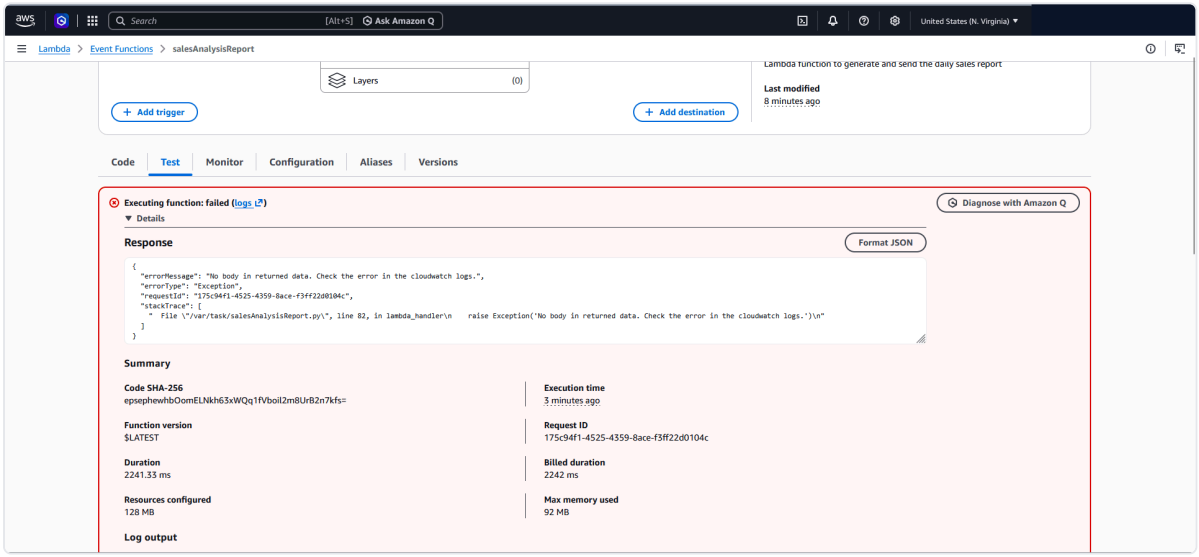


V7-E23 — AWS confirmation email received

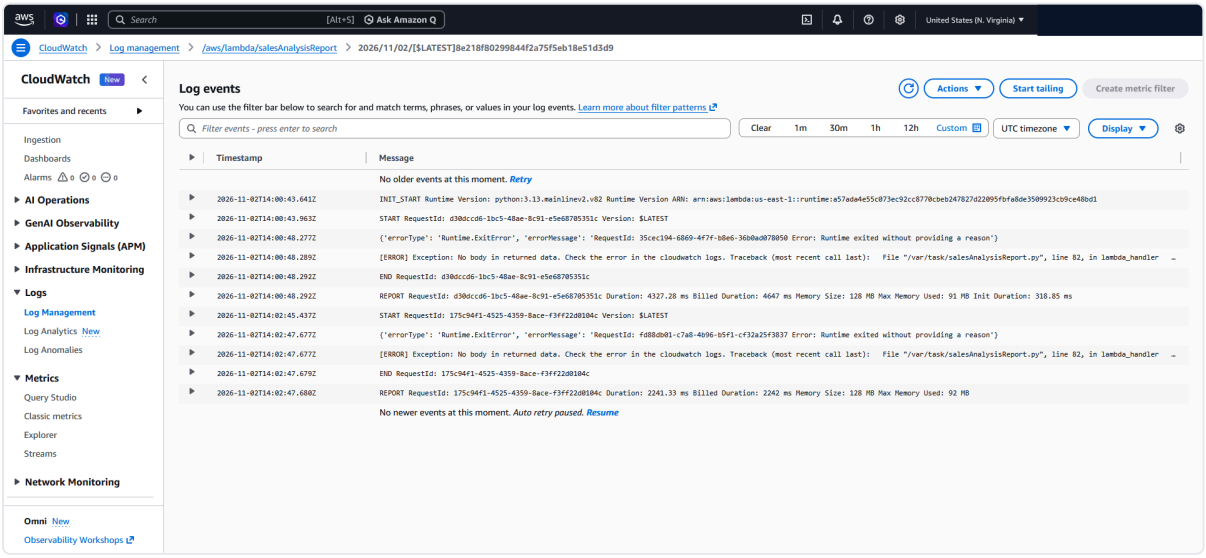


V7-E24 — Subscription confirmed

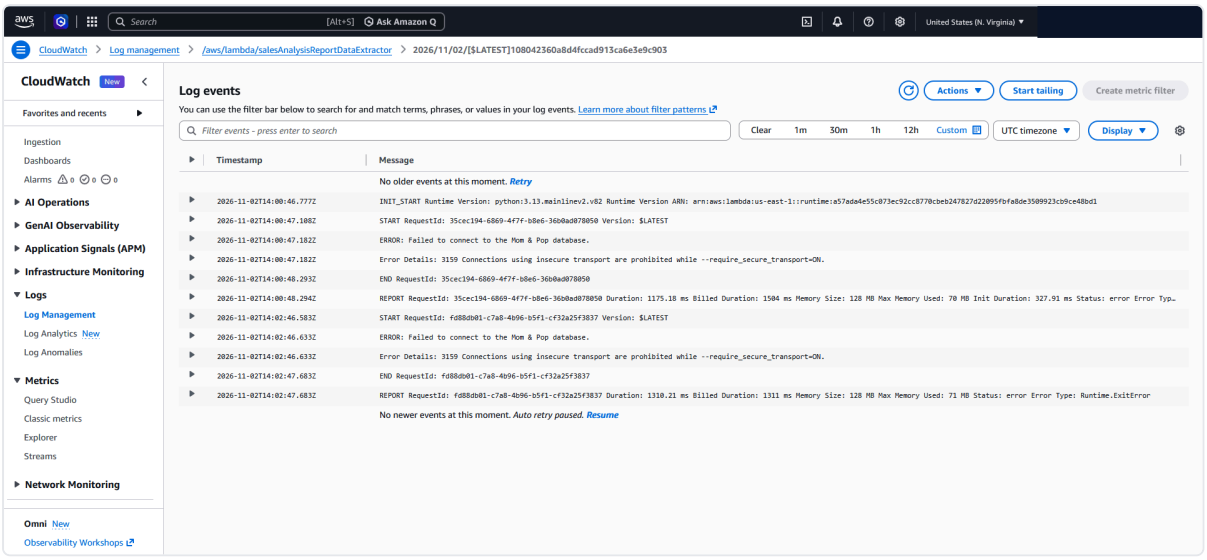
Task 6 — Testing



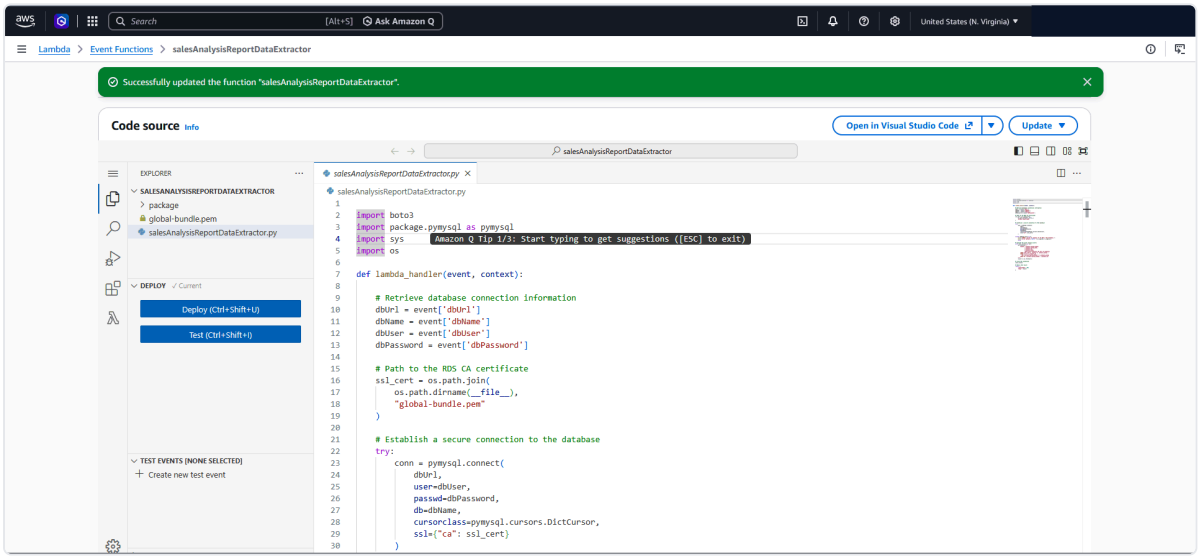
V7-E25 — First test run of salesAnalysisReport failed ("No body in returned data")



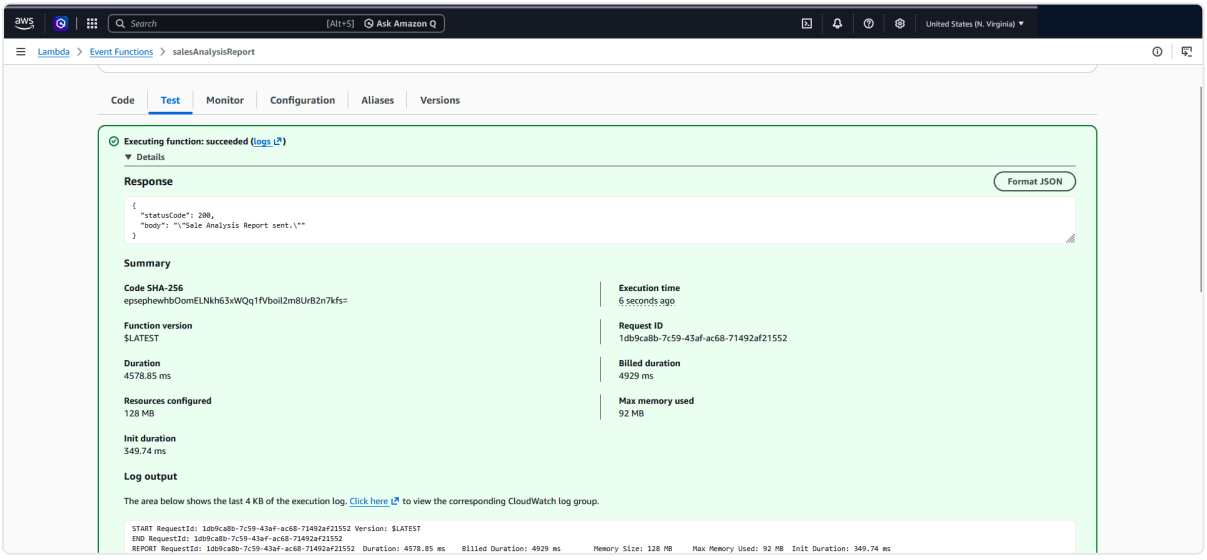
V7-E26 — CloudWatch log of salesAnalysisReport showing the same error



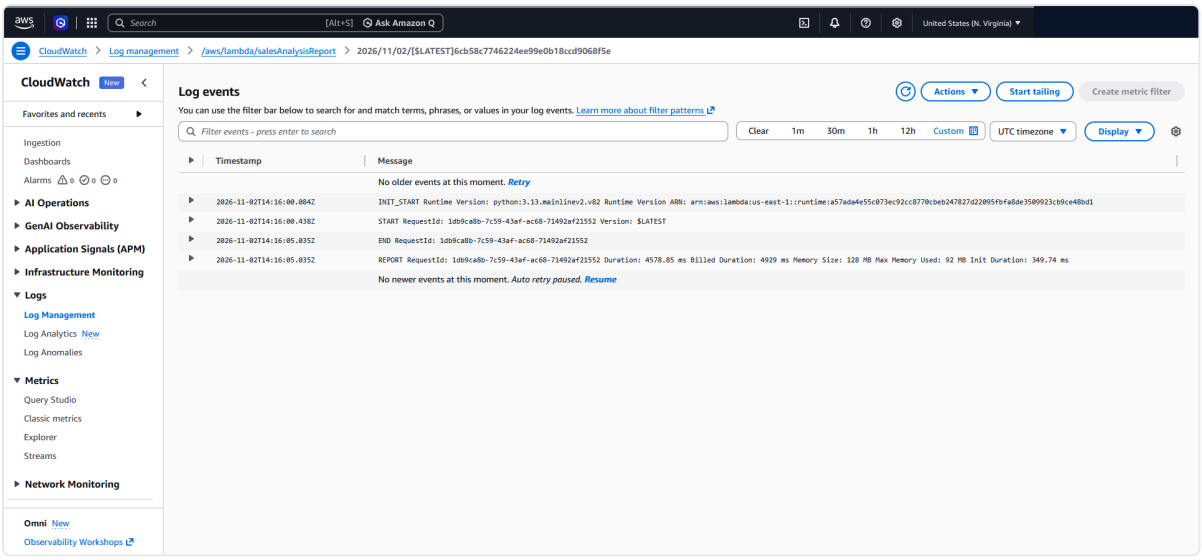
V7-E27 — DataExtractor log: database connection refused because the server requires TLS



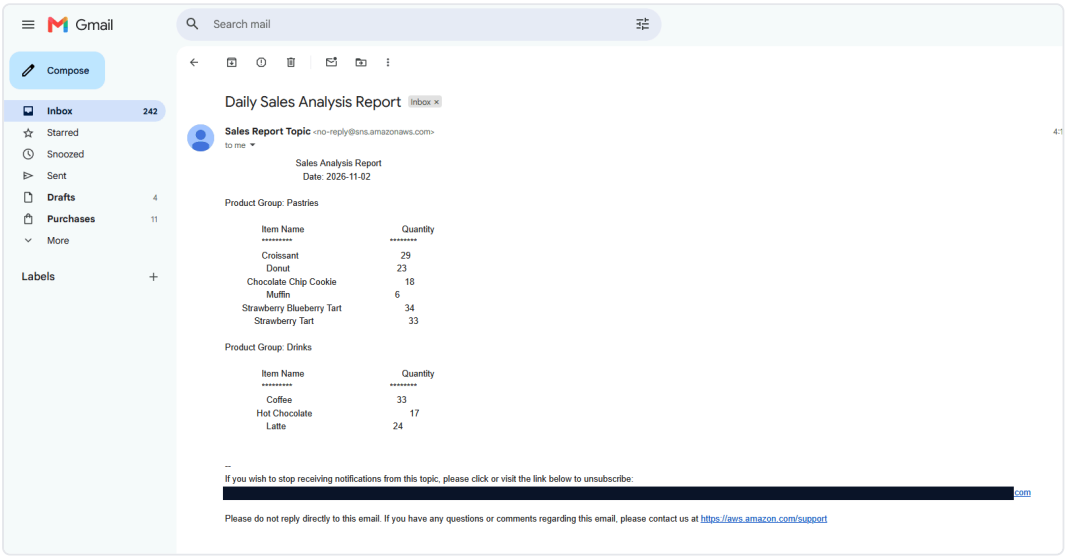
V7-E28 — DataExtractor code updated to connect over TLS using the RDS CA bundle



V7-E29 — Second test run succeeded (statusCode 200, report sent)

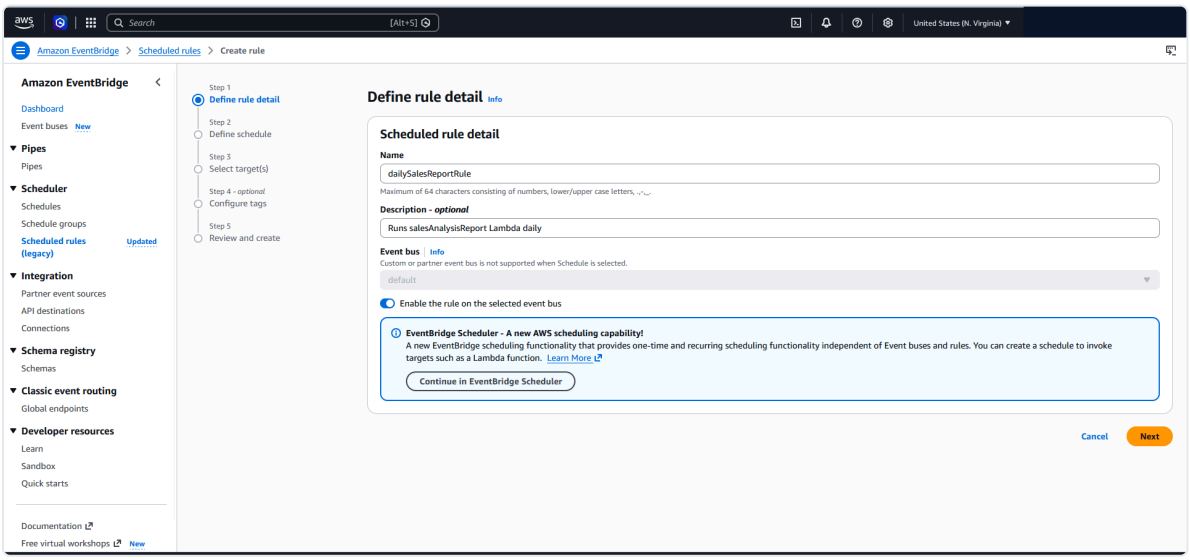


V7-E30 — CloudWatch log of the successful run

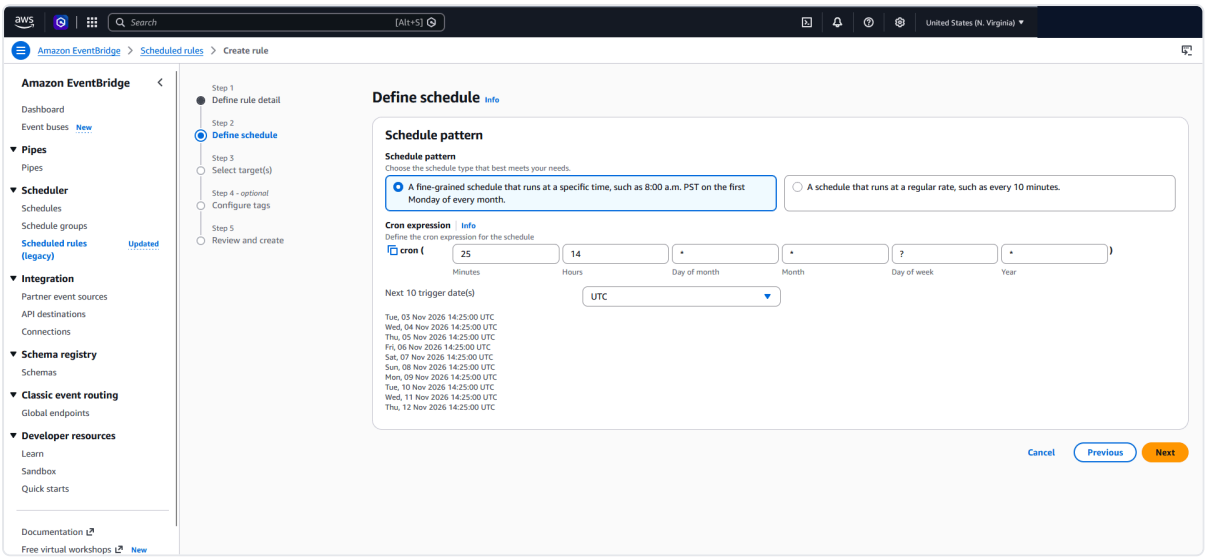


V7-E31 — Daily Sales Analysis Report email received

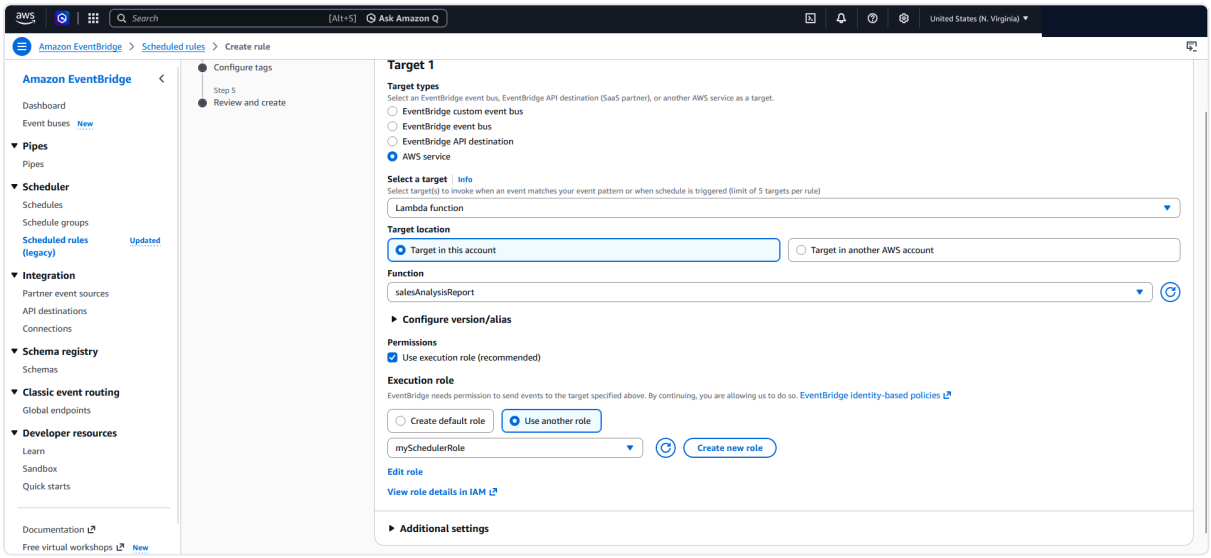
Task 7 — EventBridge



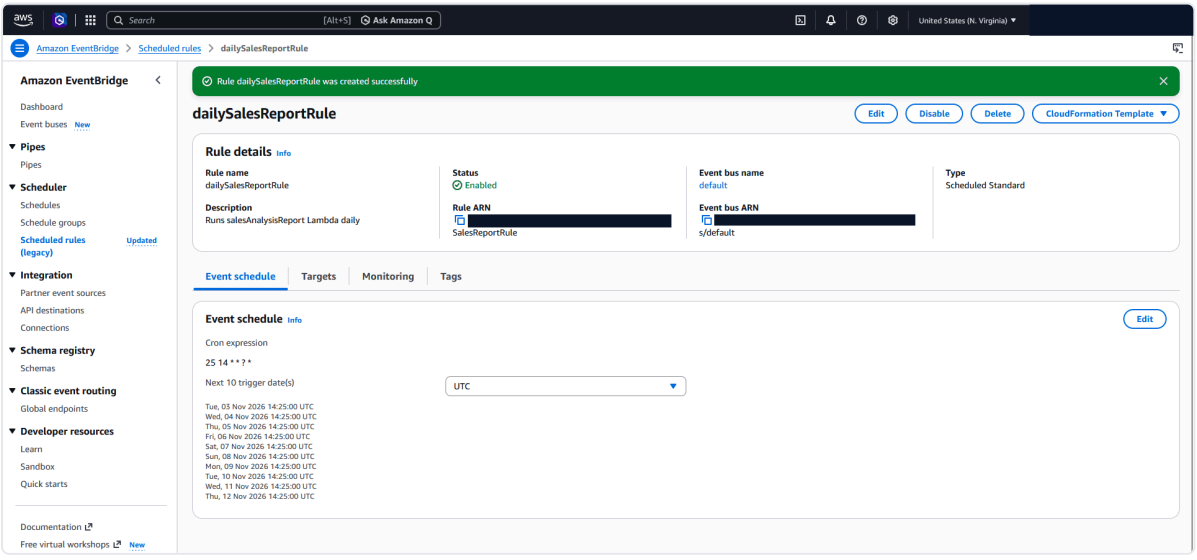
V7-E32 — Defining the scheduled rule dailySalesReportRule



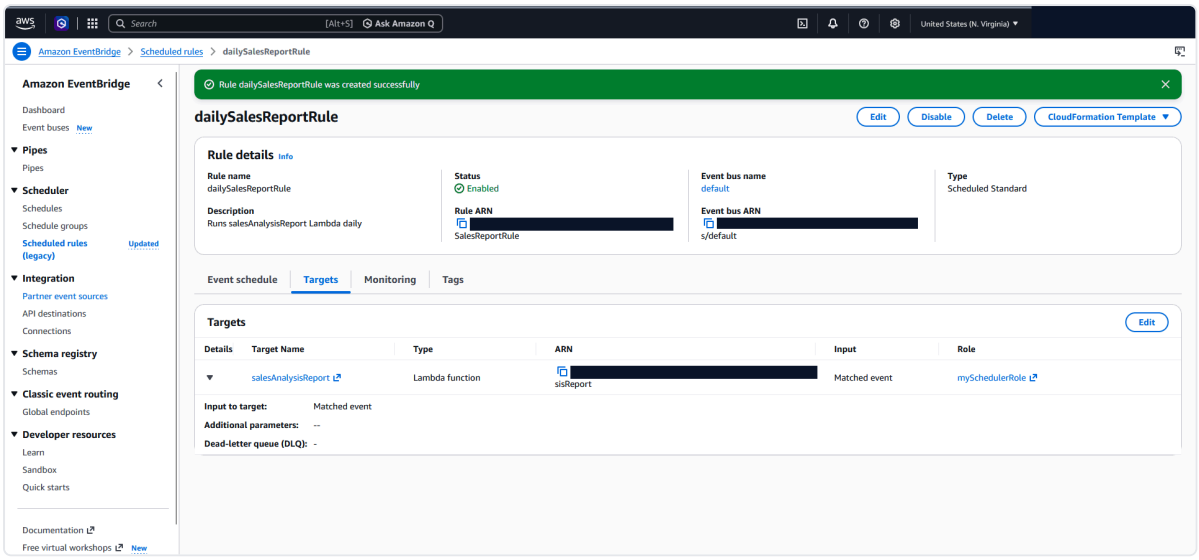
V7-E33 — Cron schedule, 14:25 UTC every day



V7-E34 — Target: salesAnalysisReport Lambda with mySchedulerRole



V7-E35 — Rule created and enabled



V7-E36 — Rule targets tab confirming the Lambda target

Outcome

Sales reporting no longer touches the production web server: a scheduled Lambda pulls the data from RDS through a VPC-attached extractor and emails the report through SNS, and the cost is only the few seconds it runs each day.

PART III

The Complete Architecture

How the stages fit together, and what I learned.

CHAPTER 9

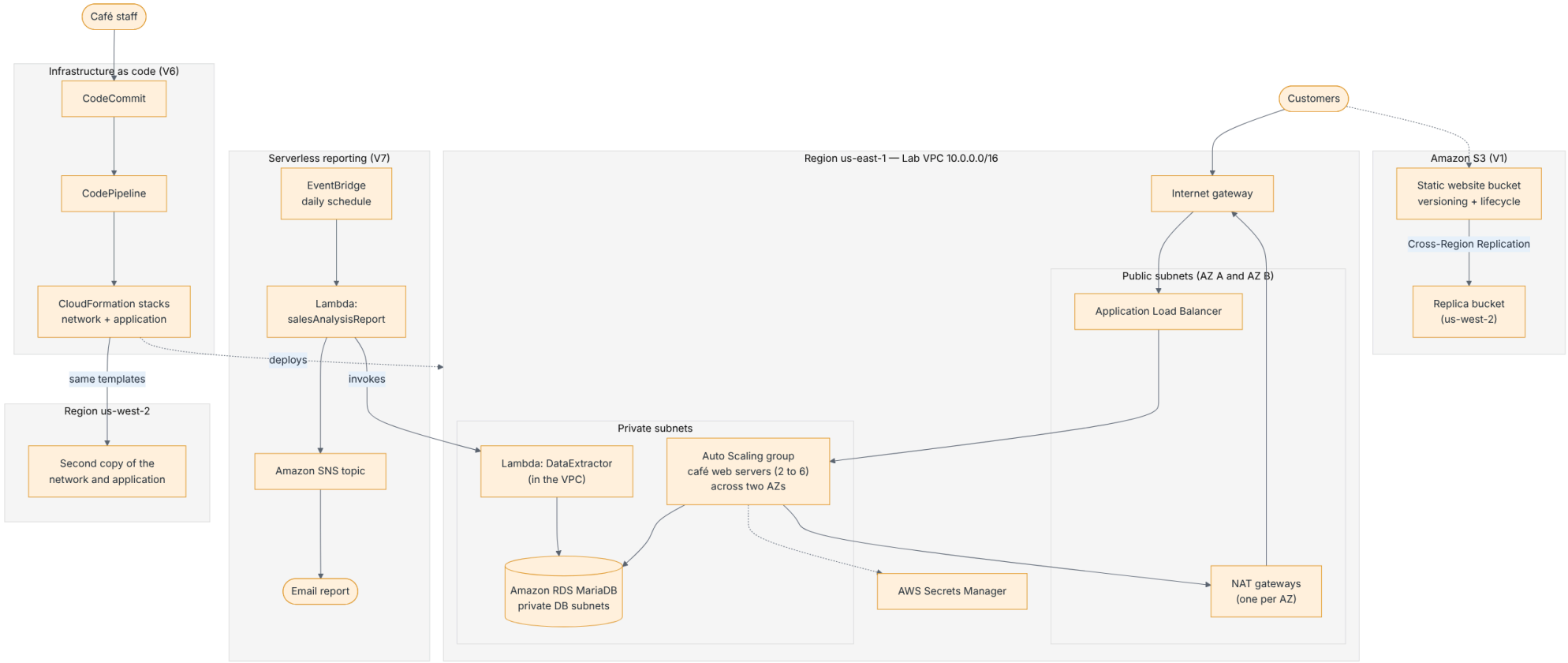
Final Architecture

9.1 Evolution V1 → V7 (Before / After)

STAGE	BEFORE	AFTER
V1	No online presence	Public static website on S3, with versioning, lifecycle rules and Cross-Region Replication
V2	Static page only	Dynamic ordering application on EC2 with Secrets Manager; dev in us-east-1 and prod in us-west-2 from one AMI
V3	Web server and database on one instance	Managed Amazon RDS MariaDB in private subnets, reachable only from the application
V4	No private network tier	VPC with public and private subnets, bastion host, NAT gateway and a custom network ACL
V5	One server in one Availability Zone	Application Load Balancer and Auto Scaling group across two AZs, scaling from 2 to 6 servers
V6	Infrastructure built by hand	CloudFormation templates in CodeCommit, deployed by pipelines; the same templates created a second copy in us-west-2
V7	Reporting would load the web server	Scheduled Lambda functions read RDS and email a daily sales report through SNS

9.2 Final Architecture Diagram

This is a composite view of how the stages fit together. Each lab ran in its own lab environment, so this diagram shows the combined design, not one environment I deployed end to end.



Composite architecture of stages V1 to V7.

9.3 Business Requirement → Solution

BUSINESS REQUIREMENT	SOLUTION
Be visible online	Static website hosted on Amazon S3 (V1)
Protect content and control storage cost	S3 versioning and lifecycle rules (V1)
Survive the loss of a Region (website content)	S3 Cross-Region Replication to us-west-2 (V1)
Take online orders	Web application and database on EC2, settings in Secrets Manager (V2)
Separate development from production	AMI copied to us-west-2 and launched as the production server (V2)
Less database maintenance and safer data	Amazon RDS MariaDB in private subnets, access limited by security group (V3)
Keep the application server away from the internet	Private subnet, bastion host, NAT gateway, network ACL (V4)
Absorb a traffic spike and survive an AZ failure	Application Load Balancer, Auto Scaling group across two AZs, second NAT gateway (V5)
Repeatable, identical deployments in any Region	CloudFormation templates in Git, deployed by CodePipeline (V6)
Daily sales report without slowing the web server or paying for an idle server	Lambda, EventBridge schedule and SNS e-mail (V7)

CHAPTER 10

Key Decisions & Lessons Learned

10.1 Challenges I Faced and How I Solved Them

STAGE	PROBLEM	WHAT I FOUND AND DID
V1	The website showed nothing after hosting was enabled.	Block all public access was still on. I turned it off and added a read-only (`s3:GetObject`) bucket policy.
V1	Replication copied only new objects.	Replication applies to new objects only. A one-time job would be needed to copy the existing files for a complete DR copy.
V2	The test page on port 8000 would not load.	The security group did not allow TCP 8000. I added an inbound rule.
V3	First connection from EC2 to RDS failed with ERROR 2002.	I added an inbound rule on the database security group for TCP 3306 with a security group as the source, then confirmed the port with nmap.
V4	SSH to the bastion stopped after my home IP changed.	I added a new inbound rule for the new IP. Next time I would remove the old rule, and use Session Manager instead of open SSH.
V4	The private instance could not reach the internet after the NAT gateway was created.	The NAT gateway must be in the public subnet and the private route table must send 0.0.0.0/0 to it.
V4	Network ACL rules depend on order and direction.	ACL rules are evaluated in number order, so the deny rule (99) has to come before the allow rule (100), and a matching rule is needed in the outbound direction because ACLs are stateless.
V6	The application pipeline failed on its first run, and the first template upload failed (NoSuchBucket).	The pipeline was run again and then succeeded; the upload was repeated and succeeded. The root cause was not recorded in my notes.
V7	The report test failed with "No body in returned data".	The extractor log showed the database required TLS (error 3159). The extractor was updated to use the RDS CA bundle, and the test then succeeded.

10.2 Architecture Trade-offs

- **Read-only public bucket policy.** The public can view the site but cannot list, change or delete anything. It is less convenient than open access and much safer.
- **Versioning versus storage growth (V1).** Versioning protects against overwrites but keeps every version, so I paired it with lifecycle rules.
- **Bastion host versus public IPs (V4).** One hardened entry point is easier to protect than many public servers, but a single-IP SSH rule is fragile when the home IP changes.
- **One NAT gateway per Availability Zone (V5).** A single NAT gateway would be a single point of failure for the second AZ. The second gateway adds cost in return for availability.

- **Minimum 2, maximum 6 instances (V5).** Two servers keep one in each AZ, and the upper limit caps cost during a traffic spike.
- **Pipelines versus manual deployment (V6).** Every change becomes a Git commit that is deployed the same way every time, at the price of setting up the pipeline first.
- **Lambda versus cron on the web server (V7).** The report no longer slows the production site and costs only the seconds it runs, at the price of a VPC-attached function that needs its own security group and TLS settings.

10.3 What I Would Improve Next

- Put a CDN with HTTPS in front of the S3 website (V1 serves HTTP only).
- Run a one-time replication job so the second Region holds the existing site files, not only new uploads.
- Replace open SSH with AWS Systems Manager Session Manager or a VPN, and remove stale security group rules.
- Keep the secret and configuration values in one managed place and rotate them.
- Verify the scheduled report end to end by capturing the e-mail that arrives from the EventBridge run, not only from a manual test.

APPENDICES

Reference Material

AWS services used, key settings and the full evidence index.

APPENDIX A

AWS Services Used

STAGE	SERVICE / FEATURE	ROLE IN THE STAGE
V1	Amazon S3	Stores the website files (`index.html`, CSS, images)
V1	S3 static website hosting	Serves the bucket content through an S3 website endpoint
V1	Bucket policy	Allows public read-only access (`s3:GetObject`) so visitors can load the site
V1	S3 Versioning	Keeps previous versions of objects; protects against overwrites
V1	S3 Lifecycle rules	Moves old versions to a cheaper class and later deletes them
V1	S3 Cross-Region Replication (CRR)	Copies new/changed objects to a bucket in a second Region (DR)
V1	IAM role	Lets S3 replicate objects between the two buckets
V2	Amazon EC2	Hosts the web server, the café application, and the database (dev and prod)
V2	Security Groups	Allow inbound TCP 8000 so the site is reachable in the browser
V2	AWS Secrets Manager	Stores the application's configuration and DB credentials outside the code
V2	Amazon Machine Image (AMI)	Captures the configured dev server so it can be reproduced
V2	IAM role (CafeRole)	Attached to the EC2 instances so the app can read its secrets
V3	Amazon RDS (MariaDB)	Managed database replacing the local MariaDB on EC2
V3	Amazon EC2	Hosts the café web application (database removed from it)
V3	AWS Secrets Manager	Holds the DB connection parameters the PHP code reads
V3	AWS Systems Manager (Session Manager)	Browser-based shell into the EC2 instance
V3	VPC security groups	Control which source can reach the database on port 3306
V4	Amazon VPC	The isolated network (`10.0.0.0/16`) holding all resources
V4	Subnets (public + private)	Separate internet-facing and internal resources
V4	Internet gateway	Connects the public subnet to the internet
V4	Route tables	Public route to the IGW; separate private route table pointing to the NAT gateway
V4	NAT gateway + Elastic IP	Lets private instances start outbound connections (patches, updates) without inbound exposure

STAGE	SERVICE / FEATURE	ROLE IN THE STAGE
V4	Amazon EC2	Bastion host, private instance and test instance (Amazon Linux 2023, `t2.micro`)
V4	Security groups	Instance-level firewall (SSH from my IP to bastion; SSH from bastion SG to private instance; ICMP for the test instance)
V4	Network ACL	Subnet-level, stateless firewall layer with explicit allow/deny rules
V4	Key pairs / SSH (PuTTY)	Key-based SSH access to the bastion and, through it, to the private instance
V5	Amazon VPC (subnets, route tables, NAT gateway)	Second NAT gateway so private instances in the second AZ can reach the internet
V5	Amazon EC2 (AMI, launch template)	Repeatable definition of the café web server (`CafeWebAppServer-temp`, t2.micro)
V5	Amazon EC2 Auto Scaling	Keeps 2 to 6 web servers running across two AZs and scales on average CPU
V5	Elastic Load Balancing (Application Load Balancer)	Public entry point that spreads HTTP requests across the private web servers
V5	Amazon EC2 security groups	`CafeAPPELBallowHTTP` allows HTTP (port 80) from the internet to the load balancer
V5	AWS Systems Manager Session Manager	Shell access to a web server to run the CPU stress test
V6	AWS CloudFormation	Defines the S3 bucket, the network layer and the application layer as templates and stacks
V6	AWS CodeCommit	Git repository (`CFTemplatesRepo`) that stores and versions the templates
V6	AWS CodePipeline	`CafeNetworkPipeline` and `CafeAppPipeline` deploy a stack whenever a template is pushed
V6	Amazon EC2 and VPC	Resources created by the templates (Cafe VPC, public subnet, security group, web server)
V6	Amazon S3	First practice stack (static website bucket) and storage for the template used in the second Region
V6	AWS CLI and Git	Creating and validating stacks, and pushing templates from the lab IDE
V7	AWS Lambda	Two functions: a data extractor (inside the VPC) and the report generator
V7	Amazon VPC / Security Groups	`LambdaSG` for the extractor; `DatabaseSG` updated to accept it
V7	Amazon RDS	Source of the café sales data (reached privately)
V7	Amazon SNS	Topic `SalesReportTopic` with an email subscription

STAGE	SERVICE / FEATURE	ROLE IN THE STAGE
V7	Amazon EventBridge	Scheduled rule that runs the report daily
V7	Amazon CloudWatch Logs	Debugging the failed test runs
V7	AWS IAM	Execution roles for the functions and `mySchedulerRole` for the schedule

APPENDIX B

Configuration Reference

Important settings only, as recorded in the stage chapters.

STAGE	SETTING	VALUE
V1	Lifecycle rules	Non-current versions move to Standard-IA after 30 days and are deleted after 365 days
V1	Replication	Cross-Region Replication from the source bucket to a bucket in us-west-2 (Oregon)
V1	Bucket policy	Public read-only (`s3:GetObject`), Block public access turned off
V2	Web server	Apache listening on port 8000; security group allows TCP 8000
V2	Secrets	Seven application parameters in AWS Secrets Manager, recreated in us-west-2
V2	Production server	`ProdCafeServer` (t2.small) launched from the `CafeServer` AMI in us-west-2
V3	Database	Amazon RDS MariaDB `CafeDatabase` (db.t3.micro, us-east-1) in private DB subnets
V3	Database access	Inbound TCP 3306 on `dbSG` , source is the application security group
V4	Network	Lab VPC `10.0.0.0/16` ; public subnet `10.0.0.0/24` ; private subnet `10.0.1.0/24`
V4	Network ACL	Deny rule number 99 evaluated before allow-all rule number 100; inbound and outbound rules
V4	Routing	Public route `0.0.0.0/0` to the internet gateway; private route `0.0.0.0/0` to the NAT gateway
V5	Auto Scaling group	Desired 2, minimum 2, maximum 6; private subnets in two AZs; scales on average CPU (target 25%, 60 s warm-up)
V5	Launch template	Café AMI, t2.micro
V5	Load balancer	`CafeAppELB` , internet-facing, in two public subnets, listener HTTP:80 to target group `CafeAPPELB-Targetgroup`
V6	Pipelines and stacks	`CafeNetworkPipeline` builds `update-cafe-network` ; `CafeAppPipeline` builds `update-cafe-app`
V6	Second Region	Network stack and `cafe-app-oregon` in us-west-2 with `t3.micro` and key pair `cafe-oregon`
V7	Lambda functions	`salesAnalysisReportDataExtractor` in the VPC (private subnets 1 and 2, `LambdaSG`) and `salesAnalysisReport` ; 128 MB memory, 30 s timeout each
V7	Database access	Second inbound rule (TCP 3306) on the database security group with `LambdaSG` as the source
V7	SNS and schedule	Topic `SalesReportTopic` with an e-mail subscription; EventBridge rule `dailySalesReportRule` , daily at 14:25 UTC

APPENDIX C

Evidence Index

All 189 screenshots. Each ID matches the caption in the stage chapter.

V1 — Static Website on Amazon S3

ID	WHAT IT SHOWS
V1-E01	Site files uploaded to the bucket (9 files, 0 failed)
V1-E02	Public-read bucket policy, Block public access off
V1-E03	Website live at the S3 website endpoint
V1-E04	Enabling bucket versioning
V1-E05	Multiple versions of `index.html`
V1-E06	Lifecycle rule: noncurrent versions → Standard-IA after 30 days
V1-E07	Lifecycle rule: delete noncurrent versions after 365 days
V1-E08	Both lifecycle rules enabled
V1-E09	Cross-Region Replication rule
V1-E10	Replicated object in the Oregon bucket

V2 — Dynamic Website on Amazon EC2

ID	WHAT IT SHOWS
V2-E01	Lab ide instance running
V2-E02	Apache running port 8000
V2-E03	Mariadb running
V2-E04	Inbound rule for TCP 8000 added to the instance's security group
V2-E05	Hello page port 8000
V2-E06	Seven application secrets created in AWS Secrets Manager
V2-E07	Database menu table
V2-E08	Café home page served from the dev instance in us-east-1
V2-E09	Test order recorded on the Order History page (dev)
V2-E10	Create ami cafeserver
V2-E11	`CafeServer` AMI available, with the copy to us-west-2 started
V2-E12	Launch prodcafeserver from ami
V2-E13	Prod instance running oregon
V2-E14	Set app parameters us west 2
V2-E15	Café app running on `ProdCafeServer` in us-west-2, with its own order history
V2-E16	Ssh to prod instance

V3 — Database Migration to Amazon RDS

ID	WHAT IT SHOWS
V3-E01	RDS console: `CafeDatabase` created and showing Available (MariaDB, db.t3.micro, us-east-1)
V3-E02	Café website home page running on the EC2 instance (public IP masked)
V3-E03	Order Confirmation page after placing a test order (order #25)
V3-E04	Order History before migration: original orders plus my test order, all served from the local database
V3-E05	Connect to instance dialog with Session Manager selected (browser-based shell, no SSH keys)
V3-E06	Secrets Manager `/cafe/dbPassword` secret (ARN and value masked)
V3-E07	Terminal: local MariaDB 10.2.38 running on the EC2 instance and root login to the SQL client
V3-E08	Source database: `cafe_db` with its four tables and 25 rows in `order`
V3-E09	Source database: rows in `order_item` (items, quantities, prices per order)
V3-E10	`mysqldump` creating `CafeDbDump.sql` and a `cat` of the start of the dump file
V3-E11	RDS connectivity details: endpoint, port 3306, Lab VPC, `lab-db-subnet-group`, `dbSG`, Publicly accessible = No
V3-E12	First connection attempt from EC2 to RDS fails with ERROR 2002 (can't connect)
V3-E13	`dbSG` inbound rule: MySQL/Aurora, TCP 3306, source = a security group
V3-E14	`nmap` shows 3306/tcp open on the RDS endpoint, then a successful login (server 10.6.25-MariaDB)
V3-E15	RDS after the import: `cafe_db` present, four tables, 25 orders
V3-E16	Café menu page (`menu.php`) used to test the application
V3-E17	Order History page showing "Connection failed: Connection refused"
V3-E18	Secrets Manager `/cafe/dbUrl` updated with the RDS endpoint
V3-E19	Terminal: end of the RDS verification output (25 rows), then `sudo service mariadb stop` on the EC2 instance
V3-E20	Secrets Manager `/cafe/dbUser` updated (`admin`)
V3-E21	Secrets Manager `/cafe/dbPassword` updated (value masked)
V3-E22	Final Order History: new order #26 placed after the switch, with all migrated orders still present

V4 — Secure VPC Networking

ID	WHAT IT SHOWS
V4-E01	Public subnet created.
V4-E02	Internet gateway attached.
V4-E03	Route `0.0.0.0/0` to the internet gateway.
V4-E04	Bastion host running with an open SSH session.
V4-E05	Private subnet created.
V4-E06	NAT gateway created.
V4-E07	Private route table with the NAT route.
V4-E08	Private route table associated with the private subnet.

ID	WHAT IT SHOWS
V4-E09	VPC resource map.
V4-E10	Private instance reaches the internet through the NAT gateway.
V4-E11	Ping to the Test Instance works before the deny rule.
V4-E12	Network ACL inbound rules.
V4-E13	Network ACL outbound rules.
V4-E14	Ping blocked by the network ACL.
V4-E15	Subnets associated with the Lab Network ACL.
V4-E16	Internet gateway created.
V4-E17	Attaching the internet gateway to the Lab VPC.
V4-E18	Main route table before adding the internet route.
V4-E19	Launching the bastion host.
V4-E20	Creating the NAT gateway.
V4-E21	Creating the Private Route Table.
V4-E22	Adding the route to the NAT gateway.
V4-E23	PuTTY session to the bastion host.
V4-E24	PuTTY authentication options (agent forwarding).
V4-E25	PuTTY private key setting (file path redacted).
V4-E26	Logged in to the bastion host.
V4-E27	SSH hop from the bastion host to the private instance.

V5 — Scalability & High Availability (ALB + Auto Scaling)

ID	WHAT IT SHOWS
V5-E01	Lab VPC resource map before any change: 4 private subnets, public subnets in two AZs, one NAT gateway.
V5-E02	Creating a zonal NAT gateway (natgw-az2) in Public Subnet 2 with its own Elastic IP.
V5-E03	Private Route Table 2 before the change: only the local route.
V5-E04	Private Route Table 2 after adding 0.0.0.0/0 pointing to the new NAT gateway.
V5-E05	VPC resource map after the change: second NAT gateway (natgw-az2) is attached to the network.
V5-E06	The Cafe WebServer Image AMI is available (created from the café web server during lab setup).
V5-E07	Launch template form: name, My AMIs, Cafe WebServer Image selected.
V5-E08	Launch template CafeWebAppServer-temp created successfully.
V5-E09	Launch templates list and details (version 1, default).
V5-E10	Create Auto Scaling group: name and launch template selected (t2.micro, key pair, security group).
V5-E11	Auto Scaling group created: desired 2, min 2, max 6, two Availability Zones, capacity updating.
V5-E12	Auto Scaling group at desired capacity: 2 instances, 2/2 healthy.
V5-E13	EC2 console: two new "webserver" instances launching, one in us-east-1a and one in us-east-1b.

ID	WHAT IT SHOWS
V5-E14	Creating the new security group for the load balancer (inbound HTTP from anywhere).
V5-E15	Security group CafeAPPELBallowHTTP created: one inbound rule, HTTP port 80.
V5-E16	Creating the target group: instance targets, HTTP port 80.
V5-E17	Target group CafeAPPELB-Targetgroup created, no targets registered yet.
V5-E18	Application Load Balancer CafeAppELB created, internet-facing, across two AZs, HTTP:80 listener forwarding to the target group.
V5-E19	Café web application loading through the load balancer DNS name at /cafe.
V5-E20	Session Manager session on a web server: installing the stress tool and starting the CPU test.
V5-E21	Top output during the test: the stress process is using about 100% of one CPU.
V5-E22	Target group with six registered targets, all healthy.
V5-E23	EC2 console after scale-out: six "webserver" instances running across us-east-1a and us-east-1b.

V6 — Infrastructure Automation (CloudFormation + CodePipeline)

ID	WHAT IT SHOWS
V6-E01	Empty VS Code IDE (code-server) on the lab EC2 instance, terminal ready.
V6-E02	Initial CloudFormation stacks created by the lab setup.
V6-E03	Initial S3 buckets created by the lab setup.
V6-E04	S3.yaml (bucket only) and the first create-stack CLI runs.
V6-E05	CreateBucket stack creation in progress.
V6-E06	CreateBucket stack CREATE_COMPLETE.
V6-E07	S3 bucket created by the stack.
V6-E08	Template tab of the CreateBucket stack.
V6-E09	CodeCommit initial commit of the CFTemplatesRepo repository.
V6-E10	Website files downloaded and copied to the bucket with the AWS CLI.
V6-E11	Website objects (index.html, css/, images/) in the bucket.
V6-E12	S3.yaml updated with retain policy, website configuration and URL output; template validated and stack update started.
V6-E13	CreateBucket stack UPDATE_IN_PROGRESS.
V6-E14	CreateBucket stack UPDATE_COMPLETE.
V6-E15	Stack Outputs tab showing the WebsiteURL.
V6-E16	Static website hosting enabled on the bucket.
V6-E17	CFTemplatesRepo repository in CodeCommit with the clone URL copied.
V6-E18	Git clone and git status: working tree clean.
V6-E19	Cafe-network.yaml created from the template with the new description.
V6-E20	CafeNetworkPipeline before the first push (Deploy shows Failed, as expected before a template exists).
V6-E21	Git add and git commit of the network template.
V6-E22	Git push of the network template to CodeCommit.

ID	WHAT IT SHOWS
V6-E23	CafeNetworkPipeline triggered: Deploy stage in progress.
V6-E24	Pipelines list: CafeNetworkPipeline In progress, CafeAppPipeline Failed (no template yet).
V6-E25	CafeNetworkPipeline Deploy stage finishing.
V6-E26	Update-cafe-network stack CREATE_COMPLETE.
V6-E27	Cafe VPC created by the pipeline.
V6-E28	Cafe Public Subnet created by the pipeline.
V6-E29	Outputs added to cafe-network.yaml and committed.
V6-E30	Git push of the network outputs.
V6-E31	CafeNetworkPipeline running the update.
V6-E32	CafeNetworkPipeline update succeeded.
V6-E33	Update-cafe-network outputs: PublicSubnet and VpcId with export names.
V6-E34	Cafe-app.yaml (instance, parameter, user data) and git push.
V6-E35	CafeAppPipeline Deploy stage failed on the first run.
V6-E36	CafeAppPipeline running again.
V6-E37	CafeAppPipeline succeeded.
V6-E38	Update-cafe-app stack CREATE_COMPLETE with the WebServerPublicIP output.
V6-E39	EC2 console: Cafe Web Server (t2.small) created by the stack next to the Lab IDE instance.
V6-E40	CafeSG security group created by the app stack.
V6-E41	Café website running in us-east-1 (server information shows Region/AZ).
V6-E42	CLI command creating the update-cafe-network stack in us-west-2.
V6-E43	Update-cafe-network stack in Oregon with its outputs.
V6-E44	Cafe VPC in us-west-2.
V6-E45	Cafe Public Subnet in us-west-2.
V6-E46	Key pair cafe-oregon created in Oregon.
V6-E47	Uploading cafe-app.yaml to the repository bucket: first attempt failed (NoSuchBucket), second succeeded.
V6-E48	Repository bucket containing cafe-app.yaml.
V6-E49	Cafe-app.yaml object details with the object URL copied.
V6-E50	Cafe-app-oregon stack creation in progress.
V6-E51	Stacks list in Oregon.
V6-E52	Cafe-app-oregon stack CREATE_COMPLETE.
V6-E53	Cafe-app-oregon outputs (WebServerPublicIP).
V6-E54	Oregon Cafe Web Server instance (t3.micro).
V6-E55	Café website running in us-west-2 (server information shows the Oregon AZ).

V7 — Serverless Reporting (Lambda + SNS)

ID	WHAT IT SHOWS
V7-E01	Creating the LambdaSG security group in the Lab VPC (all outbound allowed)
V7-E02	LambdaSG created
V7-E03	Editing the database security group to add a second inbound rule
V7-E04	DatabaseSG now allows MySQL/Aurora (3306) from both the café app SG and LambdaSG
V7-E05	Lambda console before creating the functions
V7-E06	Creating salesAnalysisReportDataExtractor with the custom execution role
V7-E07	Attaching the function to Lab VPC, Private subnet 1 + 2 and LambdaSG
V7-E08	VPC configuration summary (2 subnets, 1 security group)
V7-E09	Function code package uploaded
V7-E10	Basic settings: 128 MB memory, 30 s timeout, salesAnalysisReportDERole
V7-E11	General configuration confirmed
V7-E12	Runtime and handler settings
V7-E13	Handler code in the console editor
V7-E14	Report function code uploaded
V7-E15	Runtime, handler and code properties
V7-E16	Basic settings: 128 MB, 30 s, salesAnalysisReportRole
V7-E17	General configuration confirmed
V7-E18	Creating the standard SNS topic SalesReportTopic
V7-E19	Topic created, no subscriptions yet
V7-E20	TopicARN environment variable set on salesAnalysisReport
V7-E21	Creating the email subscription on the topic
V7-E22	Subscription created, status Pending confirmation
V7-E23	AWS confirmation email received
V7-E24	Subscription confirmed
V7-E25	First test run of salesAnalysisReport failed ("No body in returned data")
V7-E26	CloudWatch log of salesAnalysisReport showing the same error
V7-E27	DataExtractor log: database connection refused because the server requires TLS
V7-E28	DataExtractor code updated to connect over TLS using the RDS CA bundle
V7-E29	Second test run succeeded (statusCode 200, report sent)
V7-E30	CloudWatch log of the successful run
V7-E31	Daily Sales Analysis Report email received
V7-E32	Defining the scheduled rule dailySalesReportRule
V7-E33	Cron schedule, 14:25 UTC every day
V7-E34	Target: salesAnalysisReport Lambda with mySchedulerRole
V7-E35	Rule created and enabled

ID	WHAT IT SHOWS
V7-E36	Rule targets tab confirming the Lambda target